# Networks

## Physical, Application & Transport Layers

Sadi Sir's Lectures (Lectures 2-4, Chapter 2, L7-L14)

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

# Table of Contents




1 Chapter 2 - Application Layer p. 3

2 L02-L04 Physical Layer - Line Coding... p. 124

3 L7 Transport Layer Introduction p. 149

4 L8 Reliable Data Transfer p. 171

5 L9 Transmission Control Protocol p. 215

6 L10 TCP Flow Control p. 255

7 L11 TCP Error Control & Congestion C... p. 283

8 L12 TCP RTT Estimation p. 323

9 L13 UDP p. 353

10 L14 Transport Layer Security p. 376

# Chapter 2 Application Layer

A note on the use of these PowerPoint slides:

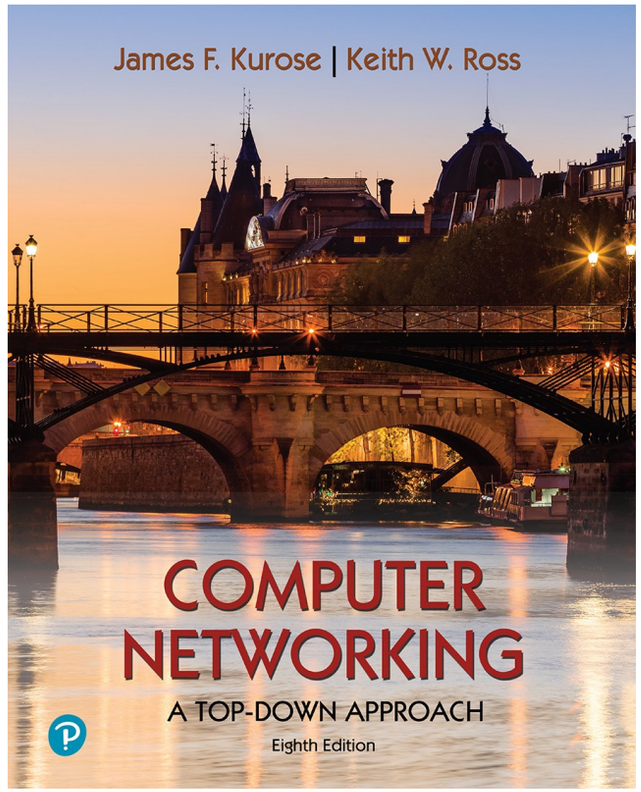
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

For a revision history, see the slide note for this page.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2023
J.F Kurose and K.W. Ross, All Rights Reserved




*Computer Networking: A Top-Down Approach*

8th edition n
Jim Kurose, Keith Ross
Pearson, 2020

# Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS

- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



# Application layer: overview

Our goals:

- conceptual *and* implementation aspects of application-layer protocols
  - transport-layer service models
  - client-server paradigm
  - peer-to-peer paradigm
- learn about protocols by examining popular application-layer protocols and infrastructure
  - HTTP
  - SMTP, IMAP
  - DNS
  - video streaming systems, CDNs
- programming network applications
  - socket API

# Some network apps

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- …

  *Q: your* favorites?

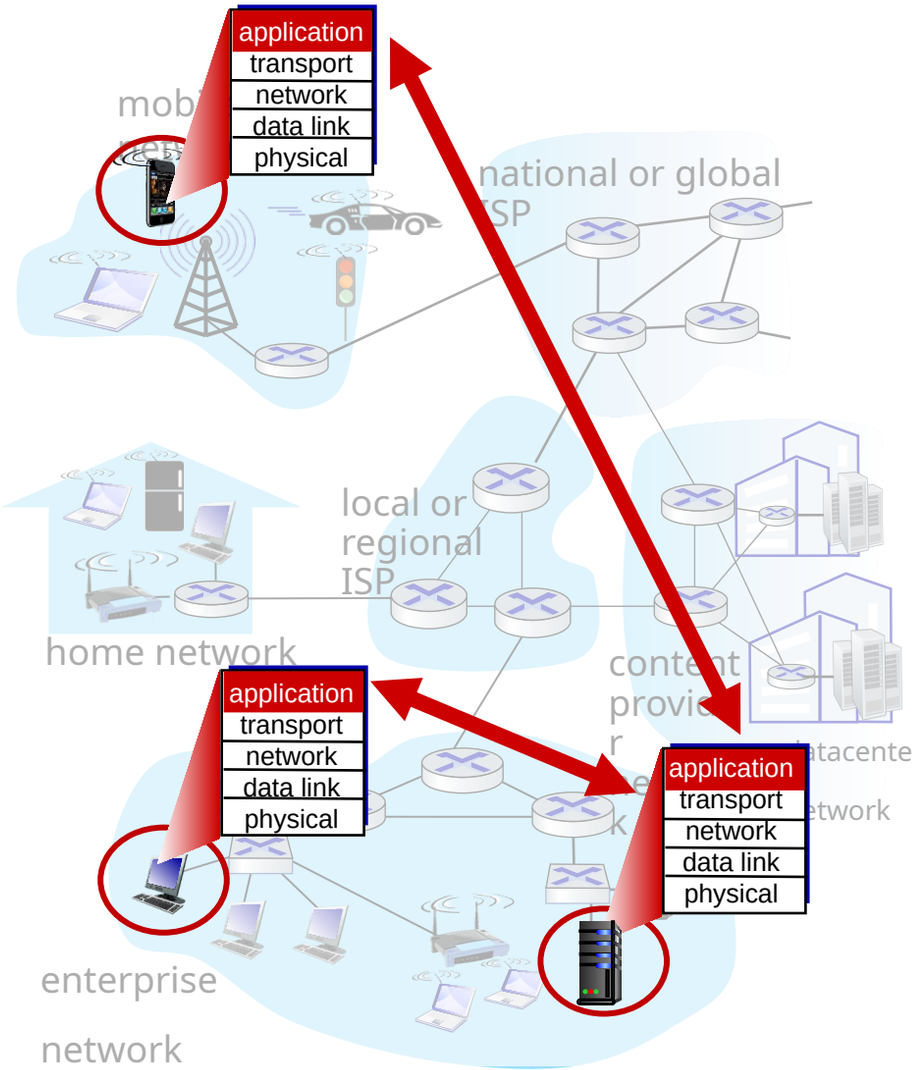
# Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



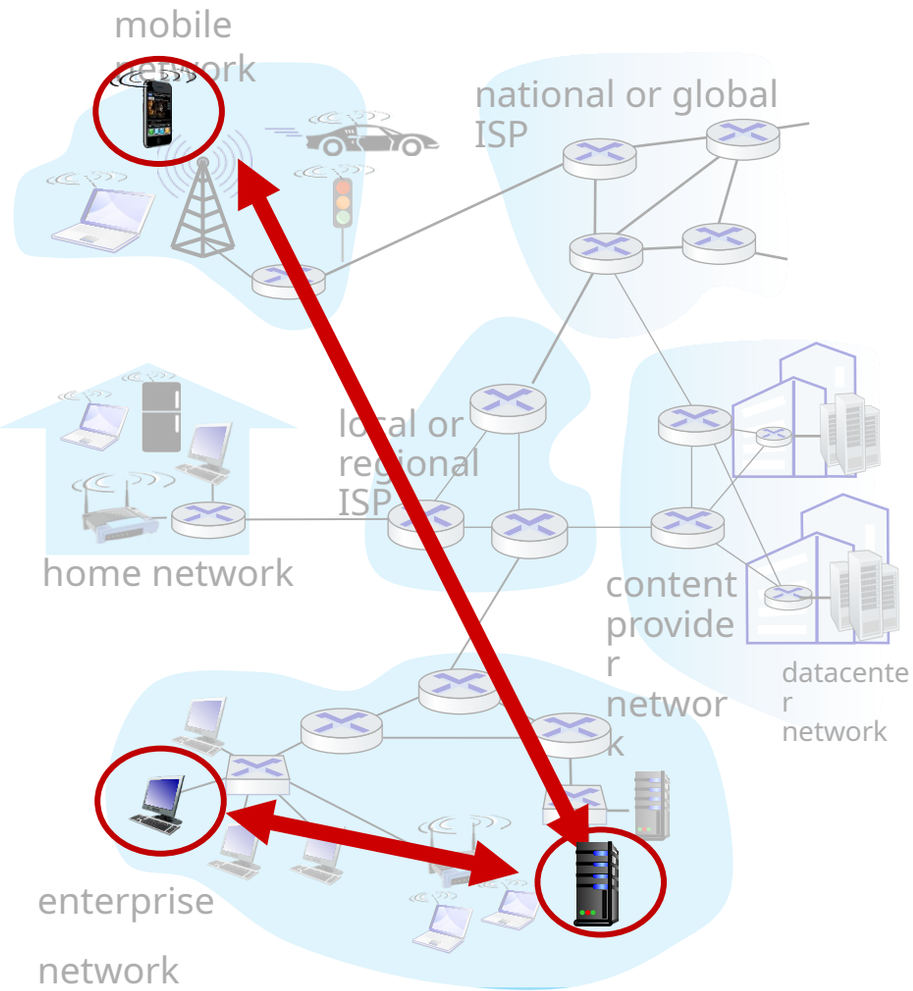
# Client-server paradigm

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

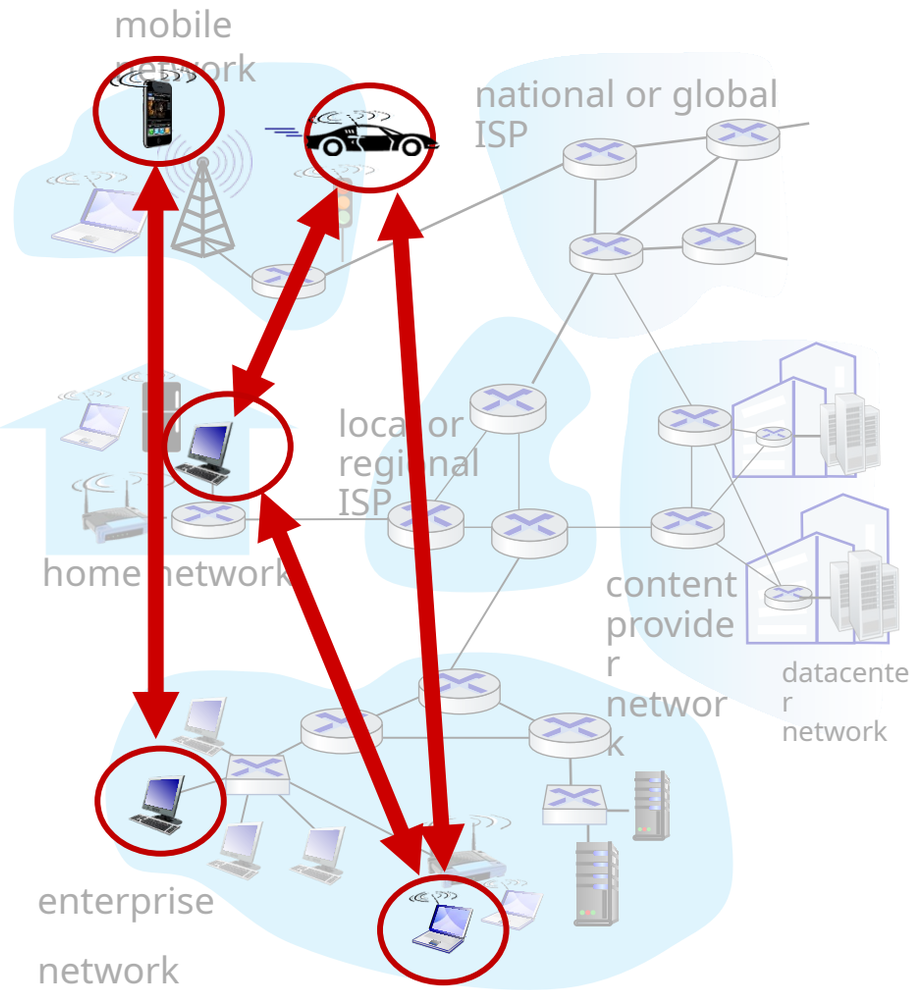
clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other



# Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- example: P2P file sharing



# Processes communicating

*process:* program running within a host

- within same host, two processes communicate using inter-process communication (defined by OS)
- processes in different hosts communicate by exchanging messages

clients, servers

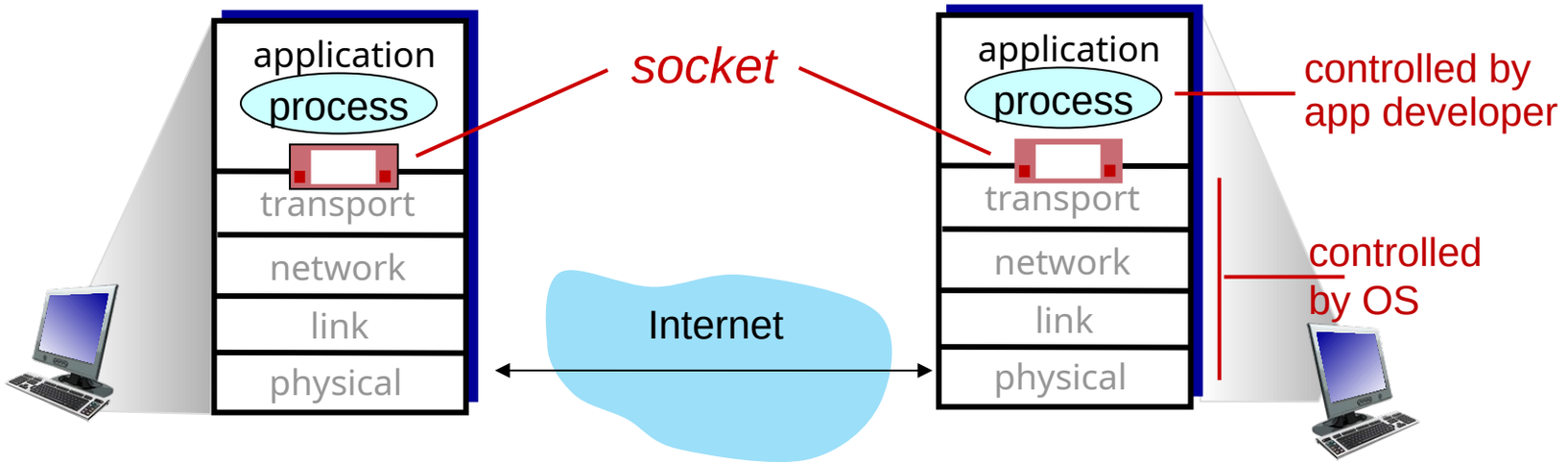
*client process:* process that initiates communication

*server process:* process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

# Sockets

- process sends/receives messages to/from its socket
- socket analogous to door
  - sending process shoves message out door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
  - two sockets involved: one on each side



# Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- *Q:* does IP address of host on which process runs suffice for identifying the process?
  - *A:* no, *many* processes can be running on same host

- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
  - HTTP server: 80
  - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
  - IP address: 128.119.245.12
  - port number: 80
- more shortly…

# An application-layer protocol defines:

- types of messages exchanged,
  - e.g., request, response
- message syntax:
  - what fields in messages & how fields are delineated
- message semantics
  - meaning of information in fields
- rules for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

# What transport service does an app need?

## data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

## timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be "effective"

## throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be "effective"
- other apps ("elastic apps") make use of whatever throughput they get

## security

- encryption, data integrity, …

# Transport service requirements: common apps

| application | data loss | throughput | time sensitive? |
|---|---|---|---|
| file transfer/download | no loss | elastic | no |
| e-mail | no loss | elastic | no |
| Web documents | loss-tolerant | audio: 5Kbps-1Mbps | yes, 10’s msec |
| real-time audio/video | | video:10Kbps-5Mbps | yes, few secs |
| | loss-tolerant | same as above | yes, 10’s msec |
| streaming audio/video | loss- | Kbps+ elastic | yes and no |

# Internet transport protocols services

*TCP service:*

- *reliable transport* between sending and receiving process
- *flow control:* sender won't overwhelm receiver
- *congestion control:* throttle sender when network overloaded
- *connection-oriented:* setup required between client and server processes
- *does not provide:* timing, minimum throughput

*UDP service:*

- *unreliable data transfer* between sending and receiving process
- *does not provide:* reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

# Internet applications, and transport protocols

| application | application layer protocol | transport protocol |
|---|---|---|
| file transfer/download | FTP [RFC 959] | TCP |
| e-mail | SMTP [RFC 5321] | TCP |
| Web documents | HTTP [RFC 7230, 9110] | TCP |
| Internet telephony | SIP [RFC 3261], RTP [RFC 3550], or proprietary HTTP [RFC 7230], DASH | TCP or UDP |
| streaming | | TCP |
| audio/video | | UDP or TCP |
| interactive games | WOW, FPS (proprietary) | |

# Securing TCP

## Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

## Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

## TSL implemented in application layer

- apps use TSL libraries, that use TCP in turn
- cleartext sent into "socket" traverse Internet *encrypted*
- more: Chapter 8

# Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



# Web and HTTP

*First, a quick review…*

- web page consists of *objects,* each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,…
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL,* e.g., `www.someschool.edu/someDept/pic.gif`

host name

path name

# HTTP overview

## HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
  - *client:* browser that requests, receives, (using HTTP protocol) and "displays" Web objects
  - *server:* Web server sends (using HTTP protocol) objects in response to requests



# HTTP overview (continued)

*HTTP uses TCP:*

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

*HTTP is "stateless"*

- server maintains *no* information about past client requests

*aside*

protocols that maintain "state" are complex!

- past history (state) must be maintained
- if server/client crashes, their views of "state" may be inconsistent, must be reconciled

# HTTP connections: two types

*Non-persistent HTTP*

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects required multiple connections

*Persistent HTTP*

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

# Non-persistent HTTP: example

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



1a. HTTP client initiates TCP connection to HTTP server (process) at www.someSchool.edu on port 80



1b. HTTP server at host www.someSchool.edu waiting for TCP connection at port 80 "accepts" connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object someDepartment/home.index

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

time

# Non-persistent HTTP: example (cont.)

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)





4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

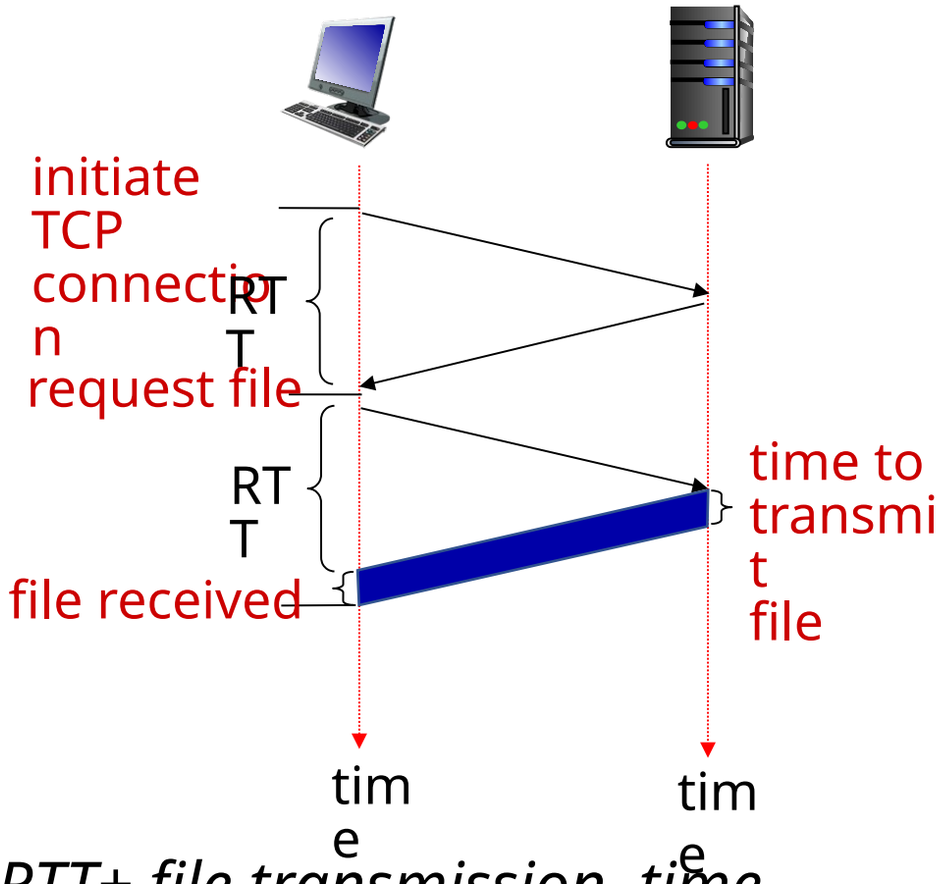
time

# Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time




*Non-persistent HTTP response time = 2RTT+ file transmission time*

# Persistent HTTP (HTTP 1.1)

*Non-persistent HTTP issues:*

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

*Persistent HTTP (HTTP1.1):*

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

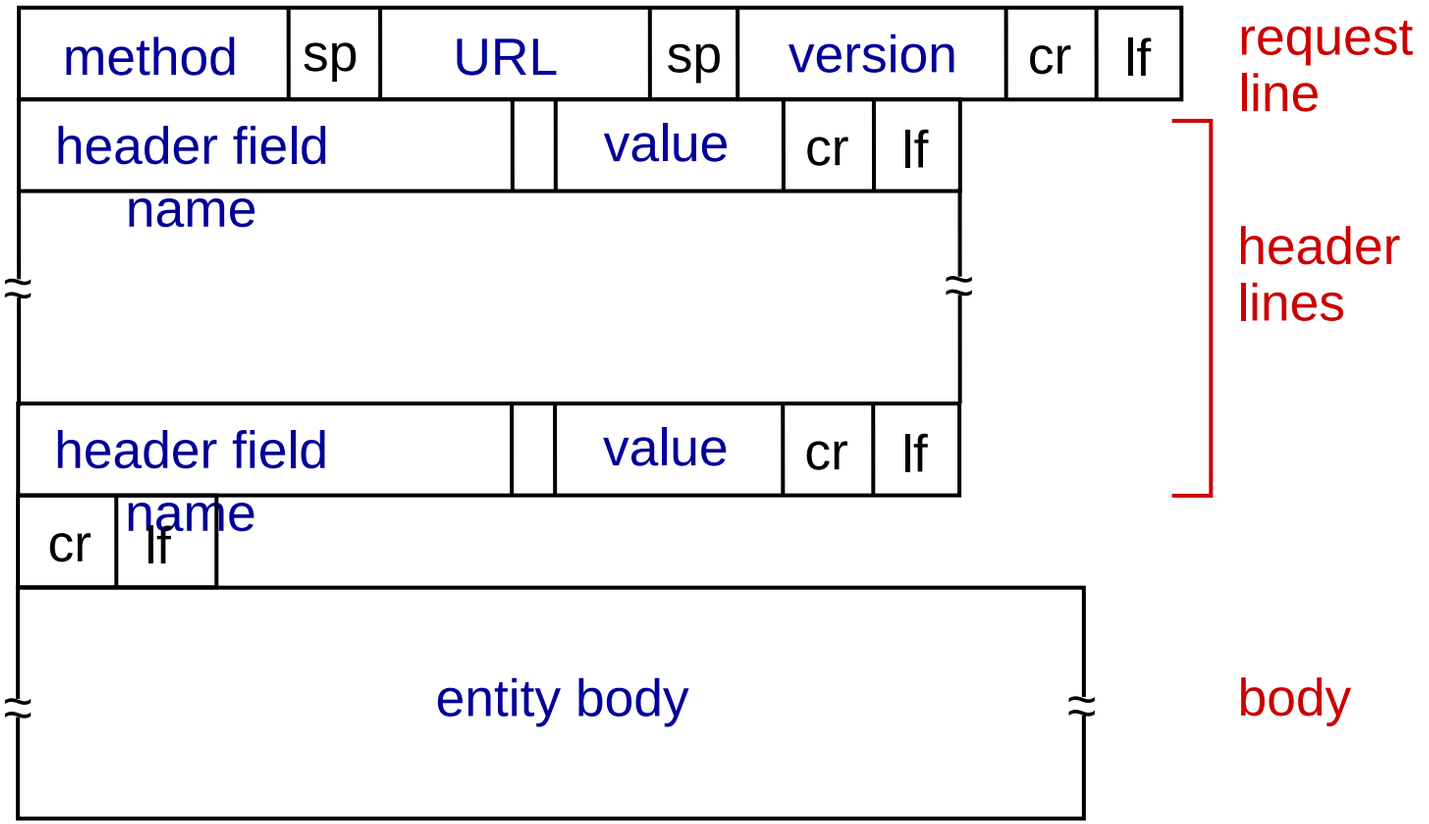
# HTTP request message

- two types of HTTP messages: *request*, *response*
- HTTP request message:
  - ASCII (human-readable format)




* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

# HTTP request message: general format



# Other HTTP request messages

## POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

## GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

```
www.somesite.com/animalsearch?monkeys&banana
```

## HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

## PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

# HTTP response message

status line (protocol
status code status phra



```
HTTP/1.1 200 OK
```

# HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK
- request succeeded, requested object later in this message

301 Moved Permanently
- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request
- request msg not understood by server

404 Not Found
- requested document not found on this server

505 HTTP Version Not Supported

# Trying out HTTP (client side) for yourself

1. netcat to your favorite Web server:

```
% nc -c -v gaia.cs.umass.edu 80
```

- opens TCP connection to port 80 (default HTTP server port) at gaia.cs.umass. edu.
- anything typed in will be sent to port 80 at gaia.cs.umass.edu

2. type in a GET HTTP request:

```
GET /kurose_ross/interactive/index.php HTTP/1.1
Host: gaia.cs.umass.edu
```

- by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

3. look at response message sent by HTTP server!

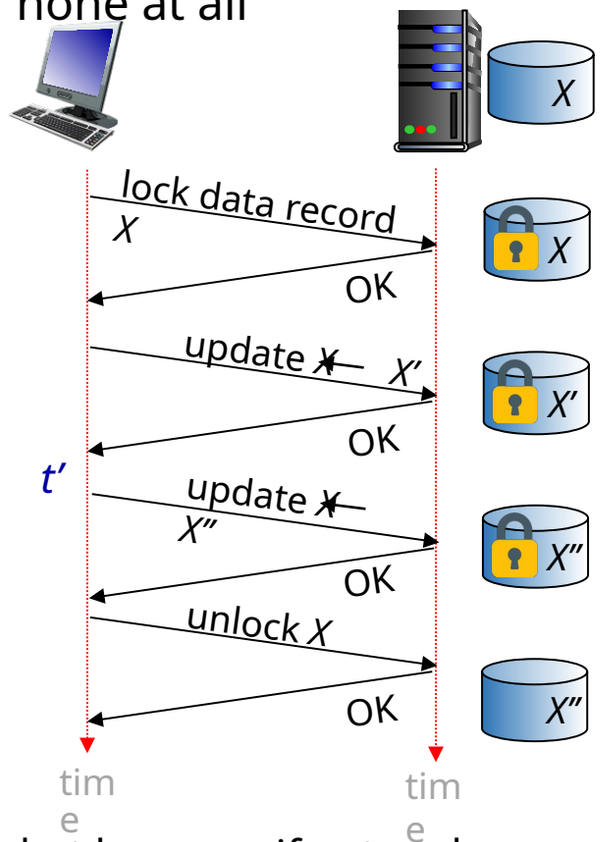
(or use Wireshark to look at captured HTTP request/response)

# Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web "transaction"
  - no need for client/server to track "state" of multi-step exchange
  - all HTTP requests are independent of each other
  - no need for client/server to "recover" from a partially-completed-but-never-completely-completed transaction

a stateful protocol: client makes two changes to X, or none at all




*Q:* what happens if network connection or client crashes at *t'*?

# Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

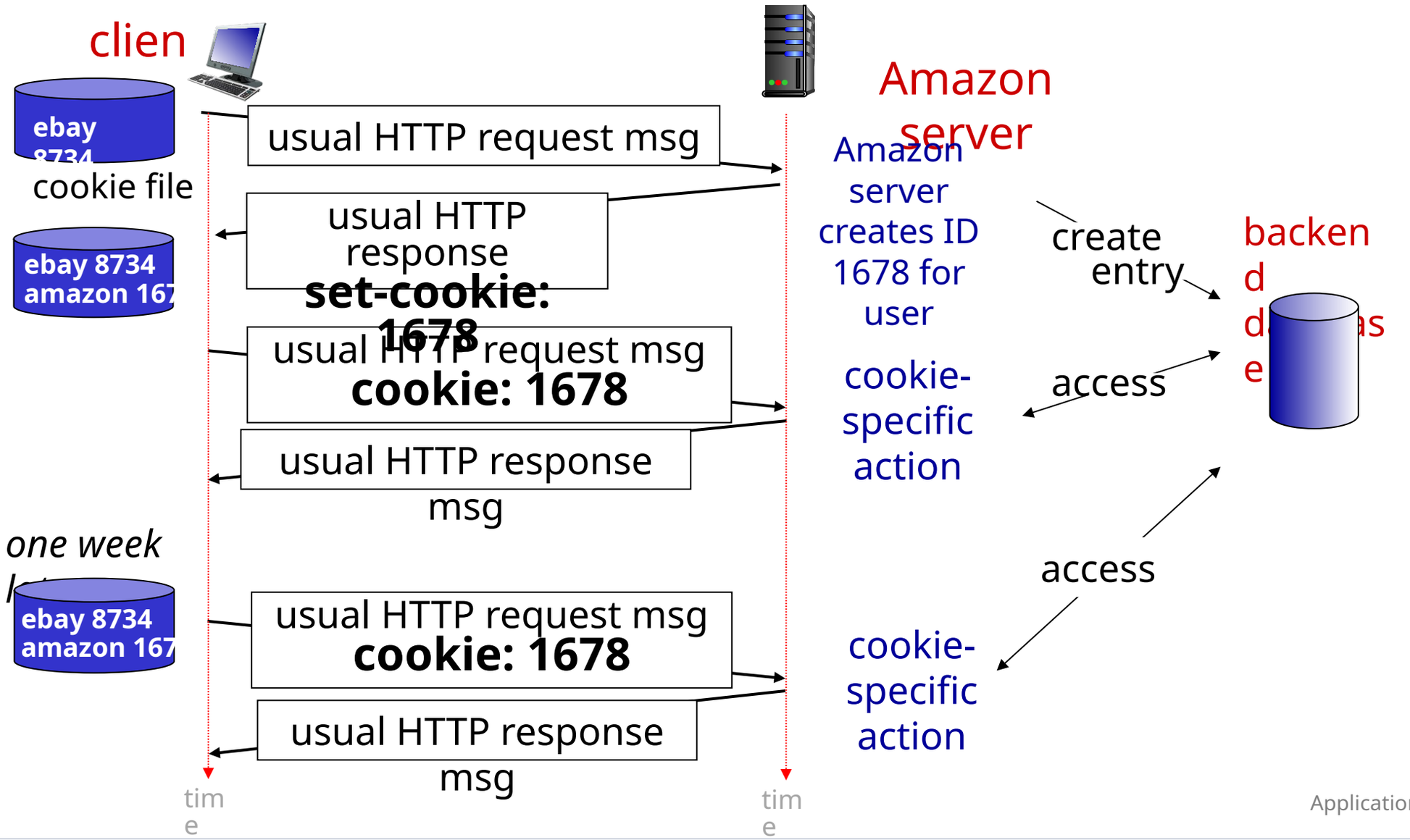
*four components:*

1) cookie header line of HTTP *response* message
2) cookie header line in next HTTP *request* message
3) cookie file kept on user's host, managed by user's browser
4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
  - unique ID (aka "cookie")
  - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to "identify" Susan

# Maintaining user/server state: cookies



# HTTP cookies: comments

*What cookies can be used for:*

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

*Challenge: How to keep state?*

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies inHTTP messages carry state

asid
e

*cookies and privacy:*

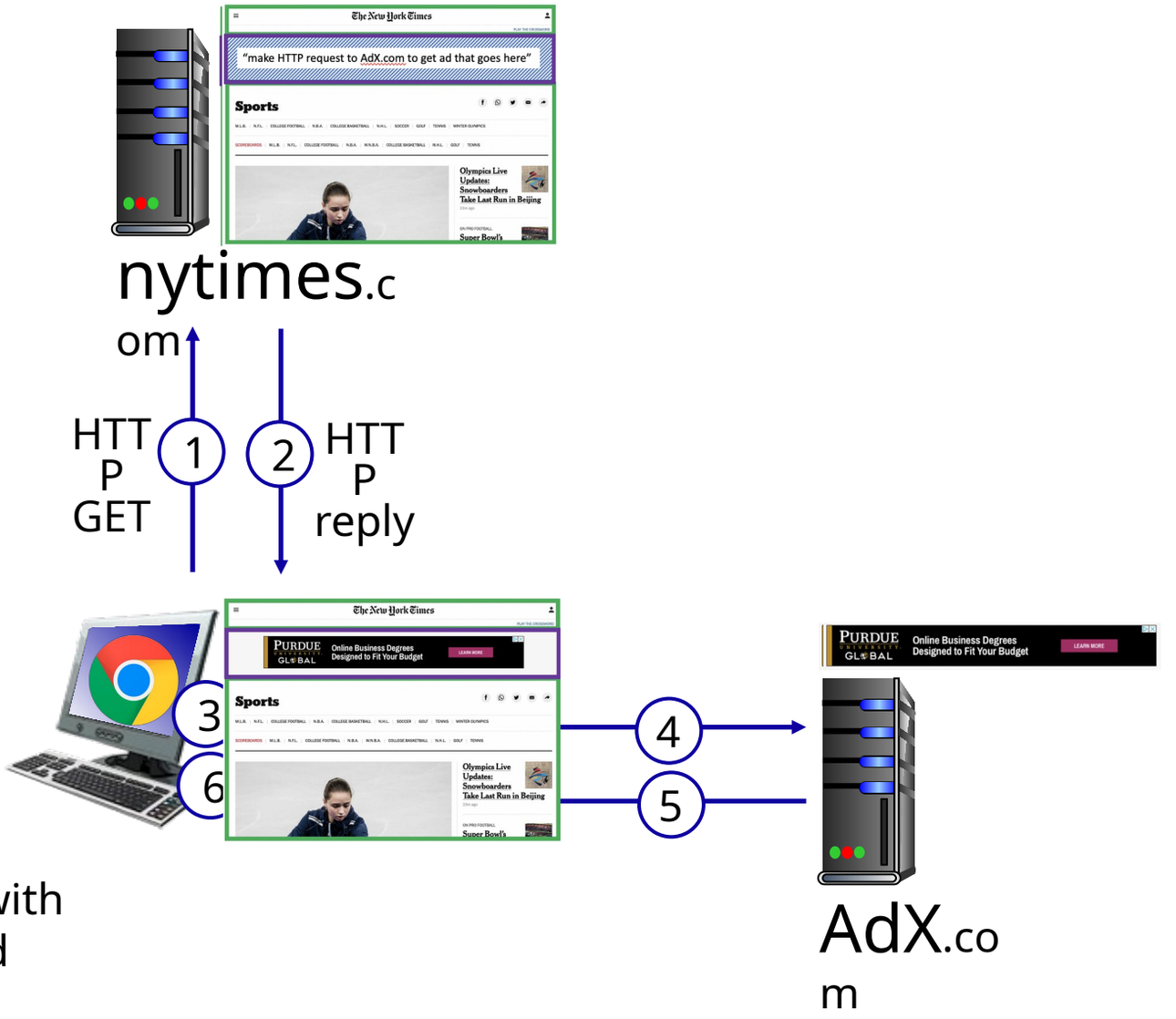
- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across ~~multiple web sites~~

# Example: displaying a NY Times web page

1 2 GET base html file from nytimes.com

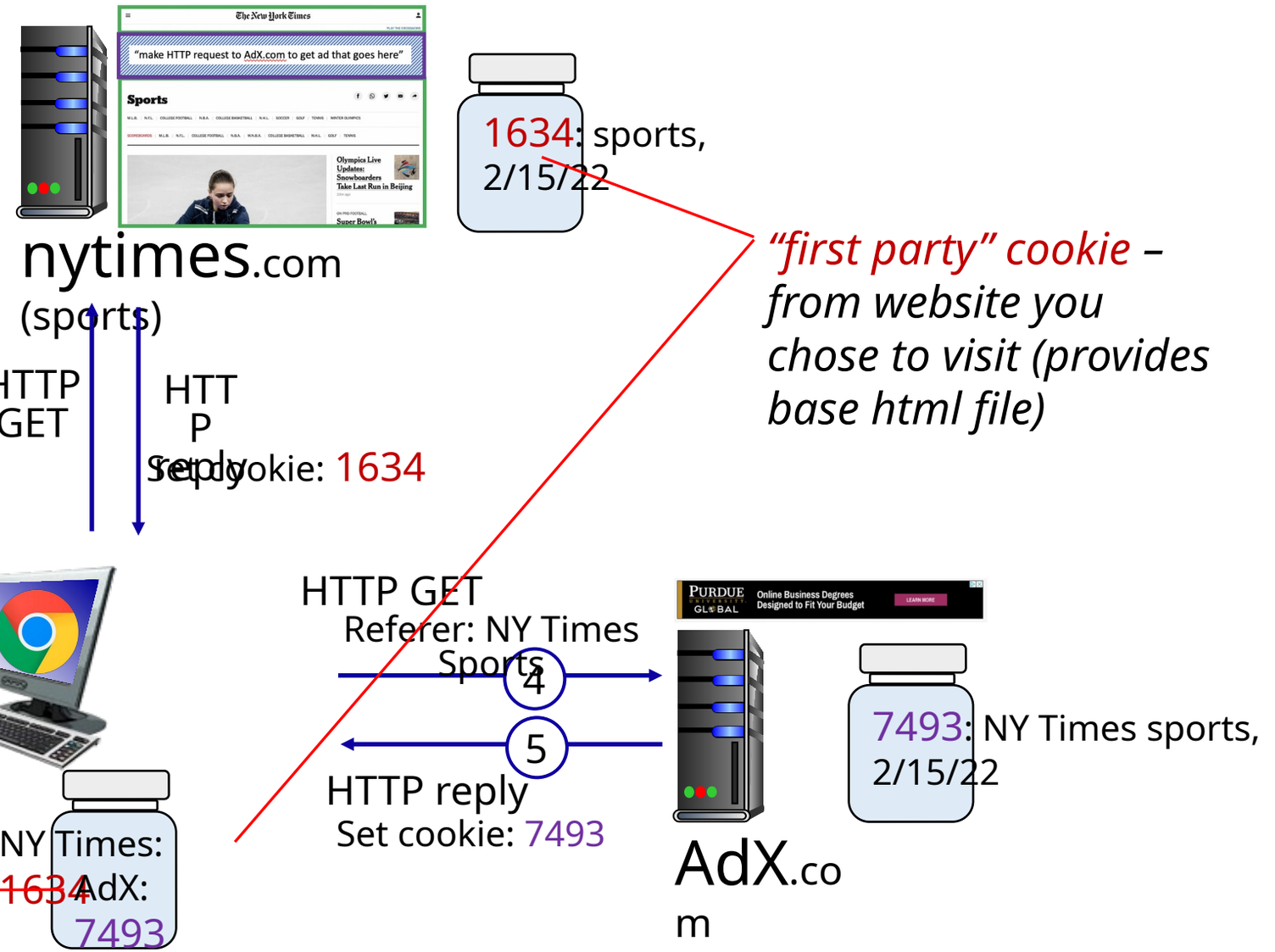
4 5 fetch ad from AdX.com

7 display composed page




NY times page with embedded ad displayed

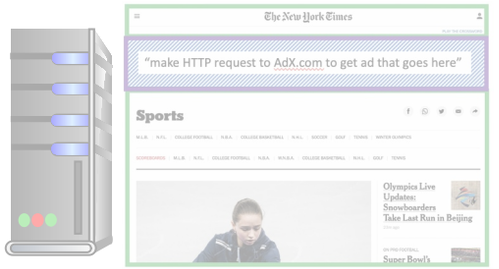
# Cookies: tracking a user's browsing behavior



# Cookies: tracking a user's browsing behavior




socks.com




nytimes.com

1634: sports, 2/15/22

HTTP GET

HTTP reply

2

1



NY Times: 1634 AdX: 7493

HTTP GET
Referer: socks.com, cookie: 7493

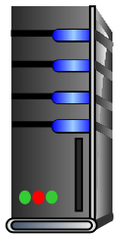
4

5

HTTP reply
Set cookie: 7493

AdX:

- *tracks my web browsing* over sites with AdX ads
- can return targeted ads based on browsing history



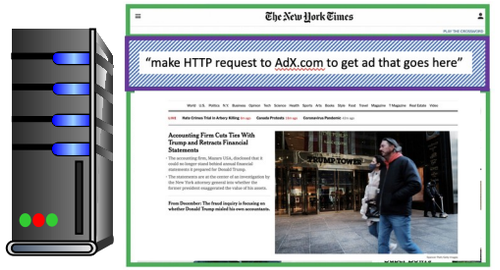
AdX.com

7493: NY Times sports, 2/15/22
7493: socks.com, 2/16/22

# Cookies: tracking a user's browsing behavior (one day later)




socks.com




nytimes.com

1634: sports, 2/13/22
1634: arts, 2/17/22

(arts)

HTTP GET cookie: 1634

HTTP reply Set cookie: 1634



HTTP GET Referer:nytimes.com, cookie: 7493

4

5

HTTP reply Set cookie: 7493

*Returned ad for socks!*

NY Times: 1634 AdX: 7493

AdX.com

7493: NY Times sports, 2/13/22
7493: socks.com, 2/18/22
7493: NY Times arts, 2/15/22

# Cookies: tracking a user's browsing behavior

Cookies can be used to:

- track user behavior on a given website (first party cookies)
- track user behavior across multiple websites (third party cookies) without user ever choosing to visit tracker site (!)
- tracking may be *invisible* to user:
  - rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

third party tracking via cookies:

- disabled by default in Firefox, Safari browsers
- to be disabled in Chrome browser in 2023

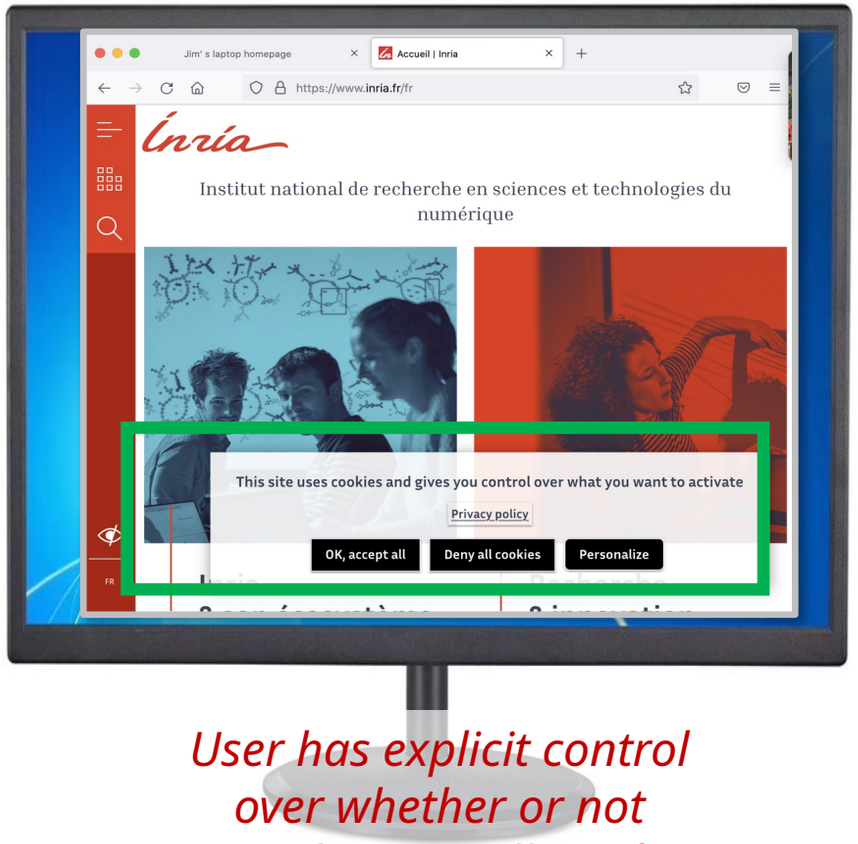
# GDPR (EU General Data Protection Regulation) and cookies

"Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...].

This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them."

GDPR, recital 30 (May 2018)

when cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations

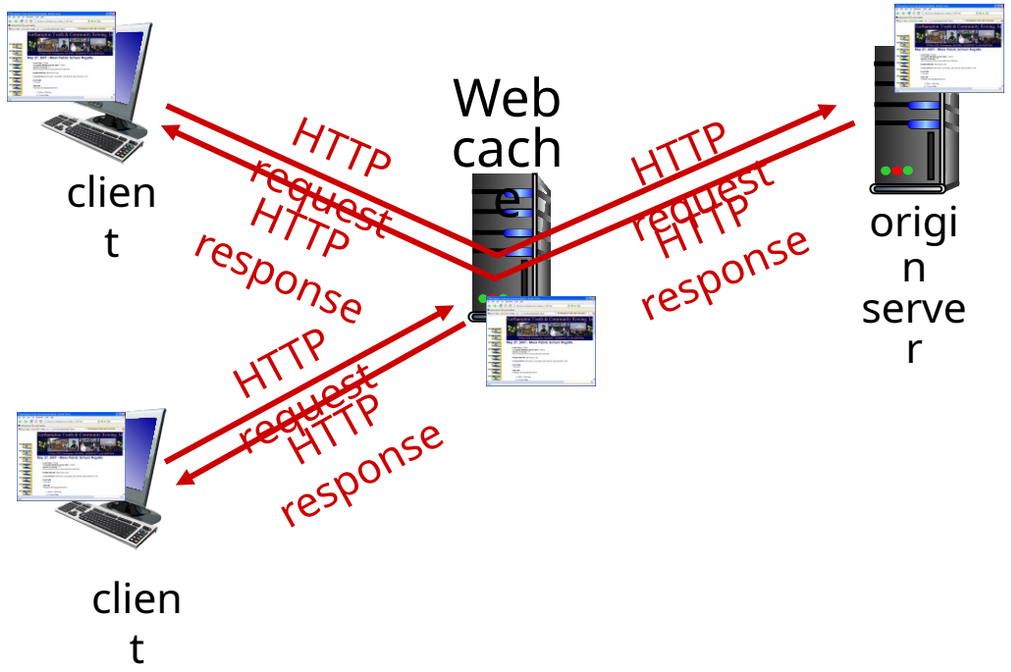



*User has explicit control over whether or not cookies are allowed*

# Web caches

*Goal:* satisfy client requests without involving origin server

- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests to cache
  - *if* object in cache: cache returns object to client
  - *else* cache requests object from origin server, caches received object, then returns object to client



# Web caches (aka proxy servers)

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

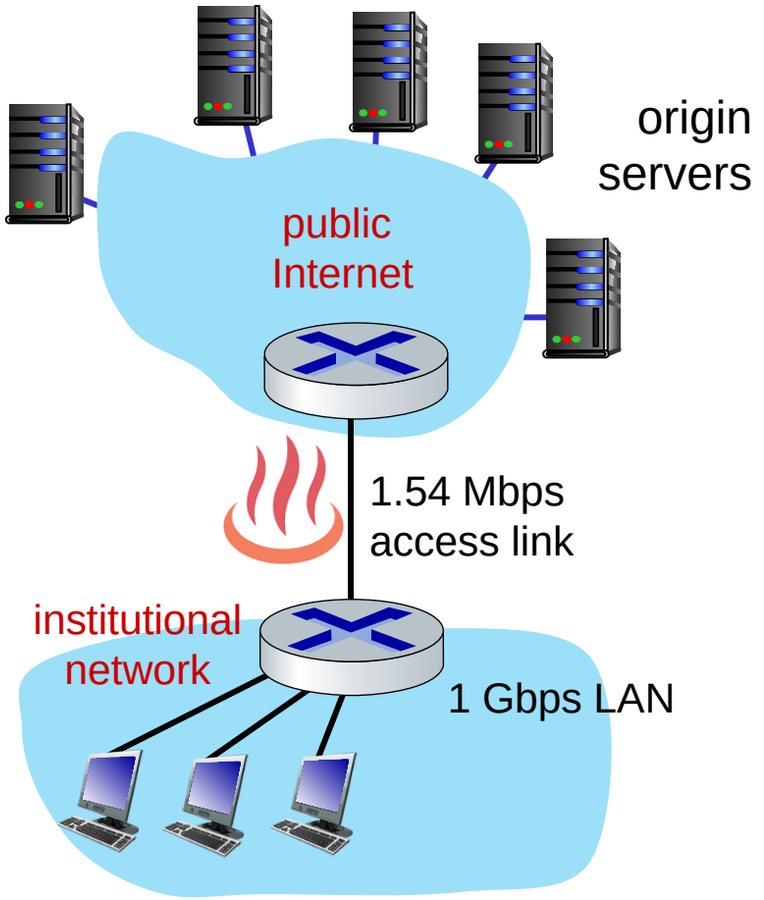
*Why* Web caching?

- reduce response time for client request
  - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
  - enables "poor" content providers to more effectively deliver content

# Caching example

*Scenario:*

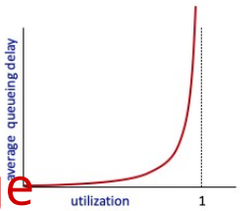
- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps




*Performance:*

- access link utilization = .97
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay

  = 2 sec + minutes + usecs




*problem:* large queueing delays at high utilization!

# Option 1: buy a faster access link

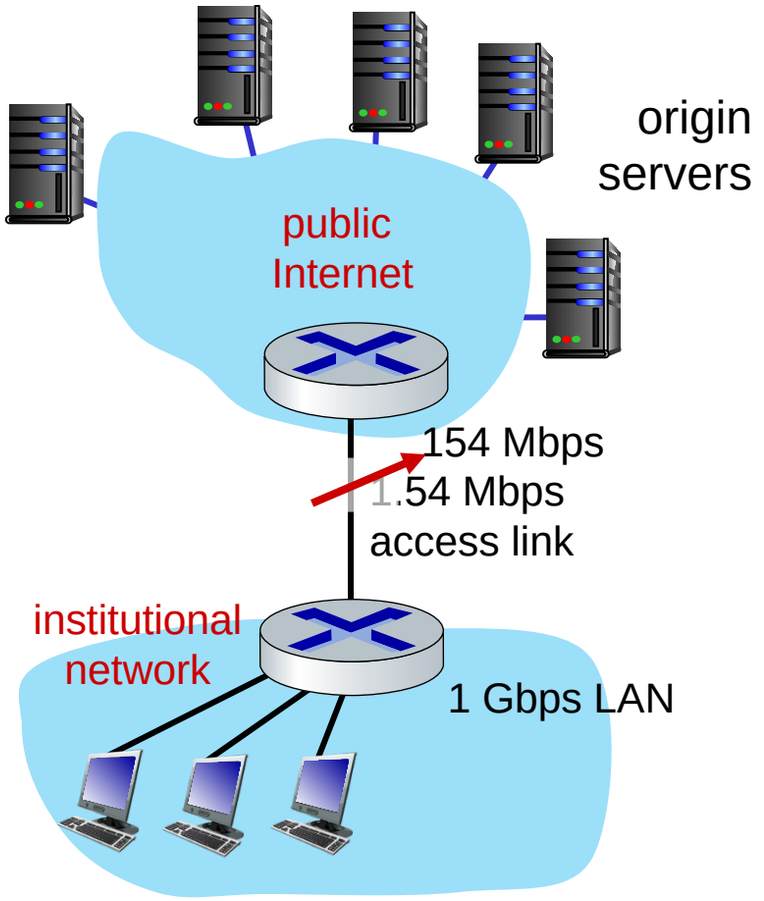
*Scenario:*

- access link rate: 1.54 Mbps → 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

*Performance:*

- access link utilization = .97 → .0097
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay
  = 2 sec + minutes + msecs

*Cost:* faster access link → usecs (expensive!)



# Option 2: install a web cache

*Scenario:*

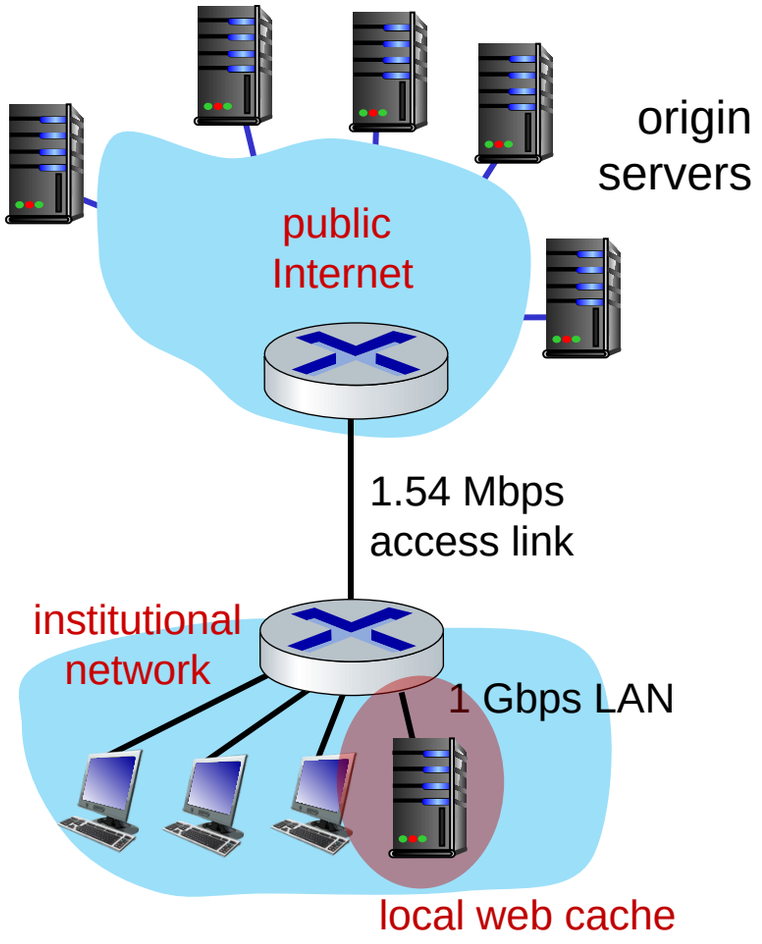
- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

*Cost:* web cache (cheap!)

*Performance:*

- LAN utilization: .?
- access link utilization = ?
- average end-end delay = ?

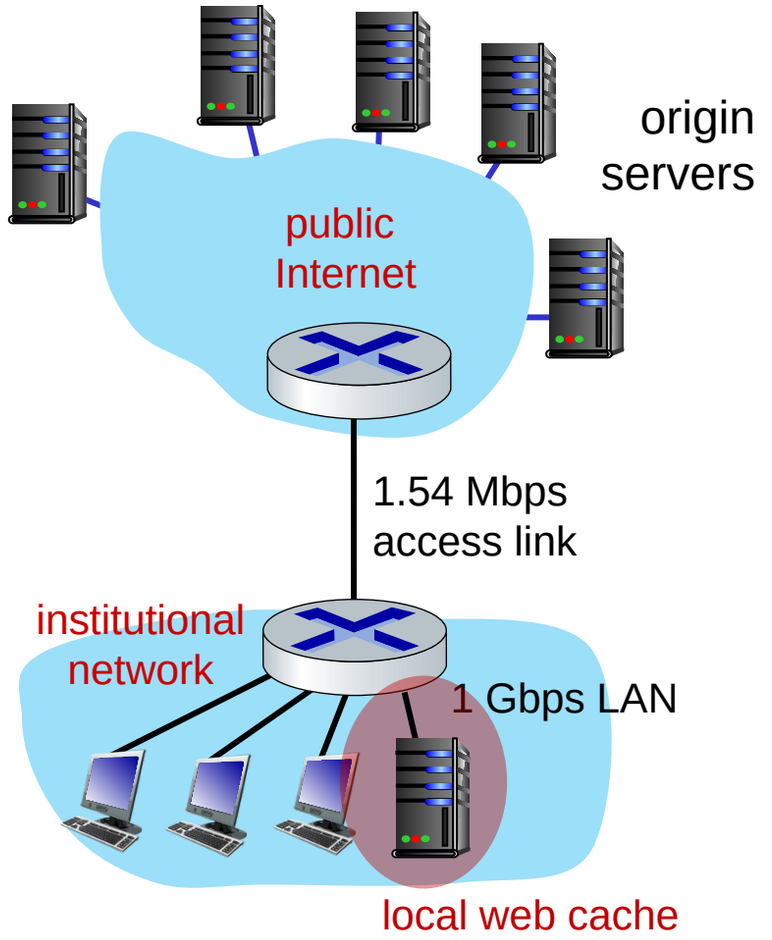
*How to compute link utilization, delay?*



# Calculating access link utilization, end-end delay with cache:

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
  - rate to browsers over access link
    = 0.6 * 1.50 Mbps = .9 Mbps
  - access link utilization = 0.9/1.54 = .58 means low (msec) queueing delay at access link
- average end-end delay:
  = 0.6 * (delay from origin servers)
  + 0.4 * (delay when satisfied at cache)
  = 0.6 (2.01) + 0.4 (~msecs) = ~ 1.2 secs




*lower average end-end delay than with 154 Mbps link (and cheaper too!)*

# Browser caching: Conditional GET

*Goal:* don't send object if browser has up-to-date cached version

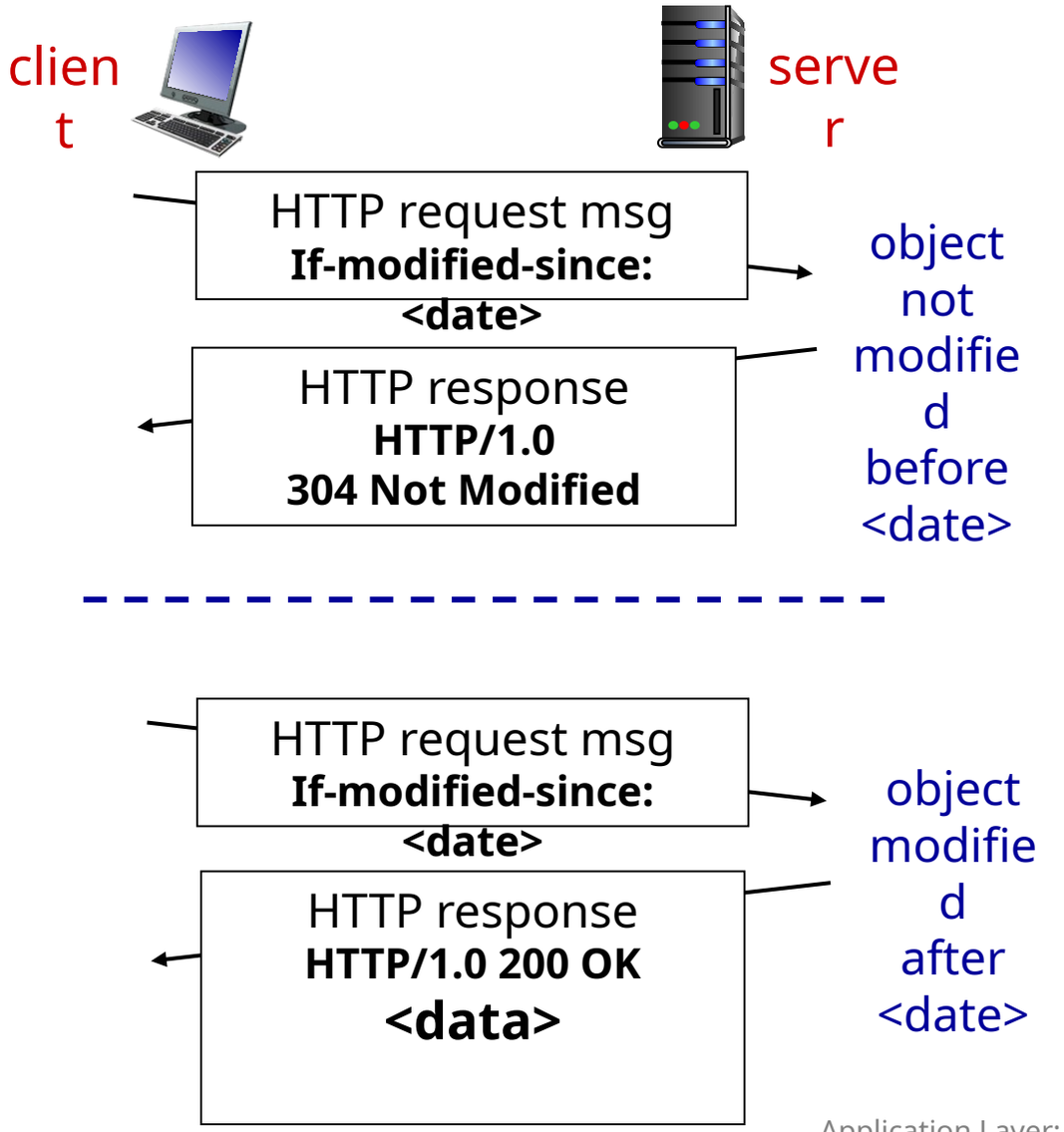
- no object transmission delay (or use of network resources)

- *client:* specify date of browser-cached copy in HTTP request

  **If-modified-since: <date>**

- *server:* response contains no object if browser-cached copy is up-to-date:

  **HTTP/1.0 304 Not Modified**



# HTTP/2

*Key goal:* decreased delay in multi-object HTTP requests

*HTTP1.1:* introduced multiple, pipelined GETs over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

# HTTP/2

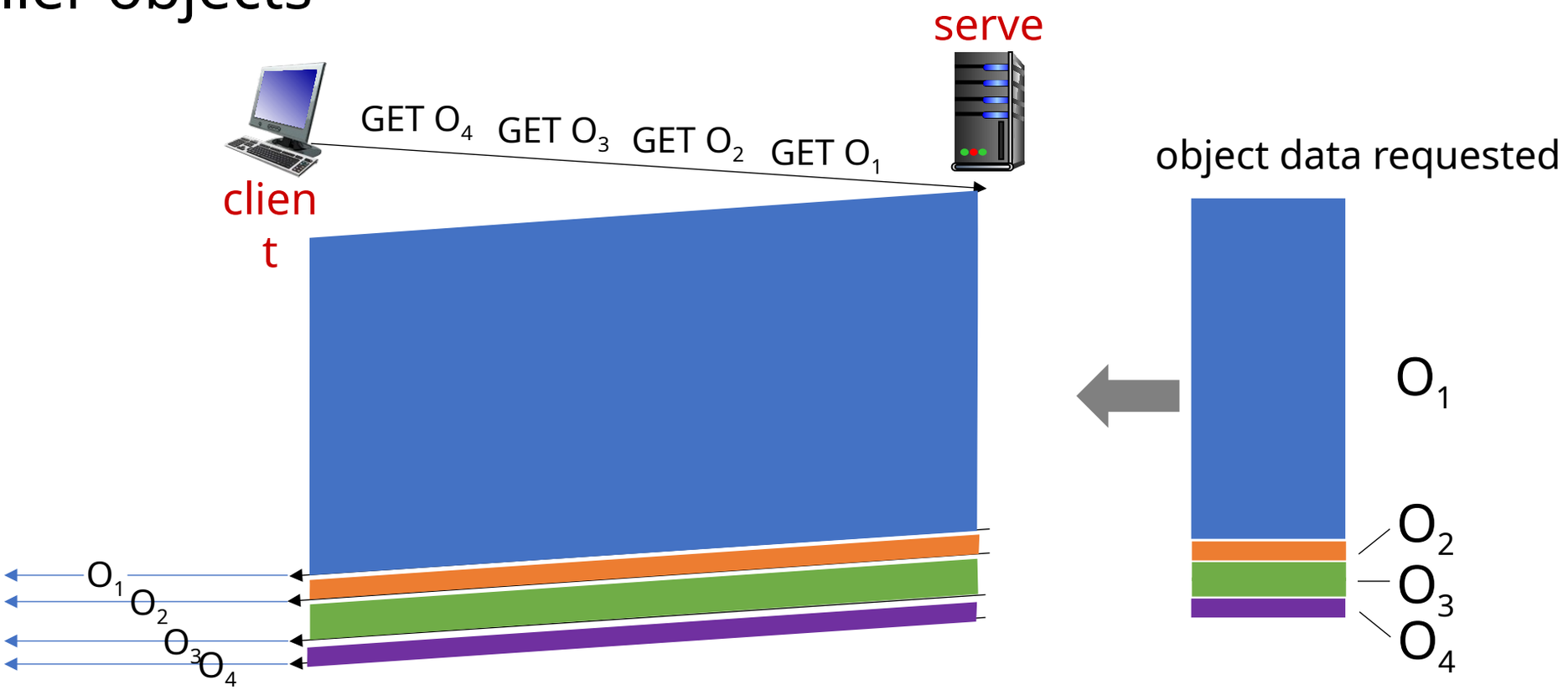
*Key goal:* decreased delay in multi-object HTTP requests

*HTTP/2:* [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

# HTTP/2: mitigating HOL blocking

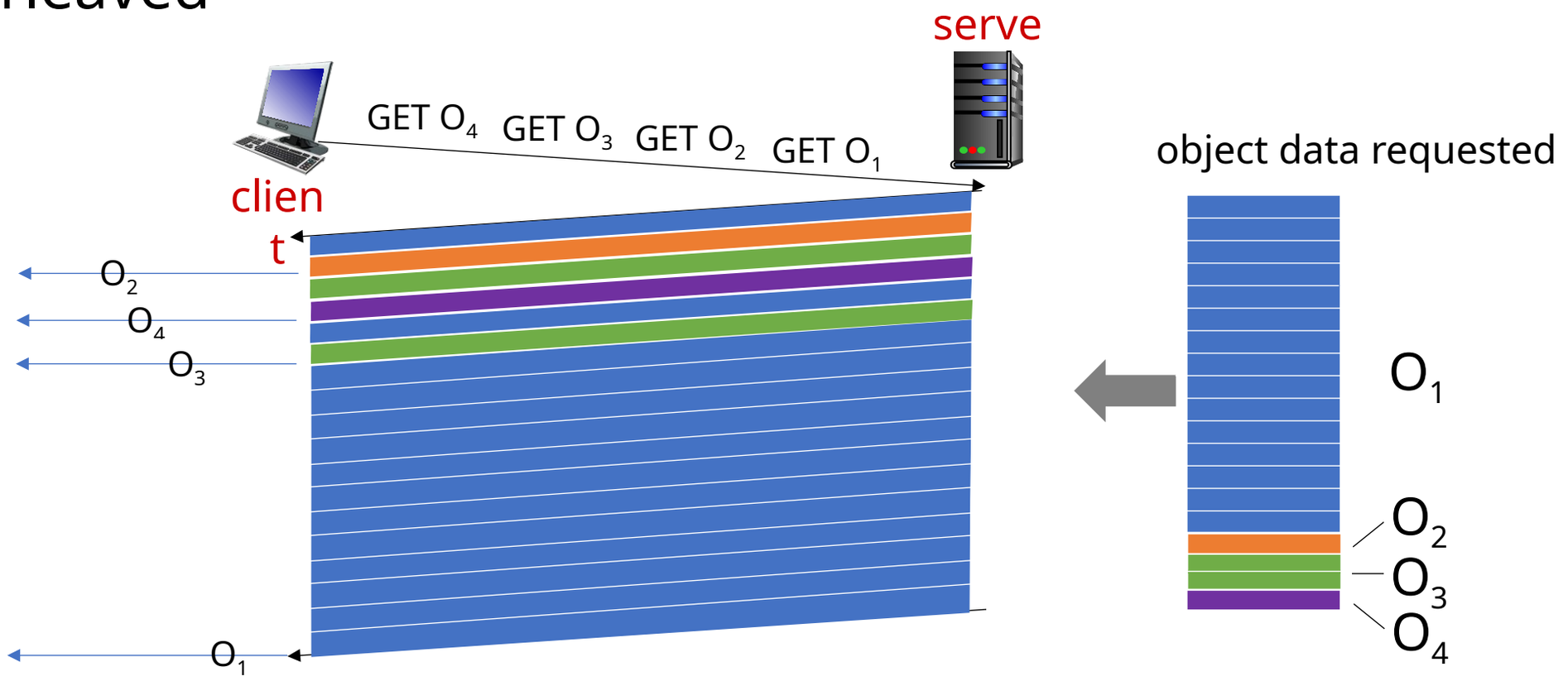
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects




*objects delivered in order requested: $O_2$, $O_3$, $O_4$ wait behind $O_1$*

# HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved




*$O_2$, $O_3$, $O_4$ delivered quickly, $O_1$ slightly delayed*

# HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
  - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- HTTP/3: adds security, per object error- and congestion-control (more pipelining) over UDP
  - more on HTTP/3 in transport layer

# Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



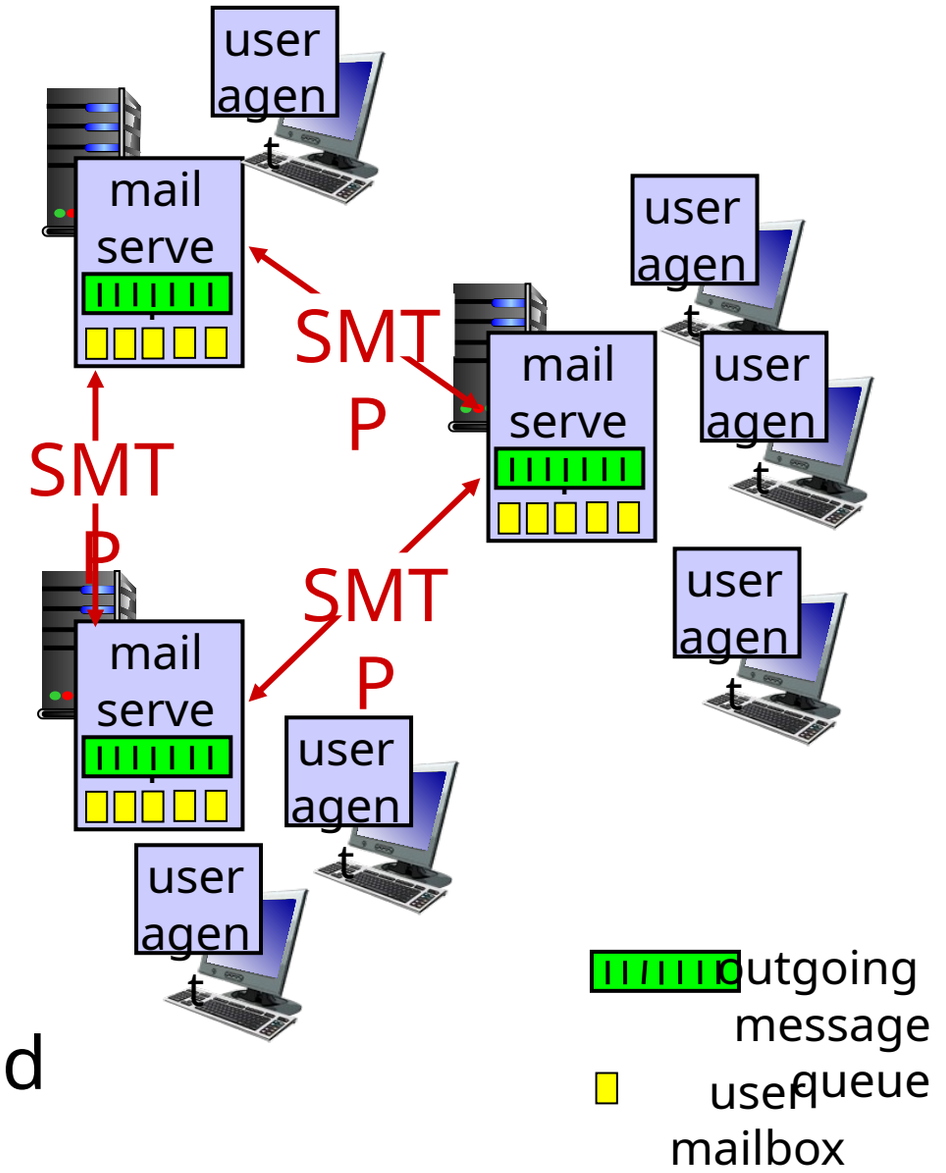
# E-mail

## Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

## User Agent

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



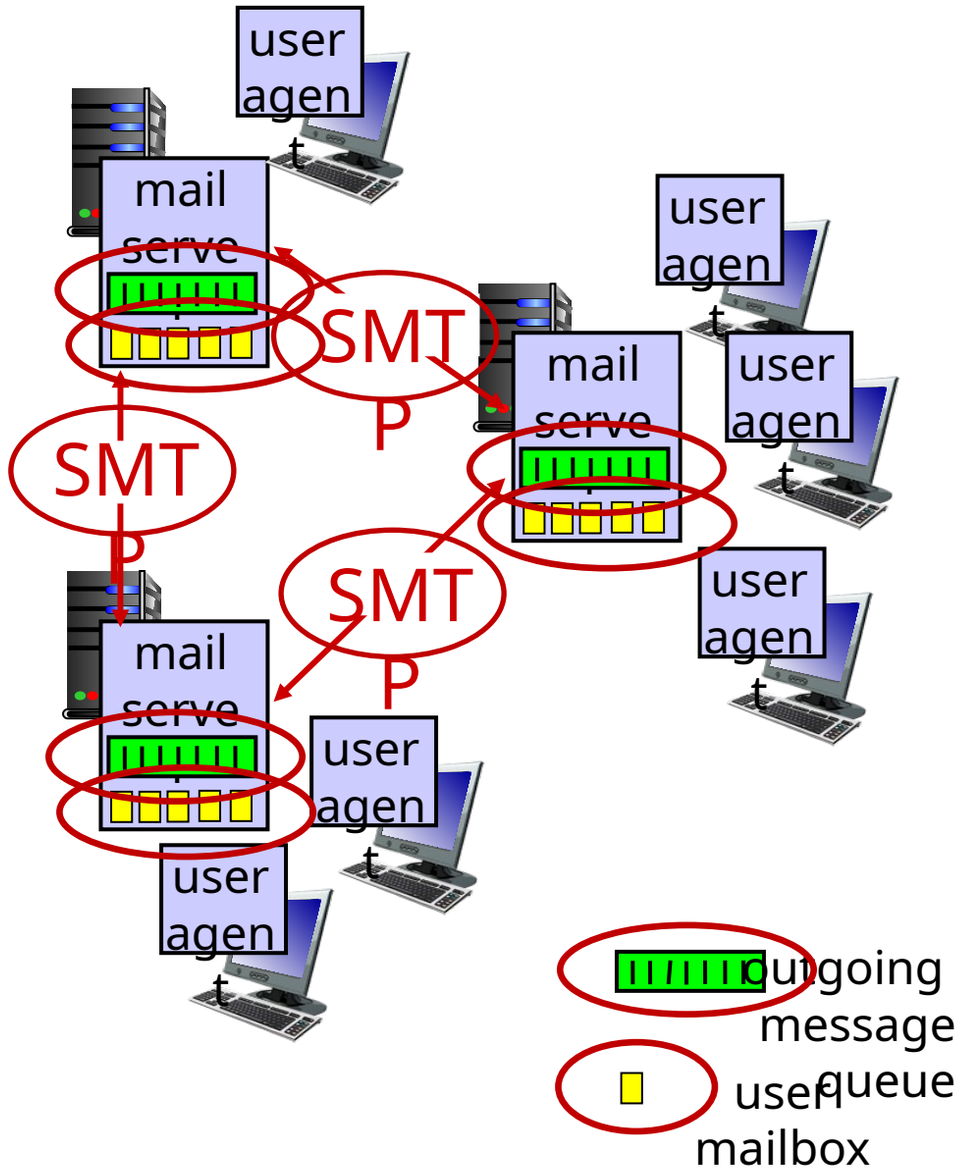
# E-mail: mail servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

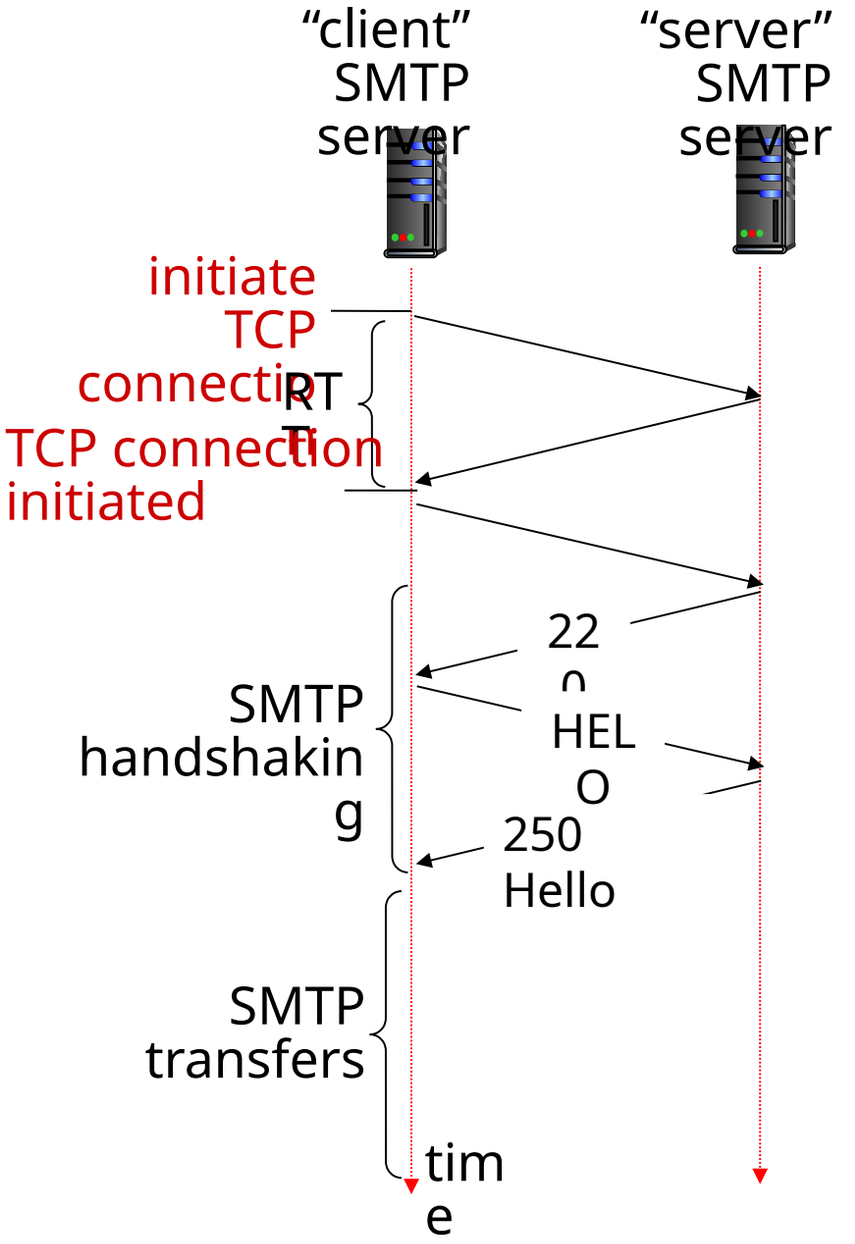
SMTP protocol between mail servers to send email messages

- client: sending mail server
- "server": receiving mail server



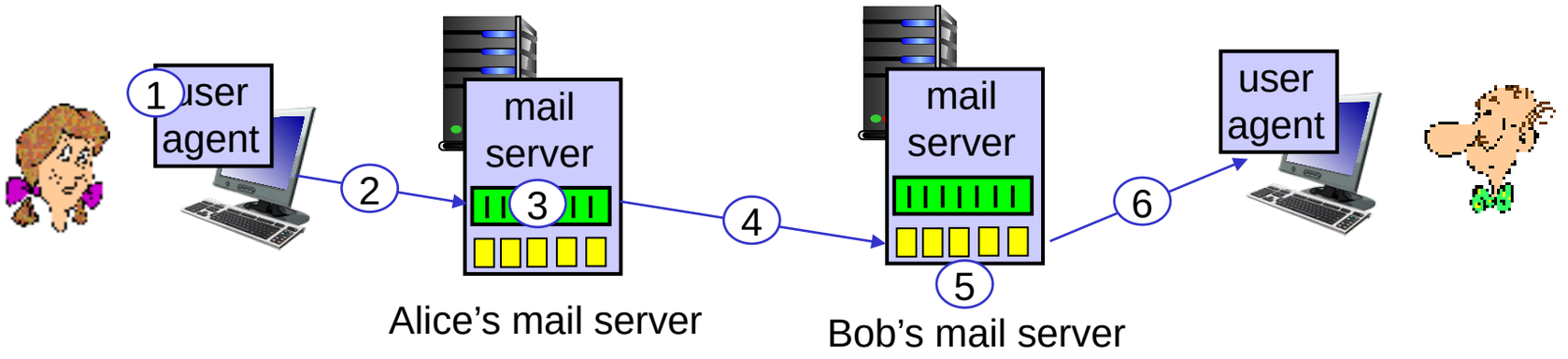
# SMTP RFC (5321)

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
  - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
  - SMTP handshaking (greeting)
  - SMTP transfer of messages
  - SMTP closure
- command/response interaction (like HTTP)
  - commands: ASCII text
  - response: status code and phrase



# Scenario: Alice sends e-mail to Bob

1) Alice uses UA to compose e-mail message “to” bob@someschool.edu
2) Alice’s UA sends message to her mail server using SMTP; message placed in message queue
3) client side of SMTP at mail server opens TCP connection with Bob’s mail server
4) SMTP client sends Alice’s message over the TCP connection
5) Bob’s mail server places the message in Bob’s mailbox
6) Bob invokes his user agent to read message



# Sample SMTP interaction

```
S: 220 hamburger.edu
```

# SMTP: observations

*comparison with HTTP:*

- HTTP: client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message

- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message

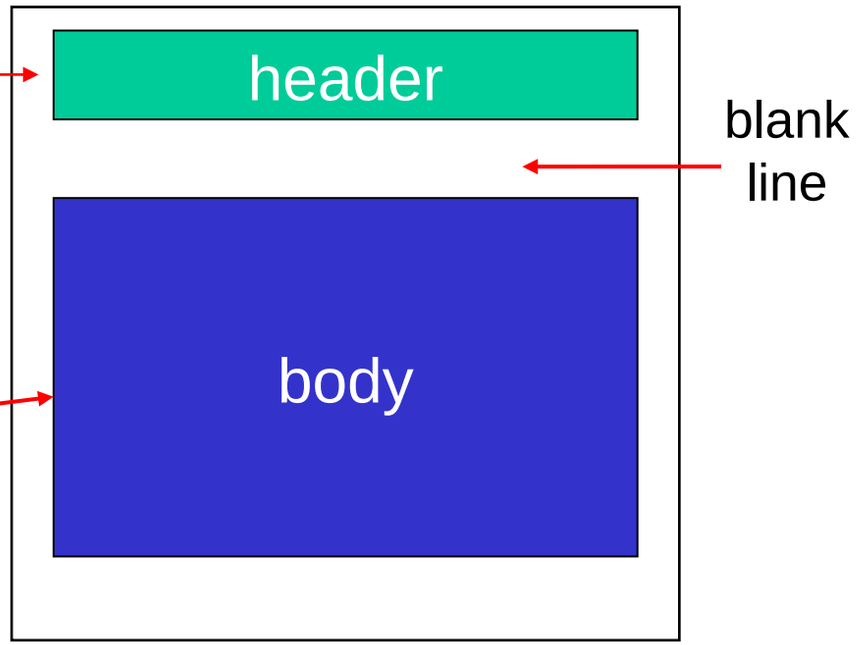
# Mail message format

SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

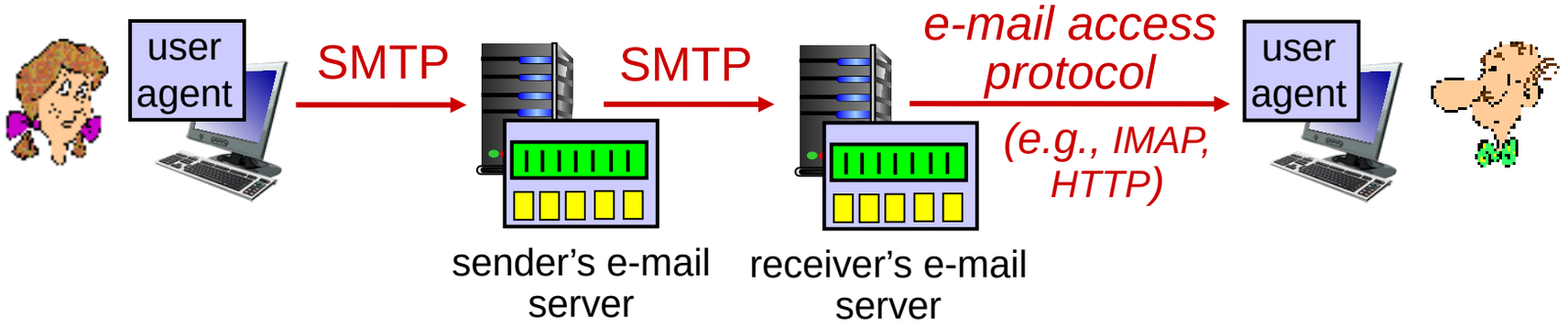
RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
  - To:
  - From:
  - Subject:

  these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the “message” , ASCII characters only



# Retrieving email: mail access protocols




- SMTP: delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
  - IMAP: Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- HTTP: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of STMP (to send), IMAP (or POP) to retrieve e-mail messages

# Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



# DNS: Domain Name System

*people:* many identifiers:

- SSN, name, passport #

*Internet hosts, routers:*

- IP address (32 bit) - used for addressing datagrams
- "name", e.g., cs.umass.edu - used by humans

*Q:* how to map between IP address and name, and vice versa ?

## Domain Name System (DNS):

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol:* hosts, DNS servers communicate to *resolve* names (address/name translation)
  - *note:* core Internet function, implemented as application-layer protocol
  - complexity at network's "edge"

# DNS: services, structure

DNS services:

- hostname-to-IP-address translation
- host aliasing
  - canonical, alias names
- mail server aliasing
- load distribution
  - replicated Web servers: many IP addresses correspond to one name

*Q: Why not centralize DNS?*

- single point of failure
- traffic volume
- distant centralized database
- maintenance
- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

*A: doesn't scale!*

# Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters:* almost every Internet transaction interacts with DNS - msecs count!

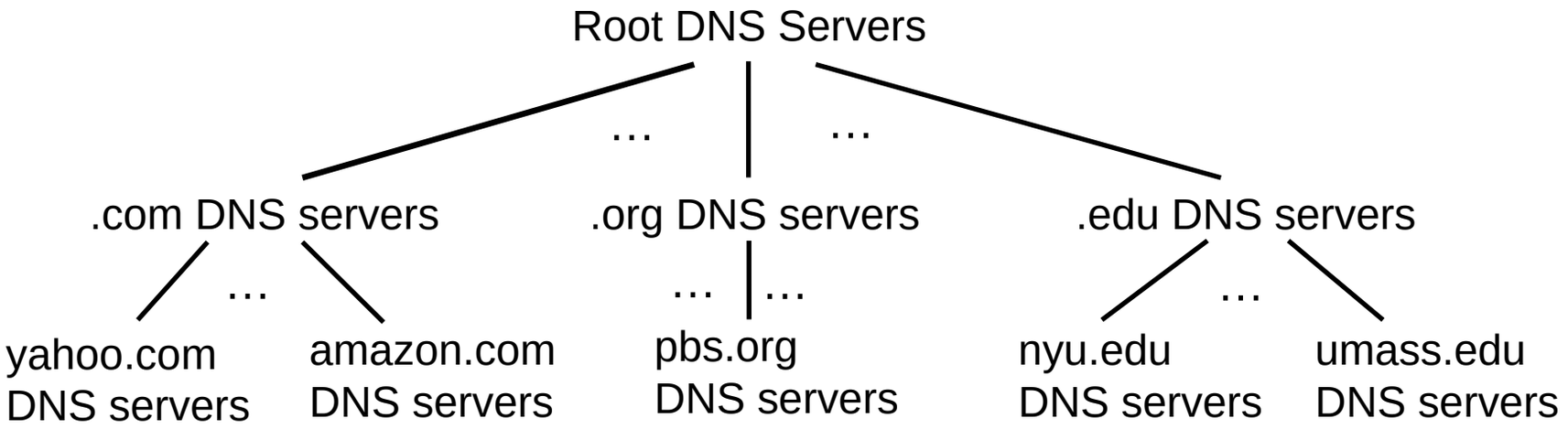
organizationally, physically decentralized:

- millions of different organizations responsible for their records

"bulletproof": reliability, security



# DNS: a distributed, hierarchical database




*Root*

*Top Level Domain*

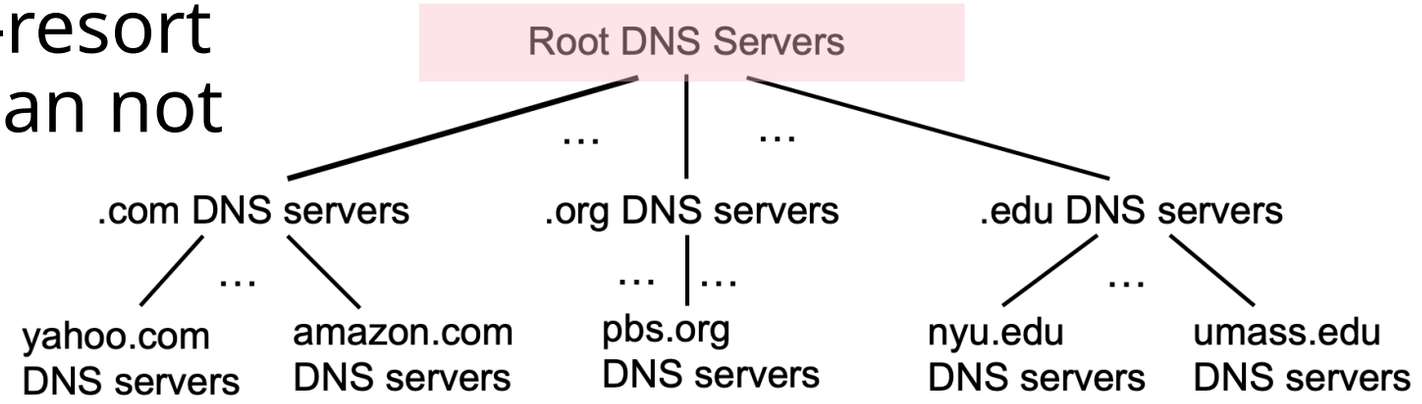
*Authoritative*

Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

# DNS: root name servers

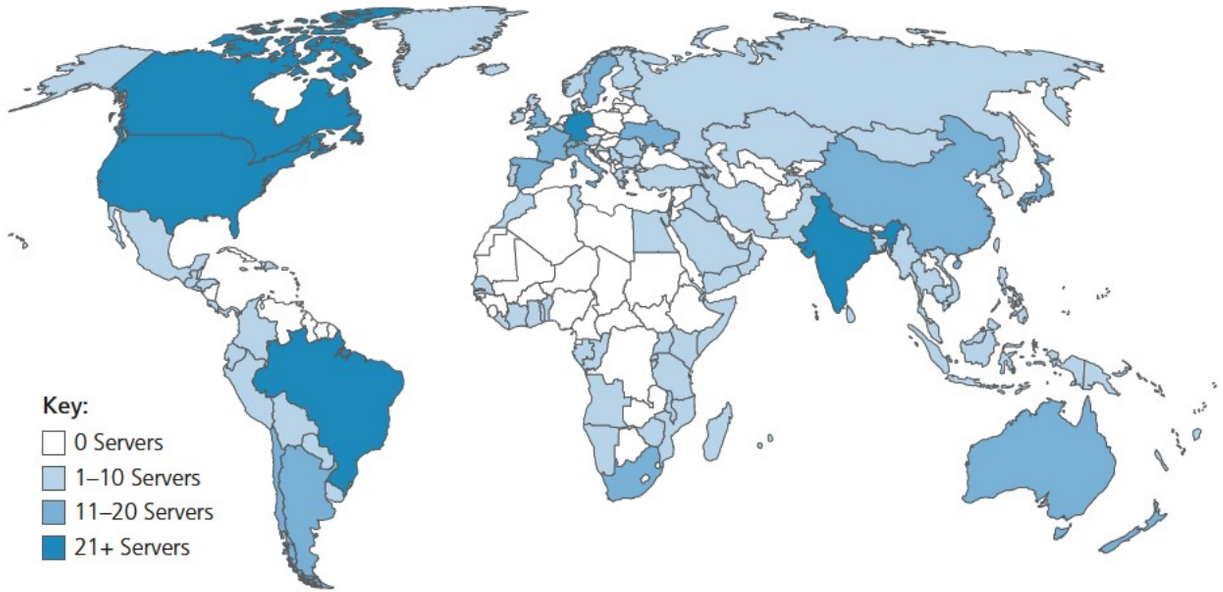
- official, contact-of-last-resort by name servers that can not resolve name



# DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
  - Internet couldn't function without it!
  - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

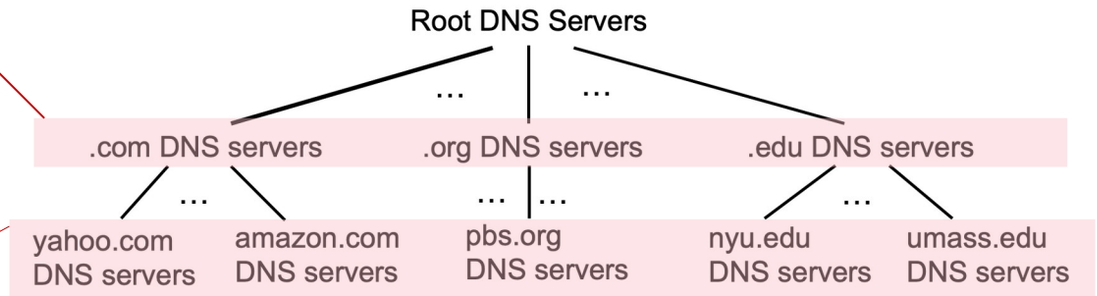
13 logical root name "servers" worldwide each "server" replicated many times (~200 servers in US)



# Top-Level Domain, and authoritative servers

## Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD




## authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

# Local DNS name servers

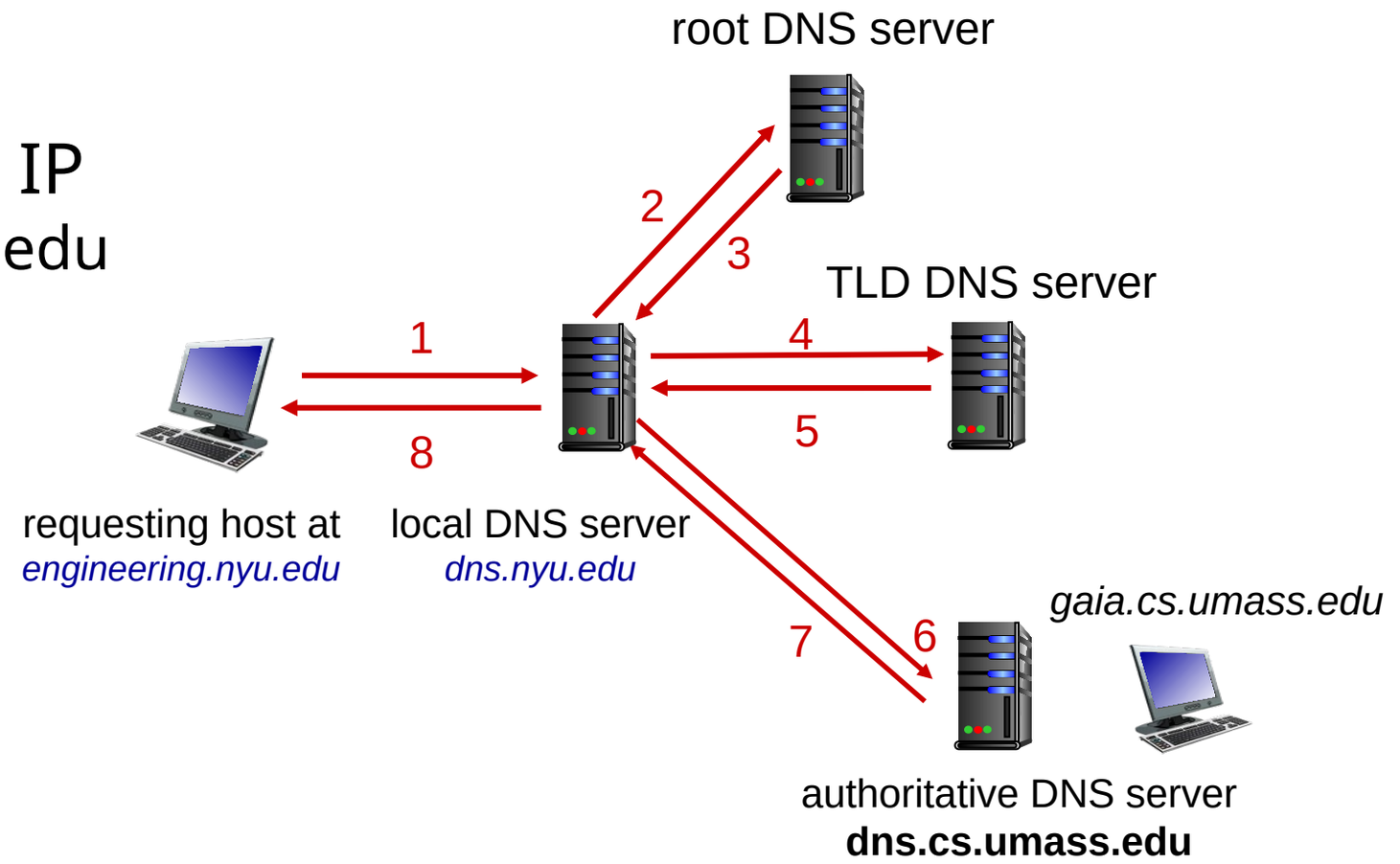
- when host makes DNS query, it is sent to its *local* DNS server
  - Local DNS server returns reply, answering:
    - from its local cache of recent name-to-address translation pairs (possibly out of date!)
    - forwarding request into DNS hierarchy for resolution
  - each ISP has local DNS name server; to find yours:
    - MacOS: `% scutil --dns`
    - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy

# DNS name resolution: iterated query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"

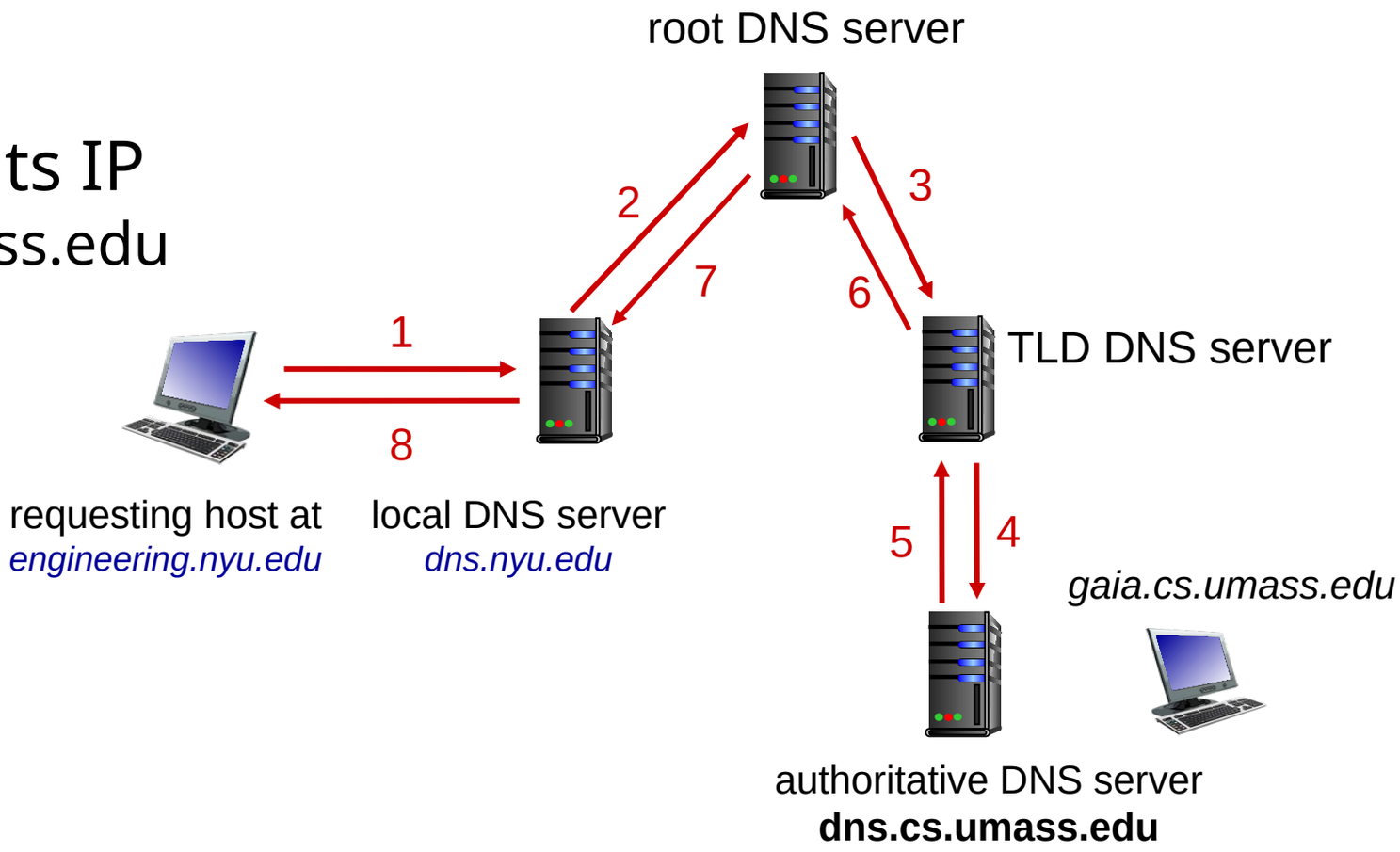


# DNS name resolution: recursive query

Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



# Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and i*mmediately* returns a cached mapping in response to a query
  - caching improves response time
  - cache entries timeout (disappear) after some time (TTL)
  - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
  - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
  - *best-effort name-to-address translation!*

# DNS records

DNS: distributed database storing resource records (RR)

RR format: (`name, value, type, ttl`)

## type=A

- `name` is hostname
- `value` is IP address

## type=NS

- `name` is domain (e.g., foo.com)
- `value` is hostname of authoritative name server for this domain

## type=CNAME

- `name` is alias name for some "canonical" (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- `value` is canonical name

## type=MX

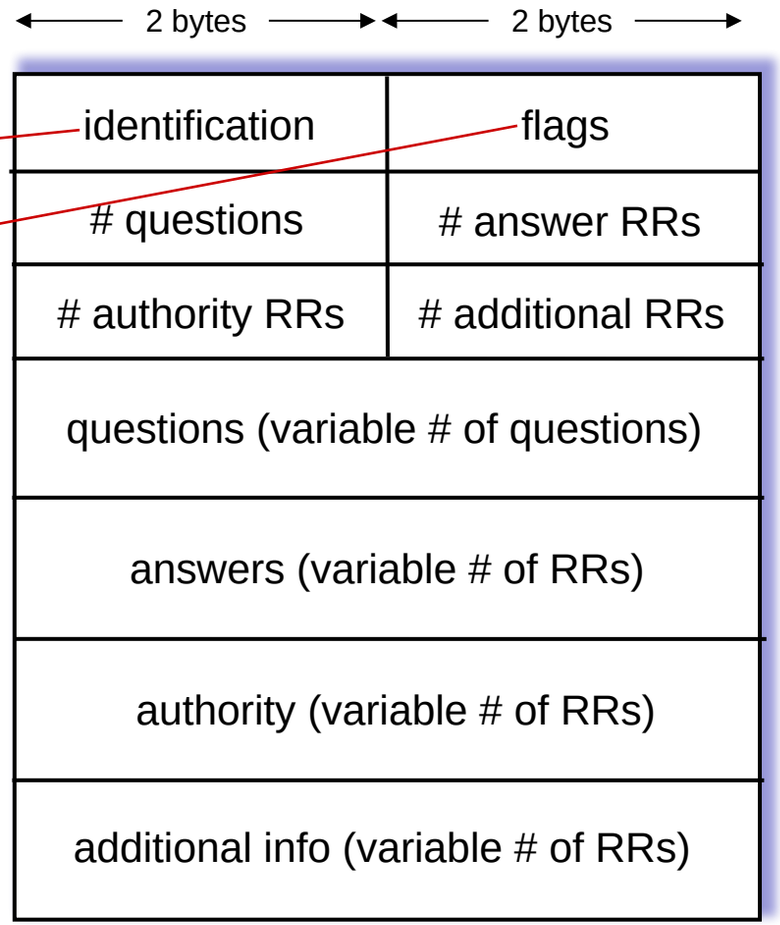
- `value` is name of SMTP mail server associated with `name`

# DNS protocol messages

DNS *query* and *reply* messages, both have same *format:*

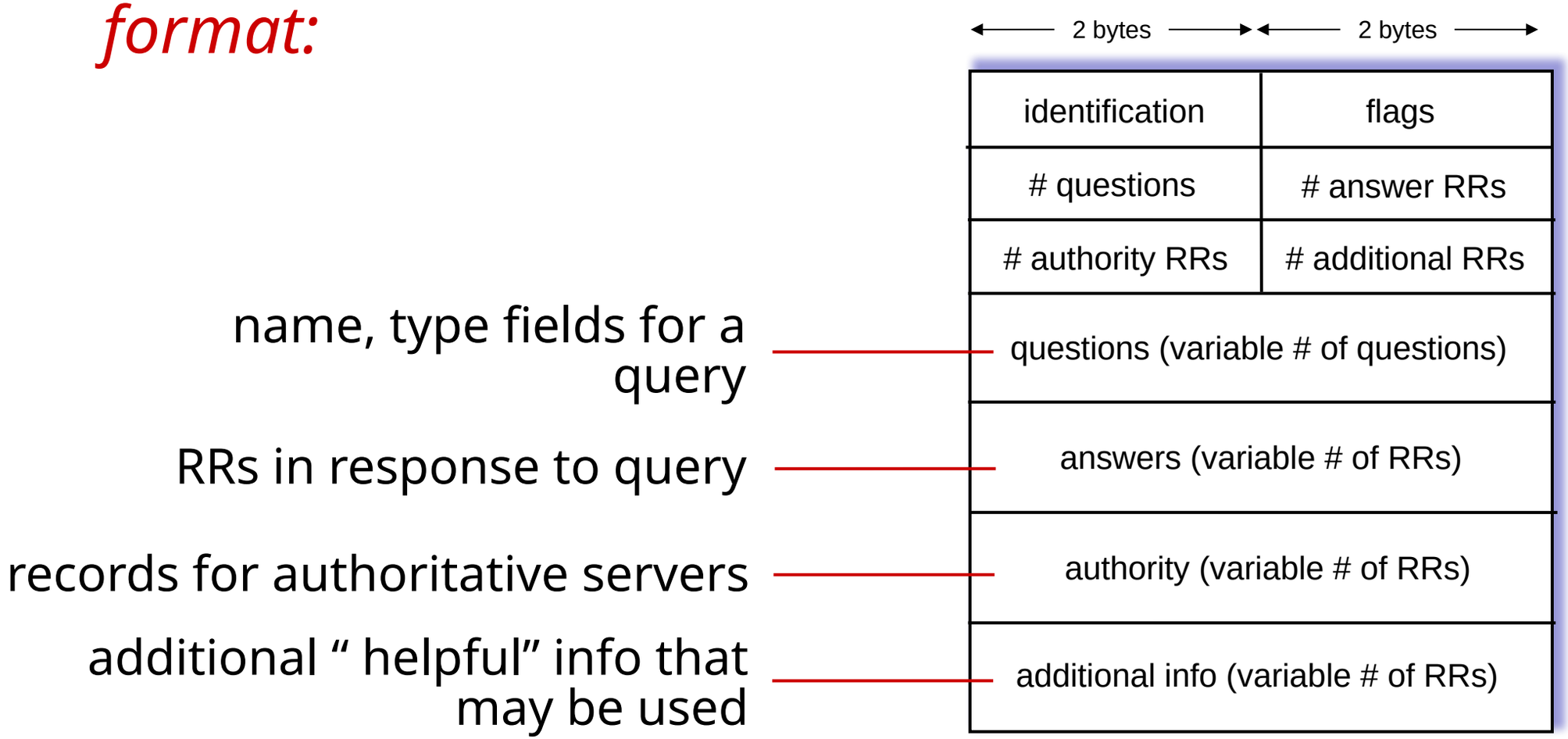
message header:

- identification: 16 bit # for query, reply to query uses same #
- flags:
  - query or reply
  - recursion desired
  - recursion available
  - reply is authoritative



# DNS protocol messages

DNS *query* and *reply* messages, both have same *format:*



# Getting your info into the DNS

example: new startup “Network Utopia”

- register name networkuptopia.com at *DNS registrar* (e.g., Network Solutions)
  - provide names, IP addresses of authoritative name server (primary and secondary)
  - registrar inserts NS, A RRs into .com TLD server:

    `(networkutopia.com, dns1.networkutopia.com, NS)`

    `(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address `212.212.212.1`
  - type A record for www.networkuptopia.com
  - type MX record for networkutopia.com

# DNS security

## DDoS attacks

- bombard root servers with traffic
  - not successful to date
  - traffic filtering
  - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
  - potentially more dangerous

## Spoofing attacks

- intercept DNS queries, returning bogus replies
  - DNS cache poisoning
  - RFC 4033: DNSSEC authentication services

# Application Layer: Overview

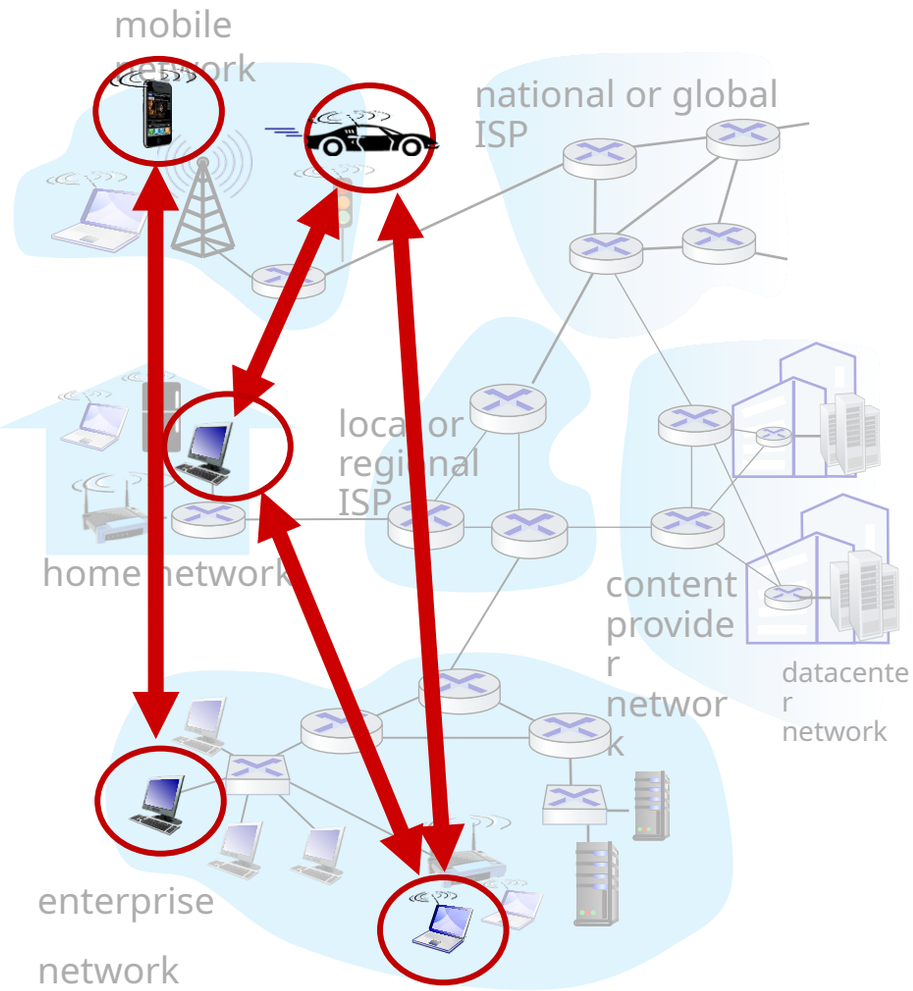
- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS

- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



# Peer-to-peer (P2P) architecture

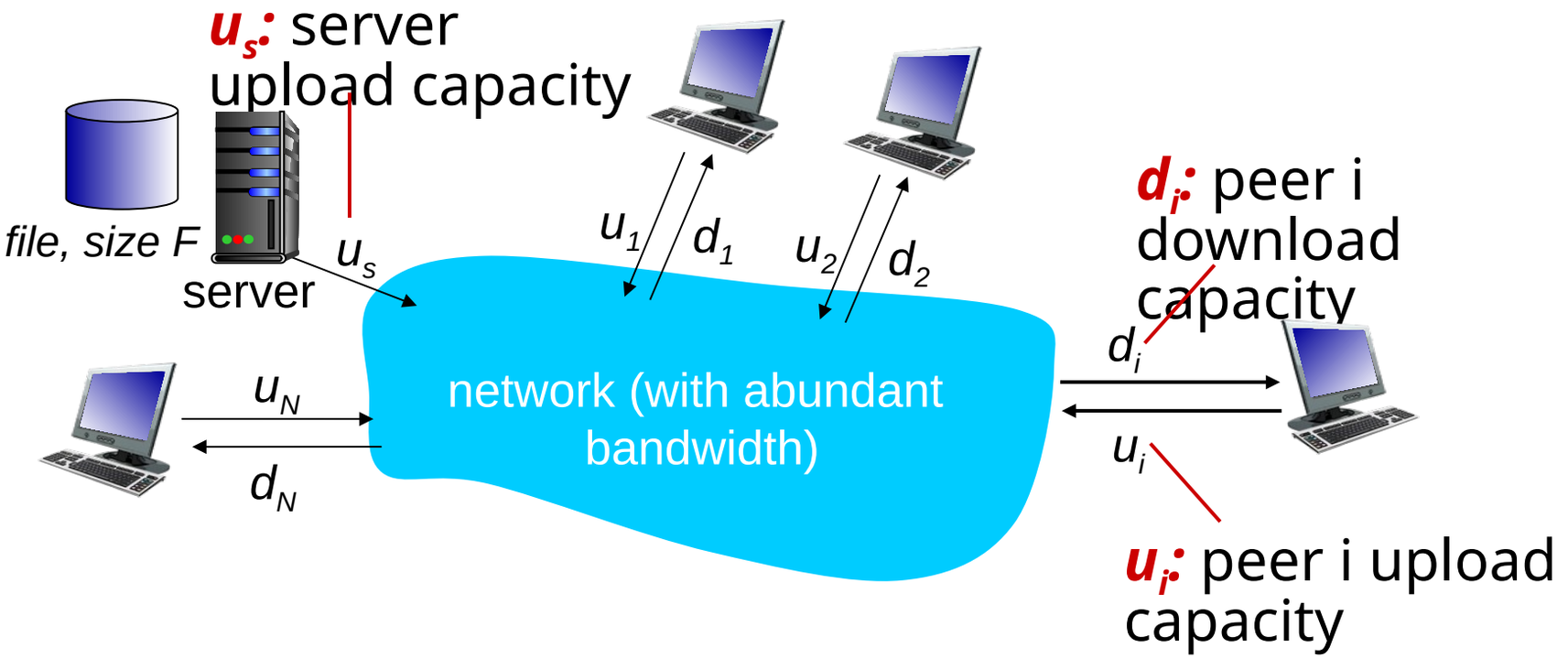
- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- examples: P2P file sharing



# File distribution: client-server vs P2P

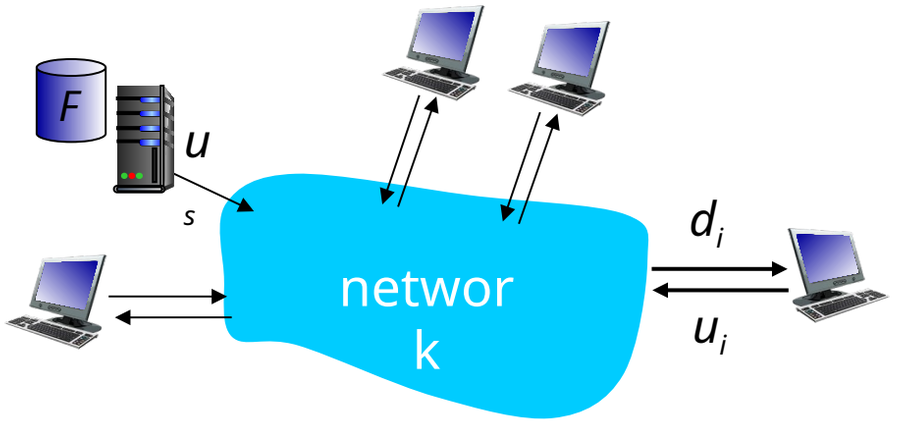
*Q:* how much time to distribute file (size *F*) from one server to *N peers*?

- peer upload/download capacity is limited resource



# File distribution time: client-server

- *server transmission:* must sequentially send (upload) *N* file copies:
  - time to send one copy: $F/u_s$
  - time to send *N* copies: $NF/u_s$
- *client:* each client must download file copy
  - $d_{min}$ = min client download rate
  - min client download time: $F/d_{min}$

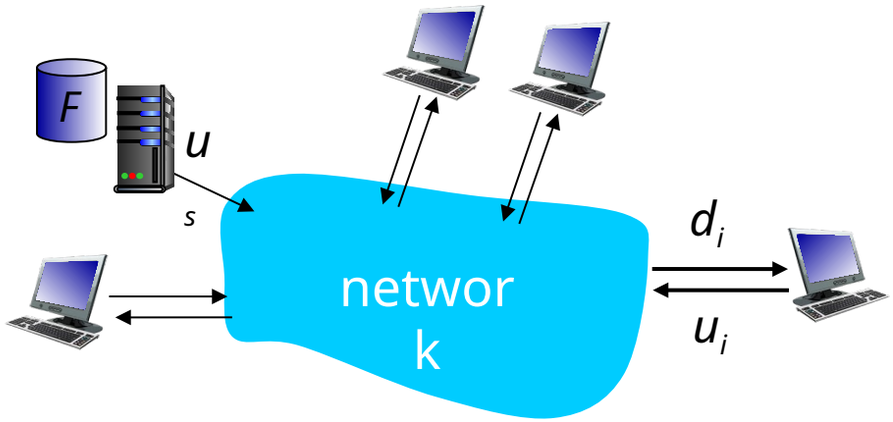



*time to distribute F to N clients using client-server approach* $D_{c\text{-}s} \geq max\{NF/u_s, F/d_{min}\}$

increases linearly in N

# File distribution time: P2P

- *server transmission:* must upload at least one copy:
  - time to send one copy: $F/u_s$
- *client:* each client must download file copy
  - min client download time: $F/d_{min}$
- *clients:* as aggregate must download $NF$ bits
  - max upload rate (limiting max download rate) is $u_s + \Sigma u_i$




*time to distribute F to N clients using P2P approach*

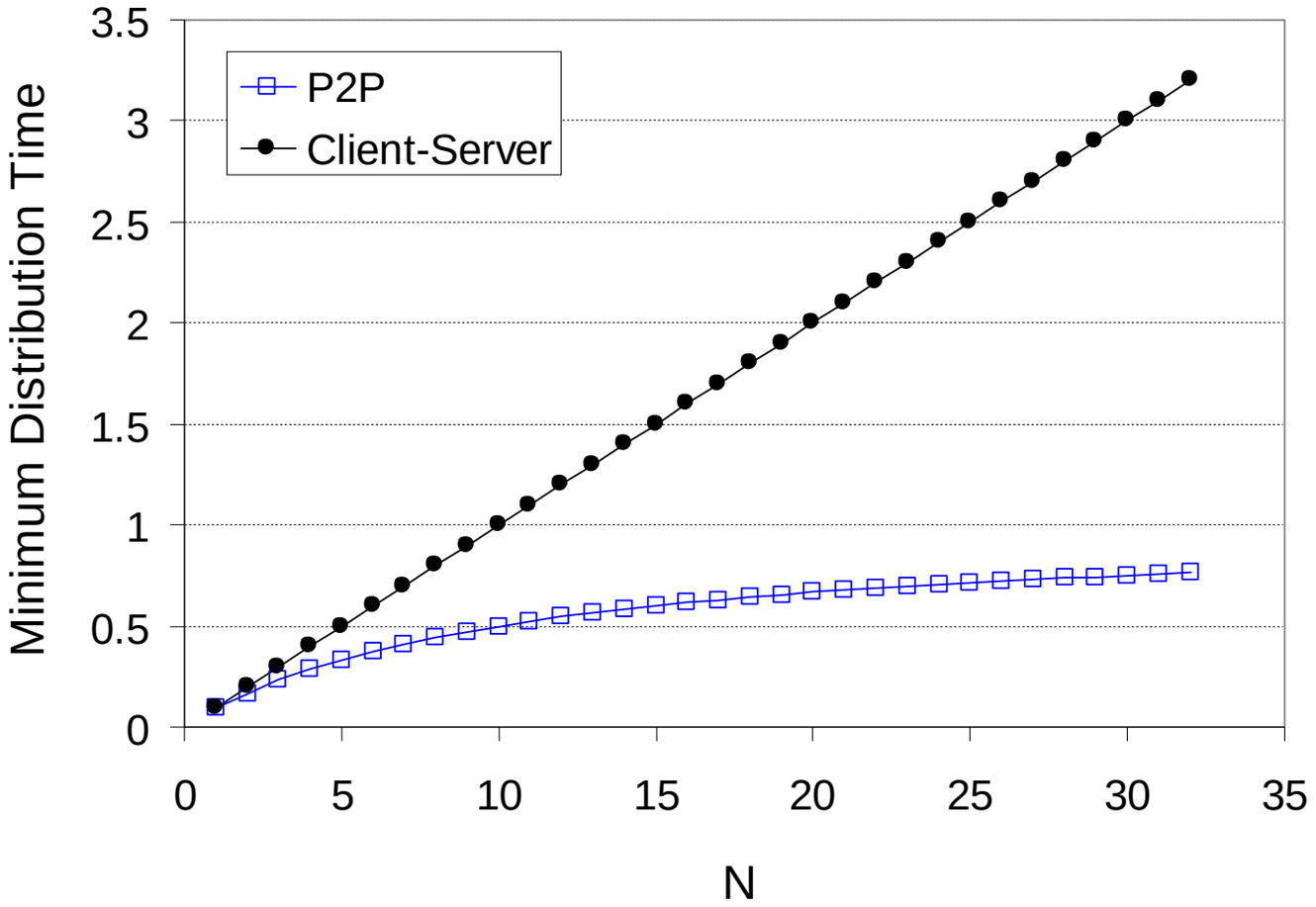
$$D_{P2P} \geq max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$$

increases linearly in $N$ ...

... but so does this, as each peer brings service capacity

# Client-server vs. P2P: example

client upload rate = $u$, $F/u$ = 1 hour, $u_s = 10u$, $d_{min} \geq$

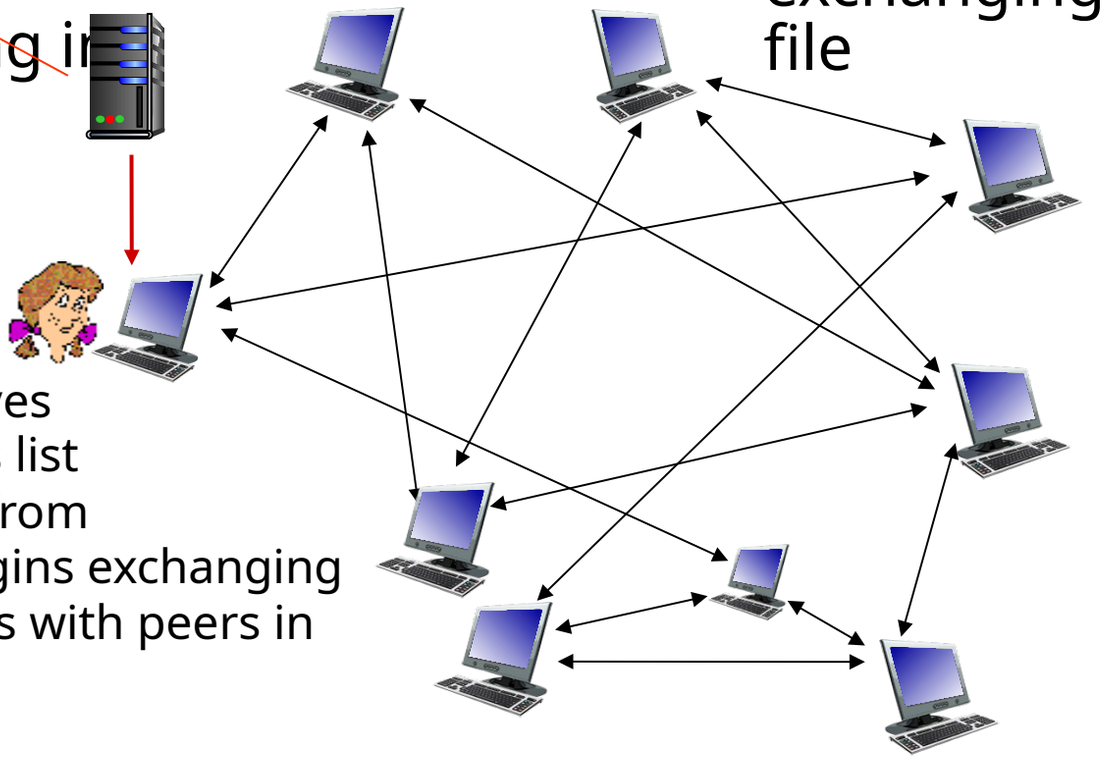


# P2P file distribution: BitTorrent

- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

*tracker:* tracks peers participating in torrent

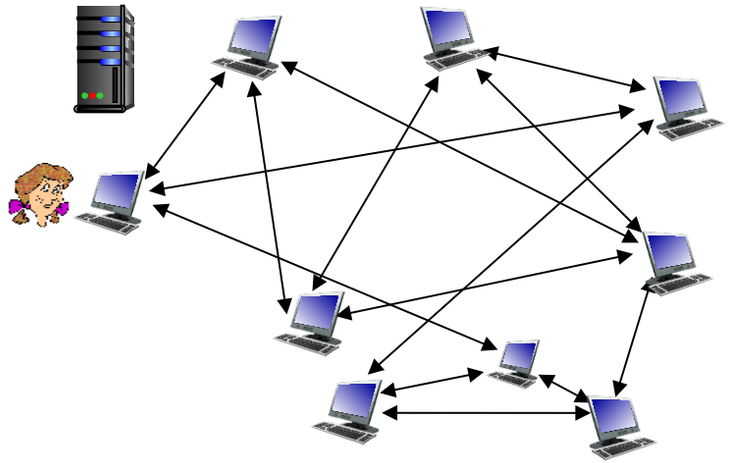
*torrent:* group of peers exchanging chunks of a file



Alice arrives
… obtains list of peers from tracker
… begins exchanging file chunks with peers in torrent

# P2P file distribution: BitTorrent

- peer joining torrent:
  - has no chunks, but will accumulate them over time from other peers
  - registers with tracker to get list of peers, connects to subset of peers ("neighbors")



- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks
- *churn:* peers may come and go
- once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent

# BitTorrent: requesting, sending file chunks

## Requesting chunks:

- at any given time, different peers have different subsets of file chunks
- periodically, Alice asks each peer for list of chunks that they have
- Alice requests missing chunks from peers, rarest first

## Sending chunks: tit-for-tat

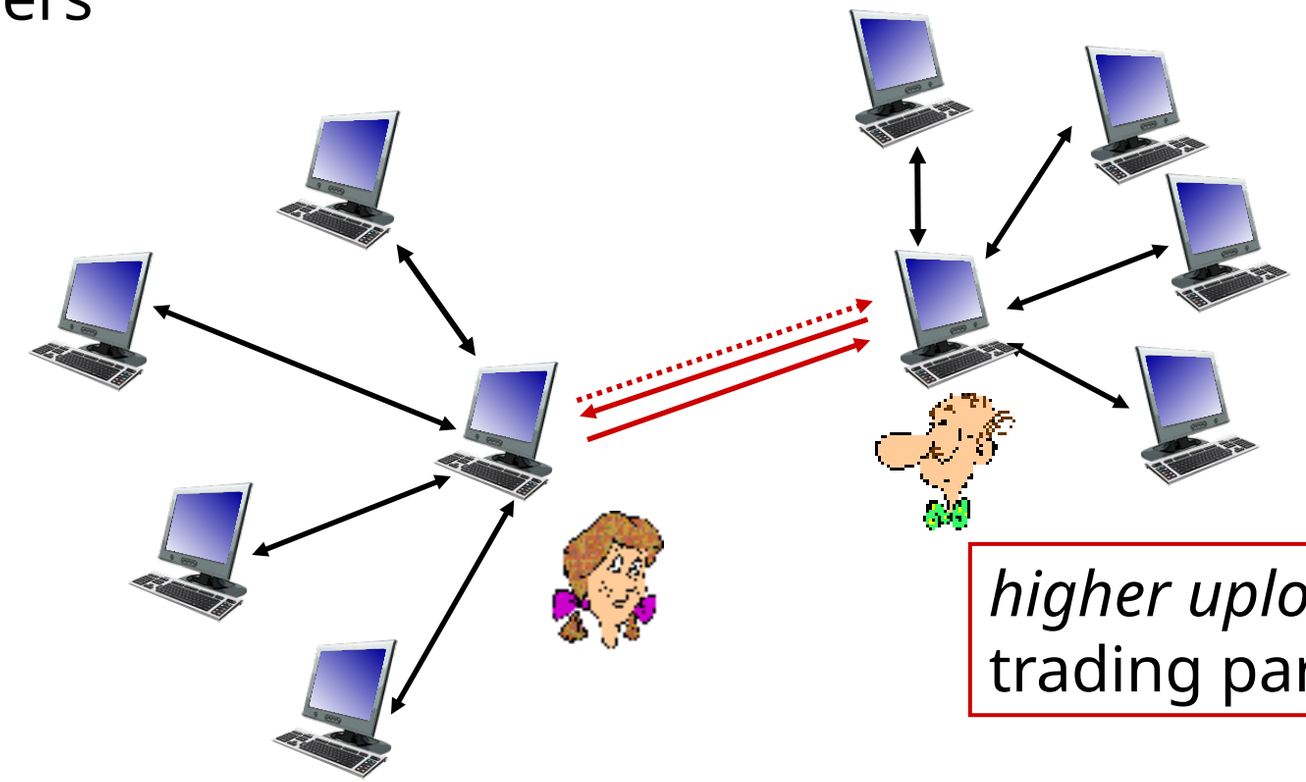
- Alice sends chunks to those four peers currently sending her chunks *at highest rate*
  - other peers are choked by Alice (do not receive chunks from her)
  - re-evaluate top 4 every10 secs
- every 30 secs: randomly select another peer, starts sending chunks
  - "optimistically unchoke" this peer
  - newly chosen peer may join top 4

# BitTorrent: tit-for-tat

(1) Alice “optimistically unchokes” Bob

(2) Alice becomes one of Bob’s top-four providers; Bob reciprocates

(3) Bob becomes one of Alice’s top-four providers



*higher upload rate:* find better trading partners, get file faster !

# Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS

- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



# Video Streaming and CDNs: context

- stream video traffic: major consumer of Internet bandwidth
  - Netflix, YouTube, Amazon Prime: 80% of residential ISP traffic (2020)
- *challenge:* scale - how to reach ~1B users?
- *challenge:* heterogeneity
  - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)
- *solution:* distributed, application-level infrastructure







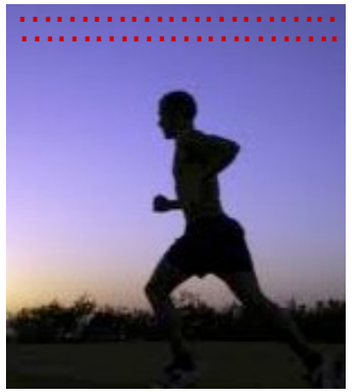





# Multimedia: video

- video: sequence of images displayed at constant rate
  - e.g., 24 images/sec
- digital image: array of pixels
  - each pixel represented by bits
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
  - spatial (within image)
  - temporal (from one image to next)

*spatial coding example:* instead of sending *N* values of same color (all purple), send only two values: color value (*purple*) *and number of repeated values (*N)



frame *i*



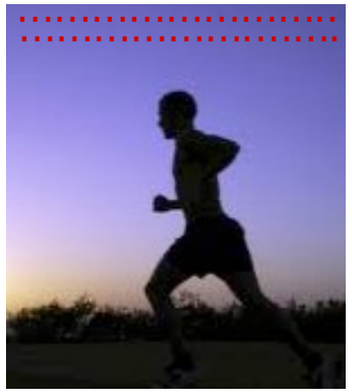
frame *i+1*

*temporal coding example:* instead of sending complete frame at i+1, send only differences from frame i

# Multimedia: video

- CBR: (constant bit rate): video encoding rate fixed
- VBR: (variable bit rate): video encoding rate changes as amount of spatial, temporal coding changes
- examples:
  - MPEG 1 (CD-ROM) 1.5 Mbps
  - MPEG2 (DVD) 3-6 Mbps
  - MPEG4 (often used in Internet, 64Kbps – 12

*spatial coding example:* instead of sending *N* values of same color (all purple), send only two values: color value (*purple*) *and number of repeated values* (N)



frame *i*

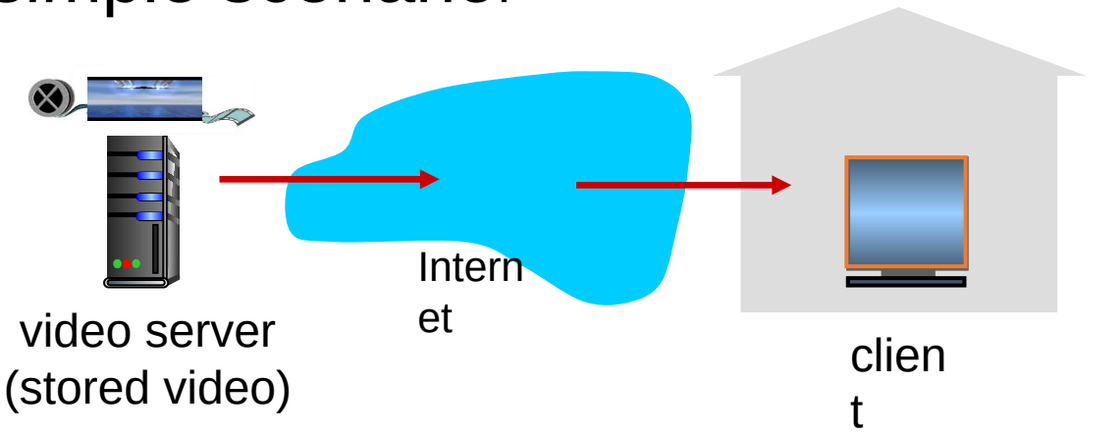


frame *i+1*

*temporal coding example:* instead of sending complete frame at i+1, send only differences from frame i

# Streaming stored video

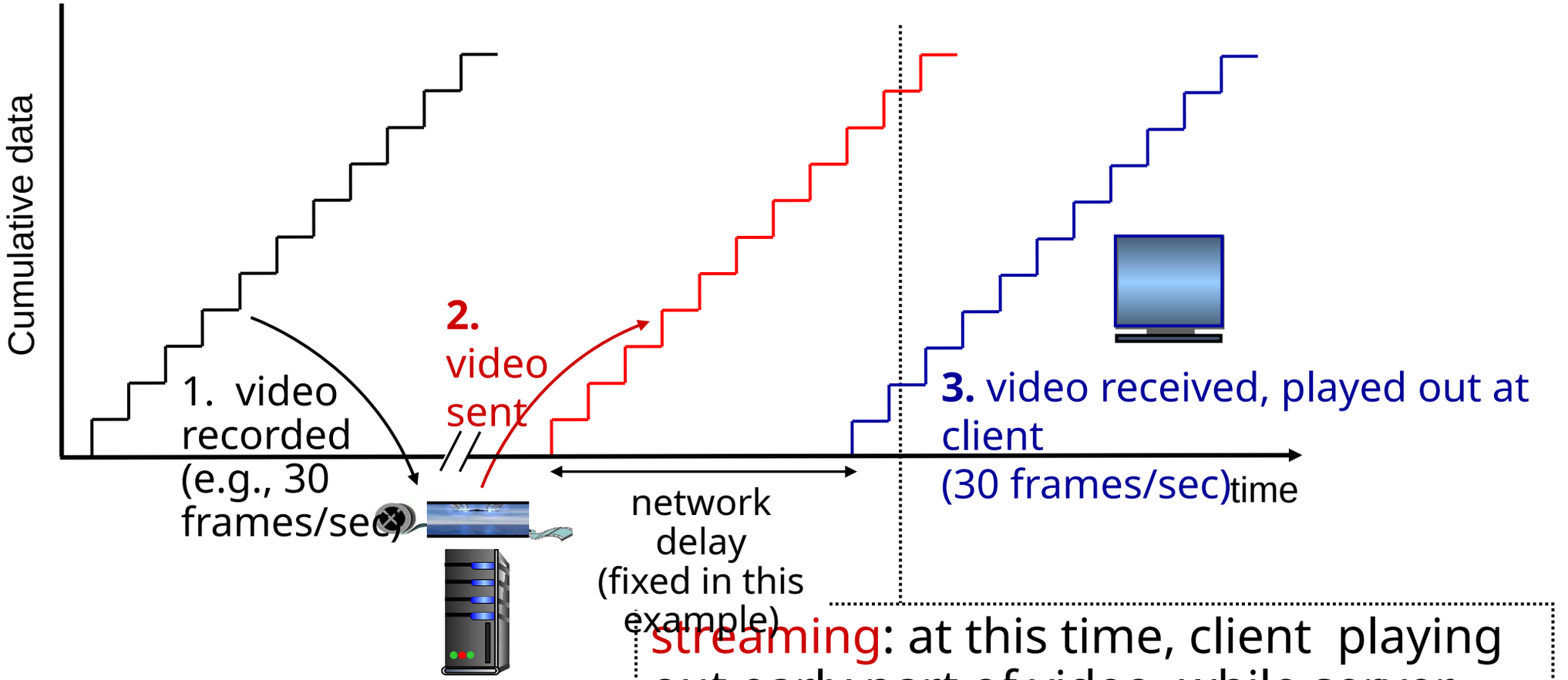
simple scenario:




Main challenges:

- server-to-client bandwidth will *vary* over time, with changing network congestion levels (in house, access network, network core, video server)
- packet loss, delay due to congestion will delay playout, or result in poor video quality

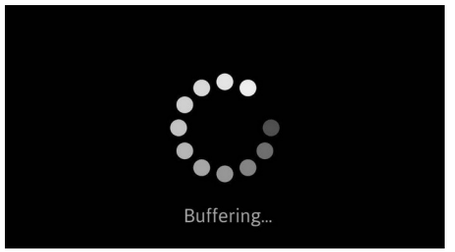
# Streaming stored video



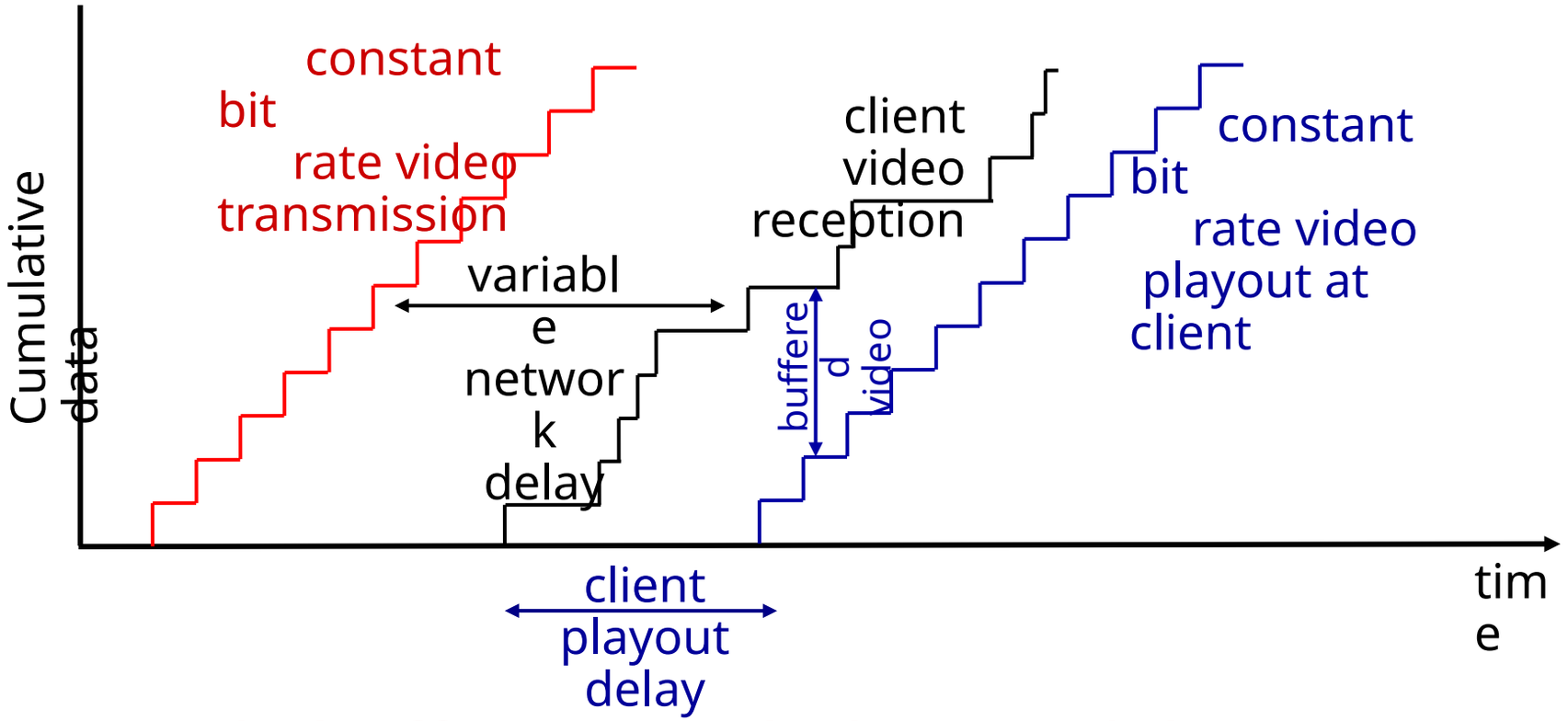

streaming: at this time, client playing out early part of video, while server still sending later part of video

# Streaming stored video: challenges

- continuous playout constraint: during client video playout, playout timing must match original timing
  - … but network delays are variable (jitter), so will need client-side buffer to match continuous playout constraint
- other challenges:
  - client interactivity: pause, fast-forward, rewind, jump through video
  - video packets may be lost, retransmitted



# Streaming stored video: playout buffering



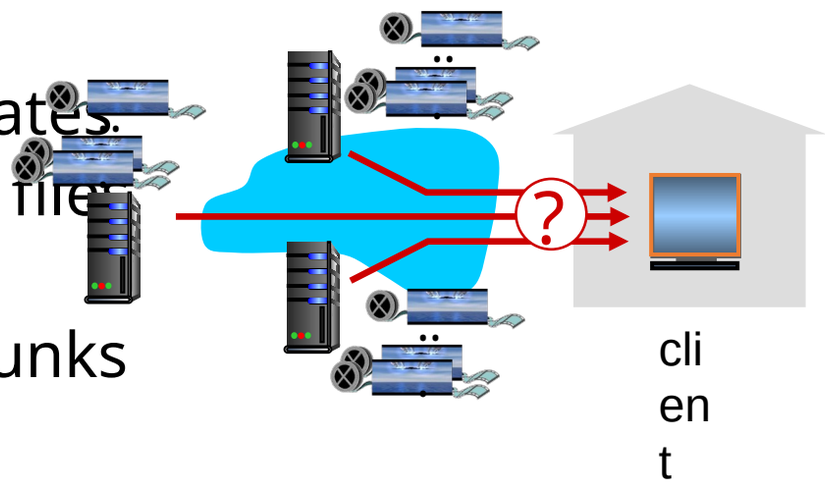

- *client-side buffering and playout delay:* compensate for network-added delay, delay jitter

# Streaming multimedia: DASH

*D*ynamic, *A*daptive *S*treaming over *H*TTP

server:

- divides video file into multiple chunks
- each chunk encoded at multiple different rates
- different rate encodings stored in different files
- files replicated in various CDN nodes
- *manifest file:* provides URLs for different chunks

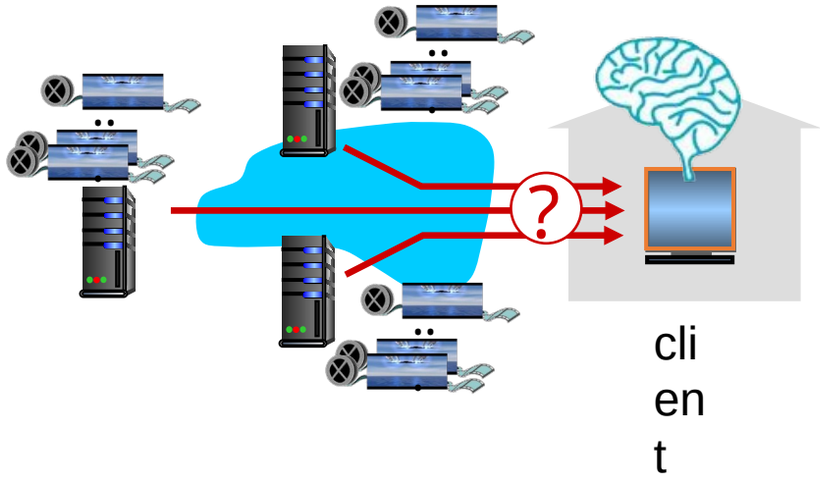



client:

- periodically estimates server-to-client bandwidth
- consulting manifest, requests one chunk at a time
  - chooses maximum coding rate sustainable given current bandwidth
  - can choose different coding rates at different points in time (depending on available bandwidth at time), and from different servers

# Streaming multimedia: DASH

- *"intelligence"* at client: client determines
  - *when* to request chunk (so that buffer starvation, or overflow does not occur)
  - *what encoding rate* to request (higher quality when more bandwidth available)
  - *where* to request chunk (can request from URL server that is "close" to client or has high available bandwidth)




Streaming video = encoding + DASH + playout buffering

# Content distribution networks (CDNs)

*challenge:* how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 1:* single, large "mega-server"
  - single point of failure
  - point of network congestion
  - long (and possibly congested) path to distant clients

....quite simply: this solution *doesn't scale*

# Content distribution networks (CDNs)

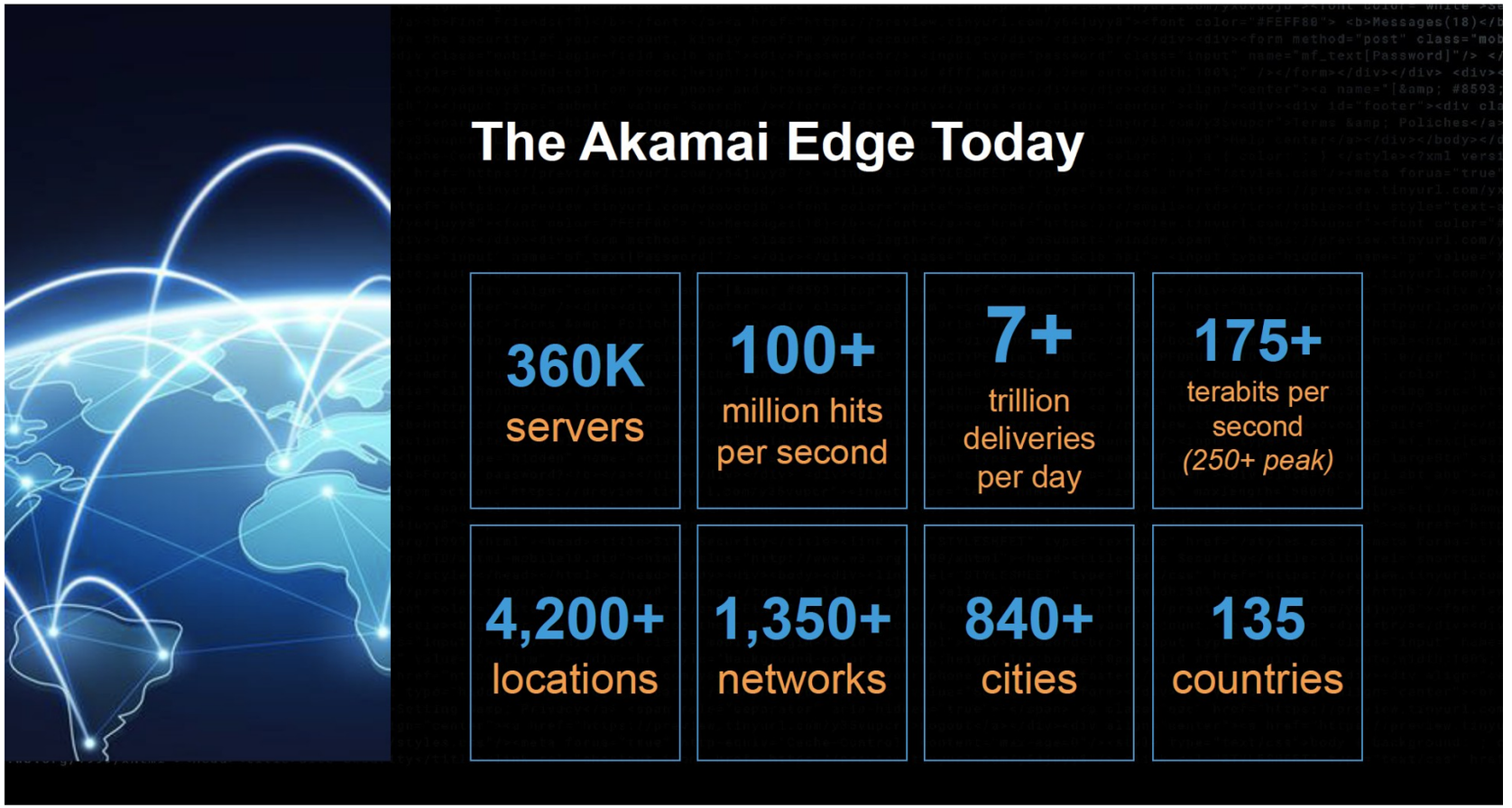
*challenge:* how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 2:* store/serve multiple copies of videos at multiple geographically distributed sites *(CDN)*
  - *enter deep:* push CDN servers deep into many access networks
    - close to users
    - Akamai: 240,000 servers deployed in > 120 countries (2015)
  - *bring home:* smaller number (10's) of larger clusters in POPs near access nets
    - used by Limelight





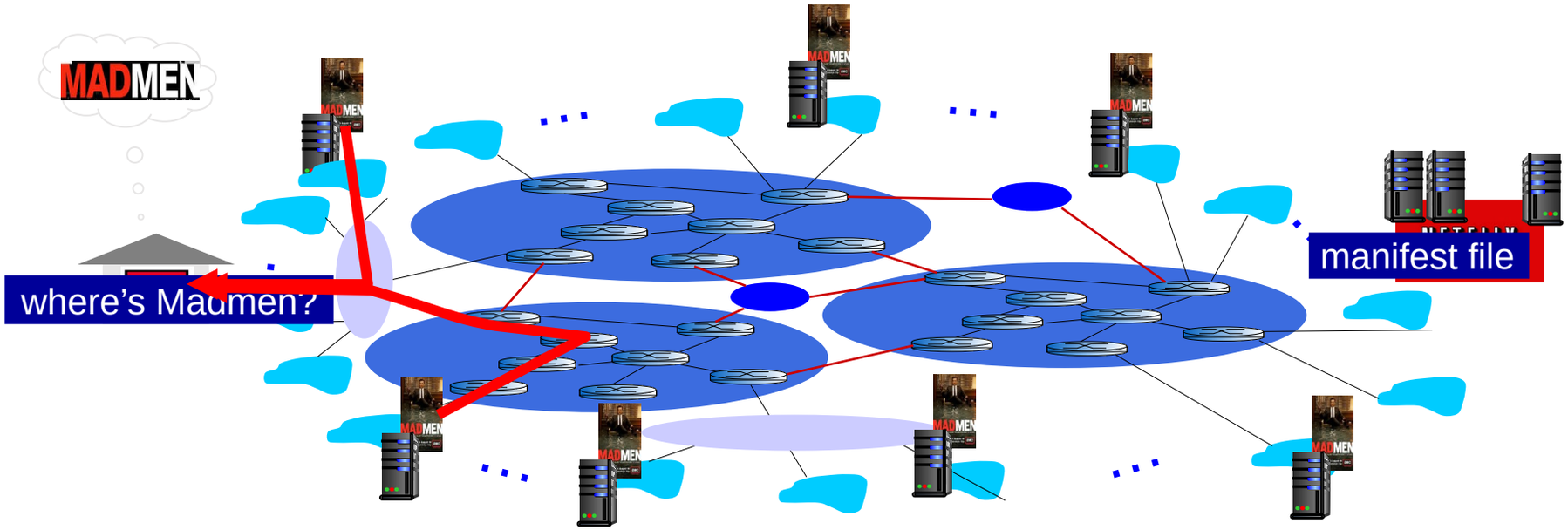

# Akamai today:



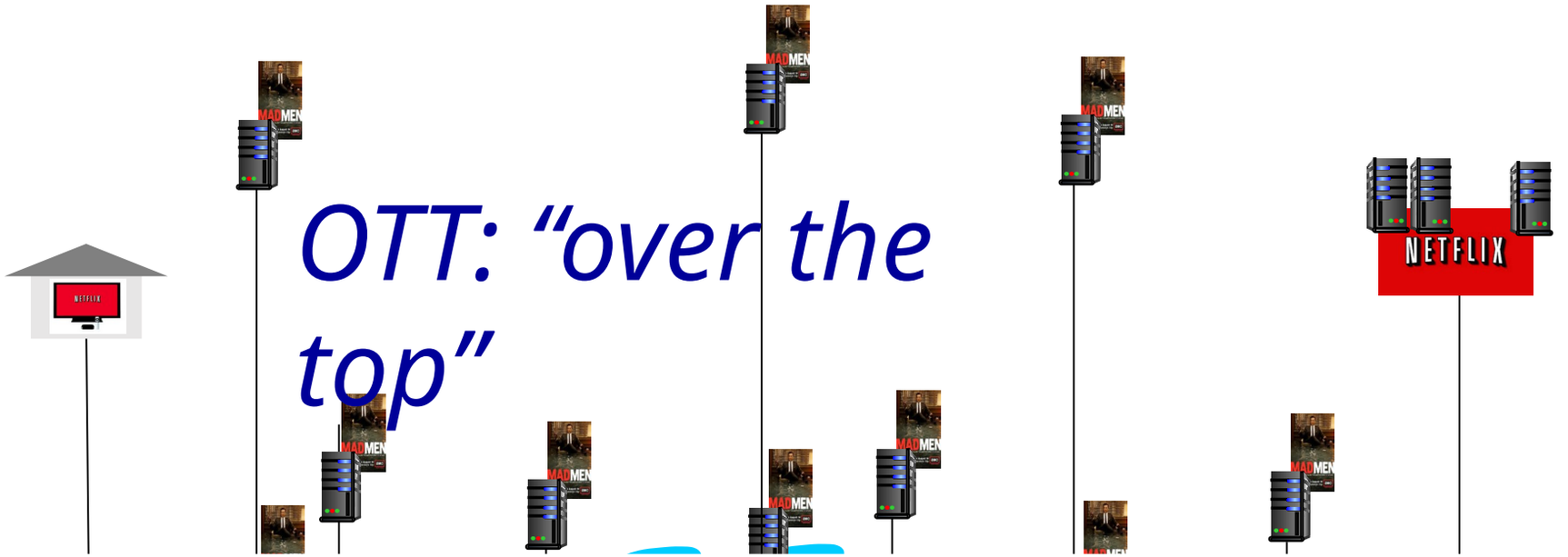

Source: https://networkingchannel.eu/living-on-the-edge-for-a-quarter-century-an-akamai-retrospective-downloads/

# How does Netflix work?

- Netflix: stores copies of content (e.g., MADMEN) at its (worldwide) OpenConnect CDN nodes
- subscriber requests content, service provider returns manifest
  - using manifest, client retrieves content at highest supportable rate
  - may choose different rate or copy if network path congested



# Content distribution networks (CDNs)




Internet host-host communication as a service

*OTT challenges:* coping with a congested Internet from the “edge”

- what content to place in which CDN node?
- from which CDN node to retrieve content? At which rate?

# Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS

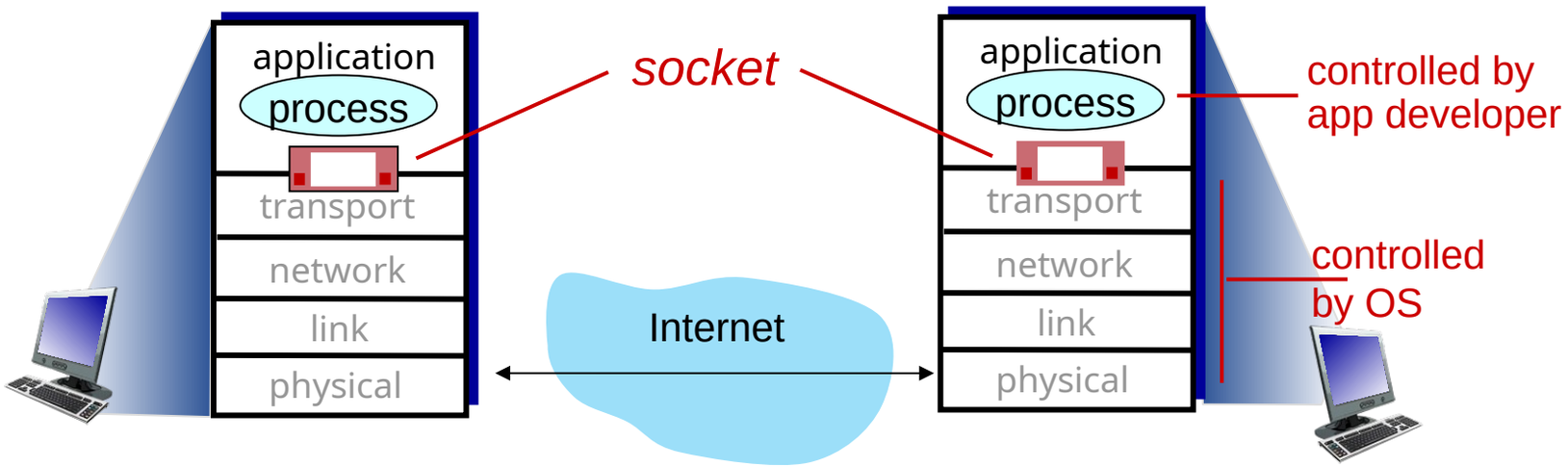
- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



# Socket programming

*goal:* learn how to build client/server applications that communicate using sockets

*socket:* door between application process and end-end-transport protocol



# Socket programming

## Two socket types for two transport services:

- *UDP:* unreliable datagram
- *TCP:* reliable, byte stream-oriented

## Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

# Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server processes

# Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client



create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with serverIP address
And port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

# Example app: UDP client

*Python UDPClient*

include Python's socket library →
```
from socket import *
```
```
serverName = 'hostname'
serverPort = 12000
```
create UDP socket for server →
```
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
```
get user keyboard input →
```
message = raw_input('Input lowercase sentence:')
```
attach server name, port to message; send into socket →
```
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
```
read reply characters from socket into string →
```
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
```
print out received string and close socket →
```
print modifiedMessage.decode()
clientSocket.close()
```

# Example app: UDP server

*Python UDPServer*

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print ("The server is ready to receive")
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

create UDP socket →

bind socket to local port number 12000 →

loop forever →

Read from UDP socket into message, getting client's address (client IP and port) →

send upper case string back to this client →

# Socket programming with TCP

## Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

## Client contacts server by:

- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
  - allows server to talk with multiple clients
  - *source* port numbers used to distinguish clients (more in Chap

Application viewpoint

n-order byte-stream transfer ("pipe") between client and server processes

# Client/server socket interaction: TCP



server (running on hostid)

client



create socket, port=**x**, for incoming request:
serverSocket = socket()

wait for incoming connection request
connectionSocket = serverSocket.accept()

TCP connection setup

create socket, connect to **hostid**, port=**x**
clientSocket = socket()

send request using clientSocket

read request from connectionSocket

write reply to connectionSocket

read reply from clientSocket

close connectionSocket

close clientSocket

# Example app: TCP client

*Python TCPClient*

```
from socket import *
serverName = ’servername’
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = raw_input(‘Input lowercase sentence:’)
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print (‘From Server:’, modifiedSentence.decode())
clientSocket.close()
```

create TCP socket for server, remote port 12000 →

No need to attach server name, port →

# Example app: TCP server

*Python TCPServer*

```
 from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind((‘’,serverPort))
serverSocket.listen(1)
print ‘The server is ready to receive’
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                                        encode())
    connectionSocket.close()
```

create TCP welcoming socket →

server begins listening for incoming TCP requests →

loop forever →

server waits on accept() for incoming requests, new socket created on return →

read bytes from socket (but not address as in UDP) →

close connection to this client (but *not* welcoming socket) →

# Chapter 2: Summary

our study of network application layer is now complete!

- application architectures
  - client-server
  - P2P
- application service requirements:
  - reliability, bandwidth, delay
- Internet transport service model
  - connection-oriented, reliable: TCP
  - unreliable, datagrams: UDP

- specific protocols:
  - HTTP
  - SMTP, IMAP
  - DNS
  - P2P: BitTorrent
- video streaming, CDNs
- socket programming: TCP, UDP sockets

# Chapter 2: Summary

## Most importantly: learned about *protocols*!

- typical request/reply message exchange:
  - client requests info or service
  - server responds with data, status code
- message formats:
  - *headers*: fields giving info about data
  - *data:* info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer
- "complexity at network edge"

# Additional Chapter 2 slides

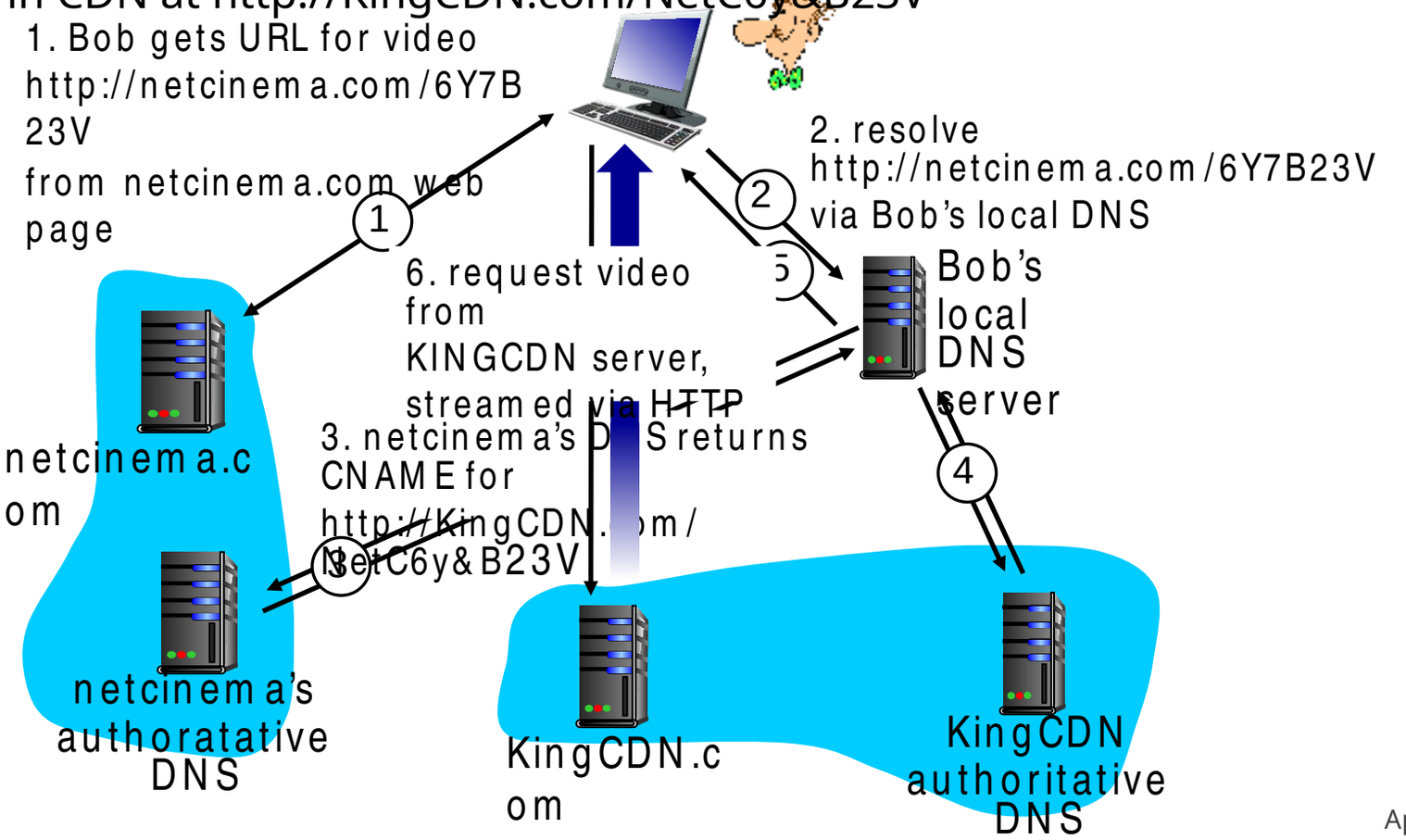
# Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

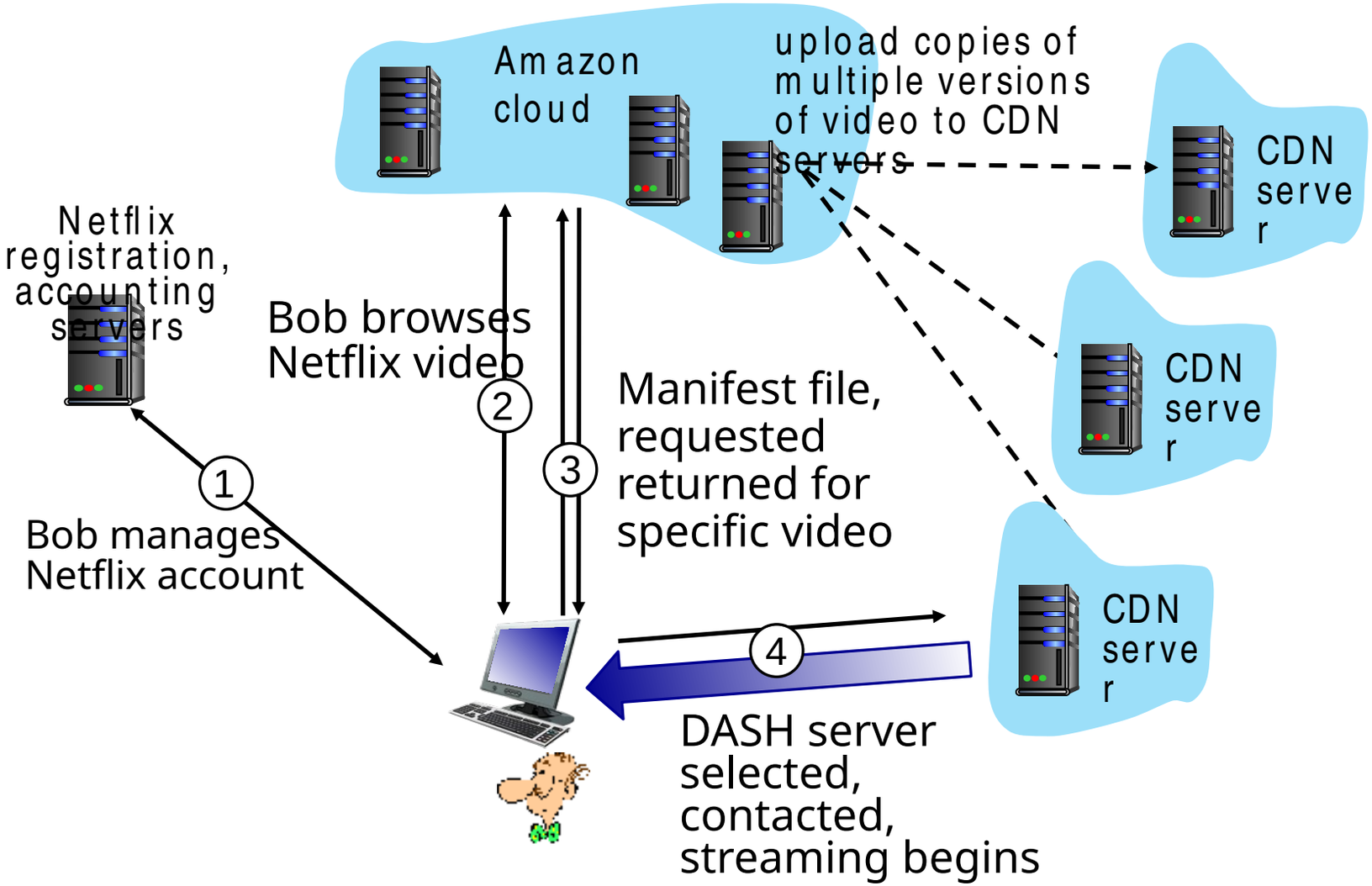
# CDN content access: a closer look

Bob (client) requests video http://netcinema.com/6Y7B23V

- video stored in CDN at http://KingCDN.com/NetC6y&B23V



# Case study: Netflix



# Computer Networks: Physical Layer

## Digital-to-Digital Transmission, Line Coding, Block Coding & Scrambling

Complete Digitized & Comprehensive Lecture Notes of Dr. Sadi Sir

**Department of Computer Science and Engineering**
Digitized from Handwritten Class Notes (Photo.pdf)

Academic Year 2026

**Abstract**

This monograph presents a rigorous, publication-grade digital reconstruction of Dr. Sadi Sir's lecture series on Physical Layer Data Transmission, encompassing Lectures 2, 3, and 4. It covers the full taxonomy of Line Coding schemes (Unipolar NRZ, Polar NRZ-L, Polar NRZ-I, Polar RZ, Biphase Manchester with both Dr. Thomas and IEEE 802.3 conventions, and Differential Manchester), Bipolar signaling (AMI and Pseudoternary), Multilevel schemes (theoretical $mBnL$ principles, 2B1Q, 8B6T with dynamic DC weight balancing, and 4D-PAM5), Block Coding mechanisms ($mB/nB$ architecture, 4B/5B run-length constraints), and Scrambling techniques (B8ZS and HDB3 state machines). All mathematical derivations, signal transitions, timing diagrams, state tables, and worked numerical examples from the original camera scans have been digitized with high-resolution vector graphics and analytical prose.

# Contents


**1 Foundations of Line Coding & Signal Design** **1**
- 1.1 Introduction to Digital-to-Digital Conversion . . . 1
- 1.2 Taxonomy of Line Coding Schemes . . . 1

**2 Unipolar and Polar Line Coding Schemes** **3**
- 2.1 Unipolar Non-Return-to-Zero (NRZ) . . . 3
- 2.2 Polar NRZ Schemes: NRZ-L and NRZ-I . . . 4
  - 2.2.1 Polar NRZ-Level (NRZ-L) . . . 4
  - 2.2.2 Polar NRZ-Invert (NRZ-I) . . . 4
- 2.3 Polar Return-to-Zero (RZ) . . . 5
- 2.4 Biphase Schemes: Manchester & Differential Manchester . . . 6
  - 2.4.1 Manchester Encoding: Dr. Thomas vs. IEEE 802.3 Conventions . . . 6
  - 2.4.2 Differential Manchester Encoding . . . 6

**3 Bipolar and Multilevel Transmission** **9**
- 3.1 Bipolar Signaling: AMI and Pseudoternary . . . 9
  - 3.1.1 Alternate Mark Inversion (AMI) . . . 9
  - 3.1.2 Pseudoternary Scheme . . . 9
- 3.2 Multilevel Coding: The $mBnL$ Architecture . . . 10
- 3.3 Multilevel 2B1Q (Two Binary, One Quaternary) . . . 11
- 3.4 Multilevel 8B6T and Dynamic DC Weight Balancing . . . 12
  - 3.4.1 DC Weight Mechanics . . . 12
- 3.5 Higher-Order Multilevel: 4D-PAM5 . . . 14

**4 Block Coding and Scrambling Techniques** **15**
- 4.1 Block Coding ($mB/nB$) Principles . . . 15
- 4.2 4B/5B Encoding and Run-Length Constraints . . . 15
- 4.3 Scrambling Principles for Bipolar AMI . . . 16
- 4.4 B8ZS (Bipolar with 8-Zero Substitution) . . . 17
- 4.5 HDB3 (High-Density Bipolar 3-Zero) . . . 18

**5 Comparative Analysis & Engineering Summary** **21**
- 5.1 Master Comparison Matrix . . . 21
- 5.2 Summary of Dr. Sadi Sir's Core Principles . . . 21

# Chapter 1

# Foundations of Line Coding & Signal Design

## 1.1 Introduction to Digital-to-Digital Conversion

Digital communication fundamentally requires representing discrete information (binary sequences of bits, $\{0, 1\}$) as physical, electromagnetic signals propagated across a guided or unguided transmission channel. The process of converting digital binary data into a digital signal is termed **Line Coding**.

A line coding scheme maps binary data words into a sequence of discrete voltage pulses. The selection of an optimal line coding technique depends on several critical engineering trade-offs:

1. **DC Component (Direct Current Bias):** When the average voltage of an encoded signal is non-zero, a constant DC component is generated. Transmission components such as transformers, repeaters, and AC-coupling capacitors cannot pass frequencies near zero (0 Hz). A strong DC component leads to signal distortion, heat dissipation, and transformer saturation. An optimal scheme should minimize or eliminate DC bias.

2. **Baseline Wandering:** The receiver computes an average signal level (the baseline) to distinguish between positive, negative, and zero pulses. A prolonged sequence of identical bits creates a drift in this reference voltage, termed *baseline wander*, resulting in decoding bit errors.

3. **Self-Synchronization (Clock Recovery):** For accurate bit detection, the receiver's sampling clock must match the transmitter's bit intervals in both frequency and phase. A self-synchronizing code incorporates guaranteed voltage transitions within each bit interval or guarantees a bounded run of non-transitioning bits, allowing the receiver to phase-lock its local oscillator without dedicated clock wires.

4. **Bandwidth Efficiency & Baud Rate ($S$):** The signaling rate (baud rate $S$) is the number of signal elements transmitted per second, whereas the data rate ($N$) is the number of bits transmitted per second. The relationship is governed by the factor $r = m/n$, representing the number of data bits carried per signal element:

$$S = c \times N \times \frac{1}{r} = c \times N \times \frac{n}{m} \quad \text{[Baud]} \tag{1.1}$$

   where $c$ is the case factor ($0 \leq c \leq 1$, typically $c = 1/2$ for average analysis). The required transmission bandwidth is directly proportional to the baud rate:

$$B_{\min} = c \times S \tag{1.2}$$

5. **Error Detection & Noise Immunity:** Certain codes possess intrinsic structural redundancy that allows instantaneous detection of physical layer transmission errors.

## 1.2 Taxonomy of Line Coding Schemes

In Dr. Sadi Sir's lecture notes, line coding schemes are classified into four principal families based on the voltage polarities and structural mappings employed:

- **Unipolar:** Uses only one polarity of voltage (typically $+V$) and zero (0 V).
- **Polar:** Uses both positive ($+V$) and negative ($-V$) voltage levels across the time axis.
- **Bipolar (Ternary):** Uses three distinct voltage levels ($+V$, 0 V, $-V$), where one bit is assigned zero volts while the other bit alternates in polarity.
- **Multilevel ($mBnL$):** Encodes a group of $m$ binary bits into $n$ signal elements using $L$ distinct voltage levels.

Figure 1.1 depicts the comprehensive taxonomy developed in the lecture notes.

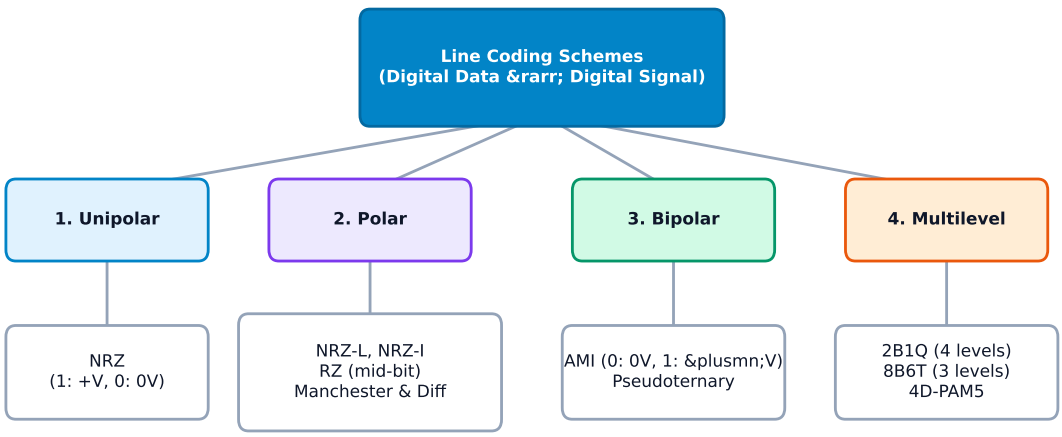



Figure 1.1: Taxonomy of Line Coding Schemes (Digital Data → Digital Signal) from Dr. Sadi Sir's Lectures.

# Chapter 2

# Unipolar and Polar Line Coding Schemes

## 2.1 Unipolar Non-Return-to-Zero (NRZ)

Unipolar NRZ is the simplest digital encoding scheme. It is called *unipolar* because all signal levels lie on a single side of the zero-voltage reference axis.

**Encoding Rule: Unipolar NRZ**

- Binary **1**: Encoded as a positive voltage ($+V$).
- Binary **0**: Encoded as zero voltage (0 V).
- **Non-Return-to-Zero (NRZ):** The voltage remains constant across the entire bit duration ($T_b$) and does not return to zero in the middle of the bit interval.

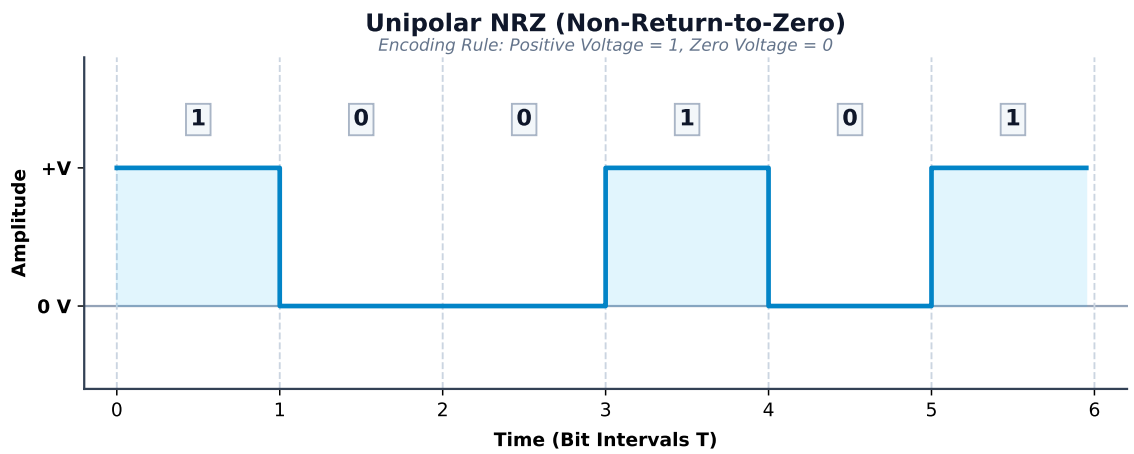



Figure 2.1: Unipolar NRZ Waveform for data stream `100101`.

**Technical Summary & Trade-offs: Unipolar NRZ Evaluation**

- **Baud Rate:** $S = N$ (since $r = 1$ bit per signal element).
- **DC Component:** Extremely high. Assuming equal probabilities for 0 and 1 ($P(0) = P(1) = 0.5$), the average DC voltage is:

$$V_{\text{DC}} = \frac{1}{2}(+V) + \frac{1}{2}(0) = \frac{V}{2} \tag{2.1}$$

  This strong DC component prevents transmission across transformer-isolated or AC-coupled circuits.

- **Baseline Wander:** High susceptibility during long runs of 0s or 1s.

- **Synchronization:** Poor. A prolonged stream of consecutive 1s or 0s produces a flat DC line with zero transitions, leading to receiver clock drift.

## 2.2 Polar NRZ Schemes: NRZ-L and NRZ-I

Polar schemes allocate voltages on both sides of the zero axis, effectively eliminating the non-zero DC baseline inherent to unipolar schemes when the data stream has balanced bit frequencies.

### 2.2.1 Polar NRZ-Level (NRZ-L)

In NRZ-L, the absolute level of the voltage specifies the binary state.

**Encoding Rule: Polar NRZ-L (Dr. Sadi Sir's Convention)**

- Binary **0**: Encoded as a positive voltage ($+V$).
- Binary **1**: Encoded as a negative voltage ($-V$).
- *Note:* The voltage level remains constant throughout the bit interval $T_b$.

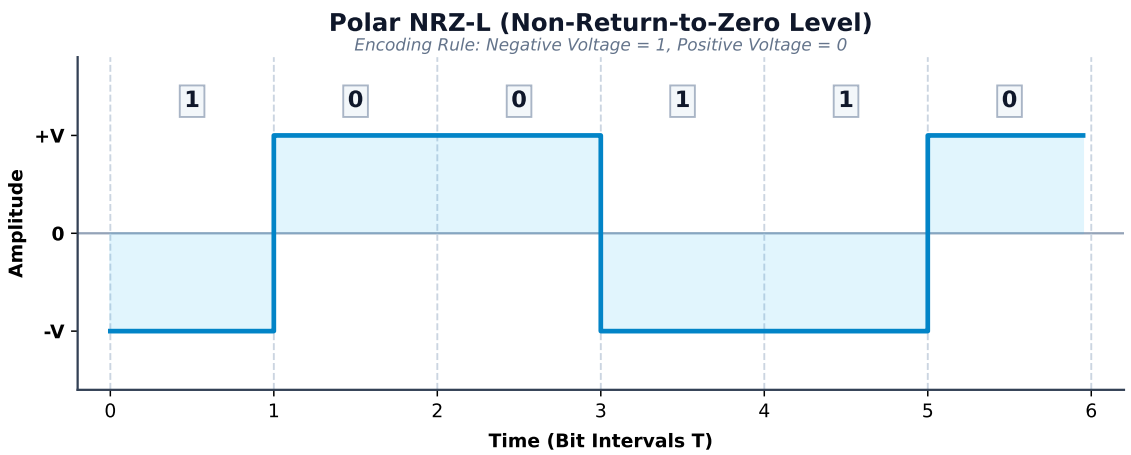



Figure 2.2: Polar NRZ-L Waveform for data stream `100110`.

### 2.2.2 Polar NRZ-Invert (NRZ-I)

Polar NRZ-I relies on *differential encoding*. Rather than associating a bit value with an absolute voltage level, NRZ-I associates bit values with the presence or absence of a voltage transition at the beginning of the bit boundary.

**Encoding Rule: Polar NRZ-I**

- Binary **1**: Triggers a **transition** (inversion of voltage level) at the start of the bit interval.
- Binary **0**: Results in **no transition**; the signal maintains its prior voltage level.
- *Note:* The initial reference level determines the starting phase (typically assumed positive $+V$).

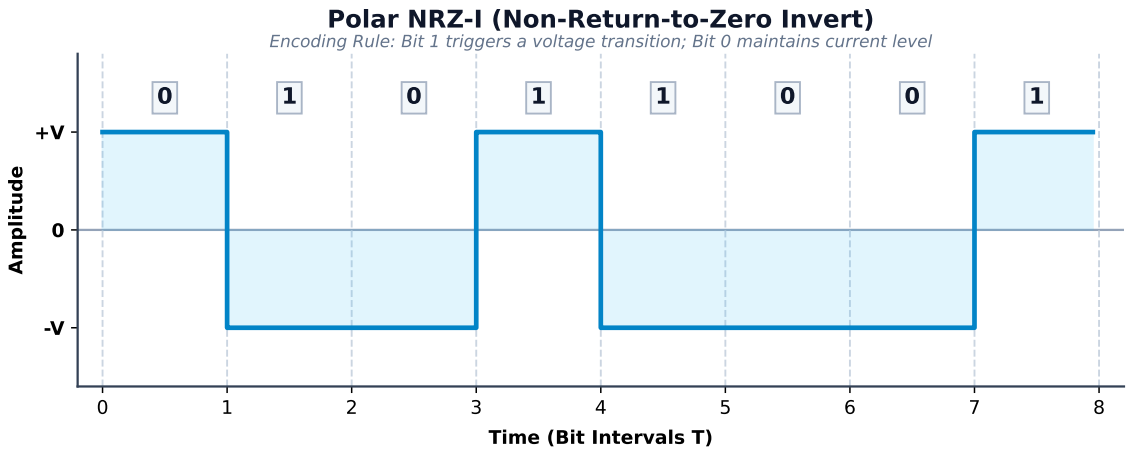



Figure 2.3: Polar NRZ-I Waveform for data stream `01011001` (Initial state: $+V$).

**Dr. Sadi Sir's Lecture Insight: NRZ-L vs. NRZ-I Synchronization**

In NRZ-L, prolonged sequences of both 0s and 1s result in flat voltage levels with zero transitions. In NRZ-I, a sequence of continuous 1s generates frequent transitions at every bit boundary, providing excellent receiver clock synchronization. However, a sequence of consecutive 0s still causes a flat line with no transitions. This remaining limitation is what motivated the creation of **Block Coding (4B/5B)** and **Scrambling**.

## 2.3 Polar Return-to-Zero (RZ)

To overcome the synchronization deficit of NRZ schemes, Polar RZ introduces an intentional mid-bit transition.

**Encoding Rule: Polar RZ**

- Binary **1**: Begins at positive voltage ($+V$) and **returns to zero** (0 V) at the midpoint of the bit interval ($T_b/2$).
- Binary **0**: Begins at negative voltage ($-V$) and **returns to zero** (0 V) at the midpoint of the bit interval ($T_b/2$).
- The scheme uses three voltage states: $+V, 0$ V, $-V$.

**Technical Summary & Trade-offs: Polar RZ Evaluation**

- **Synchronization:** Fully self-synchronizing. Every bit interval contains a guaranteed transition to zero at $t = T_b/2$, allowing continuous clock recovery regardless of data pattern.
- **Bandwidth Penalty:** Because two signal transitions can occur per bit, each bit requires two signal elements ($r = 1/2$). The baud rate is $S = 2N$, which **doubles the required channel bandwidth** compared to NRZ ($B = N$). Due to this high bandwidth overhead and the complexity of distinguishing three voltage levels, RZ is rarely used in modern high-speed physical layers.

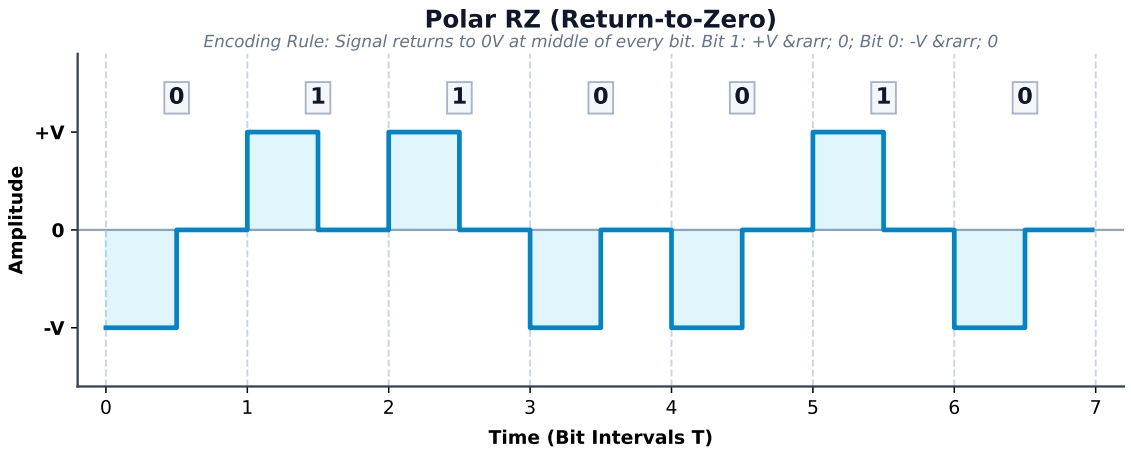



Figure 2.4: Polar RZ Waveform for data stream `0110010`.

## 2.4 Biphase Schemes: Manchester & Differential Manchester

Biphase encoding provides the self-synchronization benefits of RZ while requiring only two voltage levels ($+V$ and $-V$).

### 2.4.1 Manchester Encoding: Dr. Thomas vs. IEEE 802.3 Conventions

In Manchester encoding, every bit interval is bifurcated into two equal halves with a guaranteed transition at the center. In literature and engineering standards, two opposing conventions exist:

**Encoding Rule: Manchester Encoding Conventions**

**Dr. Thomas Convention (Original / Classical Textbook):**

- Binary **0**: Low-to-High transition ($-V \rightarrow +V$ at mid-bit).
- Binary **1**: High-to-Low transition ($+V \rightarrow -V$ at mid-bit).

**IEEE 802.3 Standard Convention (Ethernet 10BASE-T):**

- Binary **1**: Low-to-High transition ($-V \rightarrow +V$ at mid-bit).
- Binary **0**: High-to-Low transition ($+V \rightarrow -V$ at mid-bit).

### 2.4.2 Differential Manchester Encoding

Differential Manchester combines the mid-bit synchronization transition of Manchester with differential encoding at the start of the bit interval. It is used in IEEE 802.5 Token Ring networks.

**Encoding Rule: Differential Manchester Encoding**

- **Mid-Bit Transition:** Always present at the center of every bit interval ($T_b/2$) strictly for clock synchronization.
- Binary **0**: Features a **transition at the beginning** of the bit interval ($t = 0$).
- Binary **1**: Features **no transition at the beginning** of the bit interval ($t = 0$).

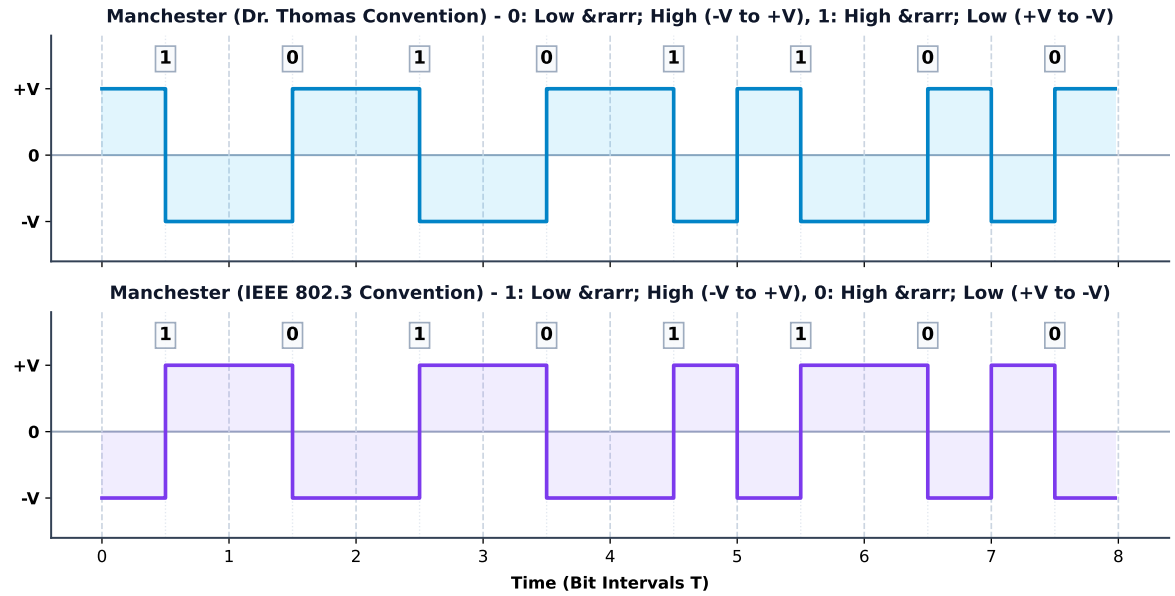



Figure 2.5: Comparison of Manchester Encoding Conventions for data stream 10101100.

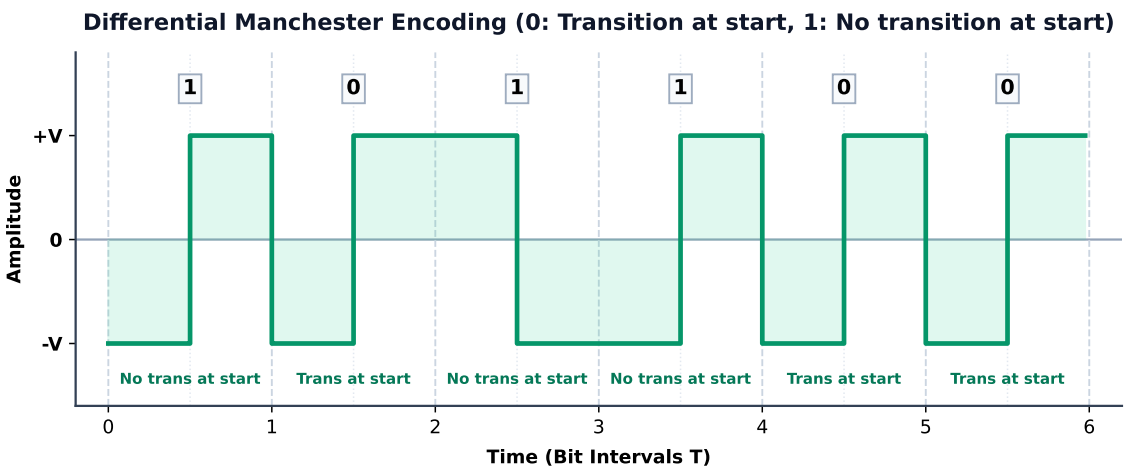



Figure 2.6: Differential Manchester Waveform for data stream 101100.

**Dr. Sadi Sir's Lecture Insight: Immunity to Inversion**

Because Differential Manchester decodes data based solely on whether a transition occurs at the bit boundary—rather than testing for absolute positive or negative polarity—it is completely **immune to line inversion** (swapping the two wires of a twisted-pair line does not corrupt the received bitstream).

# Chapter 3

# Bipolar and Multilevel Transmission

## 3.1 Bipolar Signaling: AMI and Pseudoternary

Bipolar encoding employs three voltage levels: positive ($+V$), zero (0 V), and negative ($-V$). Unlike Polar RZ, zero voltage is used to represent one binary state, while the opposing binary state alternates between positive and negative polarities.

### 3.1.1 Alternate Mark Inversion (AMI)

The term *mark* comes from telegraphy and denotes a binary 1.

**Encoding Rule: Alternate Mark Inversion (AMI)**

- Binary **0**: Encoded as constant zero voltage (0 V).
- Binary **1**: Encoded as alternating positive ($+V$) and negative ($-V$) pulses:

$$1 \implies +V, -V, +V, -V, +V, \ldots \tag{3.1}$$

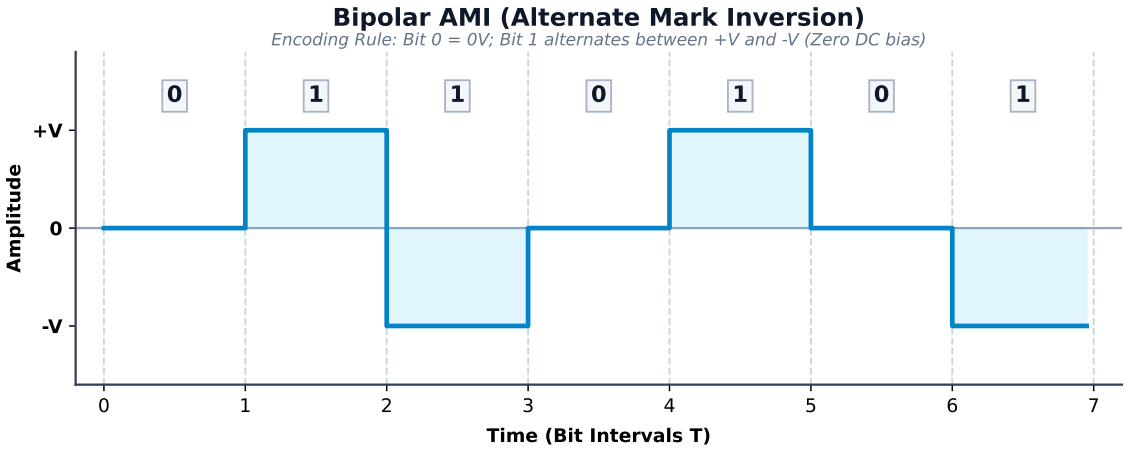



Figure 3.1: Bipolar Alternate Mark Inversion (AMI) for data stream `0110101`.

### 3.1.2 Pseudoternary Scheme

Pseudoternary is the direct functional inverse of AMI.

**Encoding Rule: Pseudoternary Scheme**

- Binary **1**: Encoded as zero voltage (0 V).
- Binary **0**: Encoded as alternating positive ($+V$) and negative ($-V$) pulses:

$$0 \implies +V, -V, +V, -V, +V, \ldots \tag{3.2}$$

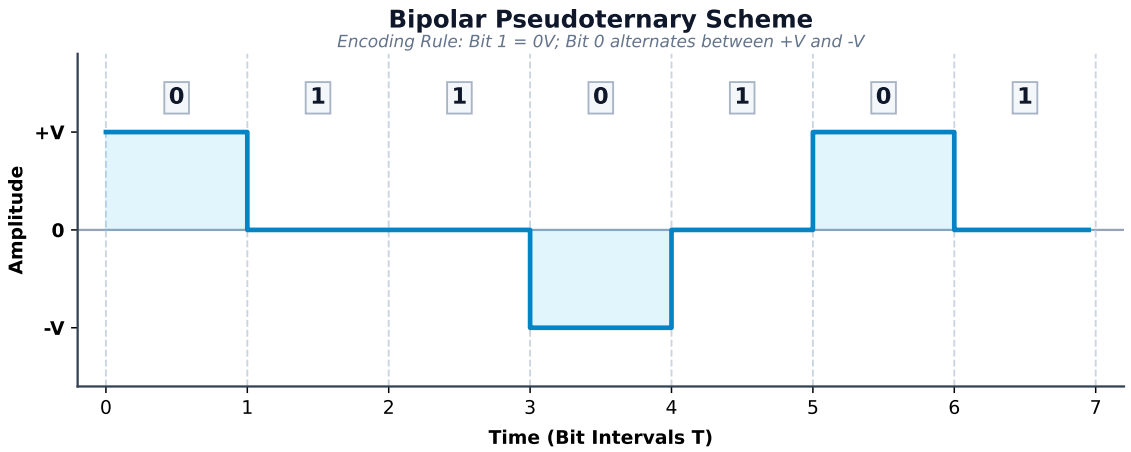



Figure 3.2: Bipolar Pseudoternary Waveform for data stream `0110101`.

**Technical Summary & Trade-offs: Bipolar AMI Technical Highlights**

- **Zero DC Component:** Since every positive pulse is strictly balanced by an equal and opposite negative pulse, the cumulative time-average DC voltage is identically zero: $\int V(t)\,dt = 0$.
- **Narrow Bandwidth:** The main spectral energy of AMI is concentrated at half the bit rate ($N/2$), making it more bandwidth-efficient than Manchester.
- **Intrinsic Error Detection:** A single bit error (e.g., noise flipping a zero into a pulse or changing pulse polarity) causes an *AMI violation* (two consecutive pulses of the same polarity), which the receiver detects immediately without error-checking parity bits.
- **Vulnerability:** A long run of binary zeros produces a sustained zero-voltage condition with no transitions, causing loss of synchronization. This vulnerability is solved by **Scrambling (B8ZS and HDB3)**.

## 3.2 Multilevel Coding: The $mBnL$ Architecture

To increase data throughput across bandwidth-limited physical media, communication engineers developed **Multilevel schemes**, formally designated as $mBnL$.

**Encoding Rule: $mBnL$ Classification Framework**

In an $mBnL$ scheme:

- $m$: Length of the binary data pattern (in bits).
- $B$: Binary data representation ($2^m$ possible data patterns).
- $n$: Length of the encoded signal pattern (number of signal elements / symbols).
- $L$: Number of distinct voltage levels available in the signaling alphabet:
  - $B$ = Binary ($L = 2$ levels)
  - $T$ = Ternary ($L = 3$ levels: $+V, 0, -V$)
  - $Q$ = Quaternary ($L = 4$ levels)

The theoretical information capacity requires that the number of possible signal combinations ($L^n$) must be greater than or equal to the number of data combinations ($2^m$):

$$L^n \geq 2^m \tag{3.3}$$

When $L^n > 2^m$, the codebook contains **redundant signal states**:

$$N_{\text{redundant}} = L^n - 2^m \tag{3.4}$$

These redundant states are strategically exploited for three crucial functions:

1. Guaranteeing frequent transitions for receiver clock synchronization.
2. Real-time error detection (any received signal pattern outside the valid $2^m$ subset triggers an error alarm).
3. **Dynamic DC baseline balancing** (canceling out accumulated electrical charge on the transmission line).

## 3.3 Multilevel 2B1Q (Two Binary, One Quaternary)

In 2B1Q, pairs of binary bits (called *dibits*, $m = 2$) are mapped into a single signal element ($n = 1$) selected from four quaternary voltage levels ($L = 4$):

$$2^m = 2^2 = 4 \quad \text{and} \quad L^n = 4^1 = 4 \tag{3.5}$$

Because $4^1 = 2^2$, there are no redundant signal states in 2B1Q. The scheme achieves a 50% reduction in baud rate ($S = N/2$).

The standard voltage levels are normalized to: $\{-3, -1, +1, +3\}$.

Table 3.1: 2B1Q Transition State Table (Dr. Sadi Sir's Formulation)

| **Dibit** | **Next Level (Prev $> 0$)** | **Next Level (Prev $< 0$)** |
|---|---|---|
| 00 | $+1$ | $-1$ |
| 01 | $+3$ | $-3$ |
| 10 | $-1$ | $+1$ |
| 11 | $-3$ | $+3$ |

**Worked Example: 2B1Q Encoding Trace**

Consider the input bitstream given in lecture: `00 11 01 10 01`.
Assuming the initial previous signal level was positive ($> 0$):

1. **Dibit 1 (00):** Previous is positive $\implies$ Output level = $+\mathbf{1}$.
2. **Dibit 2 (11):** Previous is $+1$ (positive) $\implies$ Output level = $-\mathbf{3}$.
3. **Dibit 3 (01):** Previous is $-3$ (negative) $\implies$ Output level = $-\mathbf{3}$.
4. **Dibit 4 (10):** Previous is $-3$ (negative) $\implies$ Output level = $+\mathbf{1}$.
5. **Dibit 5 (01):** Previous is $+1$ (positive) $\implies$ Output level = $+\mathbf{3}$.

The resulting transmitted sequence is: $+\mathbf{1}, -\mathbf{3}, -\mathbf{3}, +\mathbf{1}, +\mathbf{3}$.

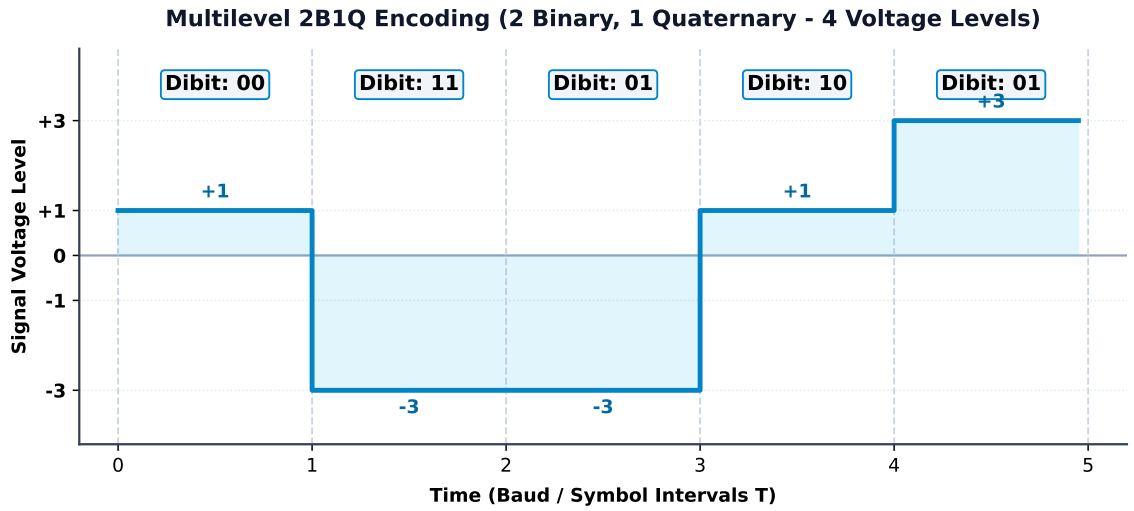



Figure 3.3: Multilevel 2B1Q Waveform for data stream `00 11 01 10 01`.

## 3.4 Multilevel 8B6T and Dynamic DC Weight Balancing

The 8B6T scheme maps an 8-bit byte ($m = 8$) into a pattern of 6 ternary signal elements ($n = 6, L = 3$), where each element takes a value from $\{-, 0, +\}$:

- Data combinations: $2^8 = 256$
- Signal combinations: $3^6 = 729$
- Redundant combinations: $729 - 256 = \mathbf{473}$

Used in **100BASE-T4 Fast Ethernet**, 8B6T reduces the baud rate to $S = N \times (6/8) = 0.75N$, allowing 100 Mbps transmission over Category 3 unshielded twisted-pair (UTP) cable.

### 3.4.1 DC Weight Mechanics

The electrical charge injected into the channel by any 6-element ternary codeword is characterized by its **Weight**:

$$\text{Weight} = (\text{Count of } `+` \text{ pulses}) - (\text{Count of } `-` \text{ pulses}) \tag{3.6}$$

- **Balanced Patterns (Weight = 0):** Equal number of positive and negative pulses (e.g., two +, two −, two 0). These inject zero net charge.
- **Unbalanced Patterns (Weight = +1):** The standard codebook includes patterns with a net weight of +1.

Table 3.2: Representative 8B6T Mapping Entries (from Dr. Sadi Sir's Lecture Notes)

| Hex | Binary Data | Codeword | Weight | Balancing Action |
|---|---|---|---|---|
| 00 | 0000 0000 | − + **0 0** − + | 0 | Transmit as-is ($W_{\text{cum}}$ unchanged) |
| 01 | 0000 0001 | + − + − **0 0** | 0 | Transmit as-is ($W_{\text{cum}}$ unchanged) |
| 11 | 0001 0001 | − **0** − **0** + + | 0 | Transmit as-is ($W_{\text{cum}}$ unchanged) |
| 53 | 0101 0011 | − + − + + **0** | +1 | Send as-is if $W_{\text{cum}} = 0$; Invert if $W_{\text{cum}} = +1$ |
| 50 | 0101 0000 | + − − + **0** + | +1 | Send as-is if $W_{\text{cum}} = 0$; Invert if $W_{\text{cum}} = +1$ |

### Encoding Rule: Dynamic DC Weight Balancing Algorithm

The transmitter maintains an internal state variable: **Cumulative DC Weight** ($W_{\text{cum}}$), initialized to 0.

1. If the incoming byte maps to a codeword with Weight $= 0$:
   - Transmit codeword as-is.
   - $W_{\text{cum}} \leftarrow W_{\text{cum}} + 0 = W_{\text{cum}}$ (unchanged).
2. If the incoming byte maps to a codeword with Weight $= +1$ and current $W_{\text{cum}} = 0$:
   - Transmit codeword as-is.
   - $W_{\text{cum}} \leftarrow 0 + 1 = +\mathbf{1}$.
3. If the incoming byte maps to a codeword with Weight $= +1$ and current $W_{\text{cum}} = +1$:
   - **Invert all polarities of the codeword** ($+ \to -$, $- \to +$, $0 \to 0$).
   - The inverted codeword has Weight $= -1$.
   - Transmit the inverted codeword.
   - $W_{\text{cum}} \leftarrow +1 + (-1) = \mathbf{0}$!

This closed-loop feedback mechanism guarantees that the cumulative DC weight never exceeds $+1$, eliminating baseline wander and DC saturation.

### Worked Example: Dr. Sadi Sir's 3-Byte 8B6T Transmission Example

Let the input digital stream consist of three bytes:

`Byte 1: 00010001 (Hex 11) Byte 2: 01010011 (Hex 53) Byte 3: 01010000 (Hex 50)`

**Step 1: Initial State**
Cumulative DC Weight $W_{\text{cum}} = 0$.
**Step 2: Process Byte 1 (`00010001` / Hex 11)**
From the 8B6T codebook, the codeword is: $-\ \mathbf{0}\ -\ \mathbf{0}\ +\ +$.
Calculation: Two '+', two '−', two '0' $\implies$ Weight $= 2 - 2 = 0$.
Action: Transmit unchanged. New $W_{\text{cum}} = 0 + 0 = \mathbf{0}$.
**Step 3: Process Byte 2 (`01010011` / Hex 53)**
From the 8B6T codebook, the codeword is: $-\ +\ -\ +\ +\ \mathbf{0}$.
Calculation: Three '+', two '−', one '0' $\implies$ Weight $= 3 - 2 = +1$.
Action: Since $W_{\text{cum}} = 0$, transmit unchanged. New $W_{\text{cum}} = 0 + 1 = +\mathbf{1}$.
**Step 4: Process Byte 3 (`01010000` / Hex 50)**
From the 8B6T codebook, the default codeword is: $+\ -\ -\ +\ \mathbf{0}\ +$.
Calculation: Three '+', two '−', one '0' $\implies$ Weight $= 3 - 2 = +1$.
Decision Check: Transmitting this as-is would yield $W_{\text{cum}} = +1 + 1 = +2$ (DC drift!).
Action: **Invert all signs** $\implies -\ +\ +\ -\ \mathbf{0}\ -$.
Inverted Weight: Two '+', three '−', one '0' $\implies$ Weight $= 2 - 3 = -1$.
Transmit inverted codeword. New $W_{\text{cum}} = +1 + (-1) = \mathbf{0}$.
**Final Cumulative Balance:** Exactly 0! Zero net DC bias across all 24 bits.

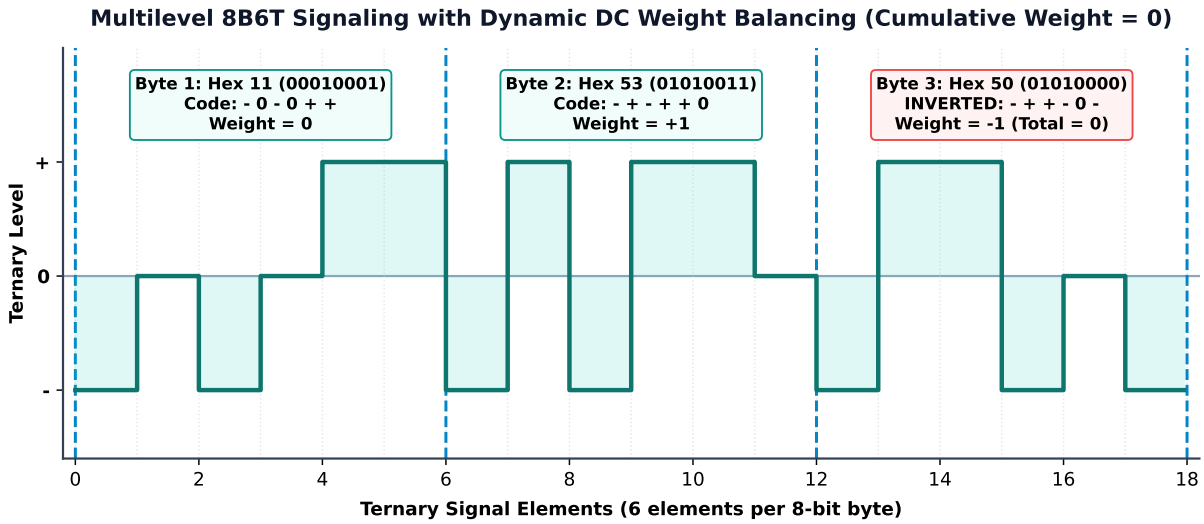



Figure 3.4: Multilevel 8B6T Dynamic DC Weight Balancing Waveform across three bytes.

## 3.5 Higher-Order Multilevel: 4D-PAM5

The **4D-PAM5** (Four-Dimensional 5-Level Pulse Amplitude Modulation) scheme is used in **1000BASE-T Gigabit Ethernet**.

- **4D (Four-Dimensional):** Data is transmitted simultaneously across all **four twisted pairs** of Category 5e UTP cable.
- **PAM5 (5 Levels):** Each wire pair carries a pulse amplitude signal taking one of 5 voltage levels: $\{-2, -1, 0, +1, +2\}$.
- **Throughput Calculation:** Each signal element carries $r = 2$ bits of user payload, plus redundancy for Trellis error-correction coding. At a symbol rate of 125 MBaud on each pair:

$$\text{Total Rate} = 125 \text{ MBaud} \times 2 \text{ bits/baud} \times 4 \text{ pairs} = \mathbf{1000} \text{ Mbps (1 Gbps)} \tag{3.7}$$

# Chapter 4

# Block Coding and Scrambling Techniques

## 4.1 Block Coding ($mB/nB$) Principles

Line coding schemes such as NRZ-I provide high bandwidth efficiency ($S = N/2$ under alternating data) and have no DC component when bits are balanced. However, they suffer from synchronization failure during sustained sequences of zeros. To resolve this without introducing the 100% bandwidth overhead of Manchester, communication systems employ a two-stage architecture: **Block Coding** followed by **Line Coding**.

Block coding is denoted as $mB/nB$, where an $m$-bit group is transformed into an $n$-bit codeword ($n > m$).

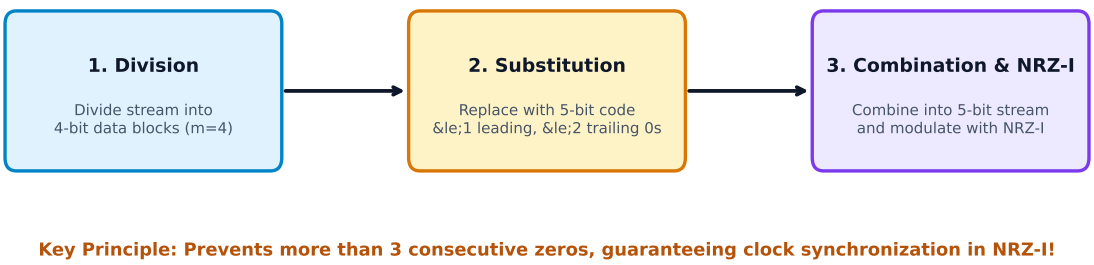



Figure 4.1: Three-step block coding transmission pipeline (4B/5B → NRZ-I).

**Encoding Rule: The Three Operational Steps of Block Coding**

1. **Division:** The input continuous bitstream is sliced into $m$-bit blocks (for 4B/5B, $m = 4$ bit nibbles).
2. **Substitution:** Each $m$-bit nibble is substituted with an $n$-bit codeword ($n = 5$) from a predefined mapping table.
3. **Combination:** The 5-bit codewords are re-serialized and passed to the line encoder (NRZ-I).

## 4.2 4B/5B Encoding and Run-Length Constraints

In 4B/5B, $2^4 = 16$ data inputs are mapped into a subset of the $2^5 = 32$ possible 5-bit output combinations. The 16 remaining unused codes are reserved for physical layer control (Quiet, Idle, Halt, Start/End delimiters) and hardware error detection.

Table 4.1: Complete 4B/5B Mapping Table (Data and Control Codes)

| Hex Symbol | Data Input (4B) | 5B Codeword | Run-Length Property |
|---|---|---|---|
| 0 | 0000 | 11110 | Zero 0s leading, one 0 trailing |
| 1 | 0001 | 01001 | One 0 leading, zero 0s trailing |
| 2 | 0010 | 10100 | Zero 0s leading, two 0s trailing |
| 3 | 0011 | 10101 | Zero 0s leading, zero 0s trailing |
| 4 | 0100 | 01010 | One 0 leading, one 0 trailing |
| 5 | 0101 | 01011 | One 0 leading, zero 0s trailing |
| 6 | 0110 | 01110 | One 0 leading, one 0 trailing |
| 7 | 0111 | 01111 | One 0 leading, zero 0s trailing |
| 8 | 1000 | 10010 | Zero 0s leading, one 0 trailing |
| 9 | 1001 | 10011 | Zero 0s leading, zero 0s trailing |
| A | 1010 | 10110 | Zero 0s leading, one 0 trailing |
| B | 1011 | 10111 | Zero 0s leading, zero 0s trailing |
| C | 1100 | 11010 | Zero 0s leading, one 0 trailing |
| D | 1101 | 11011 | Zero 0s leading, zero 0s trailing |
| E | 1110 | 11100 | Zero 0s leading, two 0s trailing |
| F | 1111 | 11101 | Zero 0s leading, zero 0s trailing |
| **Control Codes (Sample Selection)** | | | |
| I (Idle) | Line State | 11111 | Continuous transitions in NRZ-I |
| J (Start Delimiter 1) | Framing | 11000 | Beginning of packet delimiter |
| K (Start Delimiter 2) | Framing | 10001 | Second framing delimiter |
| T (End Delimiter) | Framing | 01101 | Terminating packet delimiter |

**Dr. Sadi Sir's Lecture Insight: The 4B/5B Mathematical Run-Length Proof**

In Dr. Sadi Sir's lecture notes, special emphasis is placed on the **Run-Length Constraint**:

1. **Leading Zeros Constraint:** No 5B codeword contains more than **one leading zero** ($\leq 1$).
2. **Trailing Zeros Constraint:** No 5B codeword contains more than **two trailing zeros** ($\leq 2$).

When any arbitrary sequence of 5B codewords is concatenated across boundary interfaces:

$$\text{Max Consecutive Zeros} = (\text{Max Trailing Zeros}) + (\text{Max Leading Zeros}) = 2 + 1 = \mathbf{3} \quad (4.1)$$

Within any individual codeword, there are at most two consecutive internal zeros. Therefore, across any arbitrary stream of user data, **no more than three consecutive zeros can ever occur**!
Since NRZ-I guarantees a voltage transition on every binary 1, a signal transition is mathematically guaranteed to occur **at least once every 4 bit intervals**. This absolute guarantee completely prevents loss of receiver synchronization while adding only a 25% bandwidth overhead ($5/4 = 1.25$).

## 4.3 Scrambling Principles for Bipolar AMI

While 4B/5B block coding solves synchronization, it expands the required bandwidth by 25%. In high-performance trunk lines (such as T1 and E1 carrier systems), bandwidth expansion is unacceptable.

**Scrambling** modifies the bipolar AMI bitstream directly at the physical layer to enforce synchronization **without increasing the baud rate or bandwidth**.

When a run of consecutive zeros occurs, the scrambler replaces them with a predefined sequence of pulses containing intentional **Bipolar Violations**:

- **Violation ($V$):** A pulse that has the **same polarity** as the preceding non-zero pulse. This deliberately violates the alternating polarity rule of AMI.
- **Bipolar ($B$):** A pulse that has the **opposite polarity** to the preceding pulse, strictly obeying the alternating rule of AMI.

The receiver recognizes the violation $V$ as an unmistakable physical layer marker, replaces the substitution pattern back with zeros, and prevents clock drift.

## 4.4 B8ZS (Bipolar with 8-Zero Substitution)

Standardized in North America for T1 carriers (1.544 Mbps), B8ZS replaces any run of **eight consecutive zeros** (`00000000`) with a special 8-element sequence:

$$\text{Substitution Pattern} = \mathbf{0\ 0\ 0\ V\ B\ 0\ V\ B} \tag{4.2}$$

**Encoding Rule: B8ZS Polarity Rules**

The polarity of the injected $V$ and $B$ pulses depends strictly on the polarity of the immediately preceding non-zero pulse:

- If the previous non-zero pulse was **Positive** $(+)$:

$$\text{Pattern} \implies 0 \quad 0 \quad 0 \quad + \quad - \quad 0 \quad - \quad + \tag{4.3}$$

- If the previous non-zero pulse was **Negative** $(-)$:

$$\text{Pattern} \implies 0 \quad 0 \quad 0 \quad - \quad + \quad 0 \quad + \quad - \tag{4.4}$$

**Worked Example: B8ZS Walkthrough from Lecture Notes**

Consider the bitstream: `1 0 0 0 [0 0 0 0 0 0 0 0] 0 1`.
Assume the initial non-zero pulse for the leading bit `1` was **Positive** $(+)$.

1. Leading bit `1`: Transmitted as $+V$.
2. Next 3 zeros: Transmitted as $0, 0, 0$.
3. The 8 consecutive zeros are replaced by **0 0 0 V B 0 V B**:
   - Element 1: 0
   - Element 2: 0
   - Element 3: 0
   - Element 4 ($V$): $+V$ (Violation: matches previous non-zero pulse $+V$).
   - Element 5 ($B$): $-V$ (Bipolar: opposes previous pulse $+V$).
   - Element 6: 0
   - Element 7 ($V$): $-V$ (Violation: matches previous pulse $-V$).
   - Element 8 ($B$): $+V$ (Bipolar: opposes previous pulse $-V$).
4. Following zero: Transmitted as 0.

5. Final bit 1: Must alternate with respect to the last bipolar pulse (which was $+V$) $\Longrightarrow$ transmitted as $-V$.

**DC Balance Check:** The substitution pattern contains two positive pulses and two negative pulses $(+V, -V, -V, +V)$. The net charge added is zero $(+1 - 1 - 1 + 1 = 0)$, completely preserving the DC neutrality of AMI.

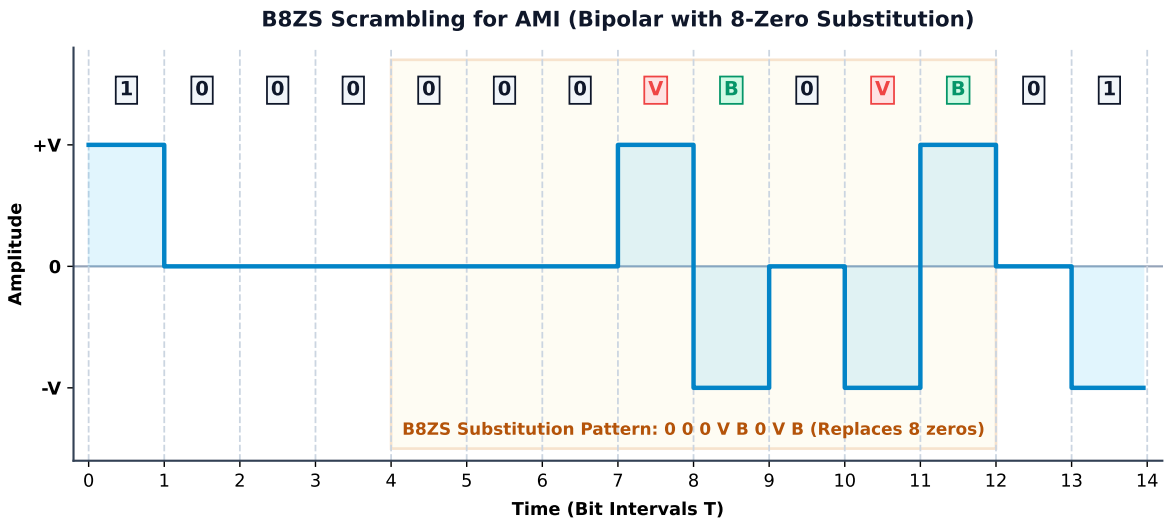



Figure 4.2: B8ZS Scrambling for AMI showing substitution of 8 consecutive zeros.

## 4.5 HDB3 (High-Density Bipolar 3-Zero)

Standardized in Europe and ITU-T for E1 carriers (2.048 Mbps), HDB3 limits the maximum number of consecutive zeros to **three**. Whenever **four consecutive zeros** (0000) occur, they are replaced.

**Encoding Rule: HDB3 Substitution State Machine**

The substitution pattern is selected based on the number of non-zero pulses that have occurred since the last substitution:

- If the count of non-zero pulses is **ODD**:

$$\text{Pattern} = \mathbf{0\ 0\ 0\ V} \tag{4.5}$$

- If the count of non-zero pulses is **EVEN**:

$$\text{Pattern} = \mathbf{B\ 0\ 0\ V} \tag{4.6}$$

**Dr. Sadi Sir's Lecture Insight: Why Does HDB3 Require Two Distinct Patterns?**

The fundamental design objective of HDB3 is to prevent consecutive violations ($V$) from having the same polarity, which would inject a net DC charge into the transmission line.

- When an **odd** number of pulses has occurred since the last substitution, the normal AMI alternating rule has naturally flipped the polarity. Thus, injecting a simple violation $V$

produces a pulse whose polarity opposes the previous violation, maintaining zero net DC.

- When an **even** number of pulses has occurred, the next normal pulse would have the same polarity as the last violation. If we used $000V$, two consecutive violations would have the same polarity! To prevent this DC accumulation, HDB3 introduces a bipolar pulse $B$ first, which flips the polarity, so that the subsequent violation $V$ strictly opposes the previous violation.

**Worked Example: HDB3 Walkthrough from Lecture Notes**

Consider the bitstream: `1 [0 0 0 0] 1 1 [0 0 0 0] 0`.
Assume initial state: last pulse was negative ($-V$) and previous substitution was balanced.

1. First bit `1`: Transmitted as $+V$.
2. First Block of 4 Zeros (`0000`):
   - Non-zero pulses since start = 1 (Odd).
   - Substitution pattern selected: **0 0 0 V**.
   - $V$ must have same polarity as last pulse ($+V$) $\implies$ $\mathbf{0}, \mathbf{0}, \mathbf{0}, +\mathbf{V}$.
3. Intermediate Bits:
   - First bit `1`: Alternates relative to previous pulse $\implies$ $-\mathbf{V}$.
   - Second bit `1`: Alternates $\implies$ $+\mathbf{V}$.
4. Second Block of 4 Zeros (`0000`):
   - Non-zero pulses since last substitution = 2 (Even: one $-V$ and one $+V$).
   - Substitution pattern selected: **B 0 0 V**.
   - $B$ must oppose last pulse ($+V$) $\implies$ $B = -\mathbf{V}$.
   - $V$ must match immediately preceding pulse ($B = -V$) $\implies$ $V = -\mathbf{V}$.
   - Resulting block: $-\mathbf{V}, \mathbf{0}, \mathbf{0}, -\mathbf{V}$.
5. Final bit `0`: Transmitted as **0** V.

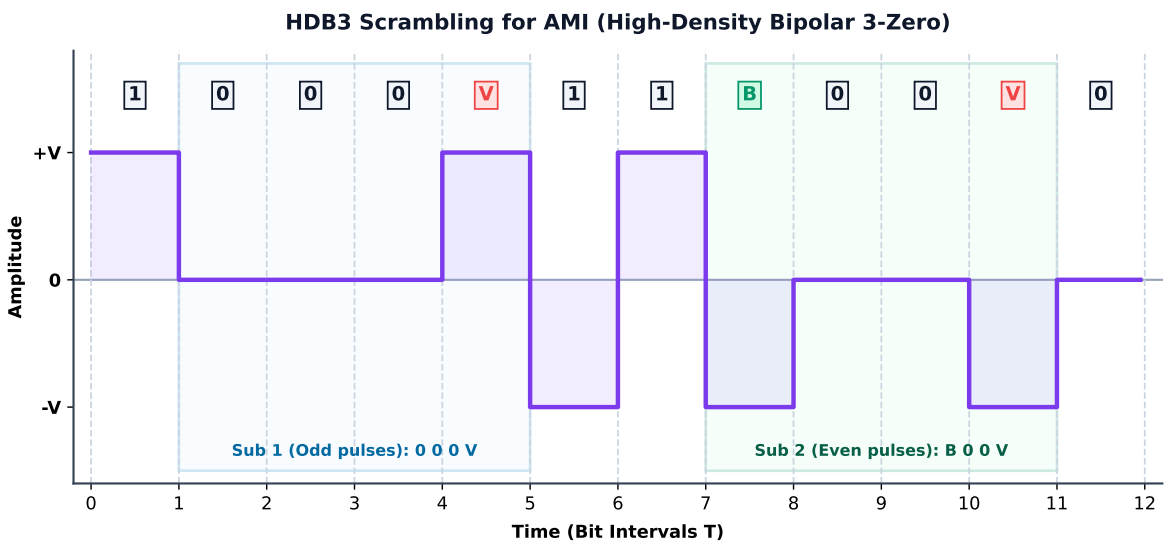



Figure 4.3: HDB3 Scrambling for AMI illustrating both Odd ($000V$) and Even ($B00V$) substitutions.

# Chapter 5

# Comparative Analysis & Engineering Summary

## 5.1 Master Comparison Matrix

Table 5.1 provides an exhaustive side-by-side comparison of all 11 line coding and scrambling techniques investigated across Dr. Sadi Sir's lectures.

Table 5.1: Comprehensive Physical Layer Line Coding Performance Matrix

| Scheme | Levels ($L$) | Baud Rate ($S$) | Bandwidth ($B$) | DC Bias | Self-Sync | Primary Application / Remarks |
|---|---|---|---|---|---|---|
| **Unipolar NRZ** | 2 $(0, +V)$ | $N$ | $N/2$ | High ($V/2$) | Poor | Legacy buses; obsolete for long-distance lines. |
| **Polar NRZ-L** | 2 $(\pm V)$ | $N$ | $N/2$ | Low (if bal.) | Poor | RS-232 serial interfaces; baseline wander issue. |
| **Polar NRZ-I** | 2 $(\pm V)$ | $N$ | $N/2$ | Low (if bal.) | Good on 1s | USB 1.1/2.0; differential; fails on long 0s. |
| **Polar RZ** | 3 $(\pm V, 0)$ | $2N$ | $N$ | None | Full | Theoretical benchmark; high $2N$ bandwidth penalty. |
| **Manchester** | 2 $(\pm V)$ | $2N$ | $N$ | Zero | Full | Ethernet 10BASE-T (IEEE 802.3); 2× bandwidth. |
| **Diff. Manchester** | 2 $(\pm V)$ | $2N$ | $N$ | Zero | Full | Token Ring (IEEE 802.5); immune to wire reversal. |
| **Bipolar AMI** | 3 $(\pm V, 0)$ | $N$ | $N/2$ | Zero | Poor on 0s | DS1/T1/E1 trunks; narrow bandwidth; built-in error check. |
| **Pseudoternary** | 3 $(\pm V, 0)$ | $N$ | $N/2$ | Zero | Poor on 1s | ISDN S-interface; functional complement of AMI. |
| **2B1Q** | 4 $(\pm 1, \pm 3)$ | $N/2$ | $N/4$ | Low | Fair | ISDN BRI, DSL local loops; cuts baud rate by 50%. |
| **8B6T** | 3 $(+, 0, -)$ | $0.75N$ | $0.375N$ | Zero | Full | 100BASE-T4 Fast Ethernet; dynamic DC balancing. |
| **4B/5B + NRZ-I** | 2 $(\pm V)$ | $1.25N$ | $0.625N$ | Low | Full | 100BASE-FX/TX FDDI; max 3 consecutive 0s; 25% overhead. |
| **B8ZS (AMI)** | 3 $(\pm V, 0)$ | $N$ | $N/2$ | Zero | Full | North American T1; replaces 8 zeros with $000VB0VB$. |
| **HDB3 (AMI)** | 3 $(\pm V, 0)$ | $N$ | $N/2$ | Zero | Full | European/ITU E1; replaces 4 zeros with $000V$ or $B00V$. |

## 5.2 Summary of Dr. Sadi Sir's Core Principles

1. **The Bandwidth vs. Synchronization Dilemma:** Simple NRZ provides high bandwidth efficiency ($S = N/2$) but fails to synchronize during runs of identical bits. Biphase (Manchester) guarantees synchronization at every bit interval but doubles the required bandwidth. Modern networking resolves this dilemma through **Block Coding** (e.g., 4B/5B with only 25%

overhead) or **Scrambling** (B8ZS/HDB3 with 0% overhead).

2. **The Importance of DC Balance:** Real-world physical channels rely on AC-coupled transformers and capacitive isolators. Unbalanced DC charge accumulates as voltage offset, causing transformer magnetic core saturation and baseline wander. Bipolar AMI inherently achieves zero DC through alternating pulses, while multilevel codes (like 8B6T) achieve it actively through runtime codeword polarity inversion.

3. **Spectral Conservation via Multilevel Signaling:** To transmit higher data rates over legacy copper wiring without exceeding transmission channel cut-off frequencies, $mBnL$ schemes encode multiple bits into fewer symbols. 2B1Q doubles the bit-per-baud capacity ($r = 2$), while 8B6T achieves $r = 1.33$ while dedicating 473 redundant states to DC balancing and error detection.

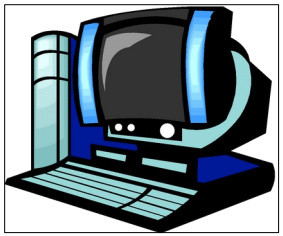
# Chapter 3
# Transport Layer
## Introduction

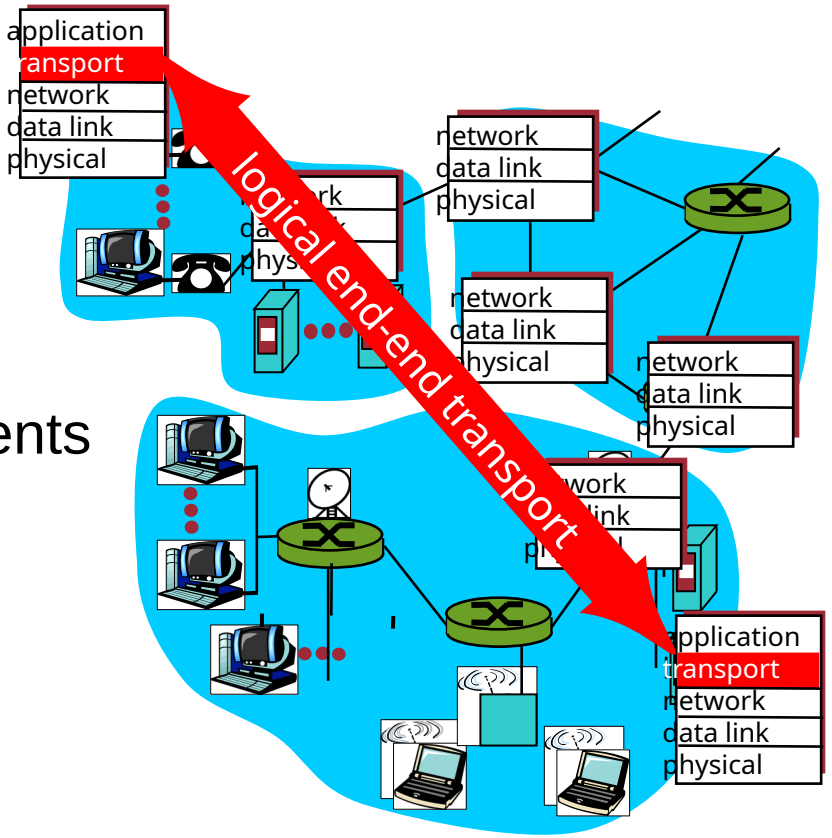
# What we'll study

- Transport layer role and services
- Develop/design some simple transport protocol

# Role of Transport Layer

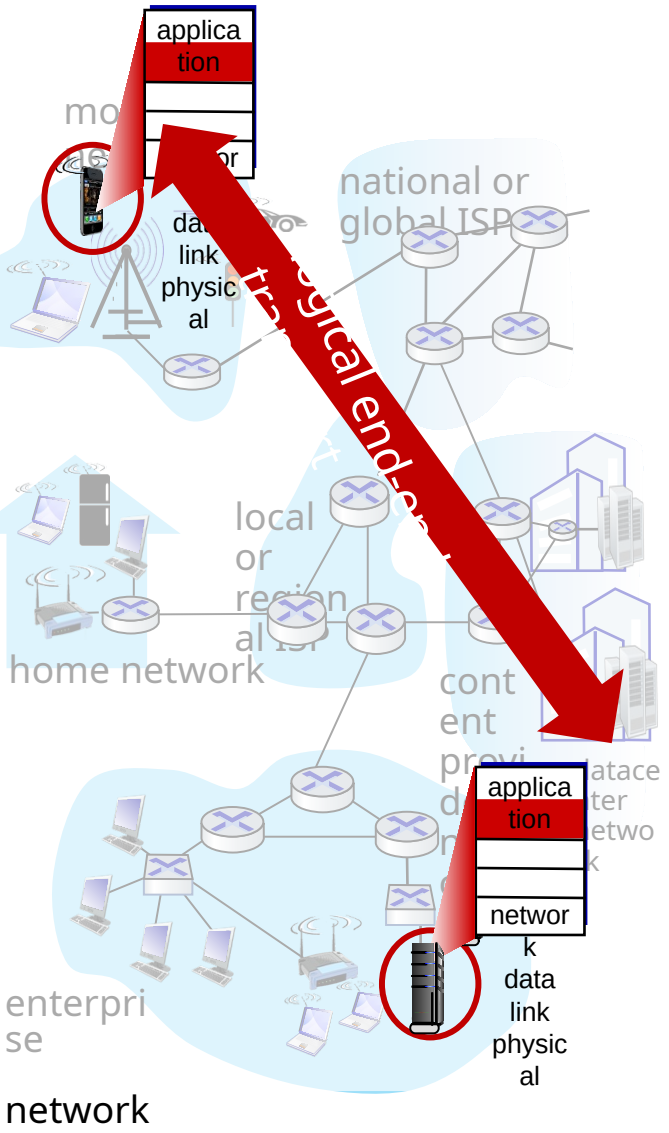
- Provide *logical communication* between application processes running on different hosts
- Run on end hosts
  - Sender: breaks application messages into segments, and passes to network layer
  - Receiver: reassembles segments into messages, passes to application layer



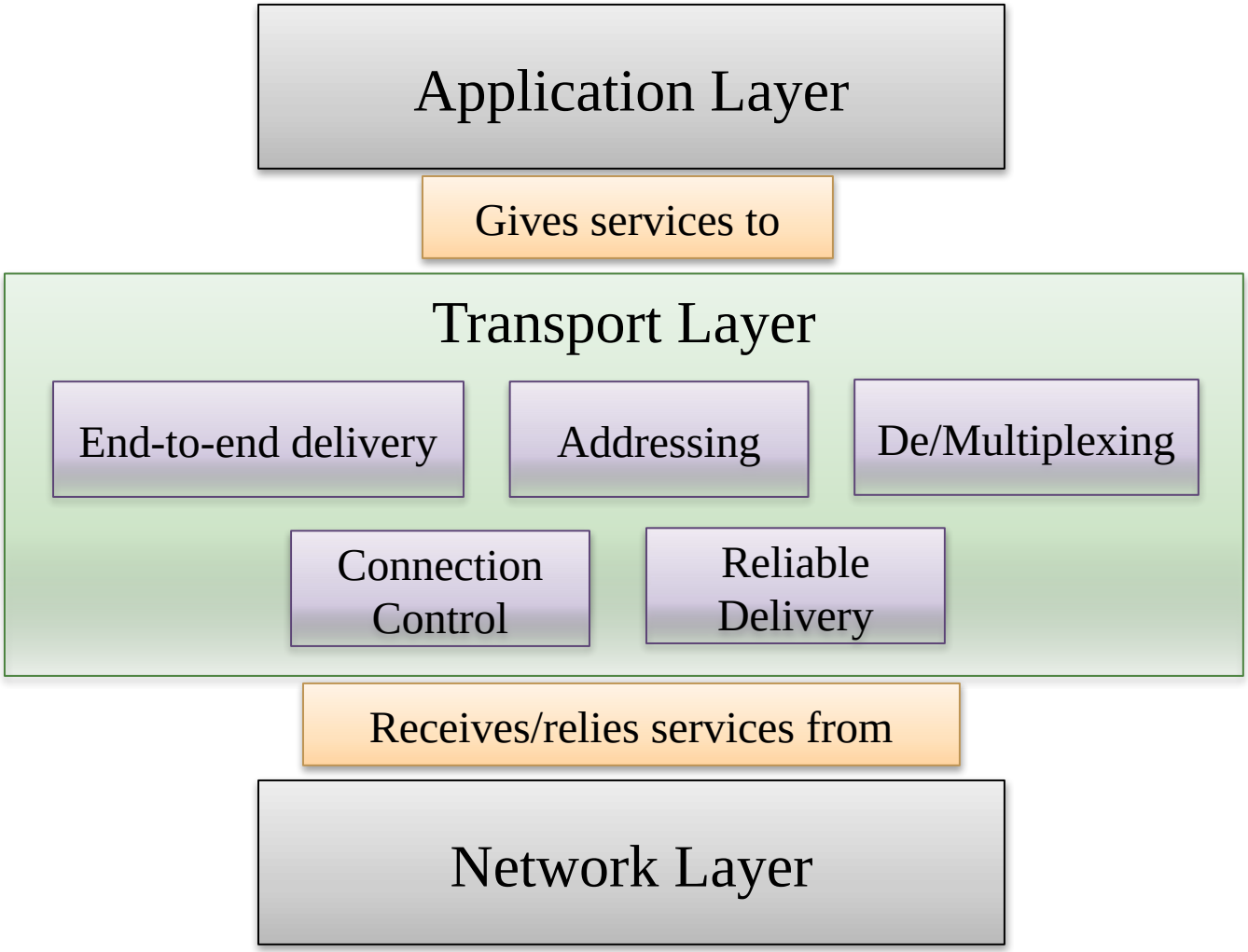


# Two principal Internet transport protocols

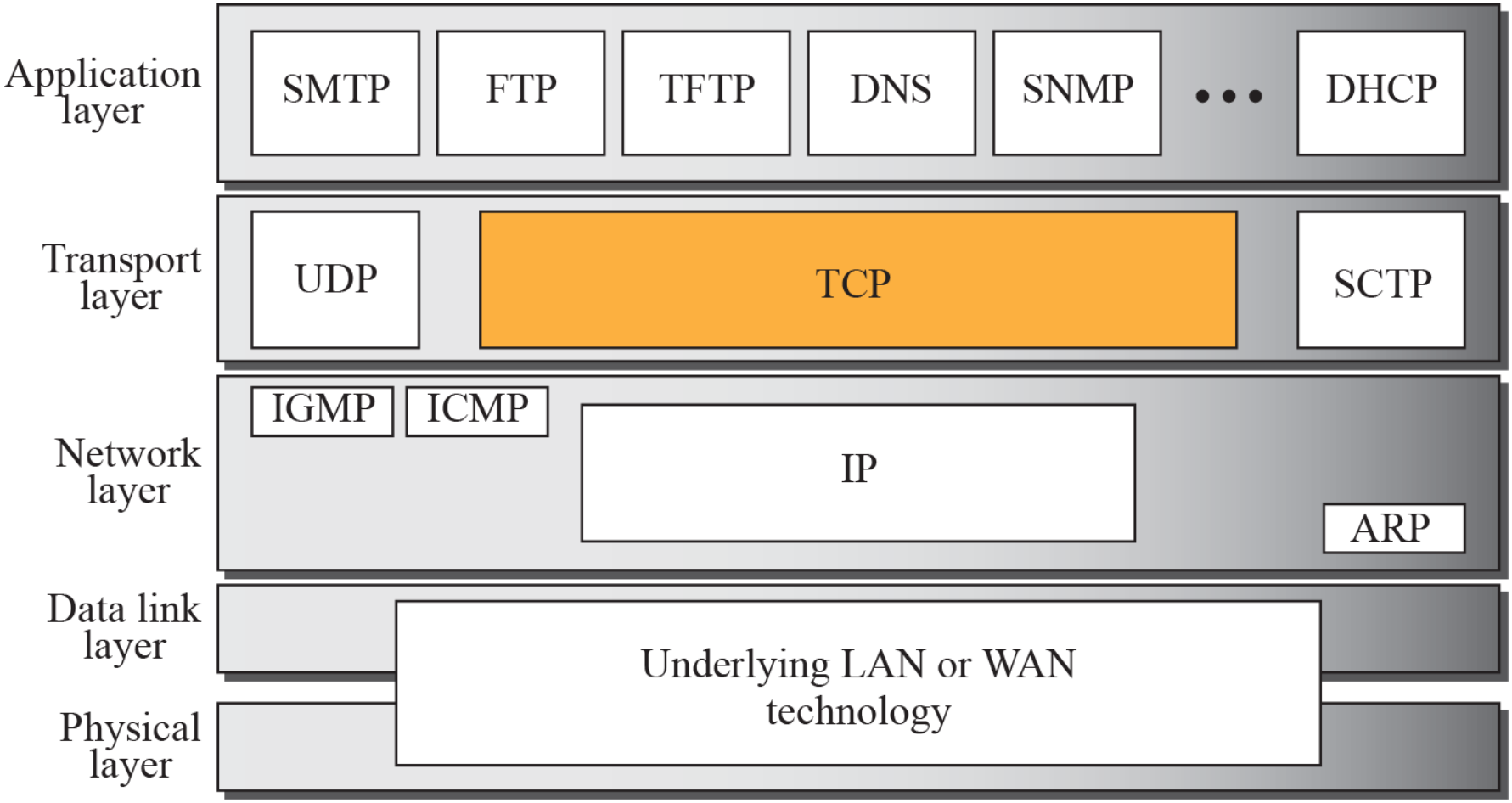
- **TCP:** Transmission Control Protocol
  - reliable, in-order delivery
  - congestion control
  - flow control
  - connection setup
- **UDP:** User Datagram Protocol
  - unreliable, unordered delivery
  - no-frills extension of “best-effort” IP
- services *not* available:
  - delay guarantees
  - bandwidth guarantees



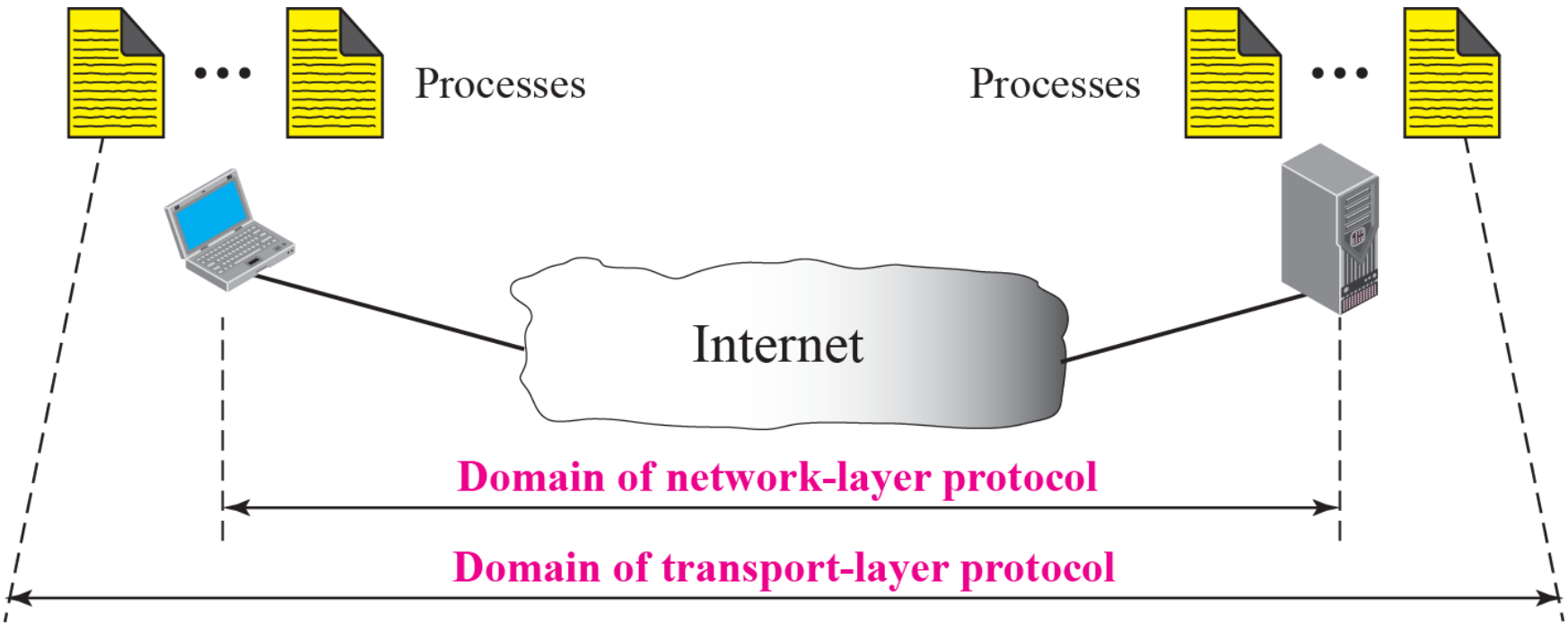
# Transport Layer Services



## TCP/IP Protocol Suite

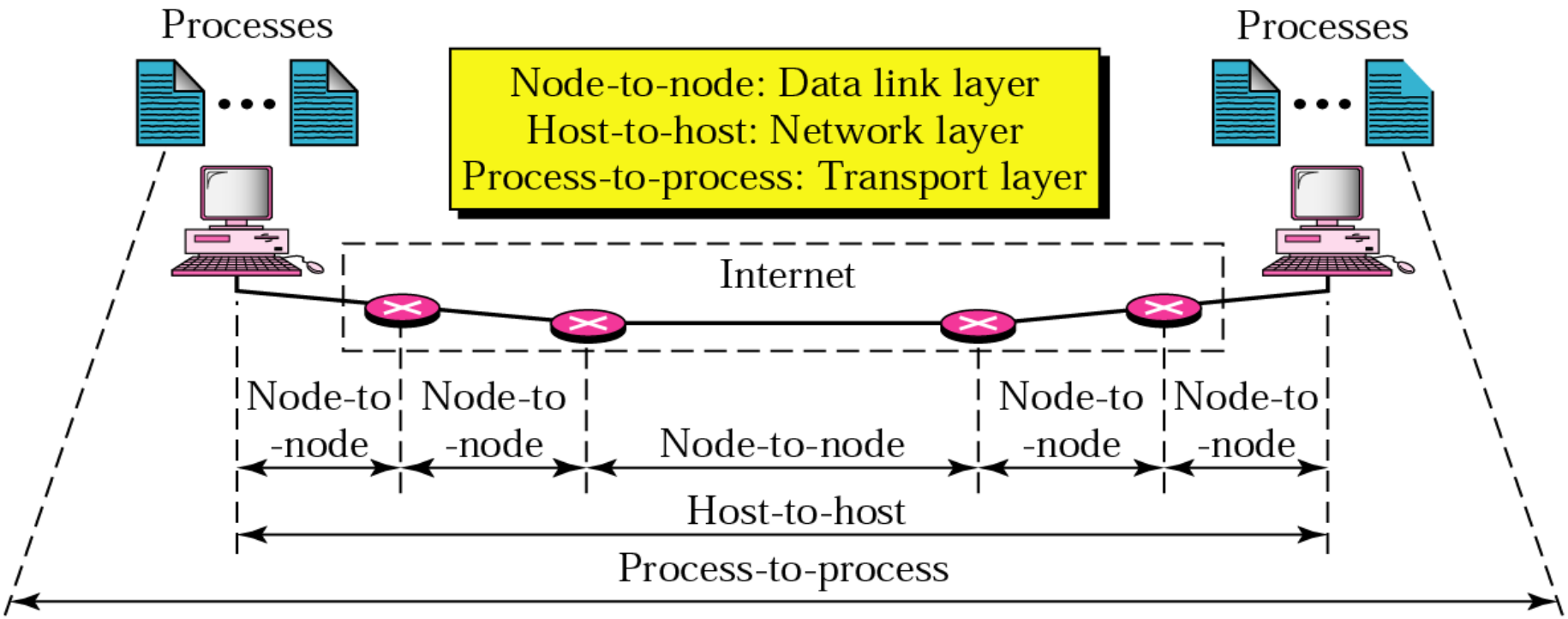


## Figure 13.1 *Network layer versus transport layer*



# Process-to-process delivery

- Several processes (application programs) can run on both the source and the destination host

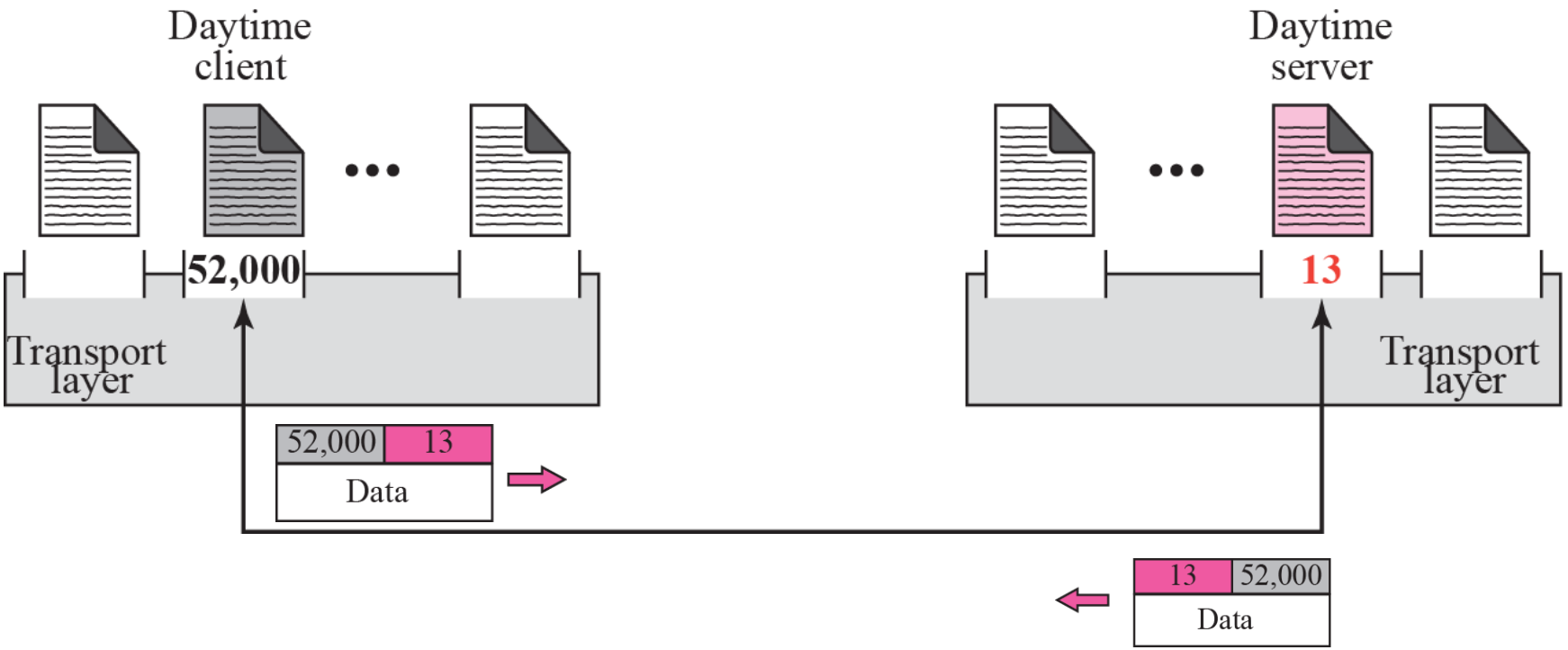


# Analogy

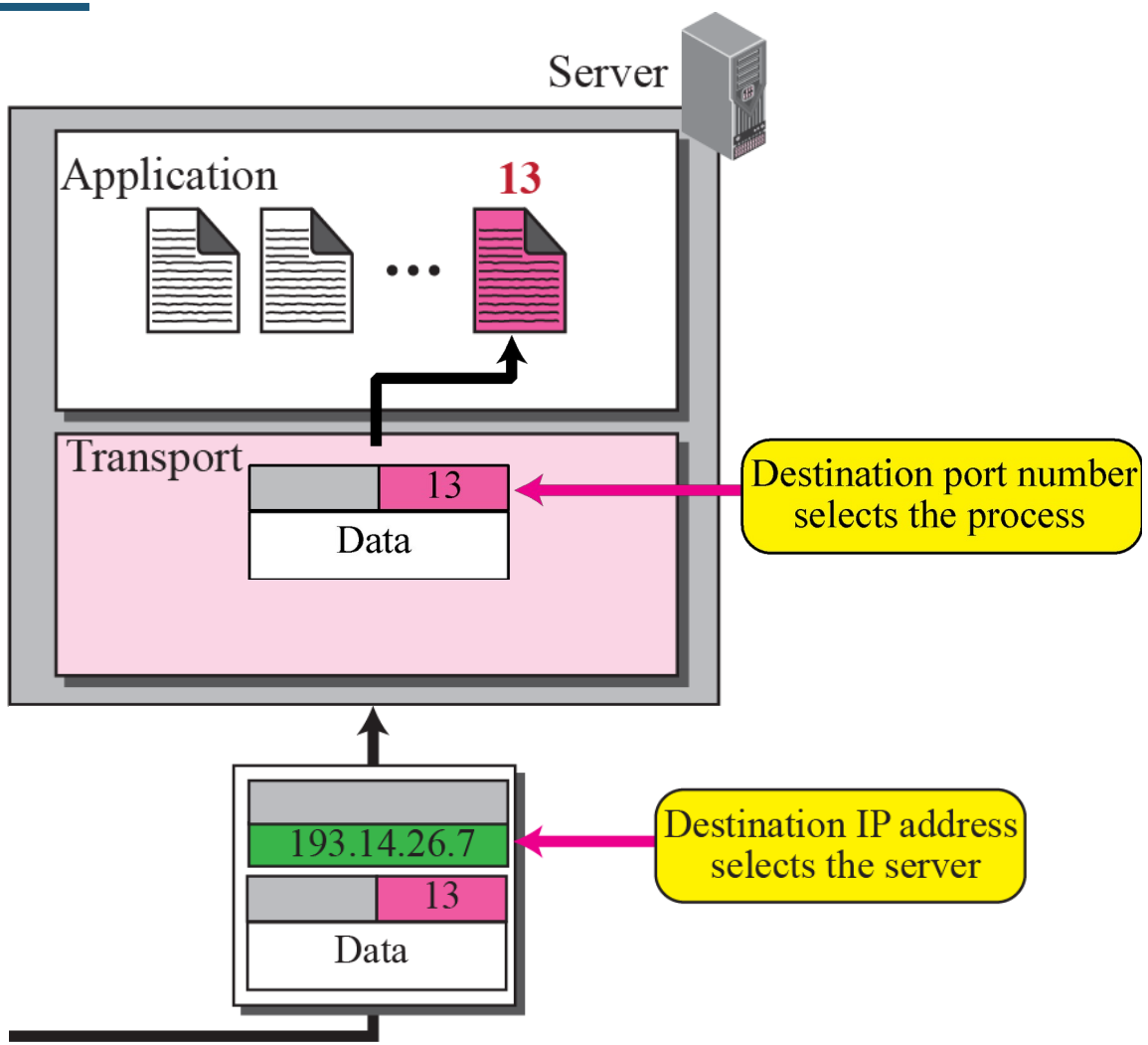
- Two houses, each with a dozen of kids, are located in Eindhoven and Amsterdam.
- All kids in different houses are relatives (cousins)
- Each kid weekly writes a letter to every other kid in another house
  - 144 letters/week in total (huge cost)
    - Dutch kids would probably use e-mail (low cost)
- In each household there is one kid (Jantje & Pietje, respectively) responsible for mail collection and mail distribution
  - each week they give all the letters to a postal-service mail carrier who makes daily visits to the house.

- Application message = letters in the envelope
- Processes = cousins
- Hosts (end systems) = houses
- Transport-layer protocol = Jantje & Pietje
- Network-layer protocol = postal service (including mail carriers)

## Figure 13.2 *Port numbers*

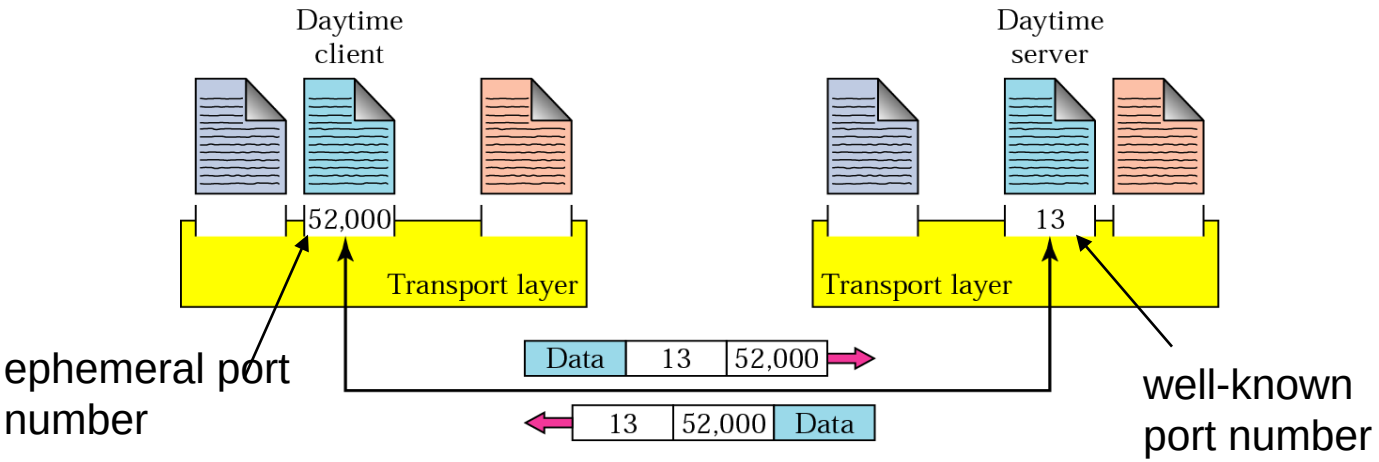


## Figure 13.3 *IP addresses versus port numbers*

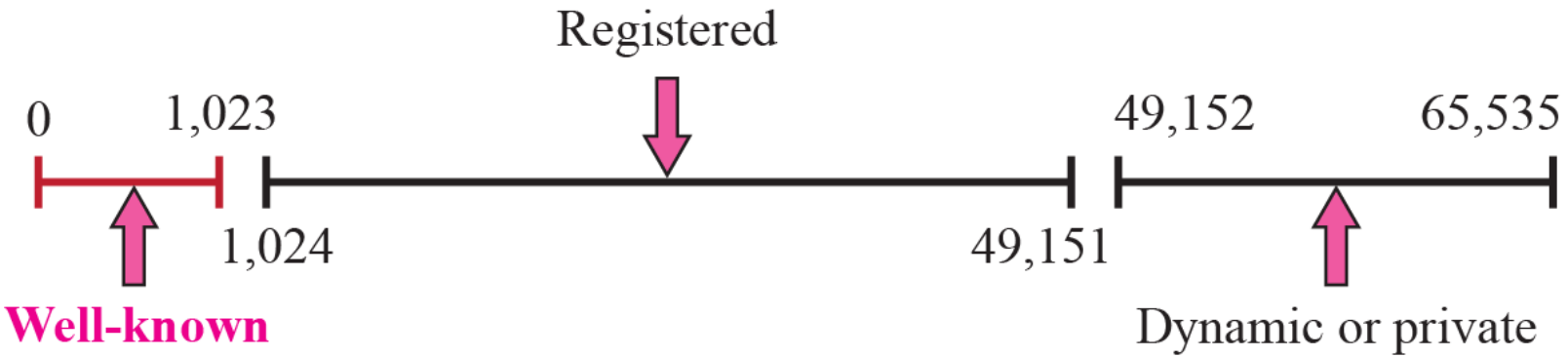


# Addressing

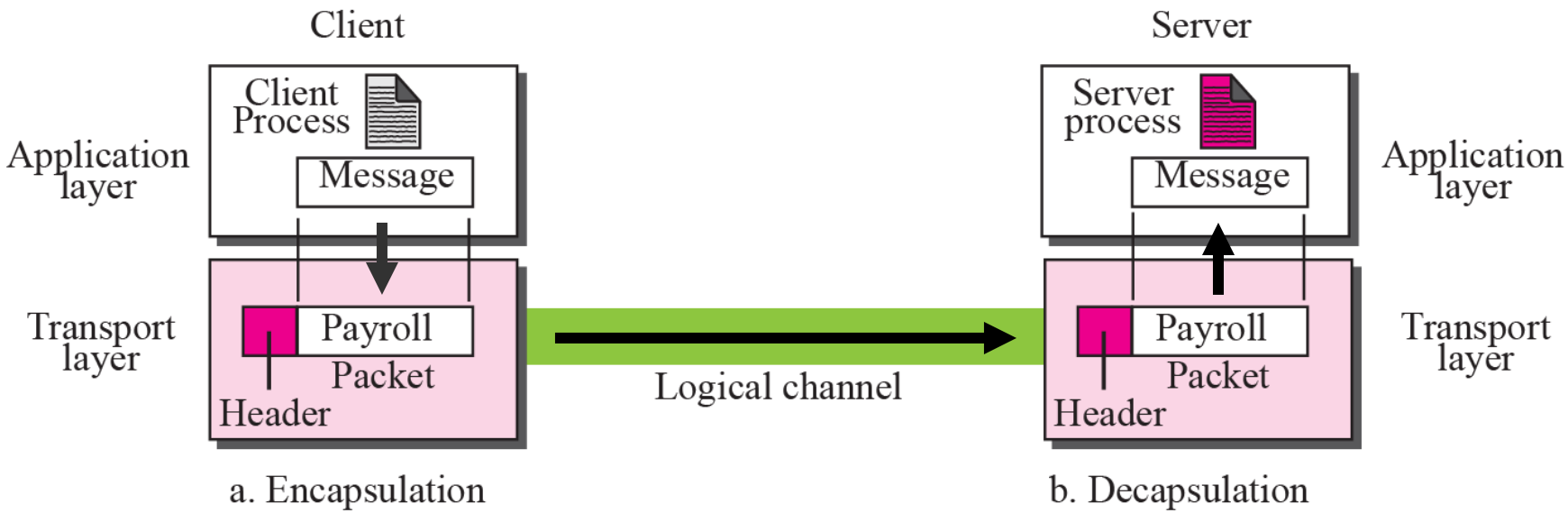
- Data link - MAC address
- Network – IP address
- Transport – port number
  - needed to choose among multiple processes running on the destination host
  - The Internet model: 16 bit integer 0 – 65 535
    - Client mostly chooses ephemeral port number
    - Server mostly uses well-known (permanent) port numbers



## Figure 13.4 *ICANN ranges*

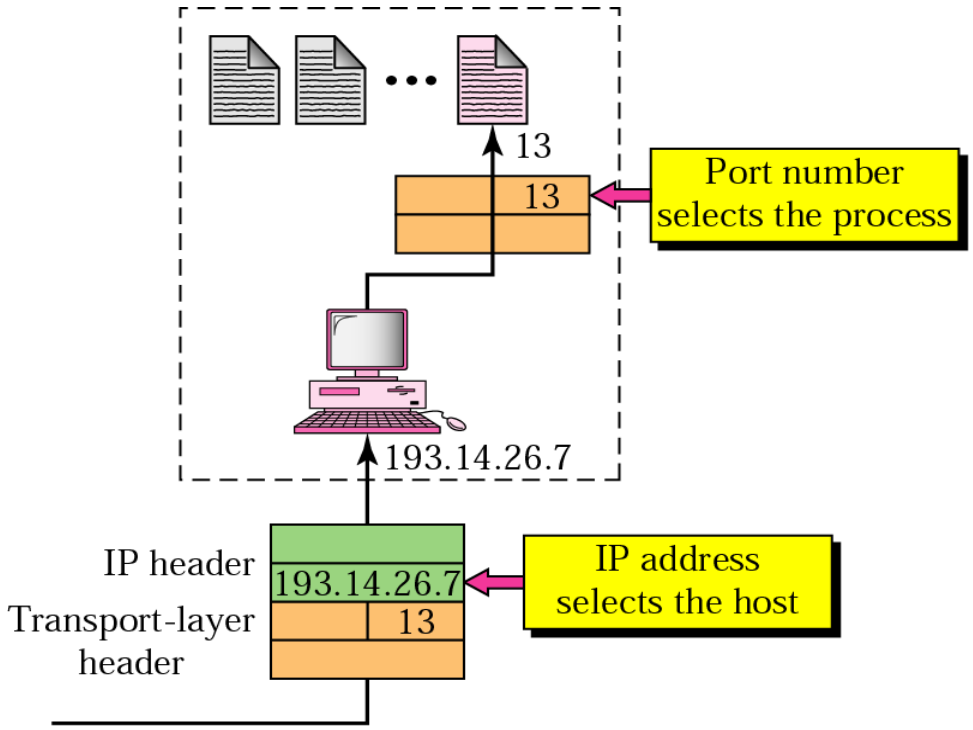


## Figure 13.6 *Encapsulation and decapsulation*



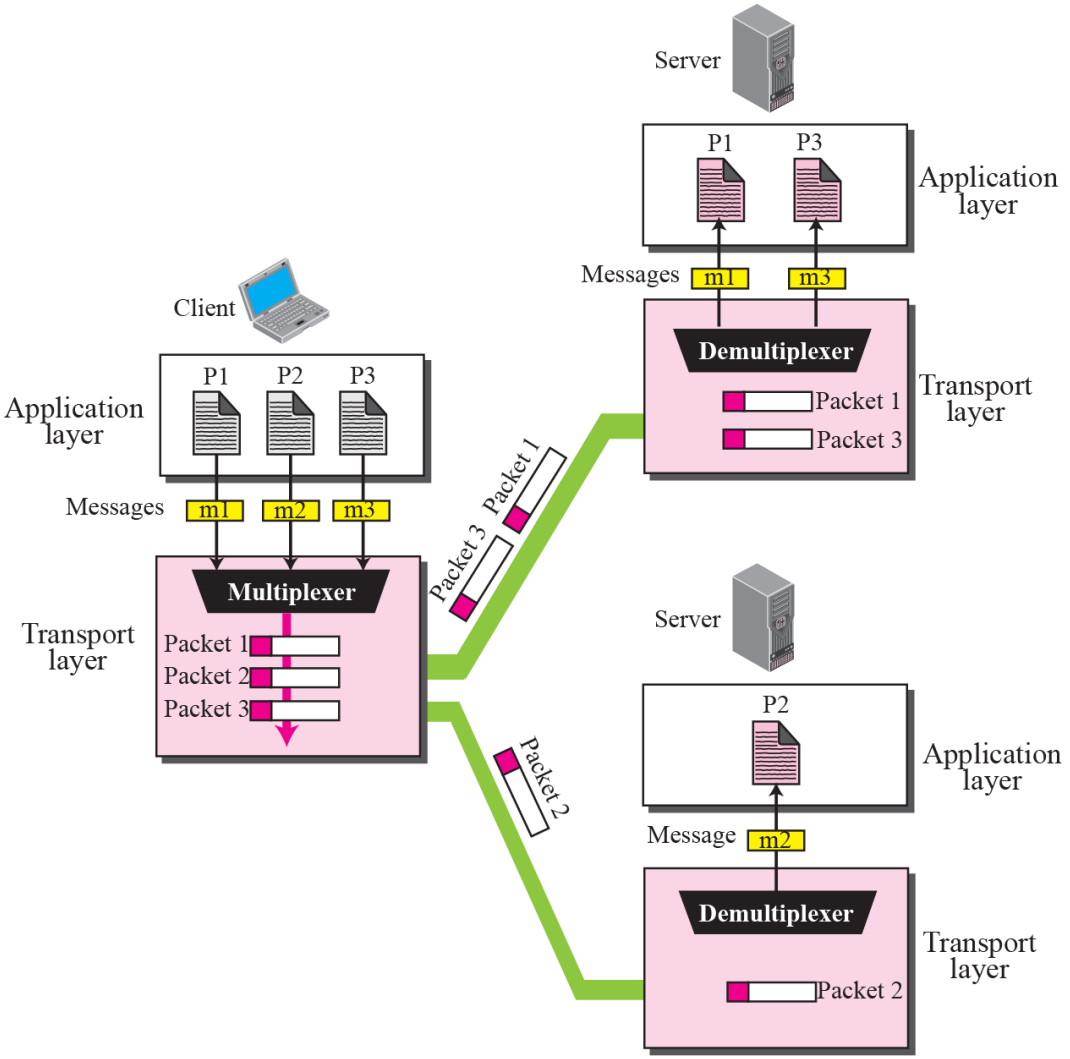
# IP addressing versus port addressing

- destination IP defines the host
- after the host has been selected:
  the port number defines one of the processes on this host



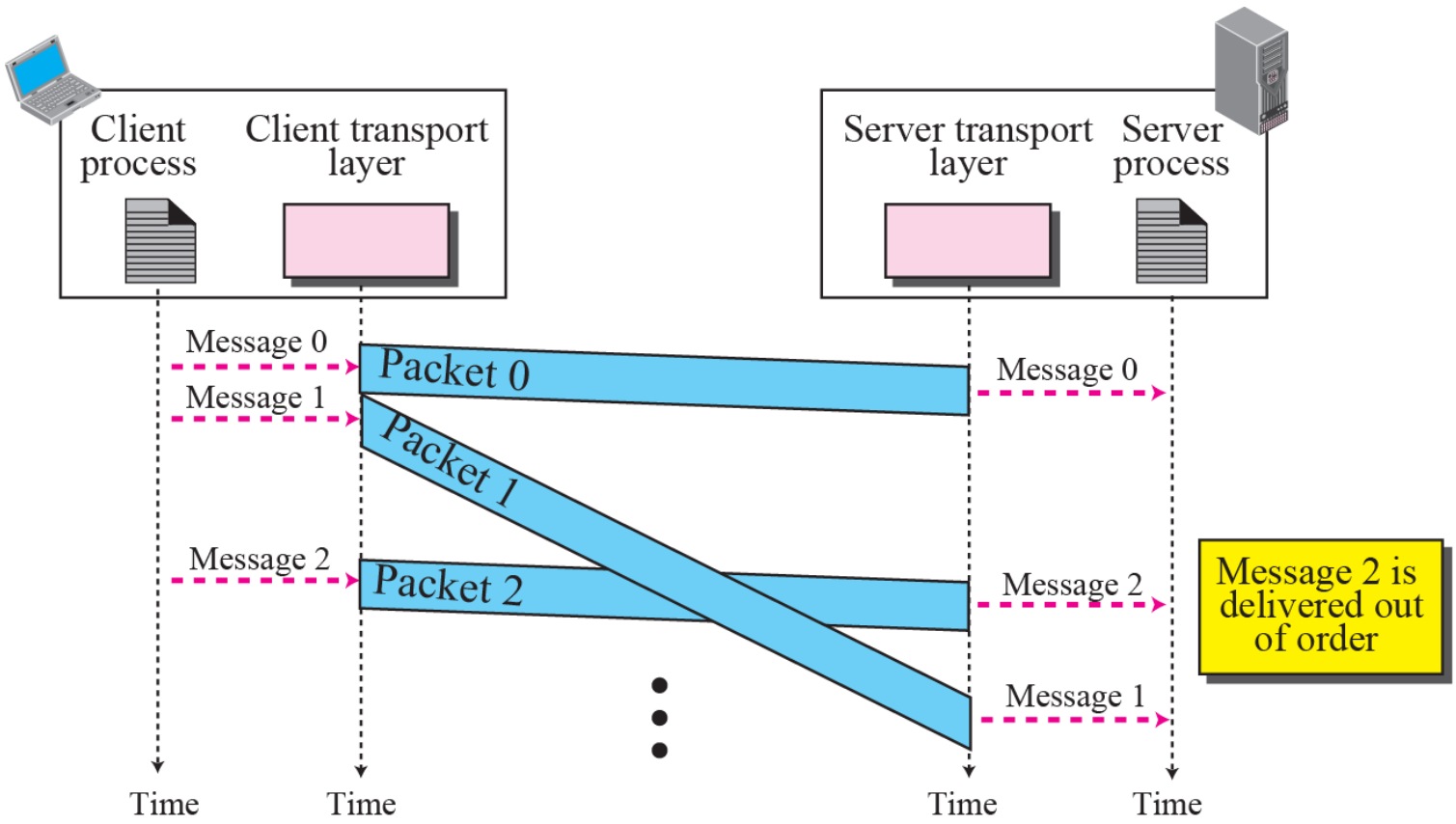
# Transport layer MUX & DeMUX

- Transport layer de/multiplexes several processes on the transport layer protocol

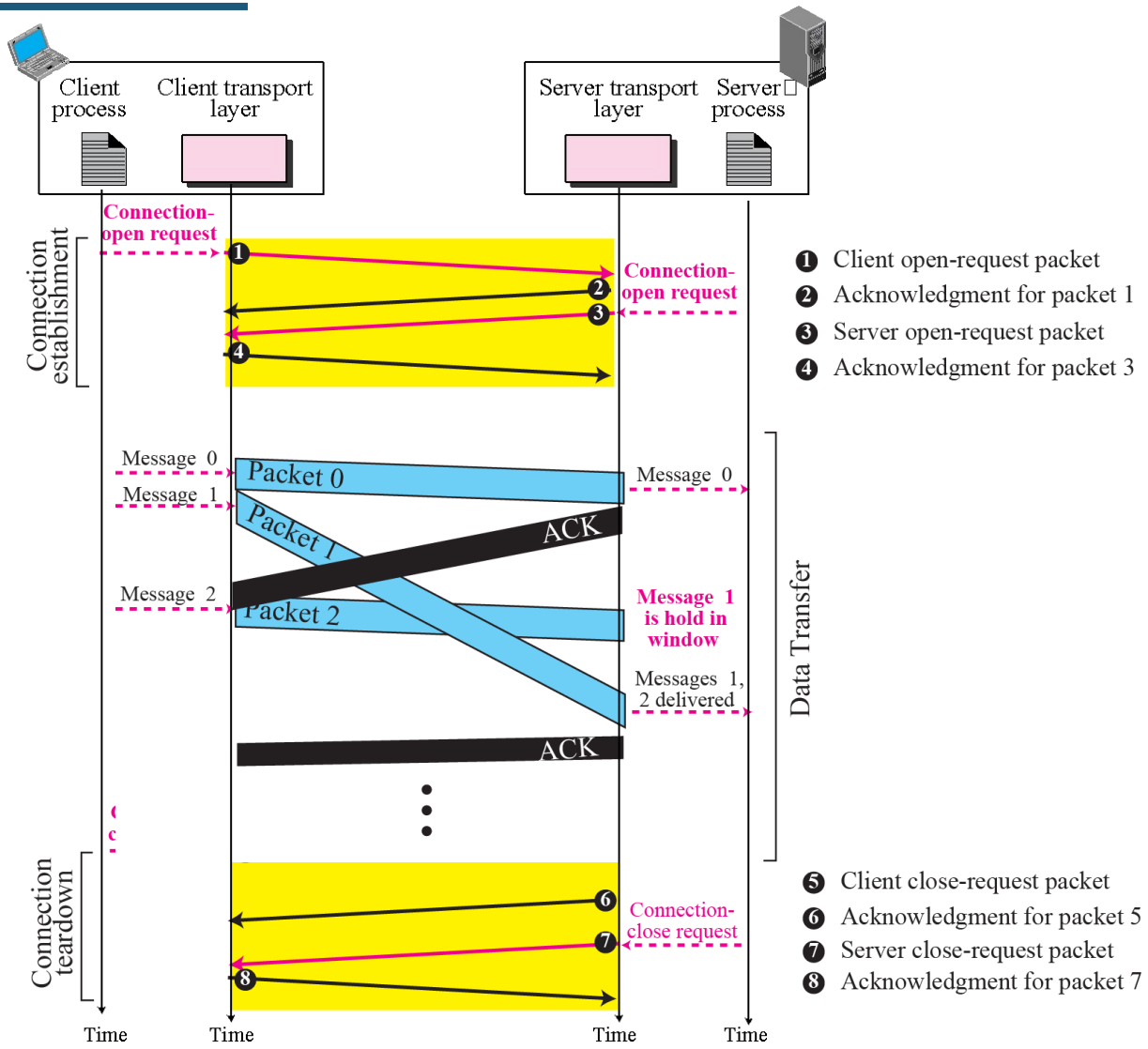


# Connection control

- Connectionless services
  - no connection establishment
  - packets are not numbered, may be delayed, lost or arrive out of order
  - UDP

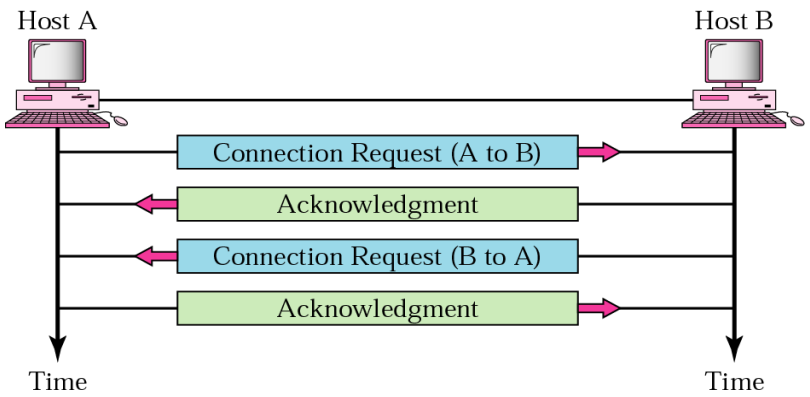


# Figure 13.14 *Connection-oriented service*

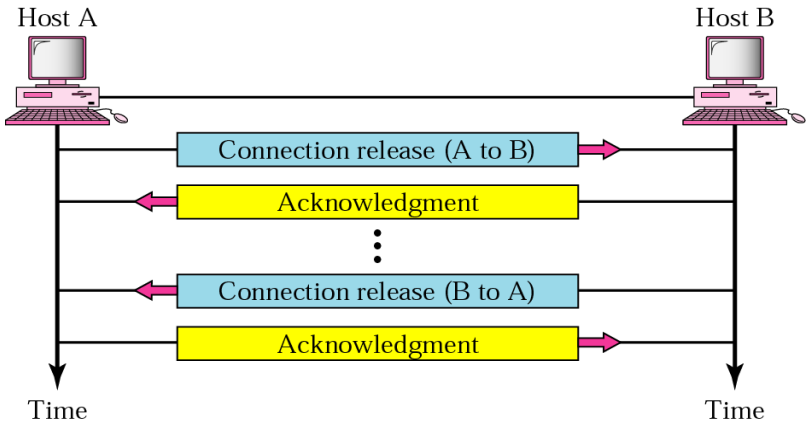


# Connection control

- Connection-oriented services
  - connection established prior to data transmission
  - TCP
- What is the difference with respect to the Network layer services?
  - transport code runs only on the user's machine and not in the routers (usually operated by the carriers)
  - Network layer can offer inadequate service (frequent packet loss, routers crash from time to time); user does not have control over the Network layer




Establishment:
three way handshake




Connection termination

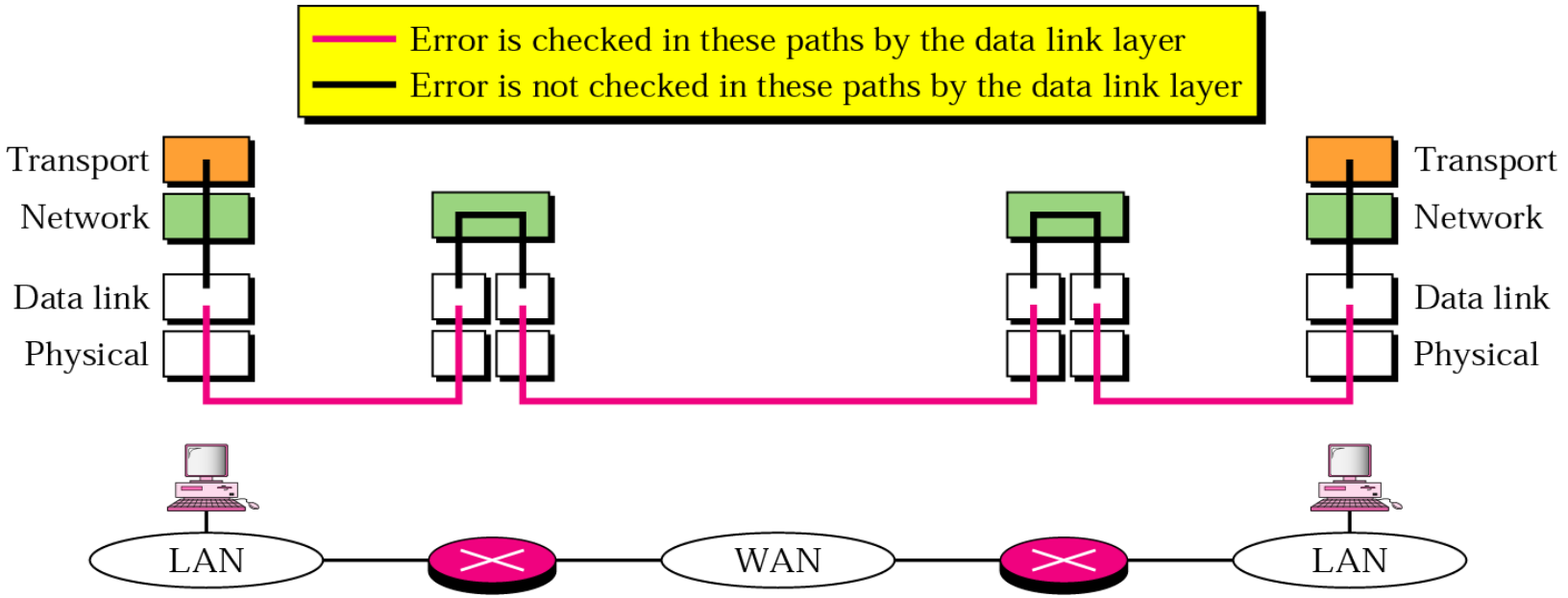
# Reliable delivery

- Reliable services
  - error control
    - Detect and discard corrupted packets.
    - Keep track of lost and discarded packets and resend them.
    - Recognize duplicate packets and discard them.
  - flow control
    - Sender doesn’t overwhelm the receiver
  - congestion control
    - Manage network load within its capacity
  - TCP
- Unreliable service
  - UDP

# Reliable delivery

- Question: Do we need reliable transport service if the Data link transport is reliable?

- Yes



# Thank You!

# Chapter 3
# Transport Layer
## Reliable Data Transfer (RDT)

# Transport layer: overview

*Our goal:*

- understand principles behind transport layer services:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control

- learn about Internet transport layer protocols:
  - UDP: connectionless transport
  - TCP: connection-oriented reliable transport
  - TCP congestion control

# Reliable delivery

- Reliable services
  - error control
    - Detect and discard corrupted packets.
    - Keep track of lost and discarded packets and resend them.
    - Recognize duplicate packets and discard them.
  - flow control
    - Sender doesn’t overwhelm the receiver
  - congestion control
    - Manage network load within its capacity
  - TCP
- Unreliable service
  - UDP

# Chapter 3: roadmap

- Transport-layer services
- Principles of reliable data transfer

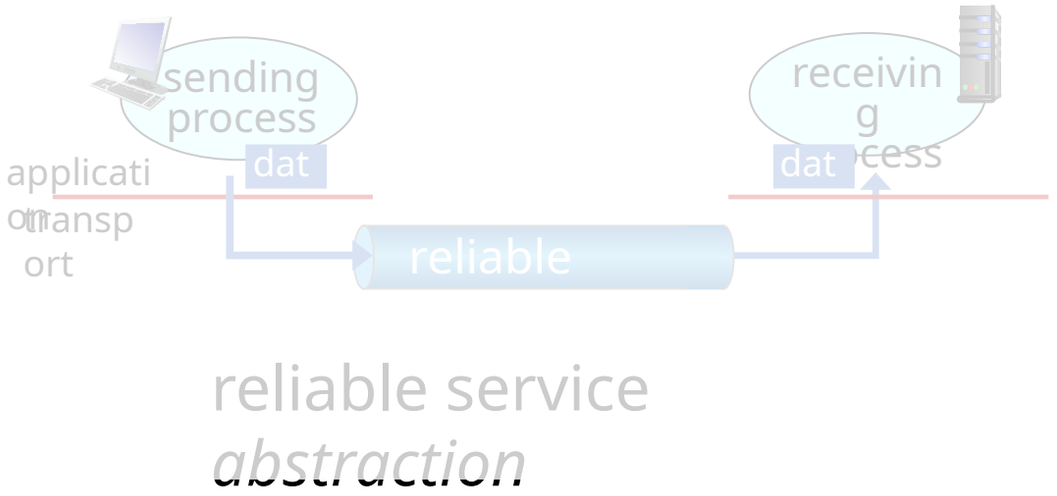


# Principles of reliable data transfer

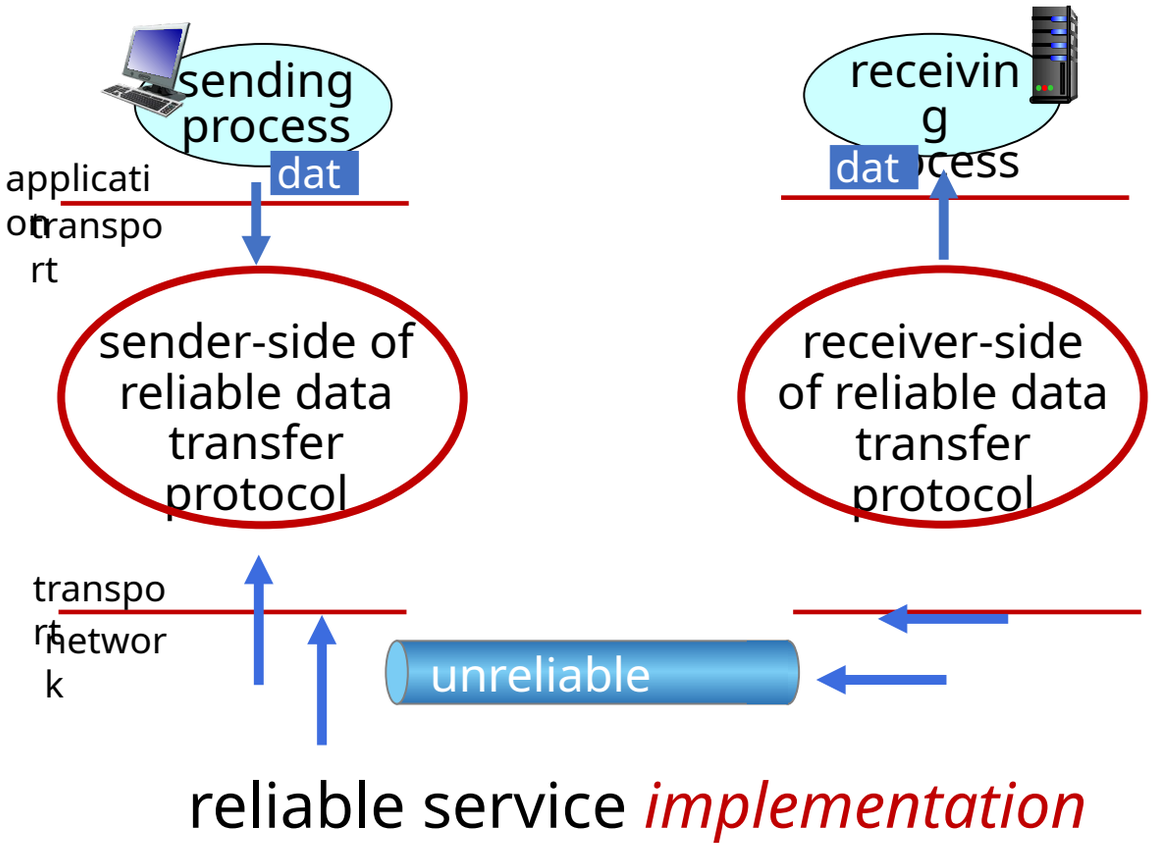



reliable service
*abstraction*

# Principles of reliable data transfer



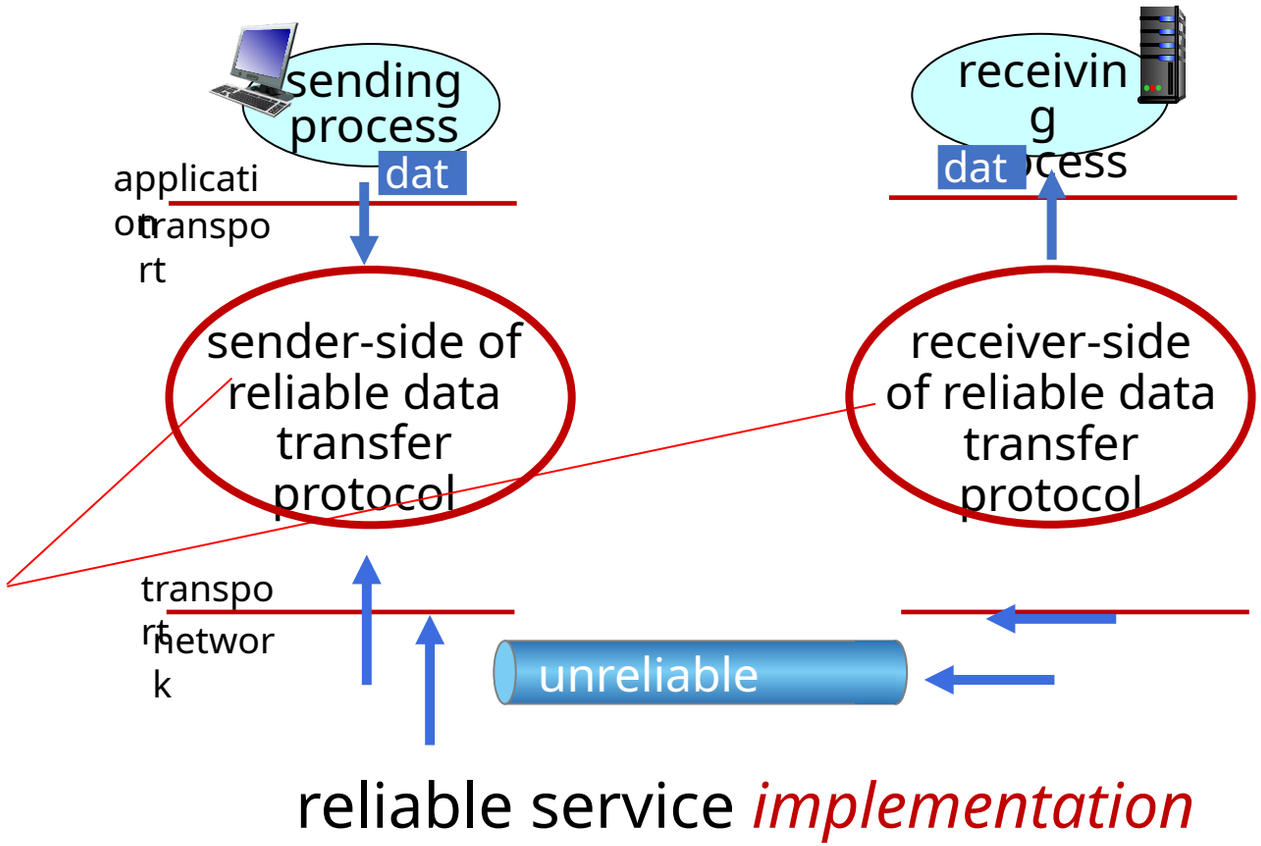

reliable service *abstraction*




reliable service *implementation*

# Principles of reliable data transfer

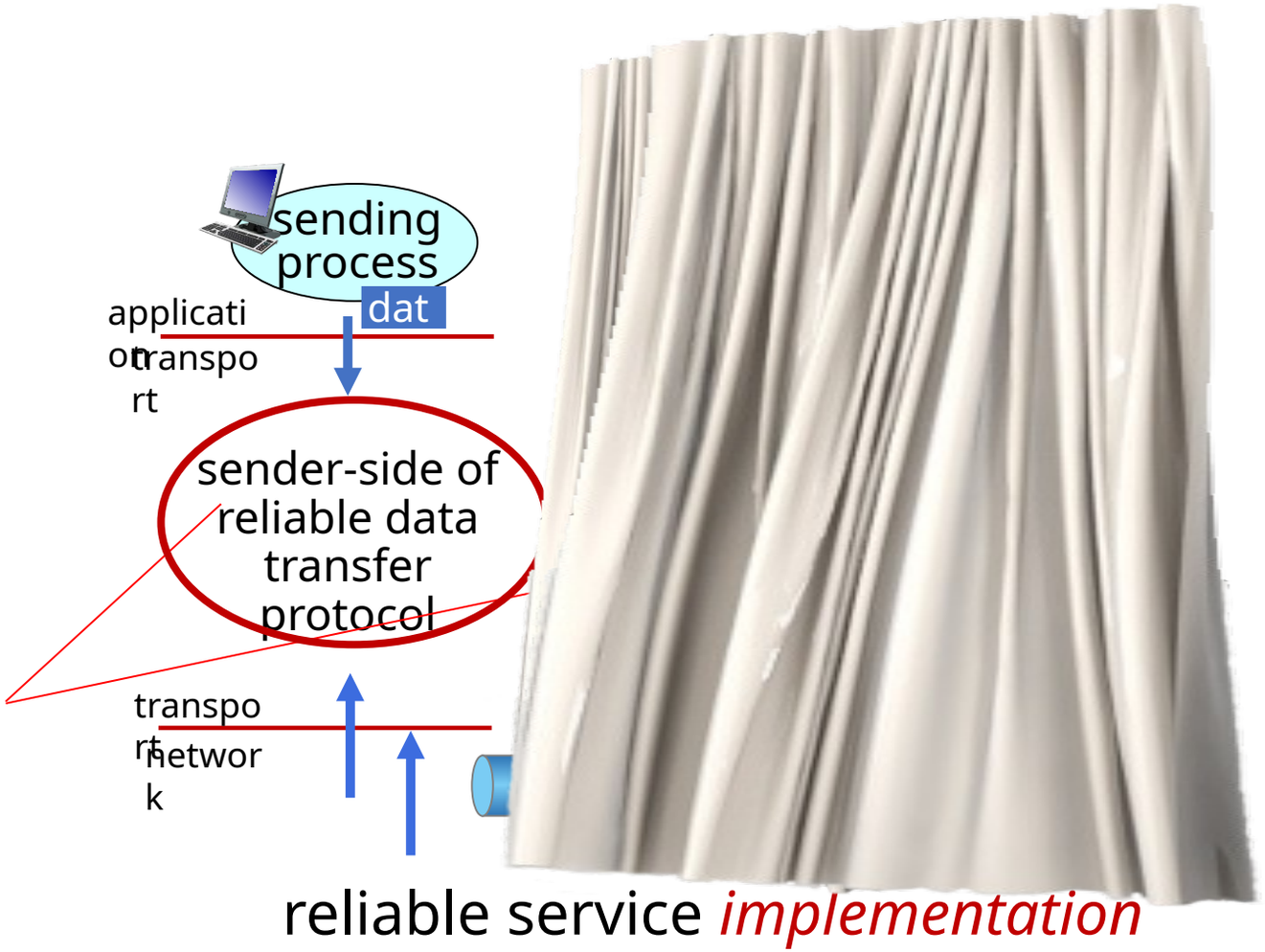
Complexity of reliable data transfer protocol will depend (strongly) on characteristics of unreliable channel (lose, corrupt, reorder data?)



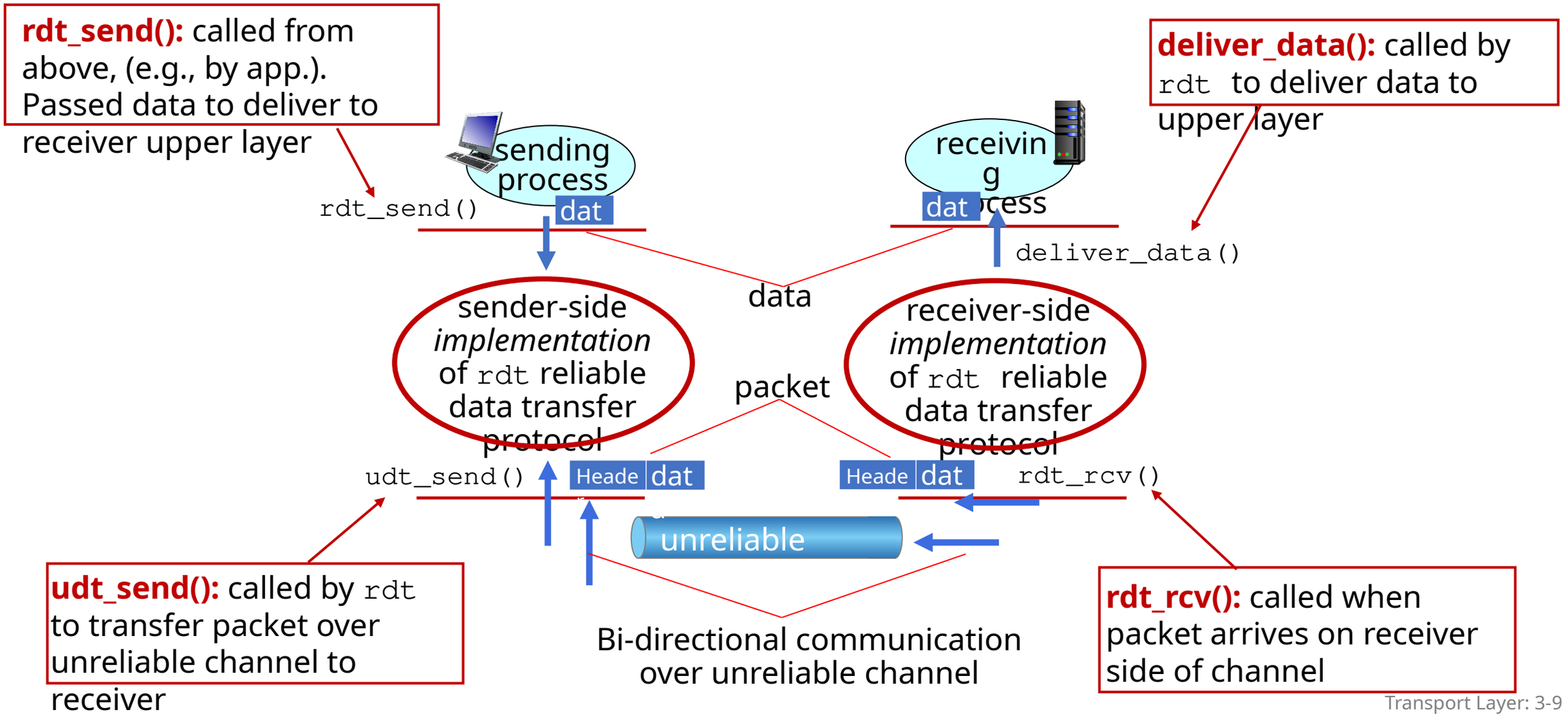
# Principles of reliable data transfer

Sender, receiver do *not* know the “state” of each other, e.g., was a message received?

- unless communicated via a message



# Reliable data transfer protocol (rdt): interfaces

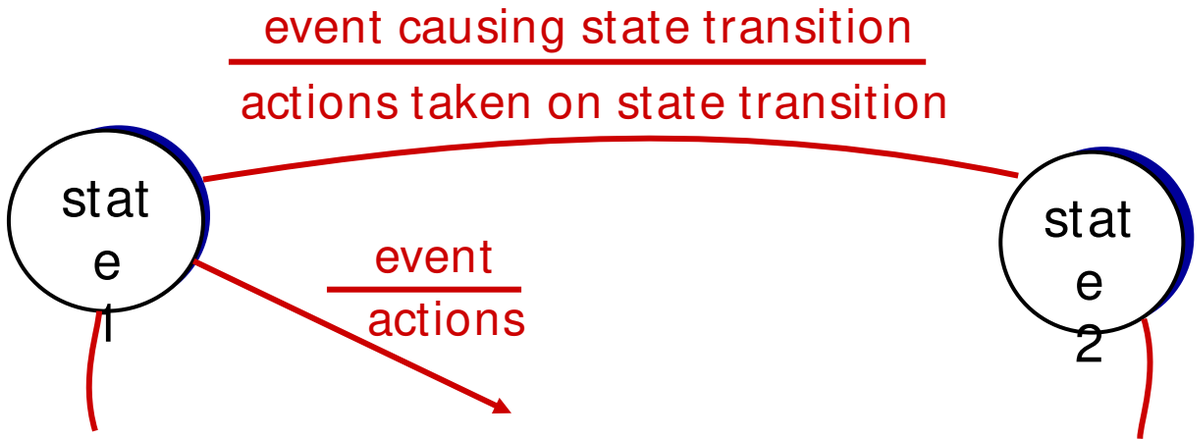


# Reliable data transfer: getting started

We will:

- incrementally develop sender, receiver sides of reliable data transfer protocol (`rdt`)
- consider only unidirectional data transfer
  - but control info will flow in both directions!
- use finite state machines (FSM) to specify sender, receiver

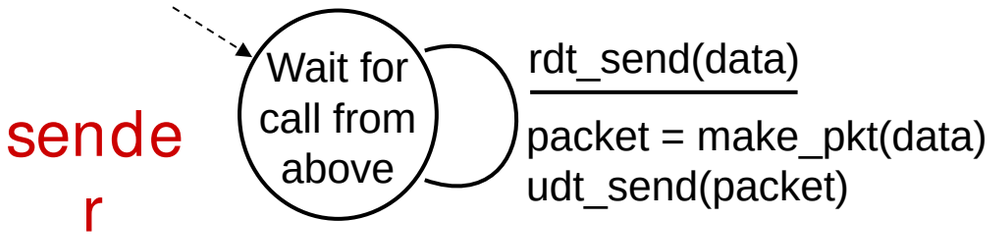
state: when in this "state" next state uniquely determined by next event

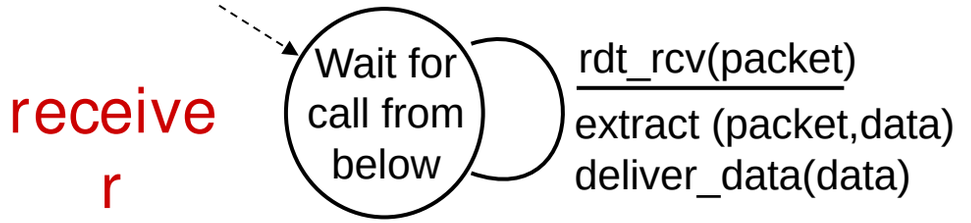


# rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
  - no bit errors
  - no loss of packets
- *separate* FSMs for sender, receiver:
  - sender sends data into underlying channel
  - receiver reads data from underlying channel



# rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
  - checksum (e.g., Internet checksum) to detect bit errors
- *the* question: how to recover from errors?

*How do humans recover from "errors" during conversation?*

# rdt2.0: channel with bit errors

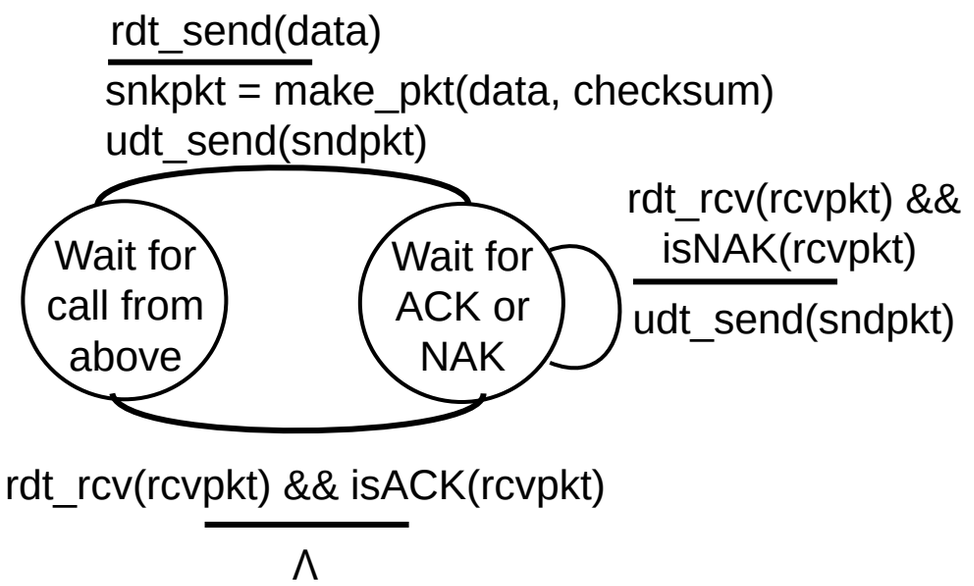
- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors?
  - *acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - sender *retransmits* pkt on receipt of NAK

stop and wait

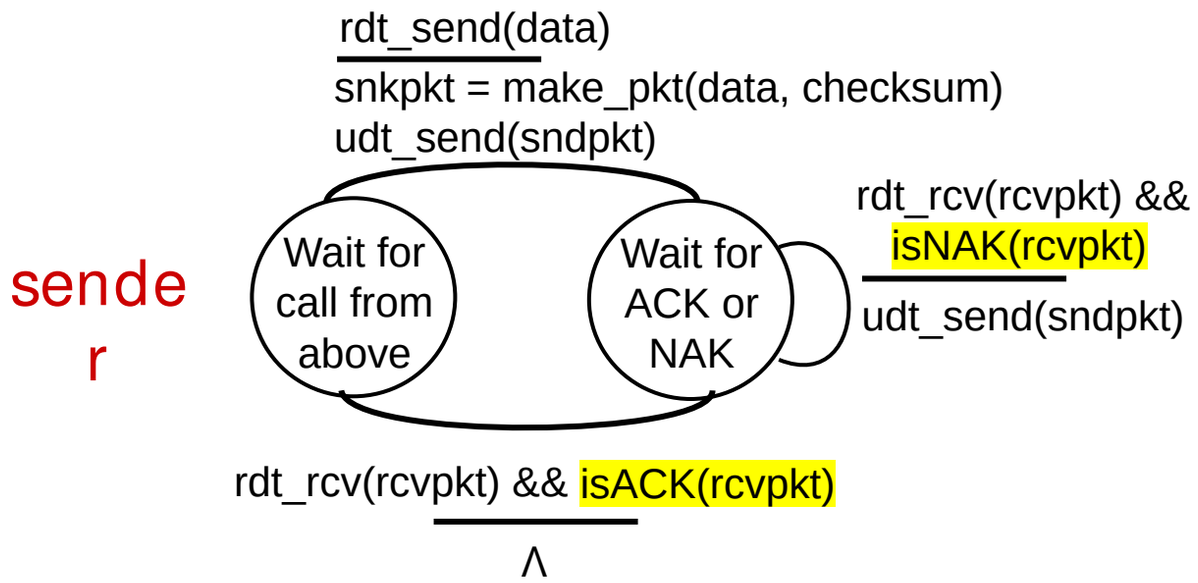
sender sends one packet, then waits for receiver response

# rdt2.0: FSM specifications

sende
r



# rdt2.0: FSM specification

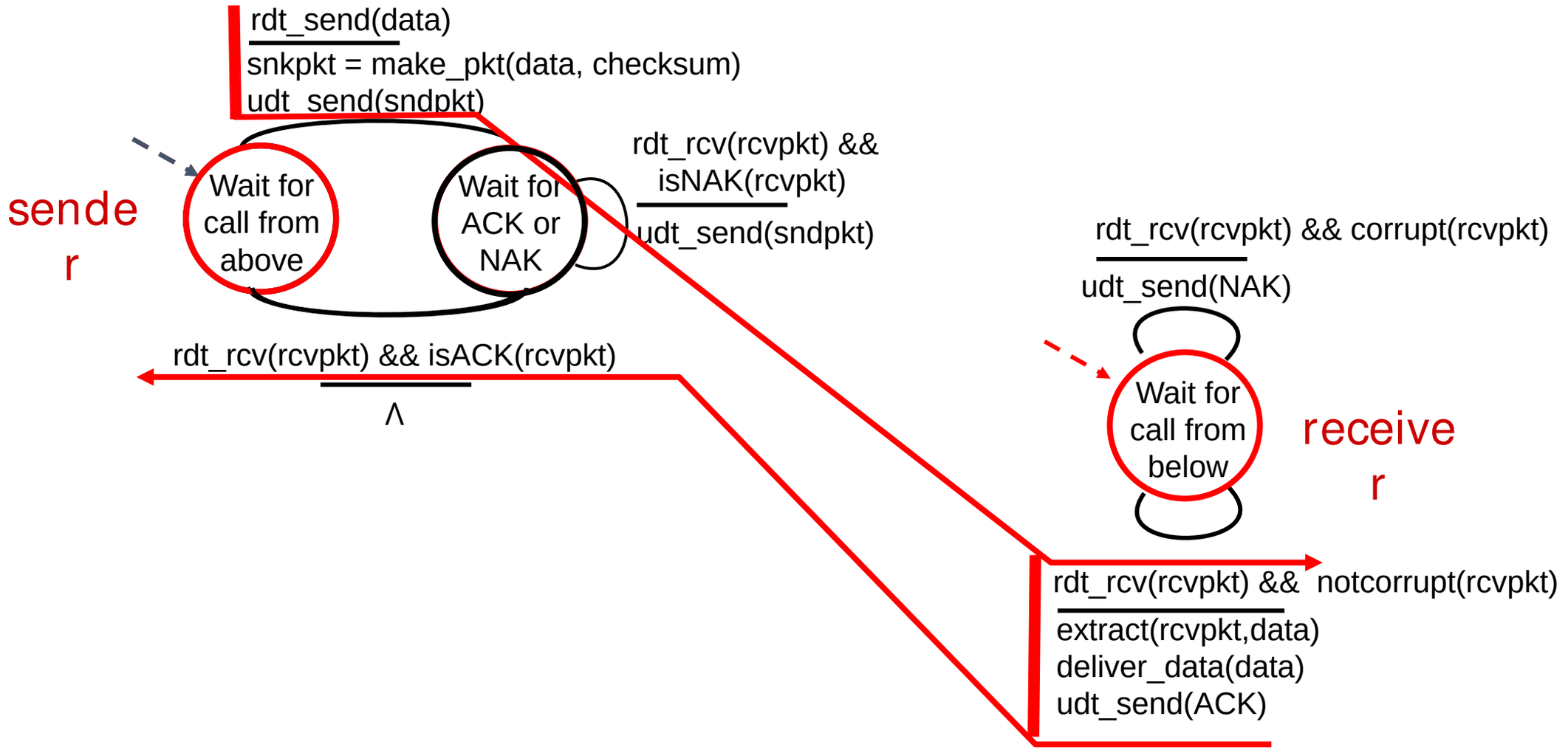



Note: "state" of receiver (did the receiver get my message correctly?) isn't known to sender unless somehow communicated from receiver to sender

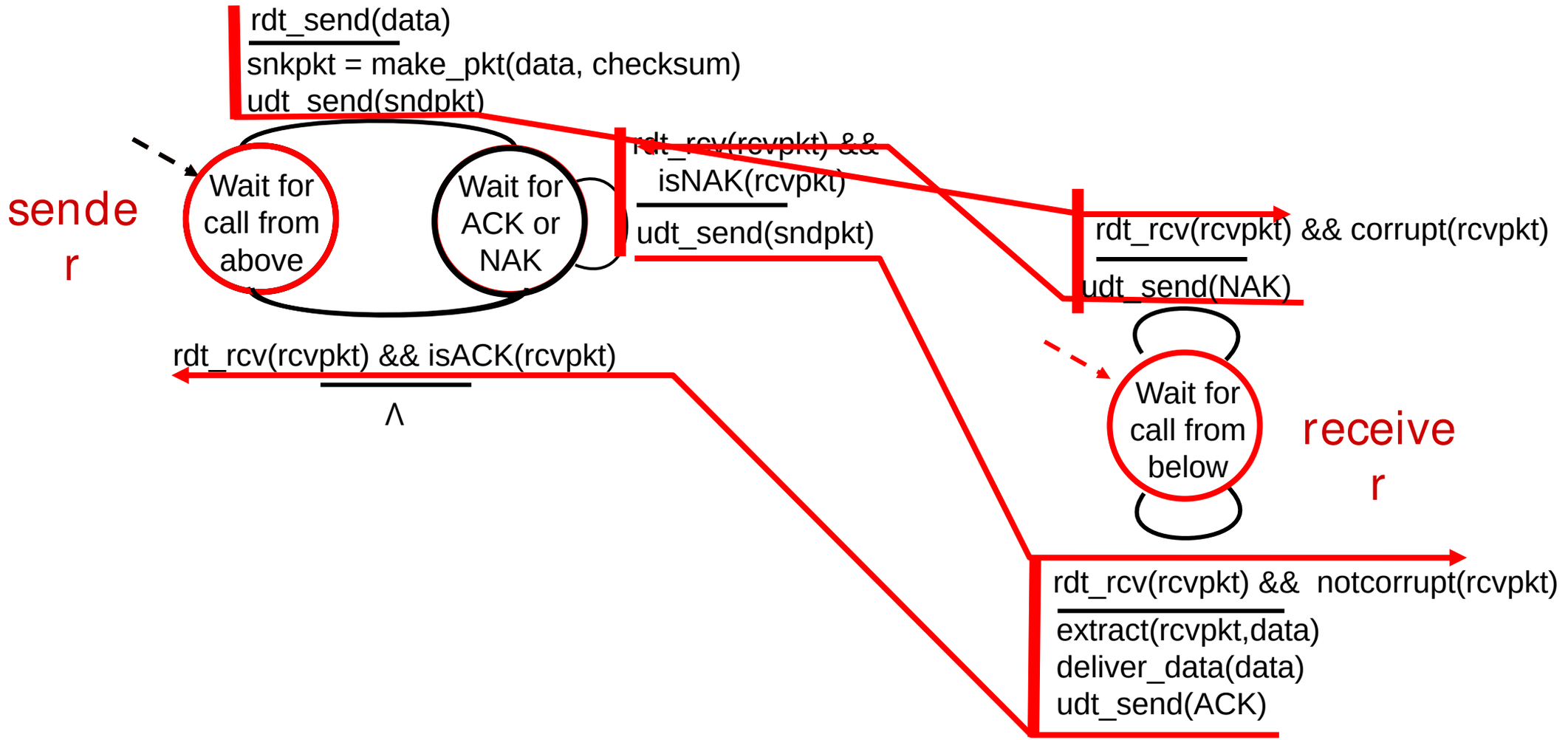
- that's why we need a protocol!



# rdt2.0: operation with no errors



# rdt2.0: corrupted packet scenario



# rdt2.0 has a fatal flaw!

## what happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

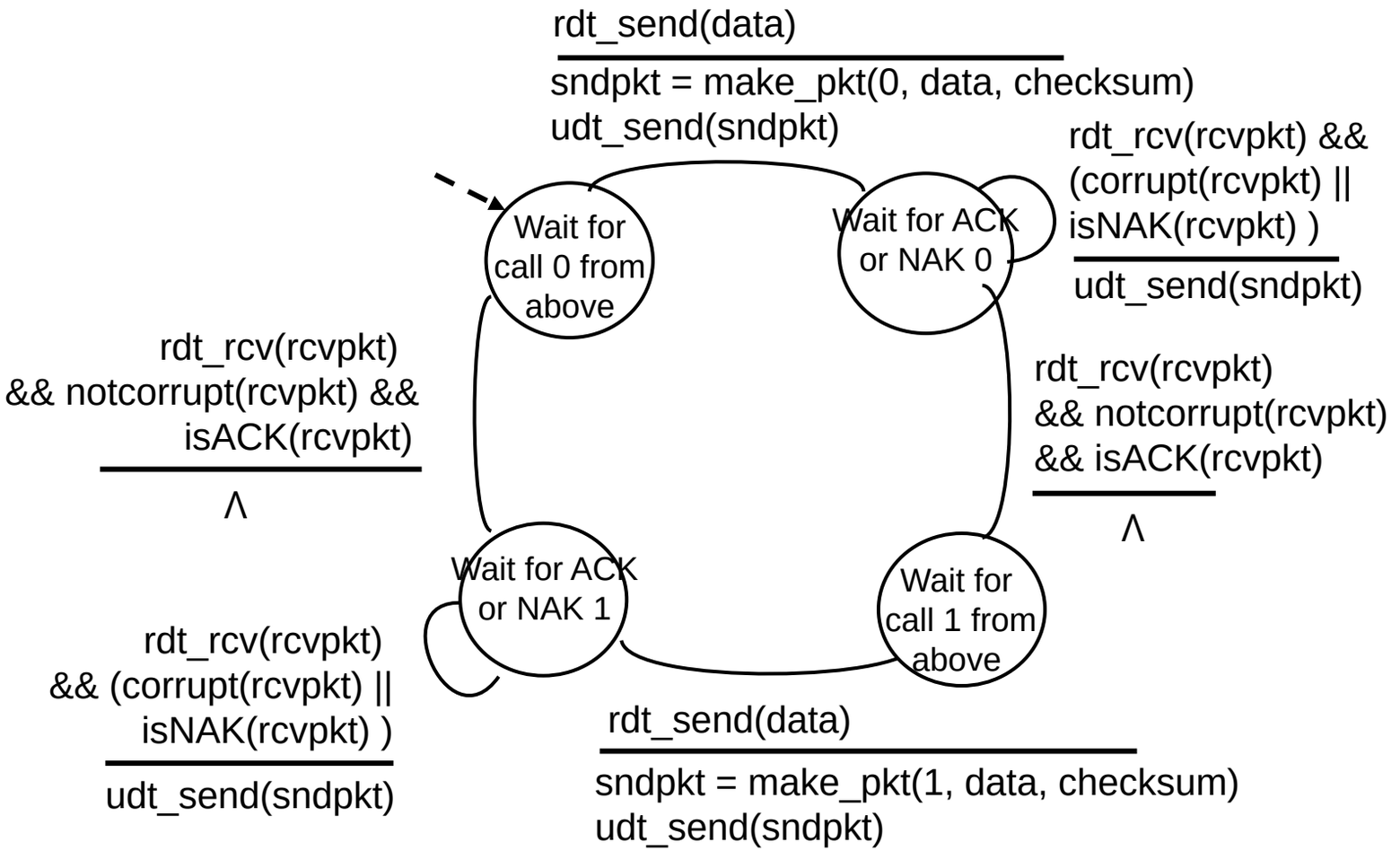
## handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

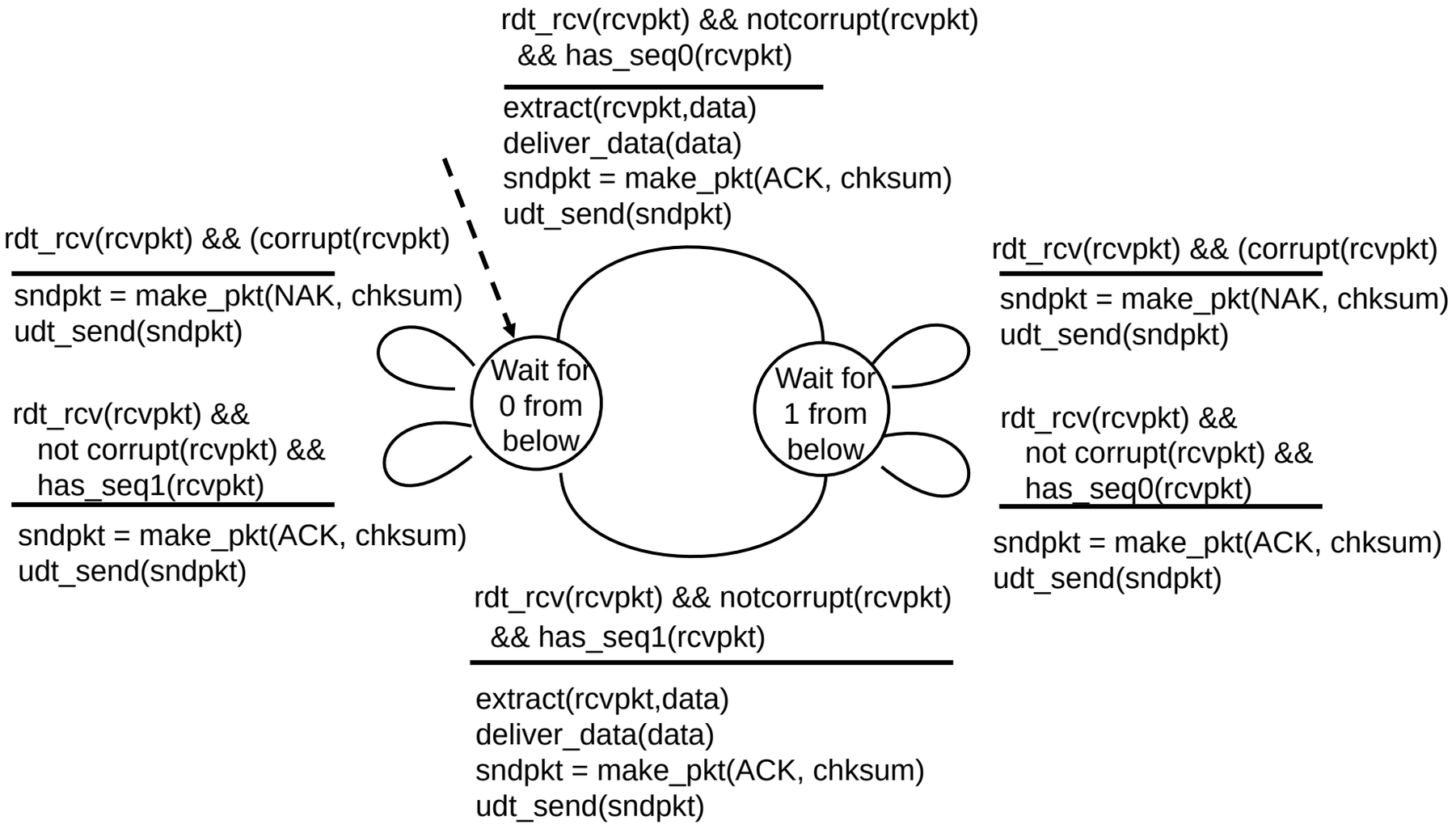
**stop and wait**

sender sends one packet, then waits for receiver response

# rdt2.1: sender, handling garbled ACK/NAKs



# rdt2.1: receiver, handling garbled ACK/NAKs



# rdt2.1: discussion

## sender:

- seq # added to pkt
- two seq. #s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states
  - state must "remember" whether "expected" pkt should have seq # of 0 or 1

## receiver:

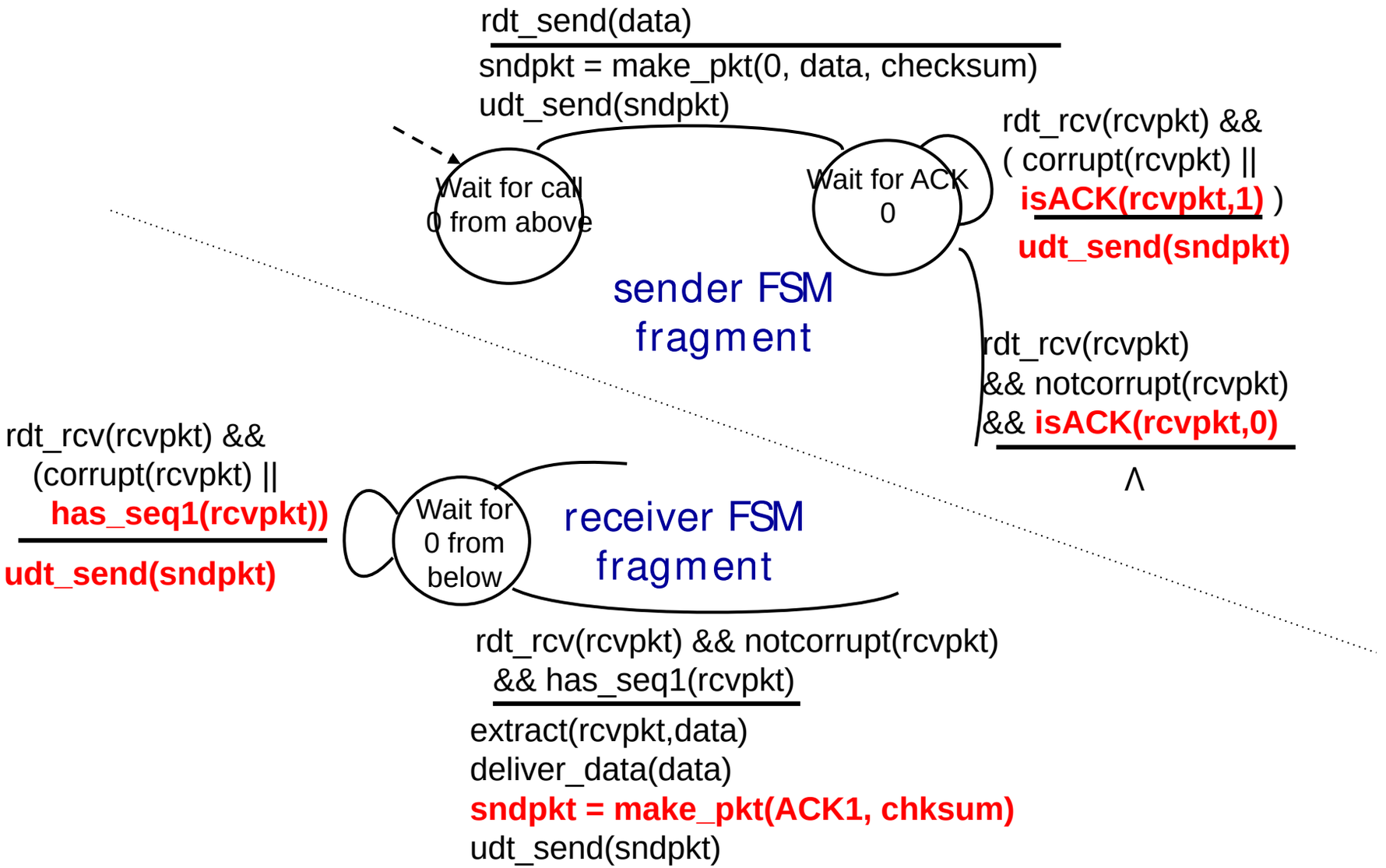
- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

# rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

As we will see, TCP uses this approach to be NAK-free

# rdt2.2: sender, receiver fragments



# rdt3.0: channels with errors *and* loss

*New channel assumption:* underlying channel can also *lose* packets (data, ACKs)

- checksum, sequence #s, ACKs, retransmissions will be of help … but not quite enough

*Q:* How do *humans* handle lost sender-to-receiver words in conversation?

# rdt3.0: channels with errors *and* loss

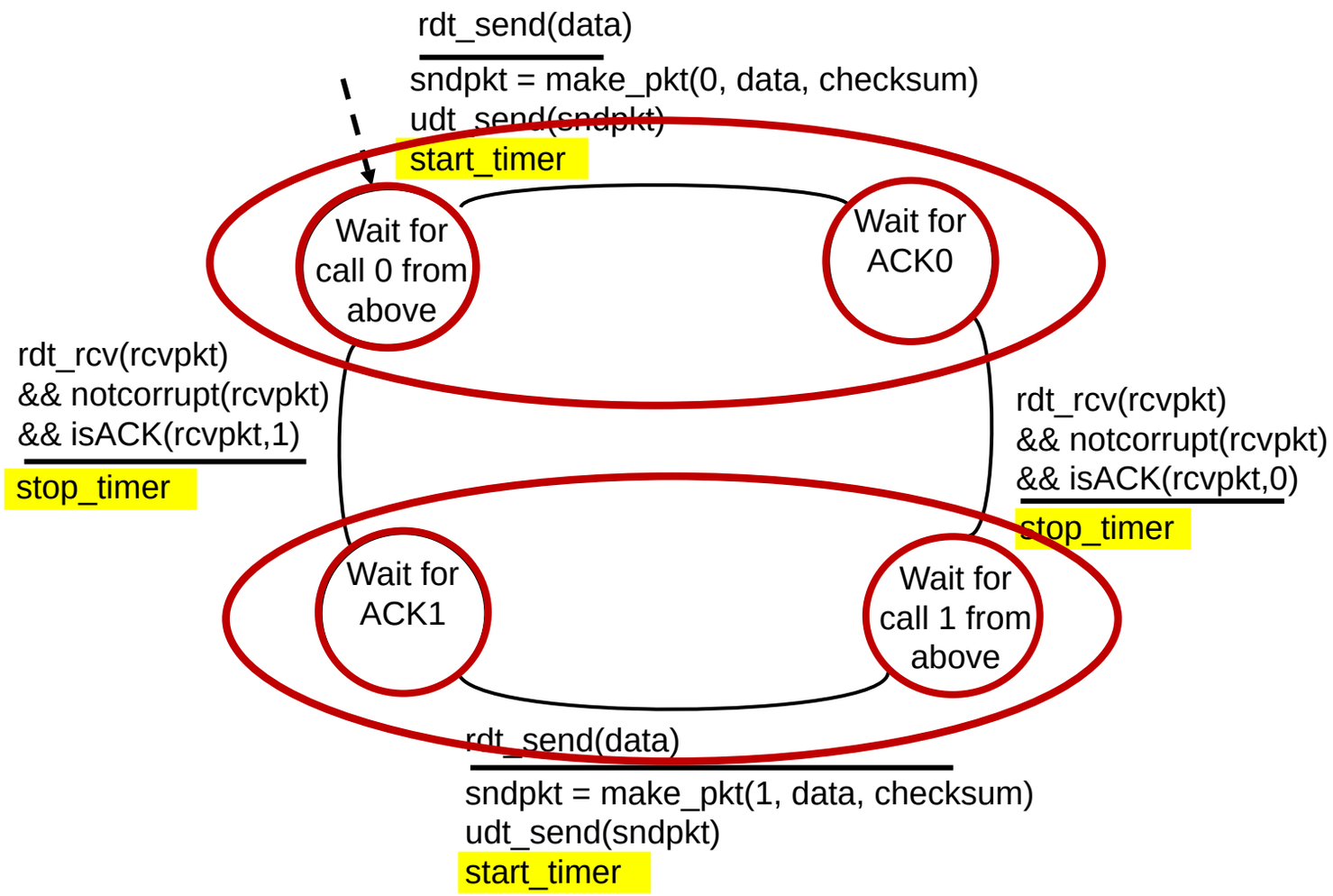
*Approach:* sender waits "reasonable" amount of time for ACK

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq #s already handles this!
  - receiver must specify seq # of packet being ACKed
- use countdown timer to interrupt after "reasonable" amount of time

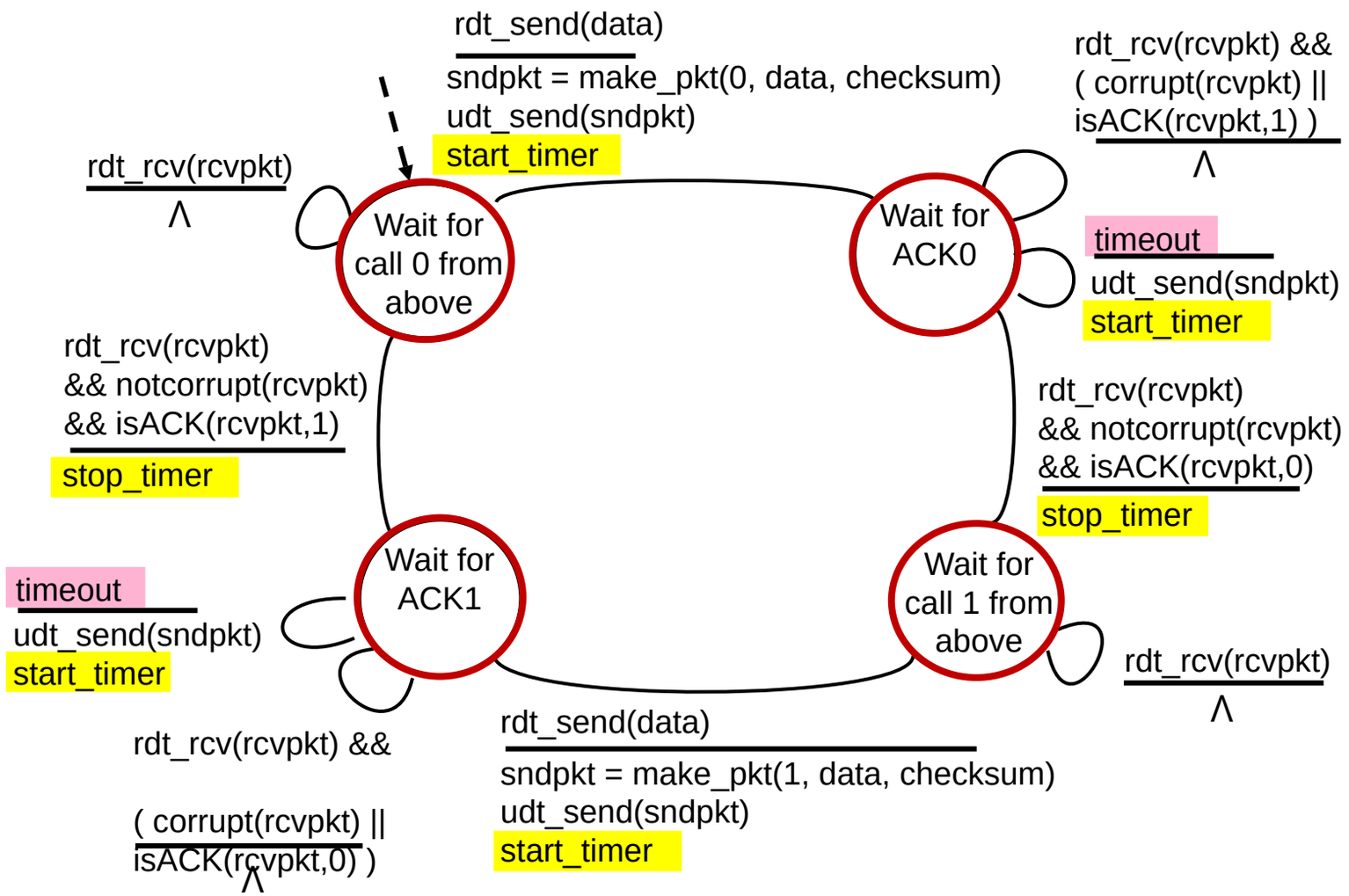


*timeout*

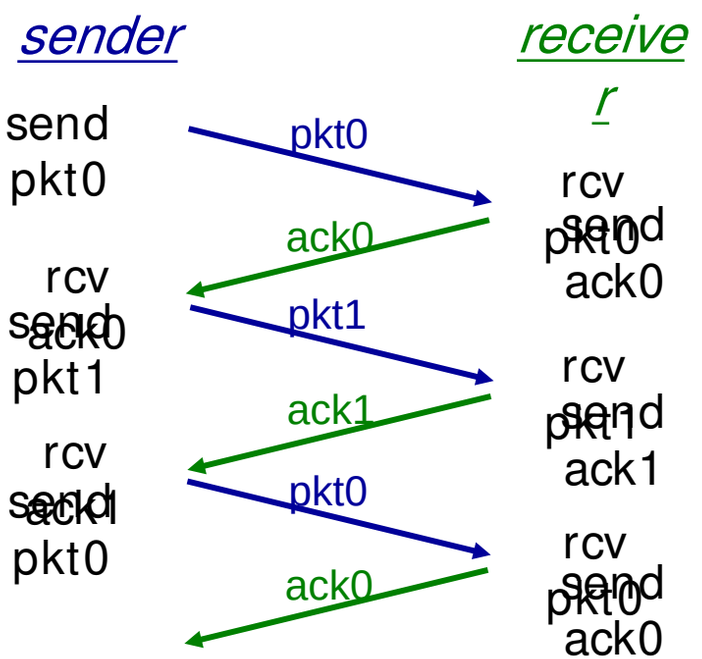
# rdt3.0 sender



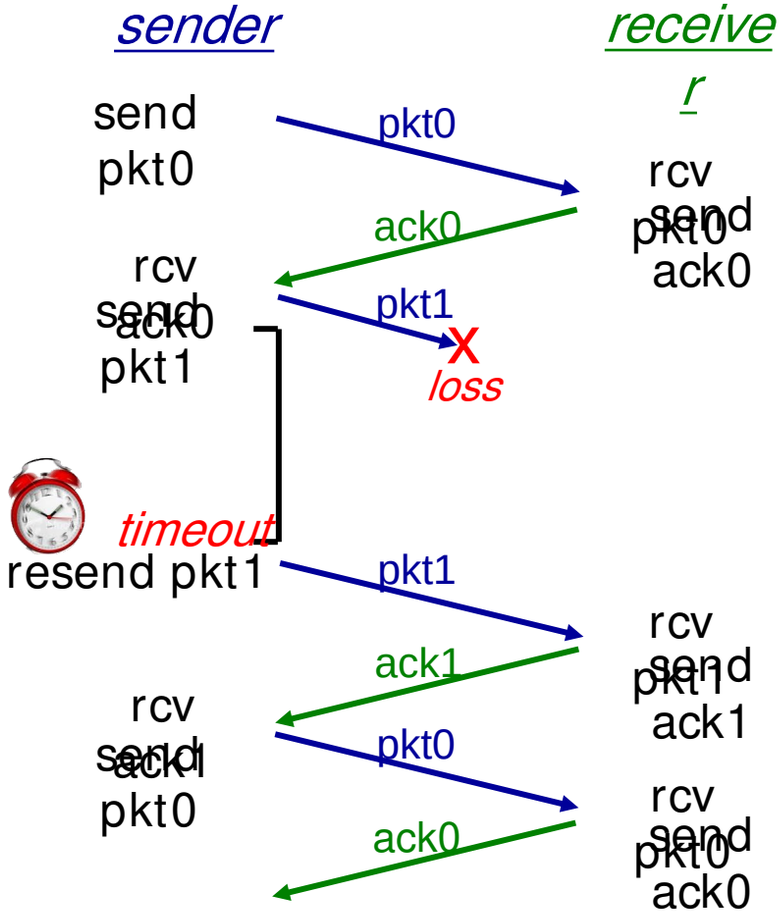
# rdt3.0 sender



# rdt3.0 in action

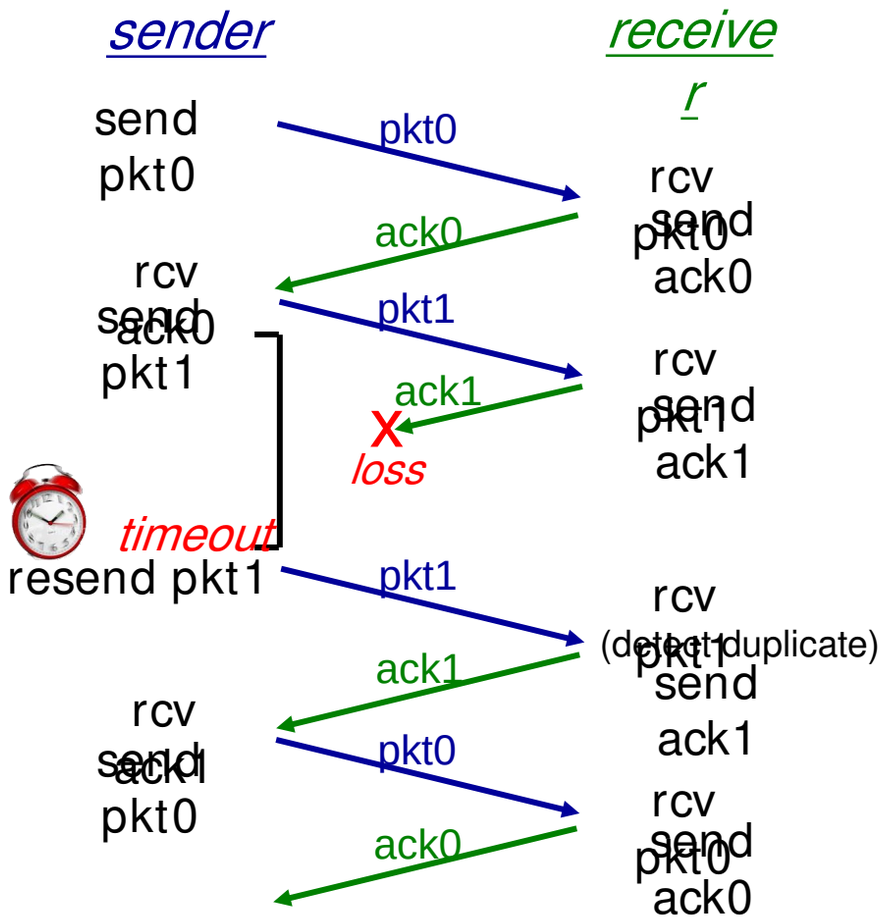



(a) no loss

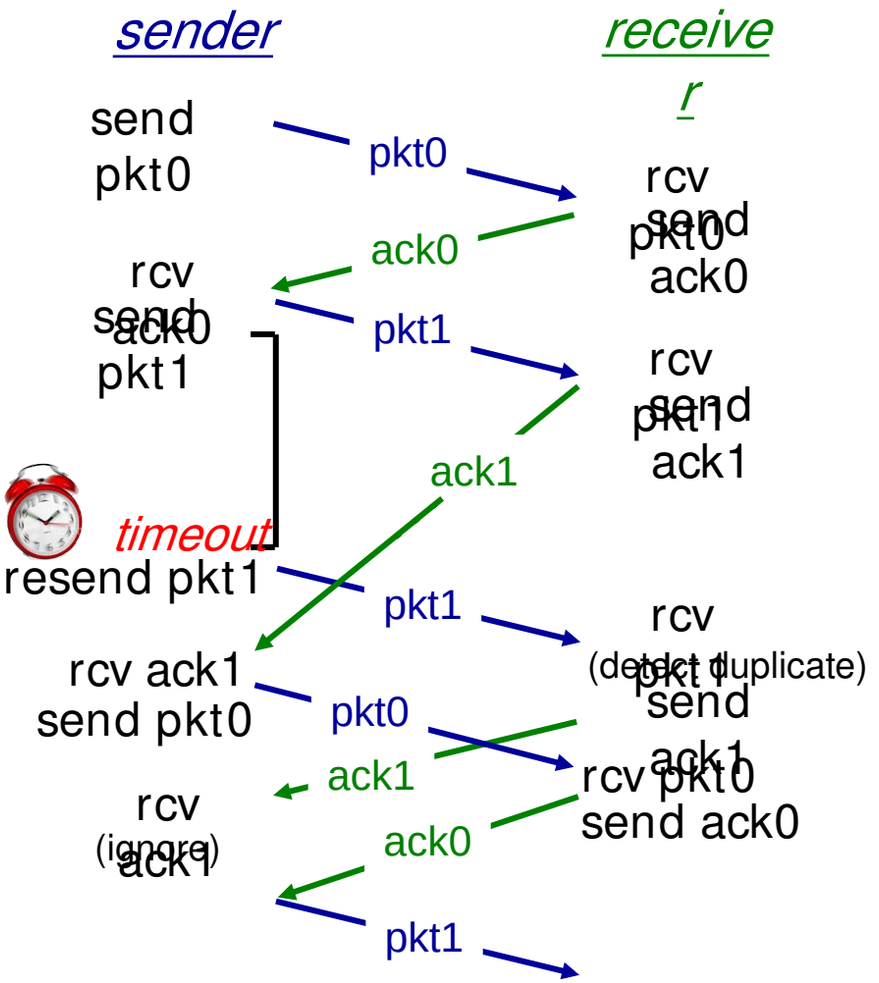



(b) packet loss

# rdt3.0 in action




(c) ACK loss



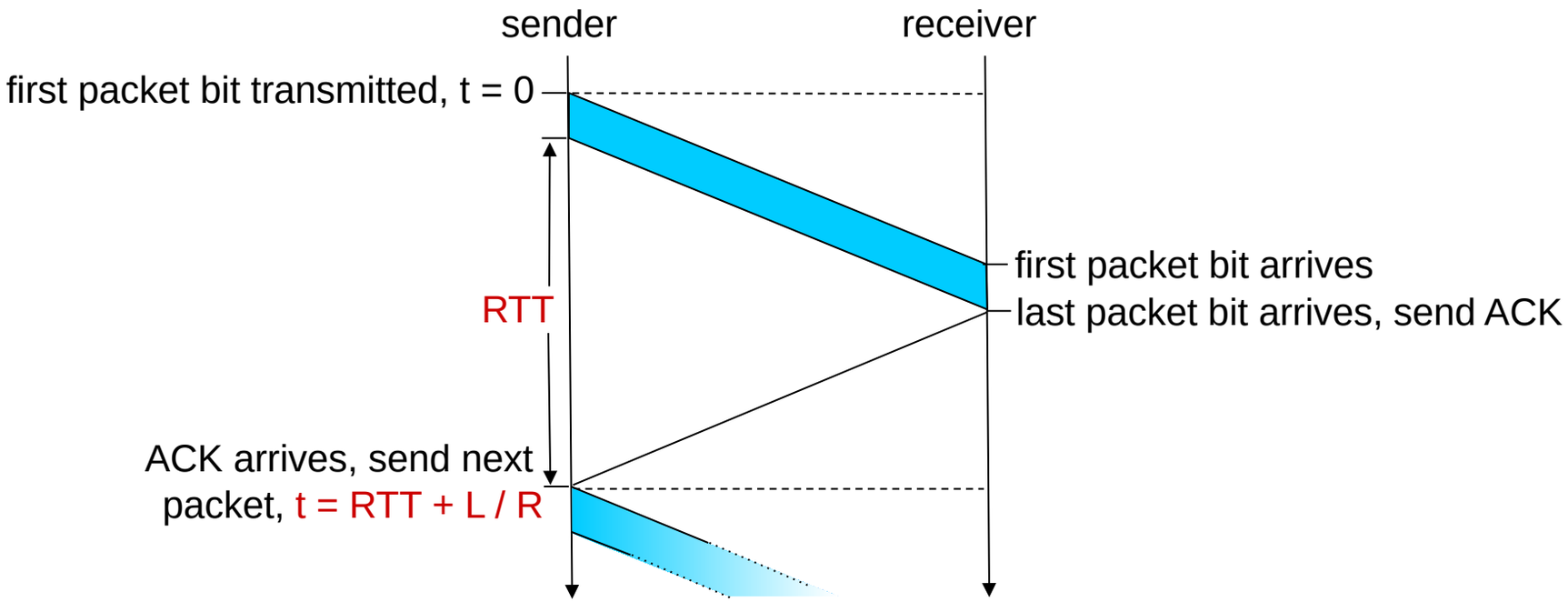

(d) premature timeout/ delayed ACK

# Performance of rdt3.0 (stop-and-wait)

- $U_{sender}$: *utilization* – fraction of time sender busy sending
- example: 1 Gbps link, 15 ms prop. delay, 8000 bit packet
  - time to transmit packet into channel:

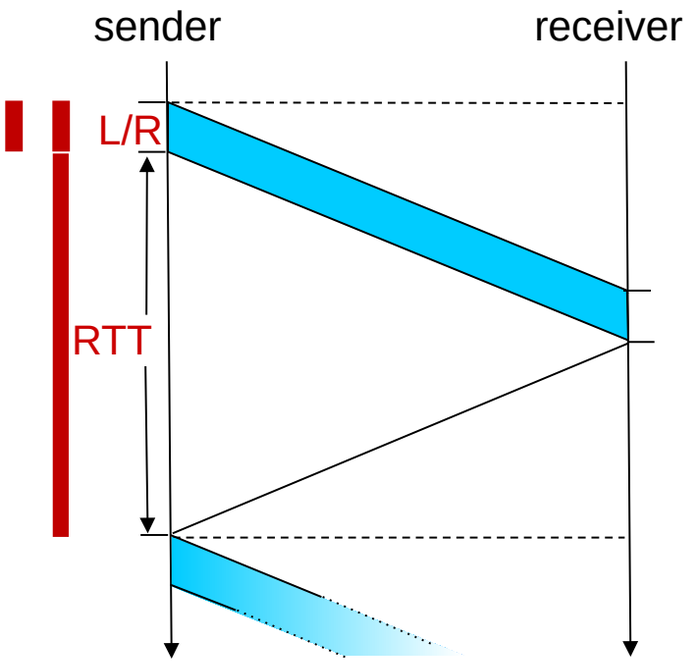
$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$

# rdt3.0: stop-and-wait operation



# rdt3.0: stop-and-wait operation

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

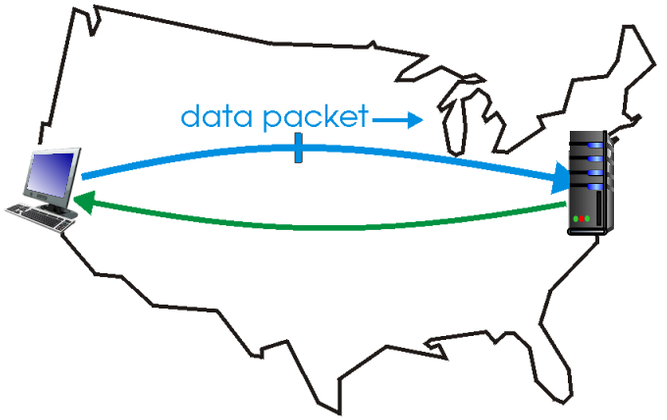



- rdt 3.0 protocol performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

# rdt3.0: pipelined protocols operation

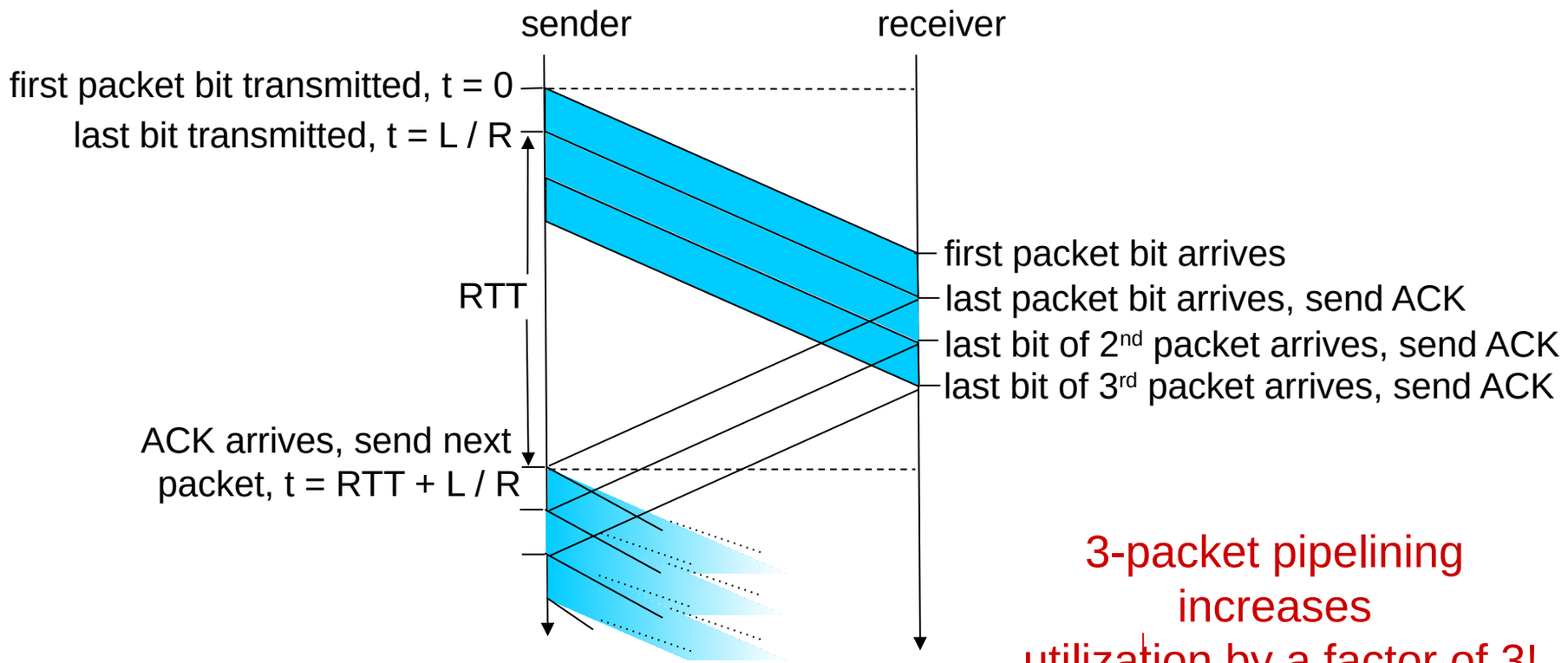
pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver




(a) a stop-and-wait protocol in operation

# Pipelining: increased utilization

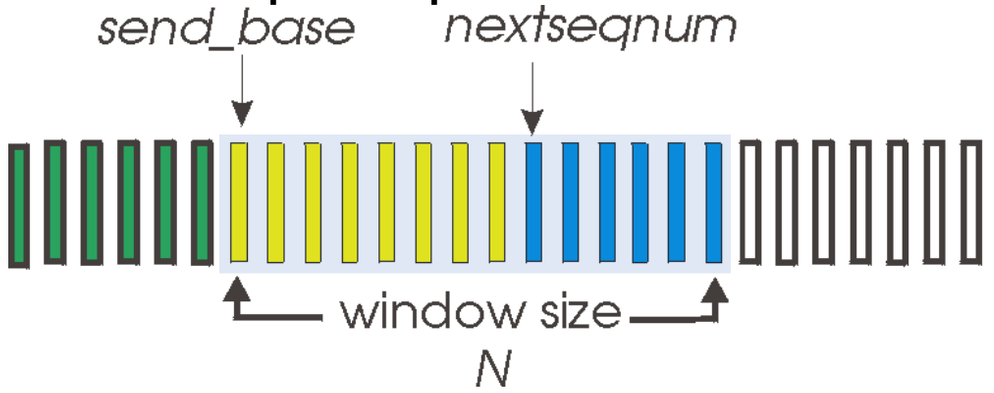


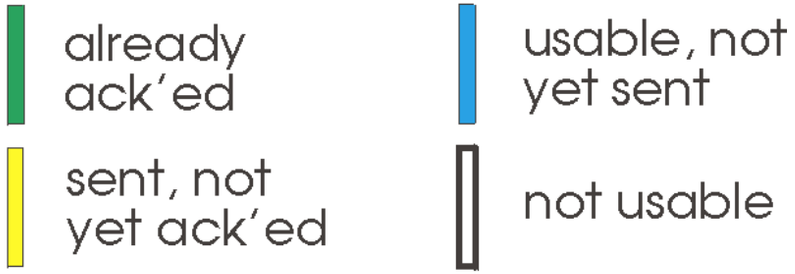

3-packet pipelining increases utilization by a factor of 3!

$$U_{sender} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$

# Go-Back-N: sender

- sender: “window” of up to N, consecutive transmitted but unACKed pkts
  - k-bit seq # in pkt header



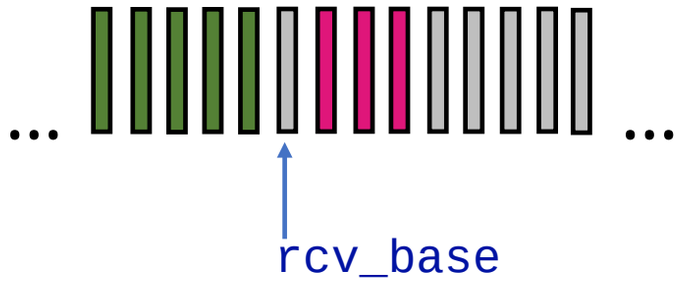



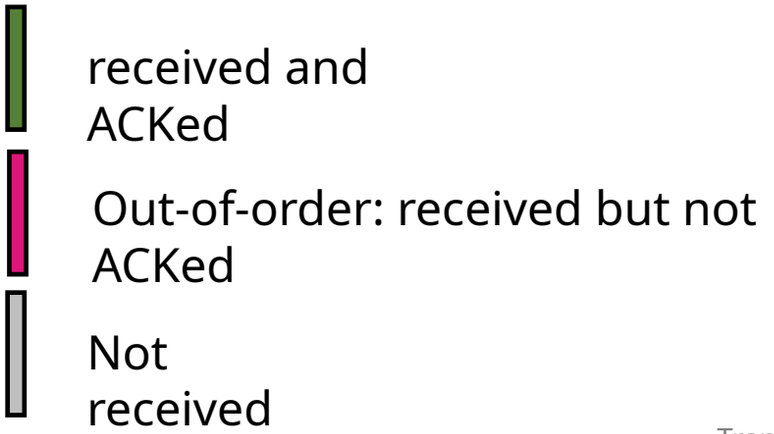

- *cumulative ACK:* ACK(*n*): ACKs all packets up to, including seq # *n*
  - on receiving ACK(*n*): move window forward to begin at *n+1*
- timer for oldest in-flight packet
- *timeout(n):* retransmit packet n and all higher seq # packets in window

# Go-Back-N: receiver

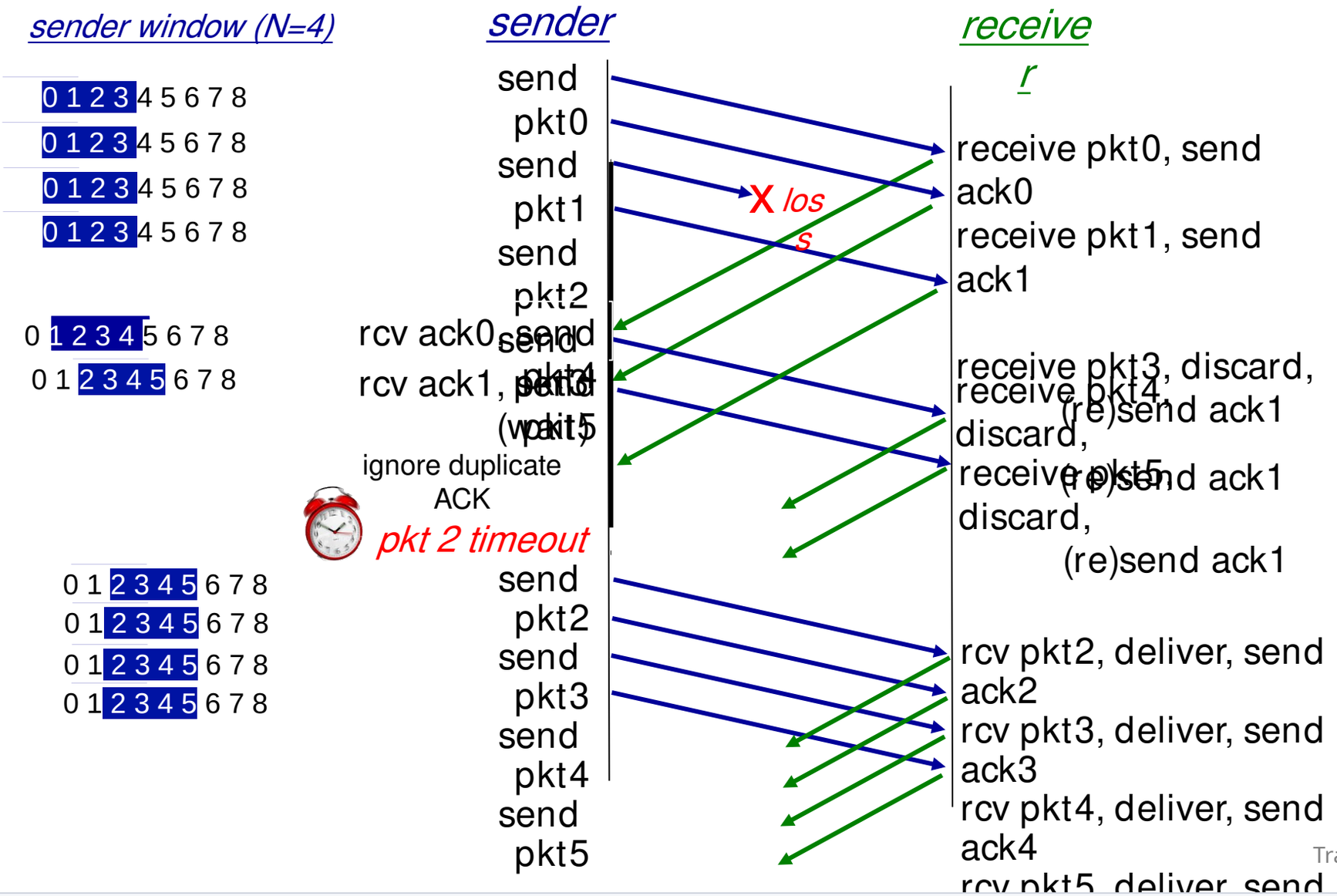
- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
  - may generate duplicate ACKs
  - need only remember `rcv_base`
- on receipt of out-of-order packet:
  - can discard (don't buffer) or buffer: an implementation decision
  - re-ACK pkt with highest in-order seq #

Receiver view of sequence number space:



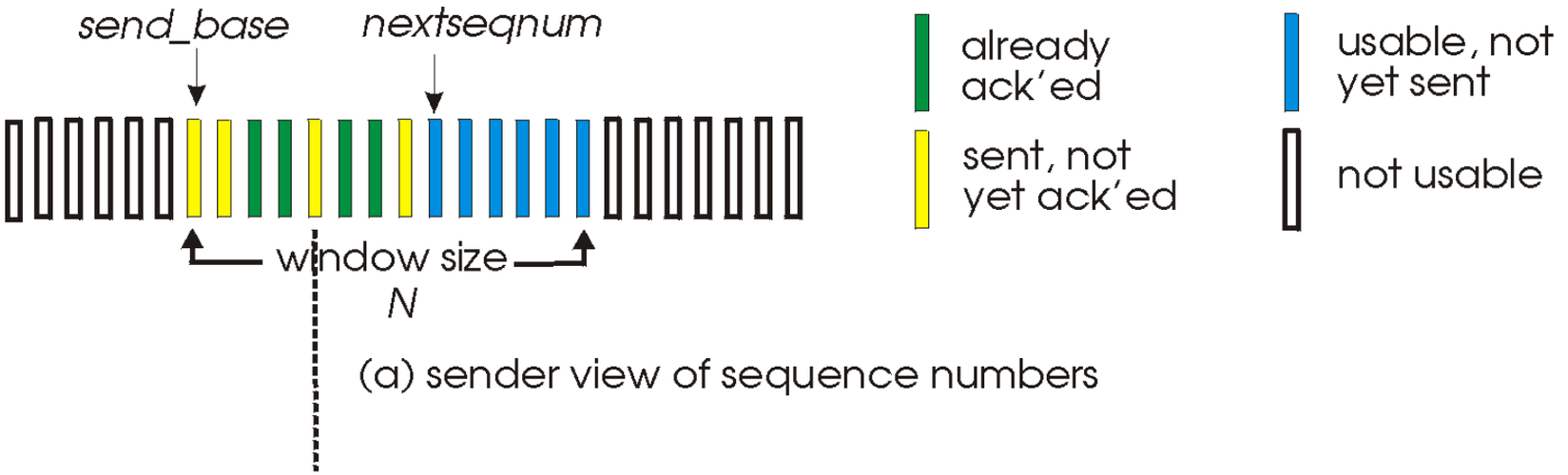
# Go-Back-N in action



# Selective repeat: the approach

- *pipelining*: *multiple* packets in flight
- *receiver individually ACKs* all correctly received packets
  - buffers packets, as needed, for in-order delivery to upper layer
- sender:
  - maintains (conceptually) a timer for each unACKed pkt
    - timeout: retransmits single unACKed packet associated with timeout
  - maintains (conceptually) "window" over *N* consecutive seq #s
    - limits pipelined, "in flight" packets to be within this window

# Selective repeat: sender, receiver windows




(a) sender view of sequence numbers

# Selective repeat: sender and receiver

## sender

data from above:

- if next available seq # in window, send packet

timeout(*n*):

- resend packet *n*, restart timer

ACK(*n*) in [sendbase,sendbase+N-1]:

- mark packet *n* as received
- if n smallest unACKed packet, advance window base to next unACKed seq #

## receiver

packet *n* in [rcvbase, rcvbase+N-1]

- send ACK(*n*)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

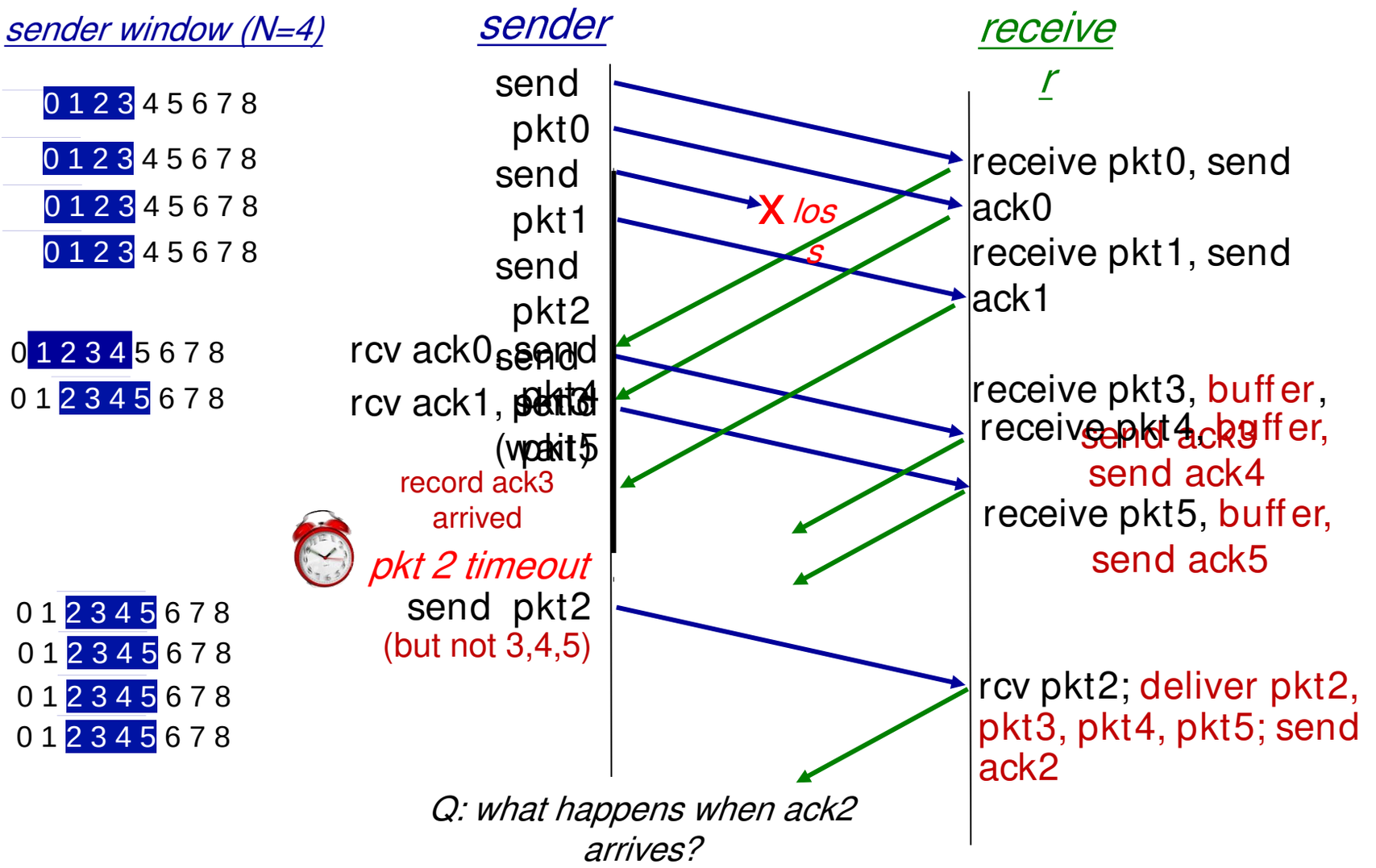
packet *n* in [rcvbase-N,rcvbase-1]

- ACK(*n*)

otherwise:

- ignore

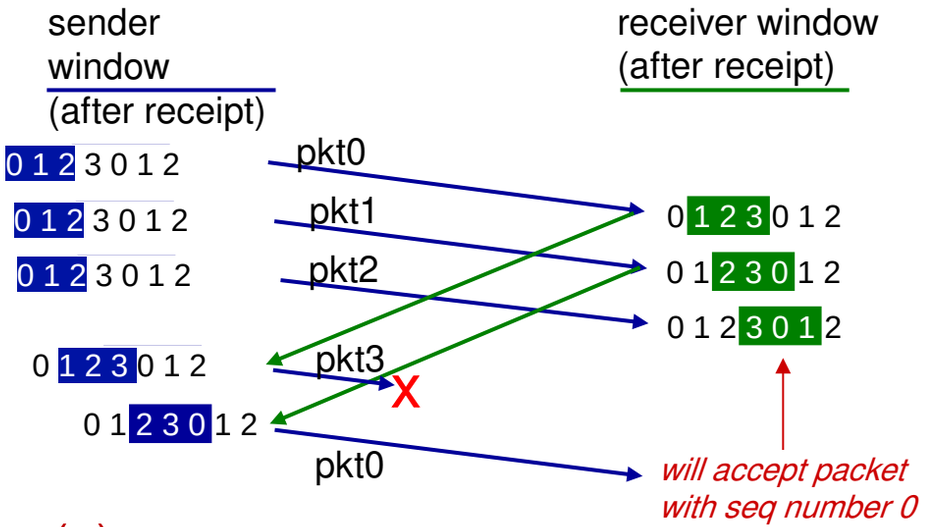
# Selective Repeat in action

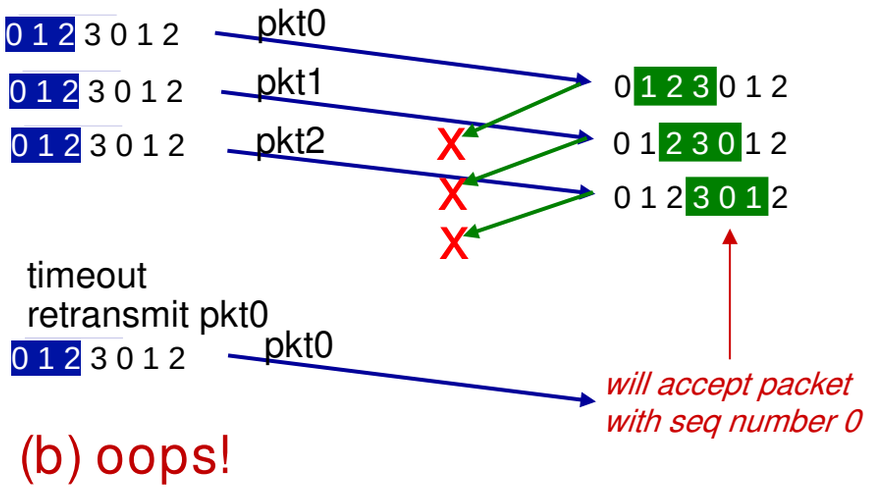


# Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3



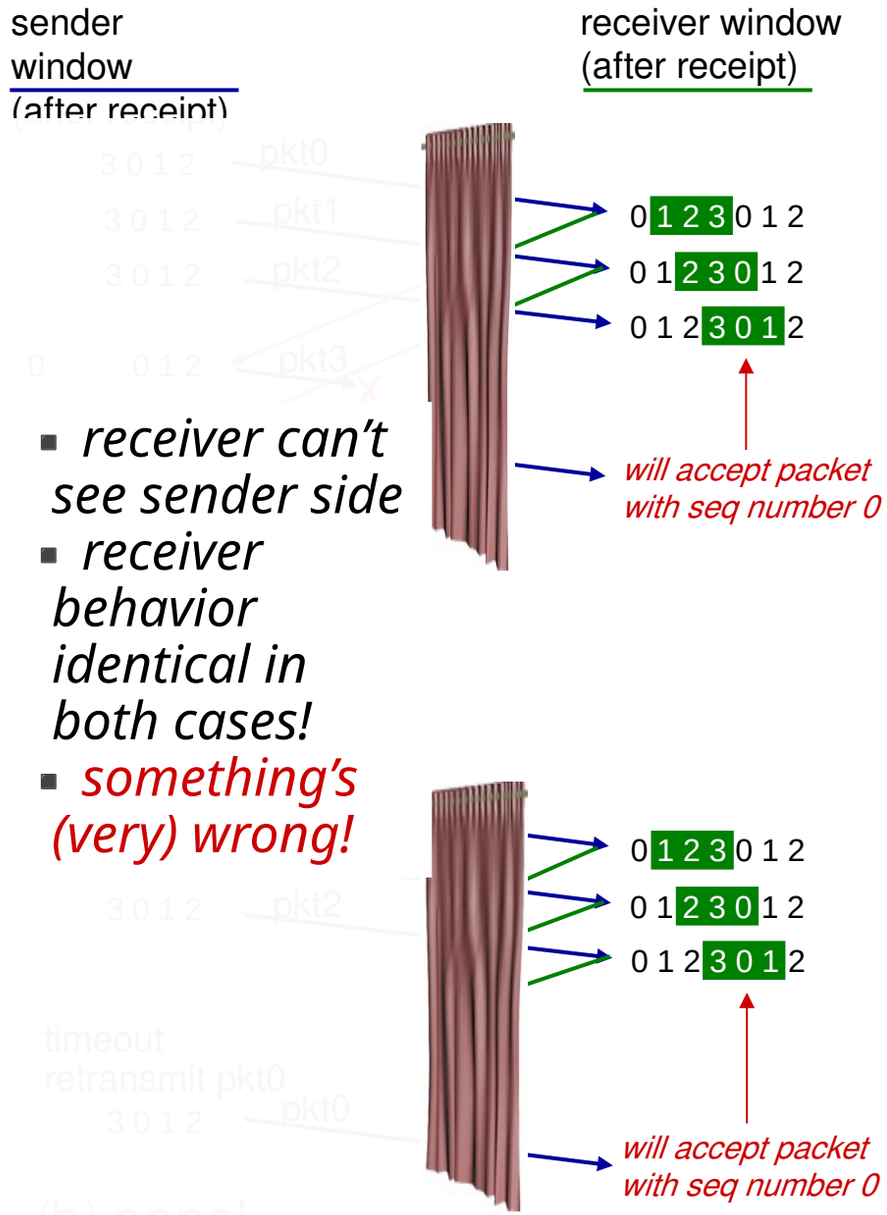
# Selective repeat: a dilemma!

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3

Q: what relationship is needed between sequence # size (N) and window size (W) to avoid problem in scenario (b)?

A: N >= 2W



# References

1.https://media.pearsoncmg.com/ph/esm/ecs_kurose_compnetwork_8/cw/#interactiveanimations

2. https://www.tkn.tu-berlin.de/teaching/rn/animations/gbn_sr/

# Chapter 3
# Transport Layer
## TCP - Connection Management

# Chapter 3: roadmap

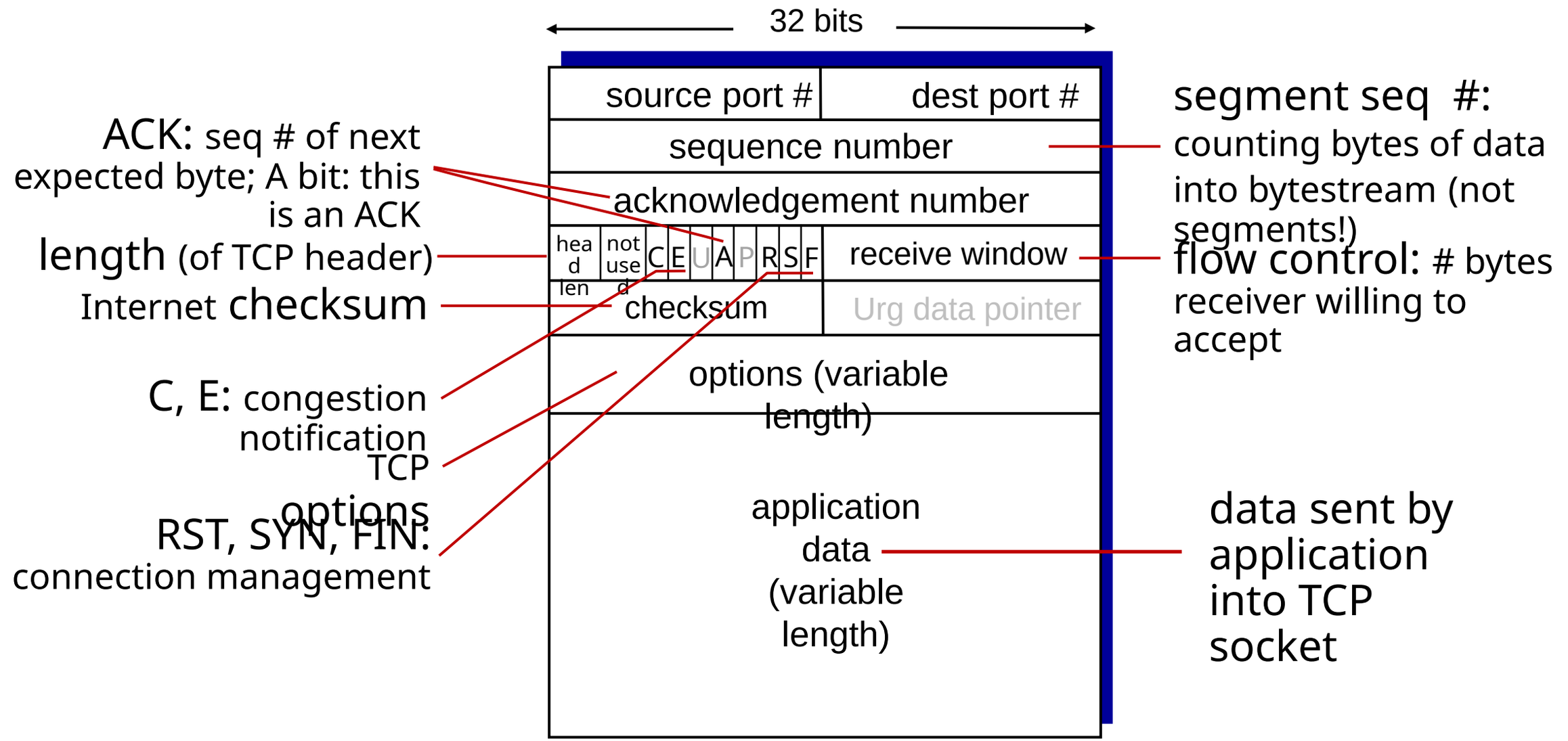
- Transport-layer services
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control



# TCP: overview RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
  - no “message boundaries"
- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- cumulative ACKs
- pipelining:
  - TCP congestion and flow control set window size
- connection-oriented:
  - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver

# TCP segment structure



# TCP sequence numbers, ACKs

*Sequence numbers:*

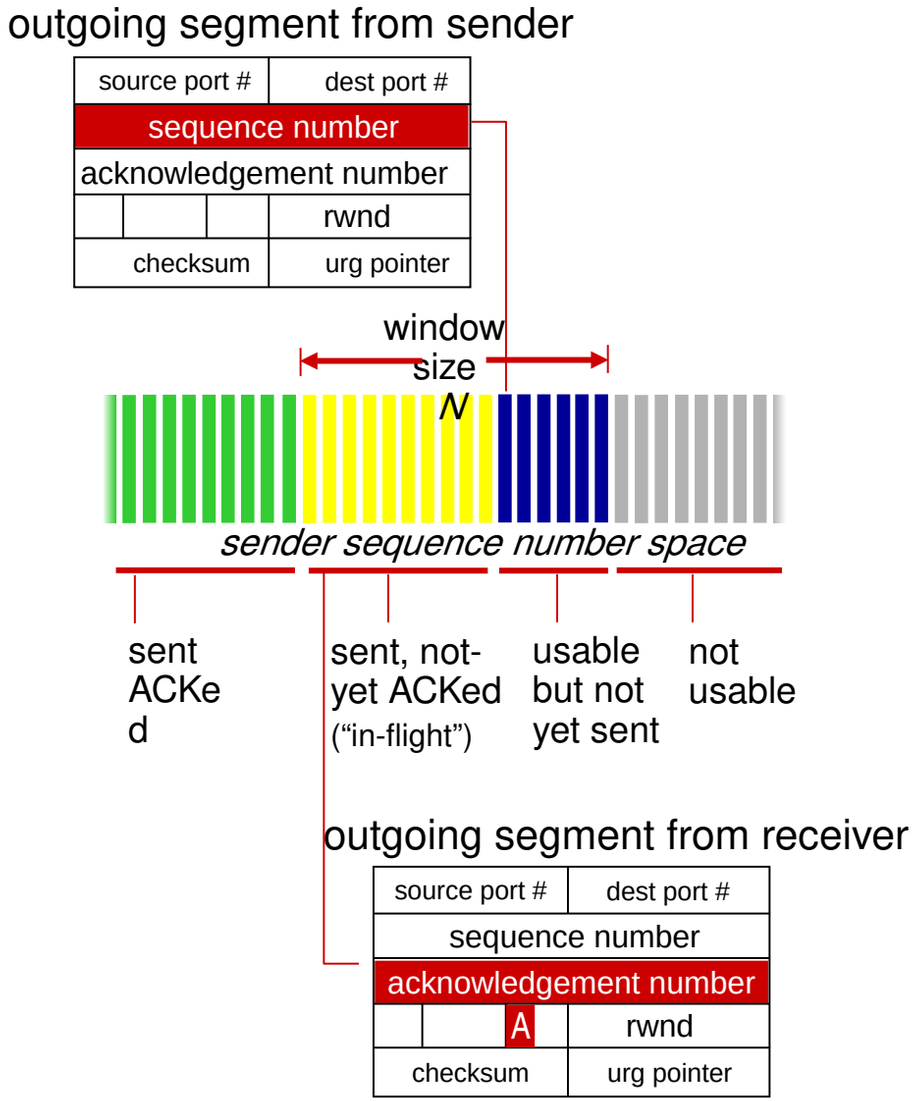
- byte stream "number" of first byte in segment's data

*Acknowledgements:*

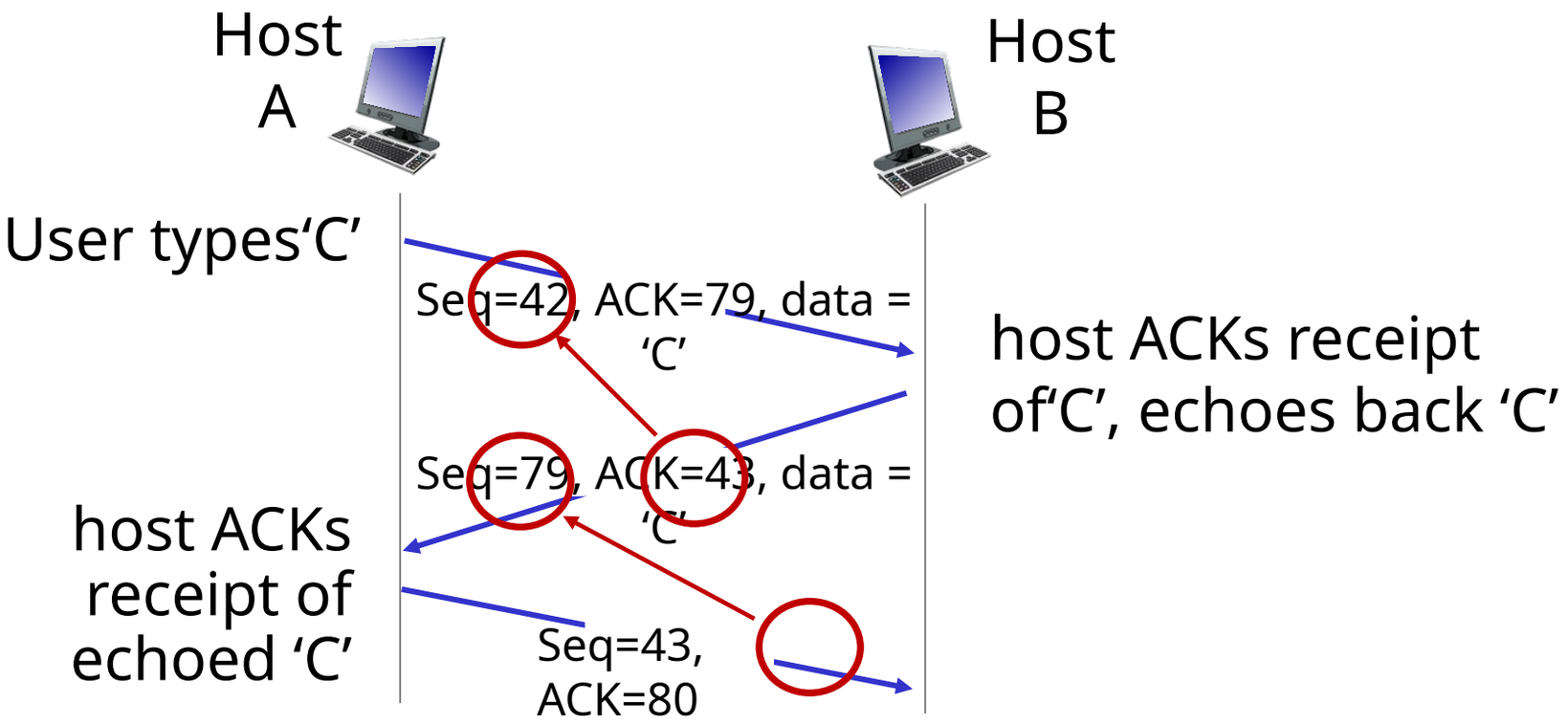
- seq # of next byte expected from other side
- cumulative ACK

*Q*: how receiver handles out-of-order segments

- *A:* TCP spec doesn't say, - up to implementor



# TCP sequence numbers, ACKs




simple telnet scenario

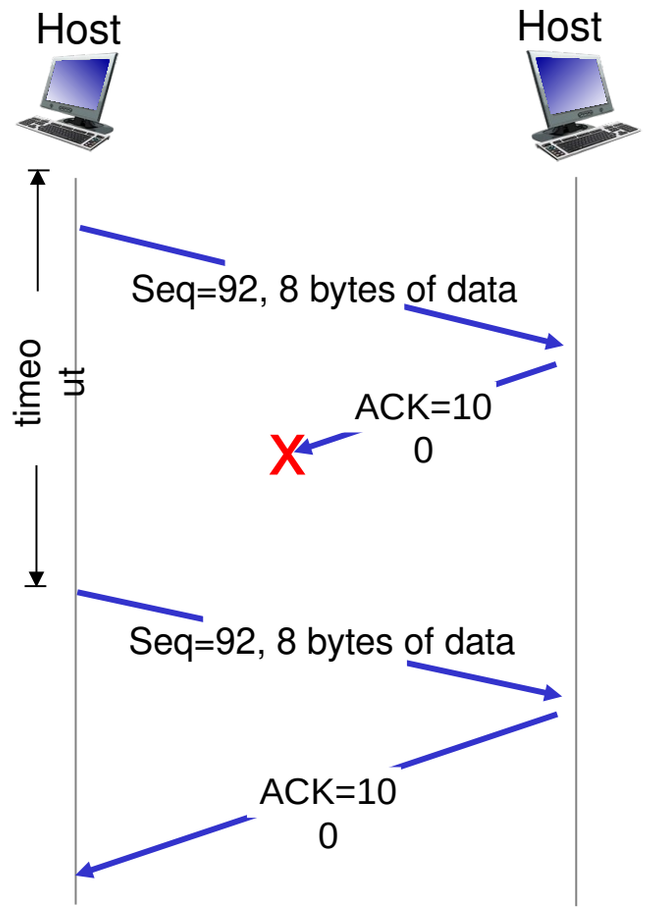
# TCP Receiver: ACK generation [RFC 5681]

| *Event at receiver* | *TCP receiver action* |
| --- | --- |
| to | send ACK |
| arrival of in-order segment | ACK, ACKing both in-order |
| | immediately send *duplicate ACK* |

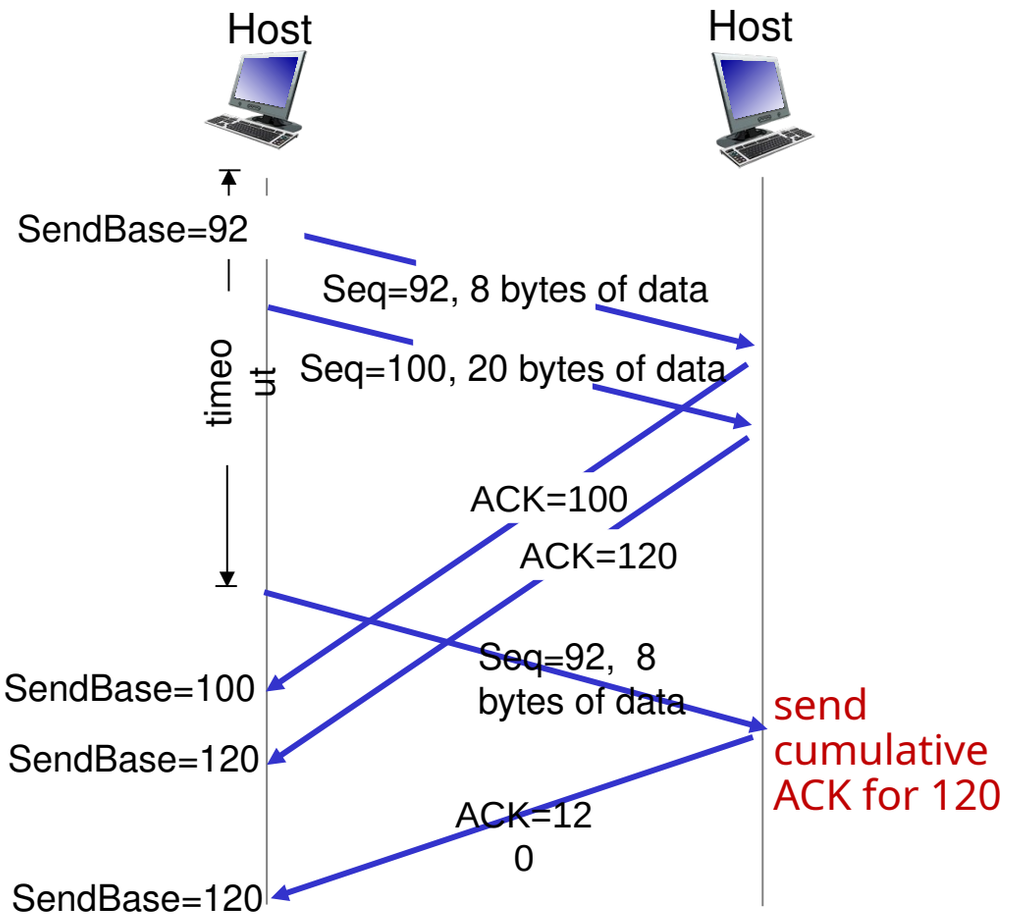
Gap detected

immediate send ACK, provided

# TCP: retransmission scenarios

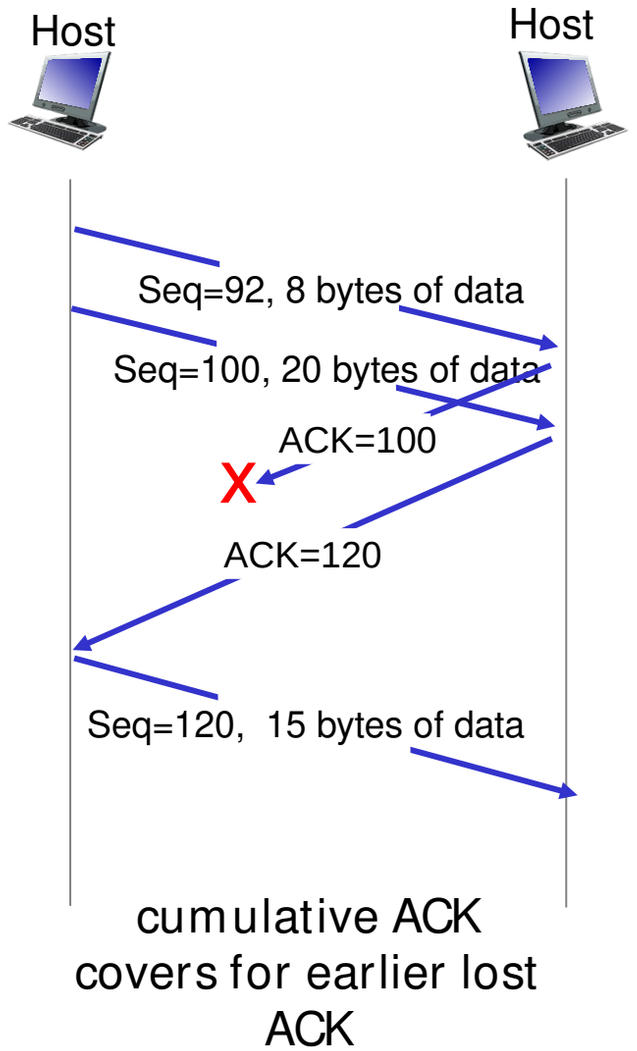



lost ACK scenario



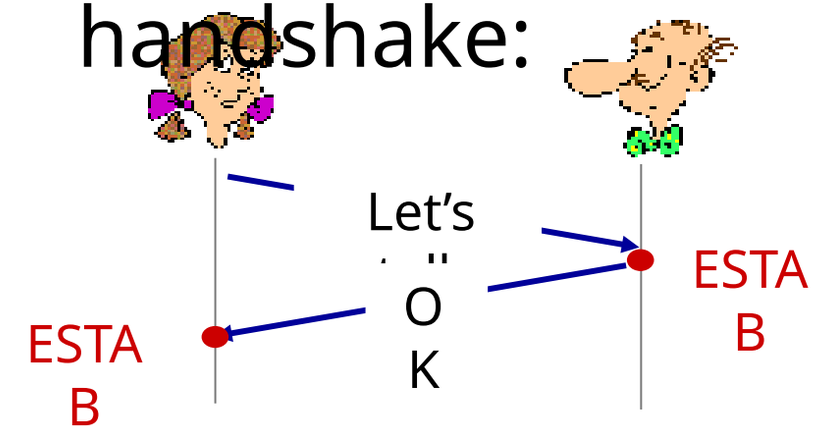

premature timeout

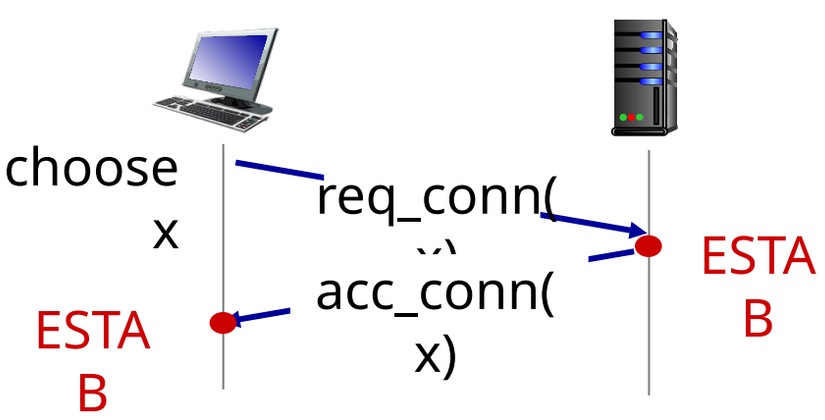
# TCP: retransmission scenarios



# Agreeing to establish a connection

2-way handshake:

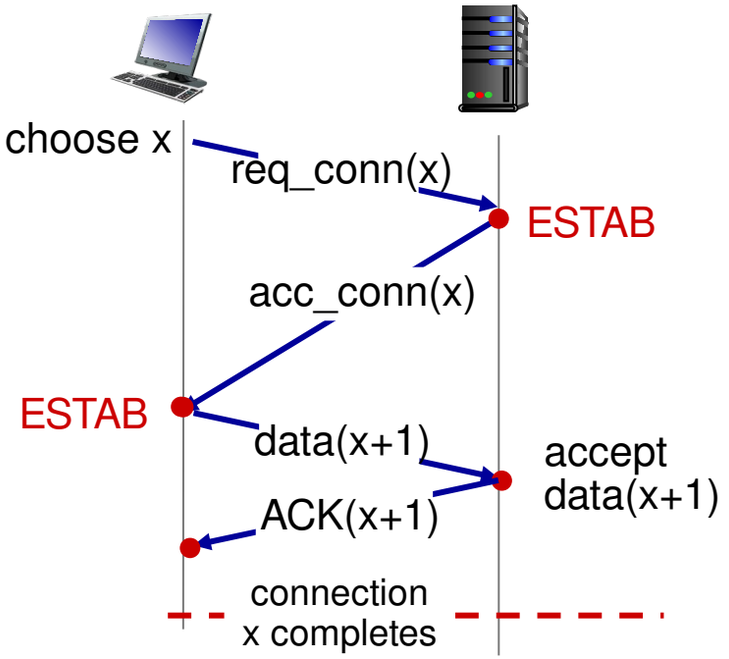



*Q:* will 2-way handshake always work in network?

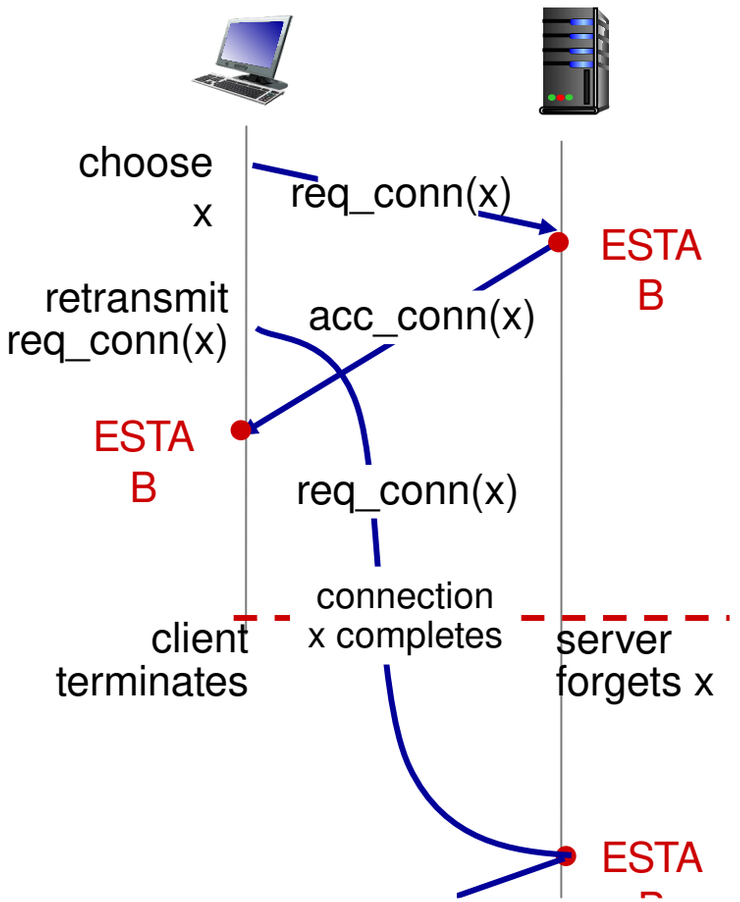
- variable delays
- retransmitted messages (e.g. req_conn(x)) due to message loss
- message reordering
- can't "see" other side

# 2-way handshake scenarios



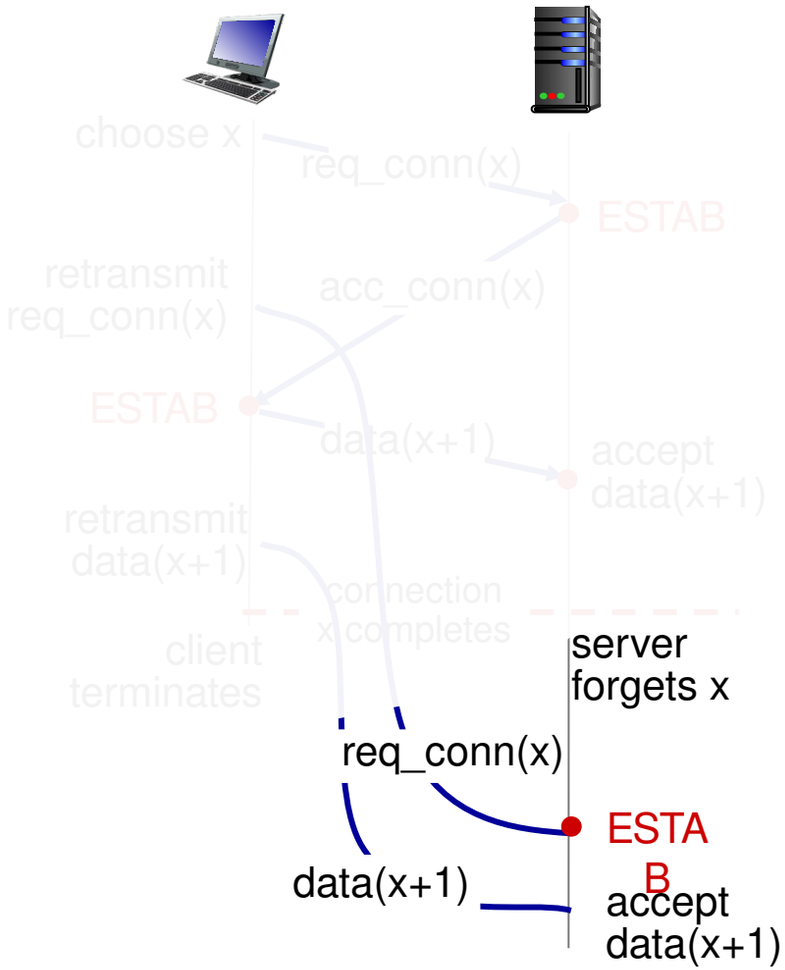

No problem!

# 2-way handshake scenarios



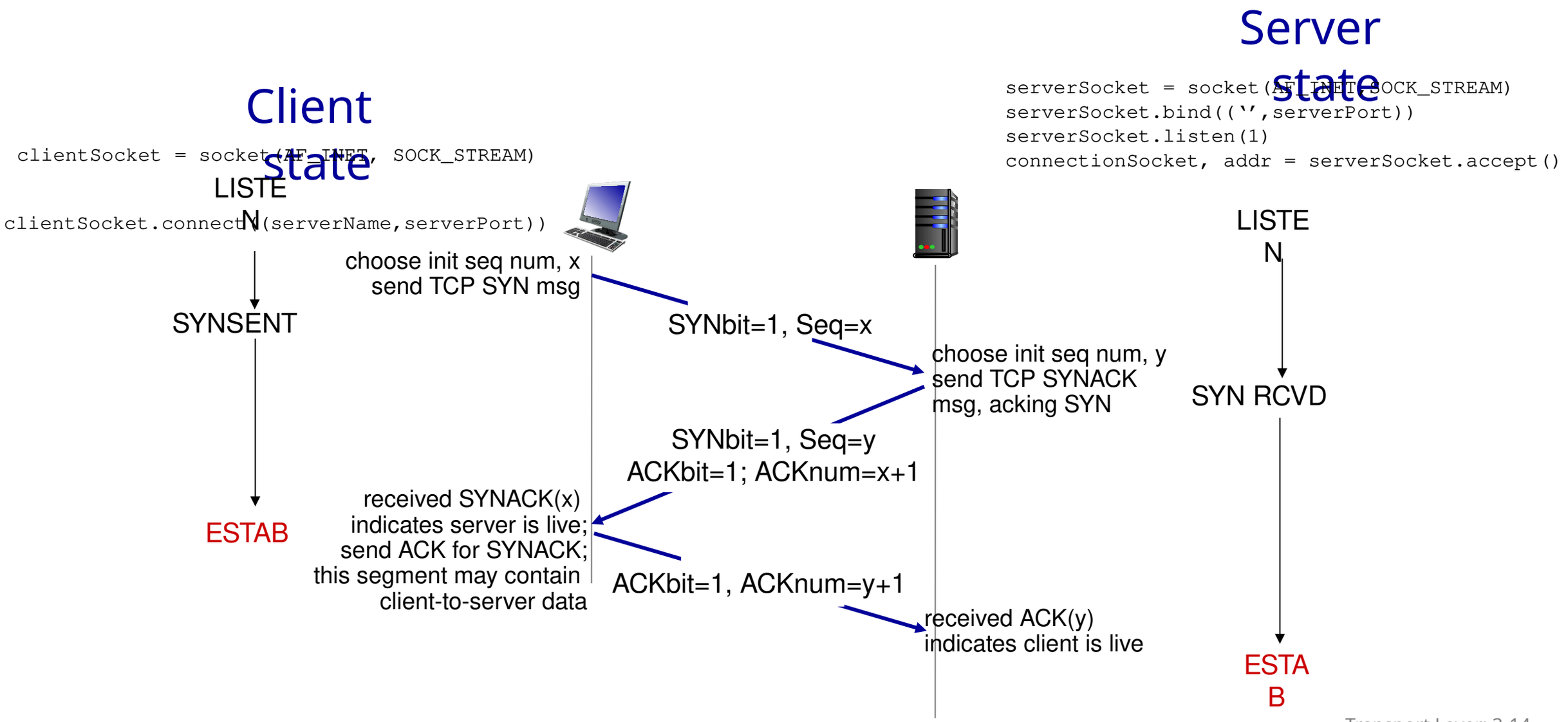

Problem: half open connection! (no client)

# 2-way handshake scenarios




Problem: dup da
accepted!

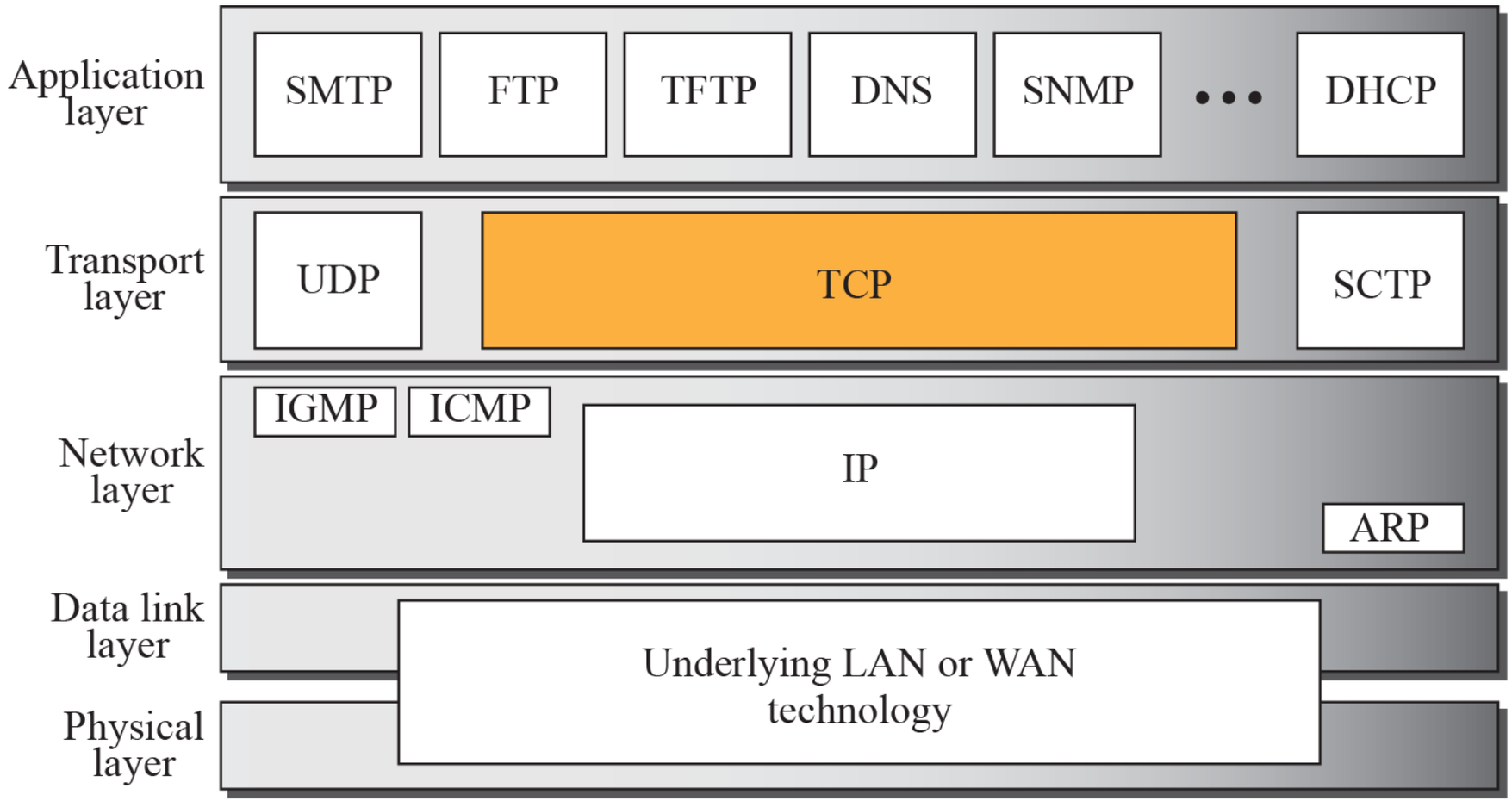
# TCP 3-way handshake



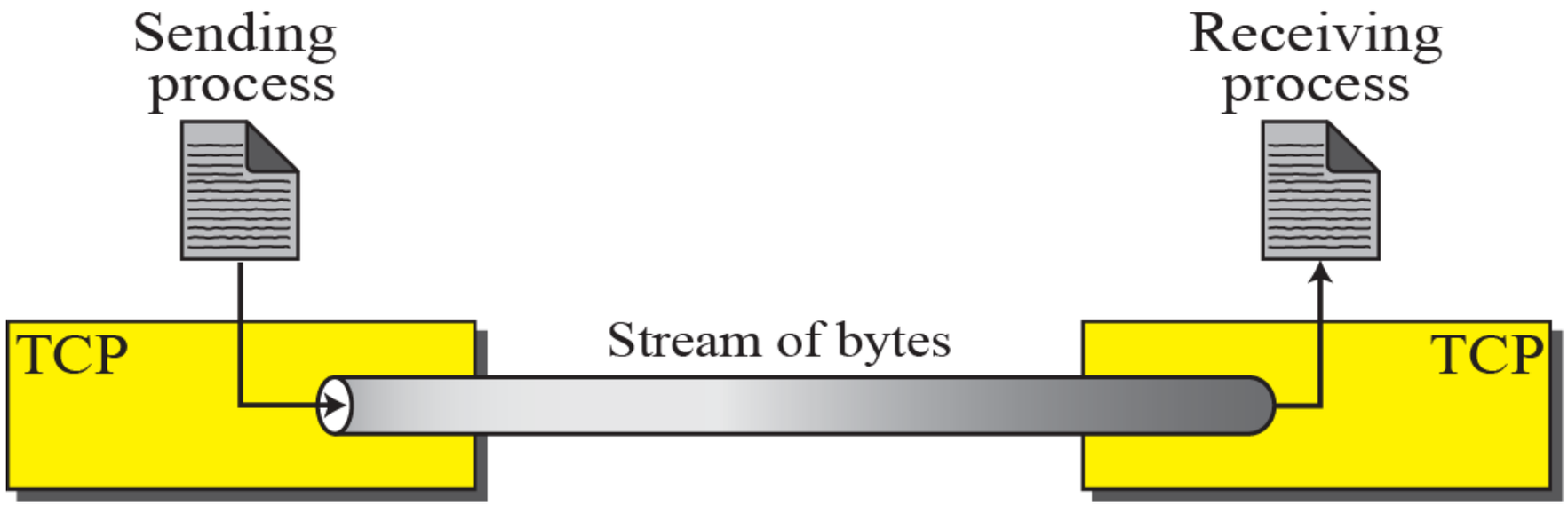
# Closing a TCP connection

- client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
  - on receiving FIN, ACK can be combined with own FIN
- simultaneous FIN exchanges can be handled

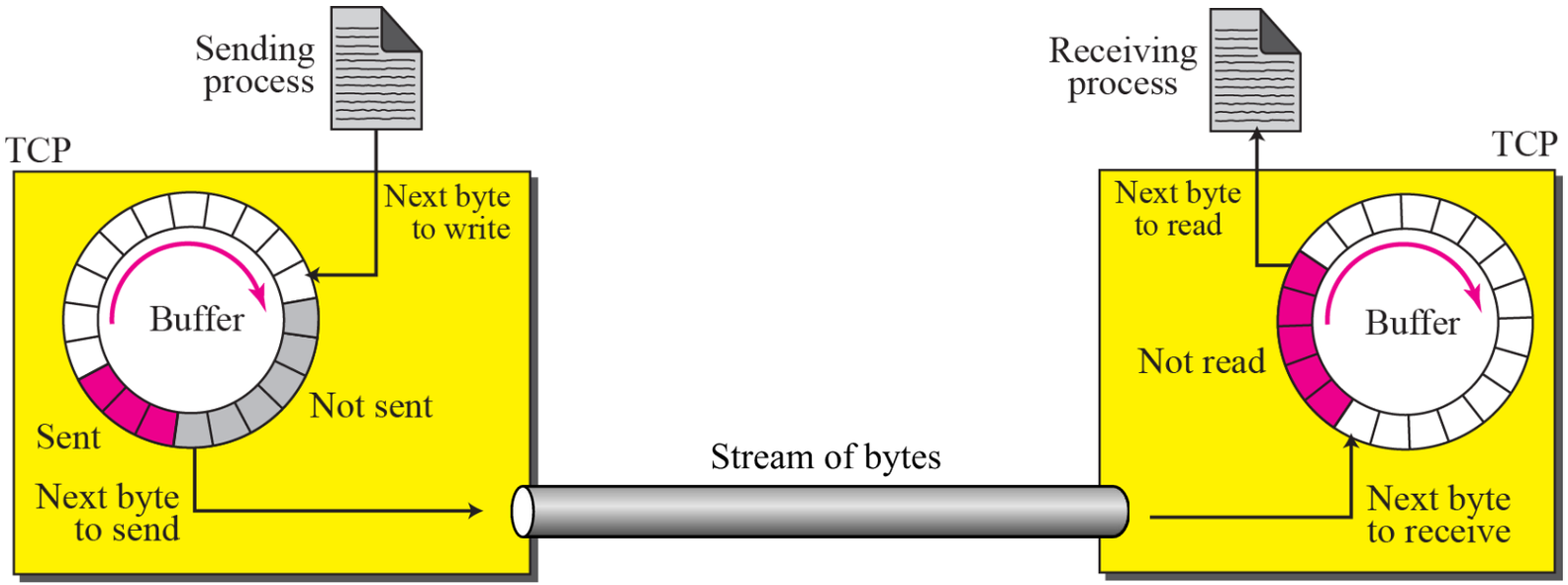
**Figure 15.1** *TCP/IP protocol suite*



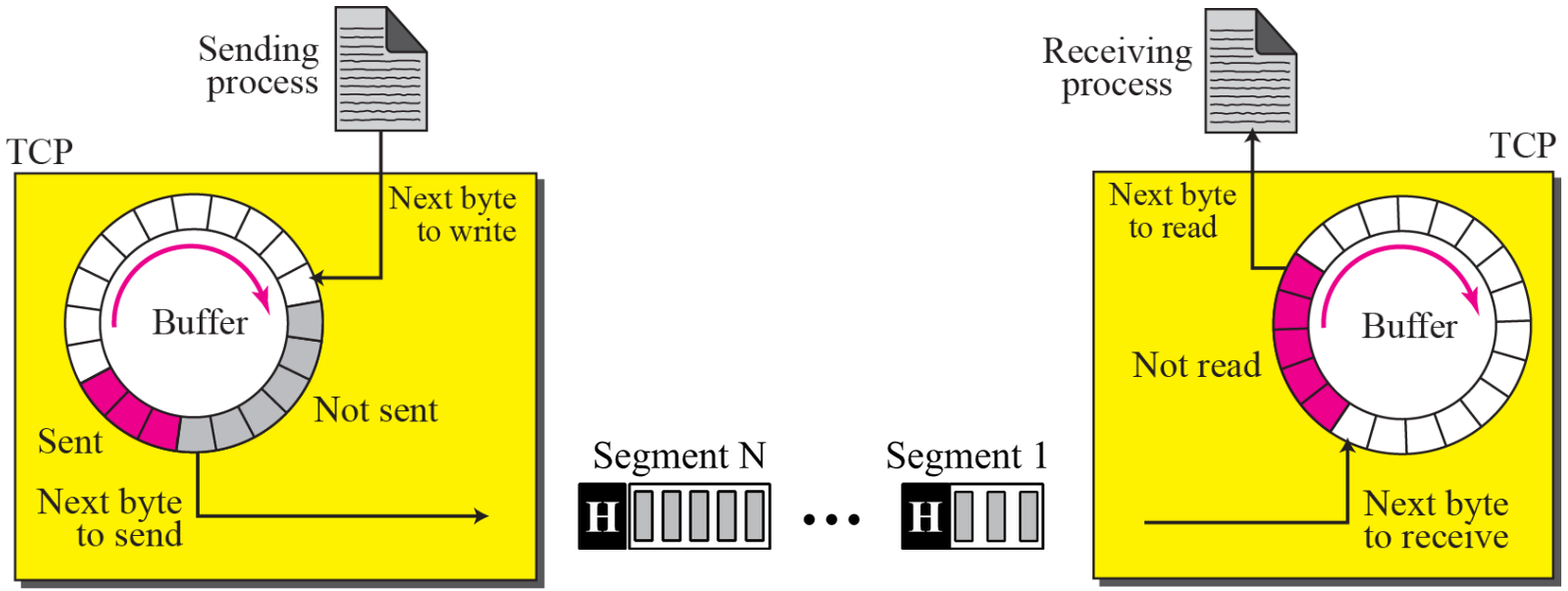
## Figure 15.2 *Stream delivery*



## Figure 15.3 *Sending and receiving buffers*



## Figure 15.4 *TCP segments*



# *Topics Discussed in the Section*

- ✔ **Numbering System**
- ✔ **Flow Control**
- ✔ **Error Control**
- ✔ **Congestion Control**

**Note**

***The bytes of data being transferred in each connection are numbered by TCP.***

***The numbering starts with an arbitrarily generated number.***

# Example 15.1

**Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?**

***Solution***

**The following shows the sequence number for each segment:**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Segment 1 | → | Sequence Number: | 10,001 | Range: | 10,001 | to | 11,000 |
| Segment 2 | → | Sequence Number: | 11,001 | Range: | 11,001 | to | 12,000 |
| Segment 3 | → | Sequence Number: | 12,001 | Range: | 12,001 | to | 13,000 |
| Segment 4 | → | Sequence Number: | 13,001 | Range: | 13,001 | to | 14,000 |
| Segment 5 | → | Sequence Number: | 14,001 | Range: | 14,001 | to | 15,000 |

**Note**

***The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.***

**Note**

***The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive.***

***The acknowledgment number is cumulative.***

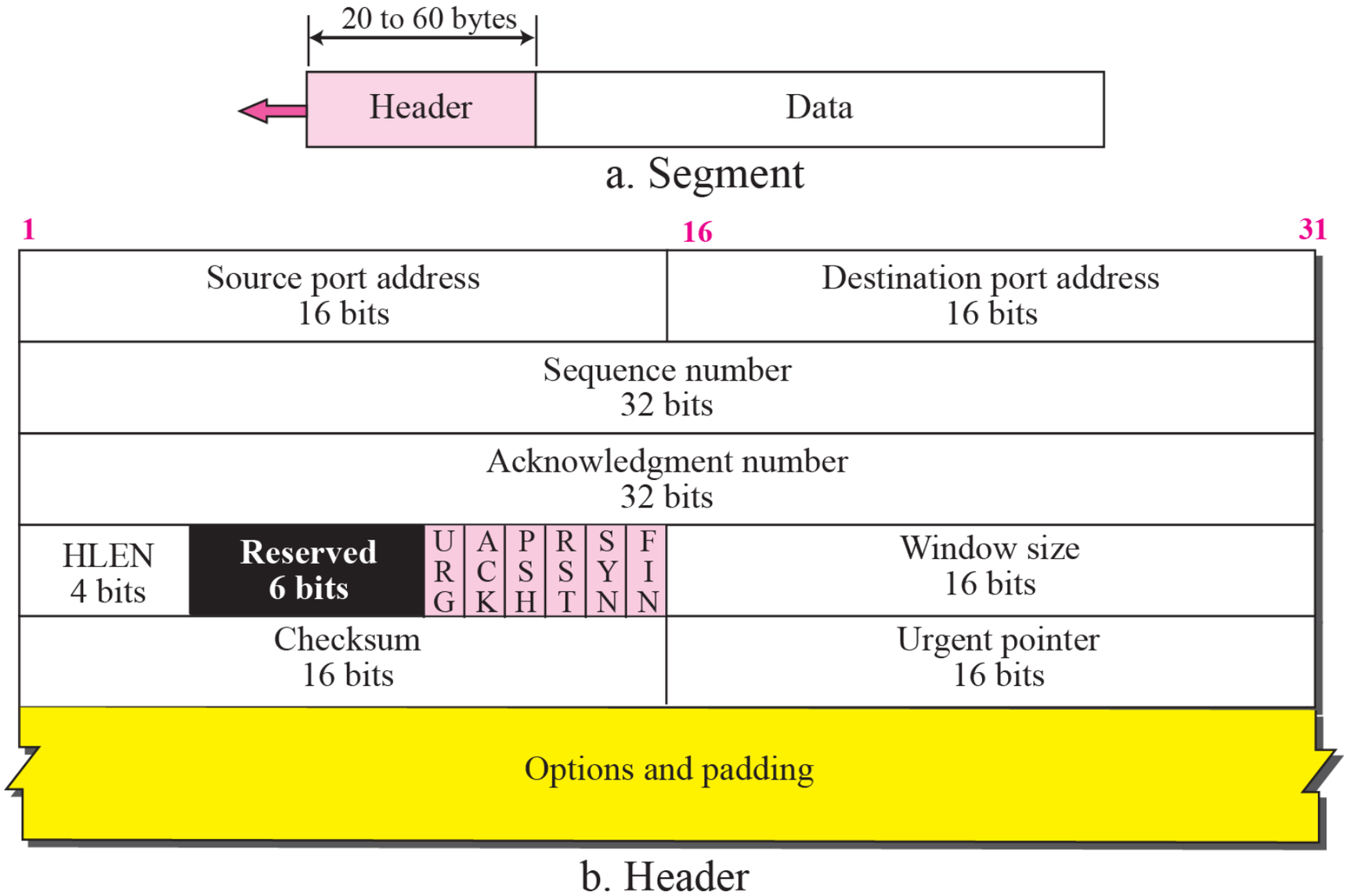
# 15-3 SEGMENT

**Before discussing TCP in more detail, let us discuss the TCP packets themselves. A packet in TCP is called a segment.**

# *Topics Discussed in the Section*

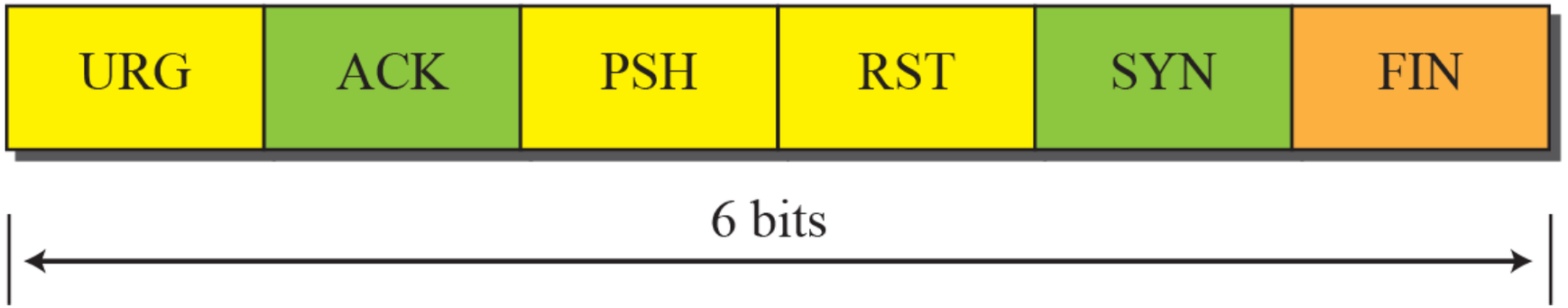
- ✔ **Format**
- ✔ **Encapsulation**

## Figure 15.5 *TCP segment format*

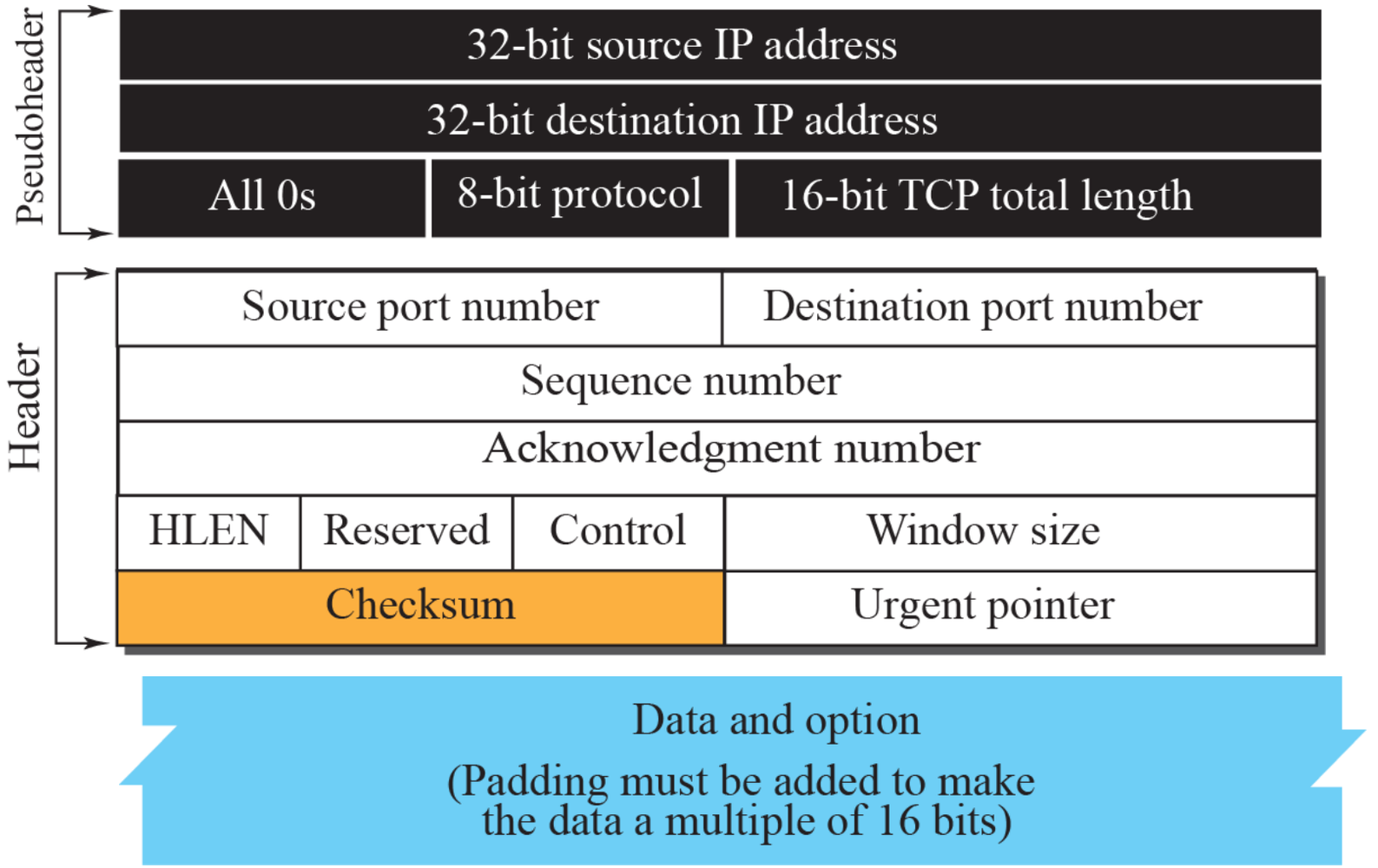


## Figure 15.6 *Control field*

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push
RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



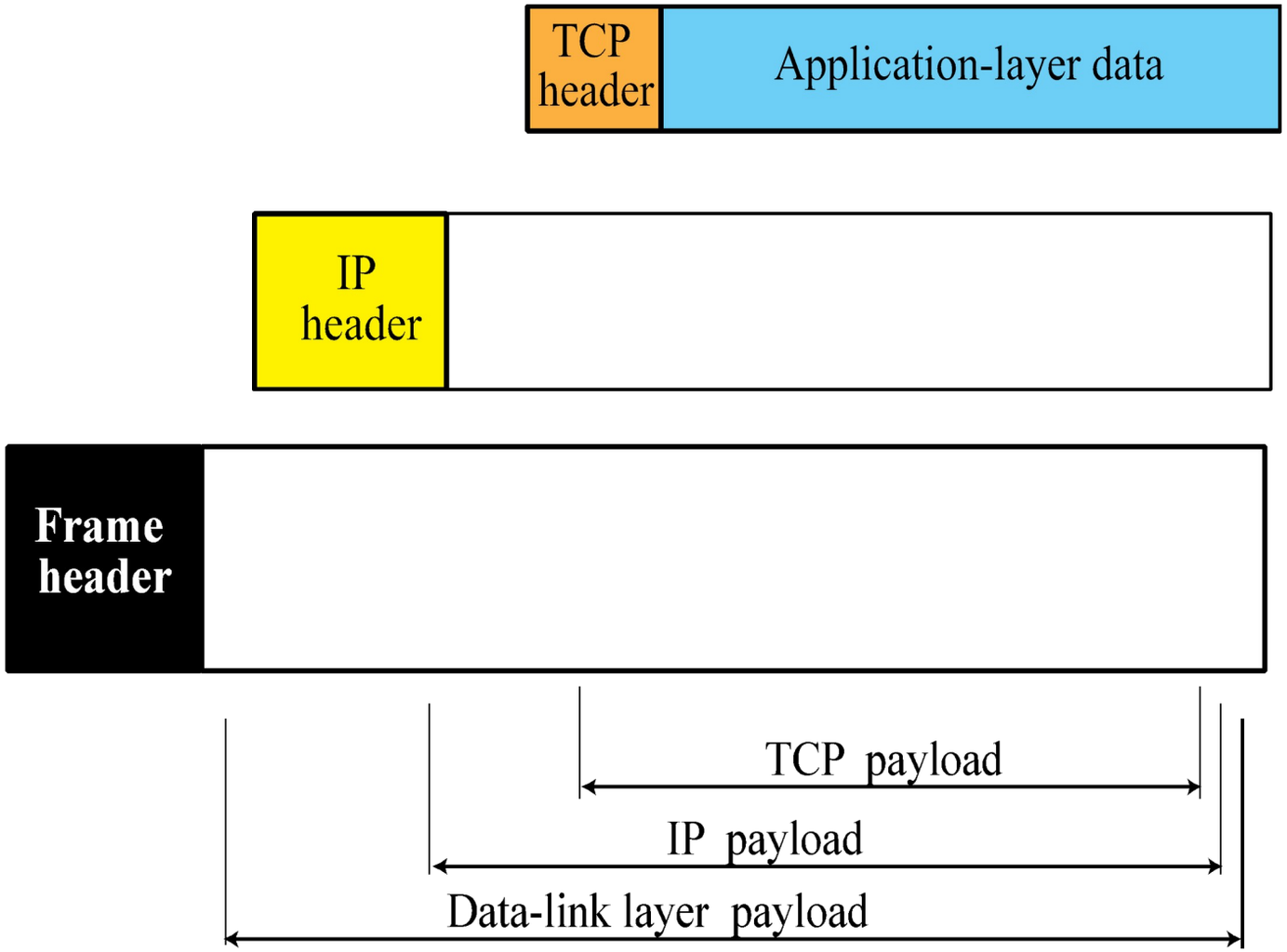
## Figure 15.7 *Pseudoheader added to the TCP segment*



**Note**

***The use of the checksum in TCP is mandatory.***

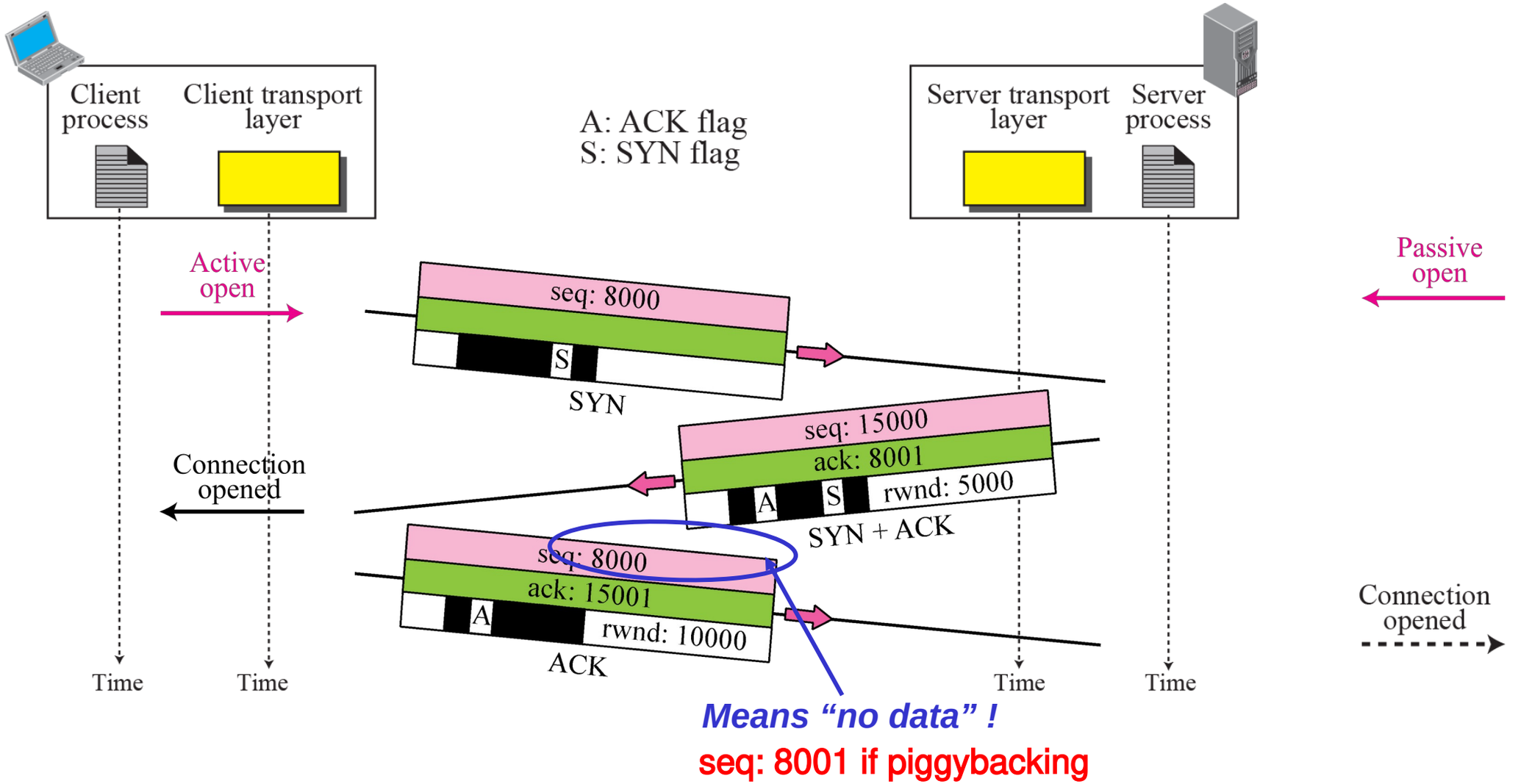
## Figure 15.8 *Encapsulation*



# *Topics Discussed in the Section*

- ✔ **Connection Establishment**
- ✔ **Data Transfer**
- ✔ **Connection Termination**
- ✔ **Connection Reset**

## Figure 15.9 *Connection establishment using three-way handshake*



**Note**

***A SYN segment cannot carry data, but it consumes one sequence number.***

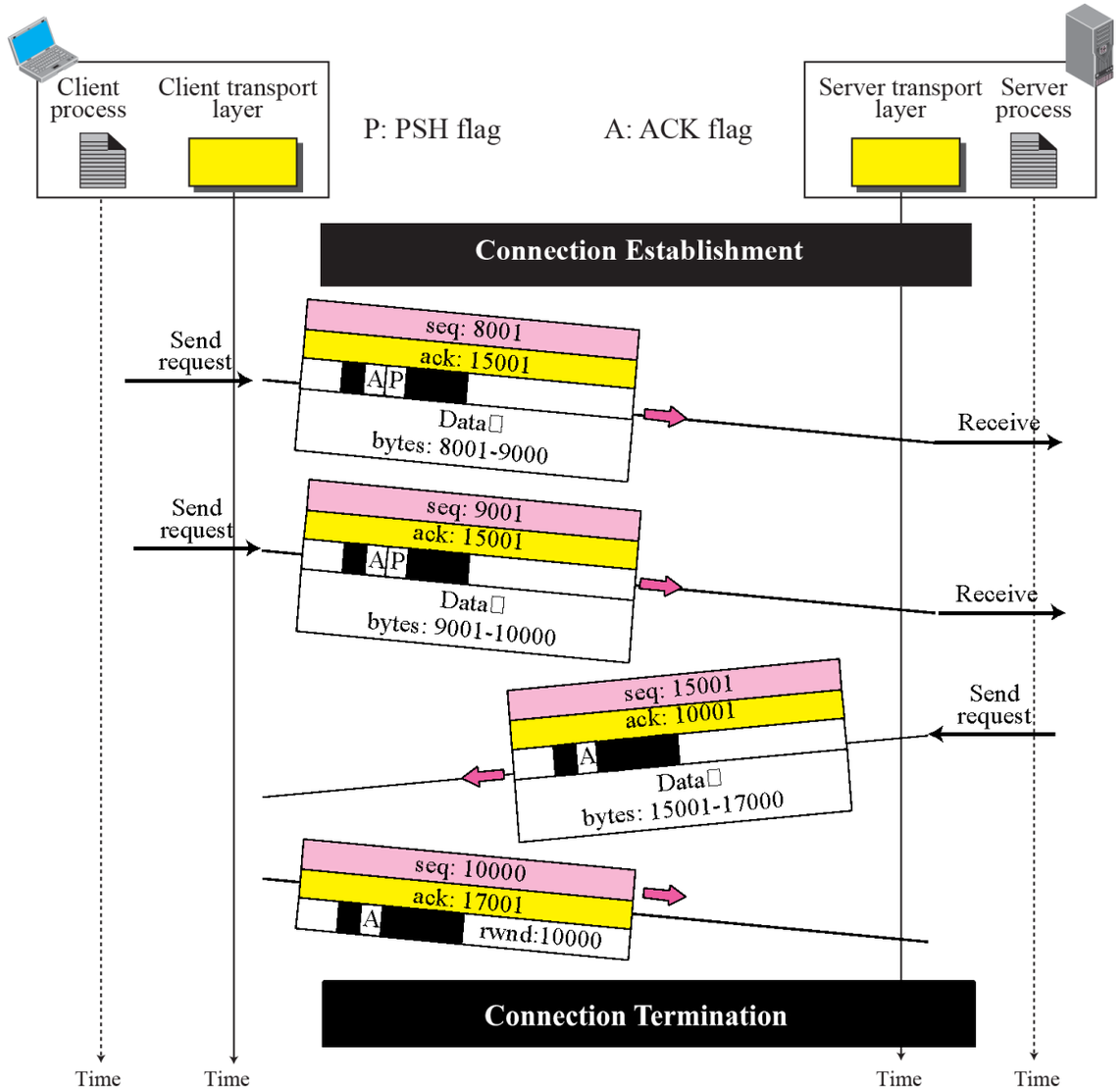
**Note**

***A SYN + ACK segment cannot carry data, but does consume one sequence number.***

**Note**

***An ACK segment, if carrying no data, consumes no sequence number.***

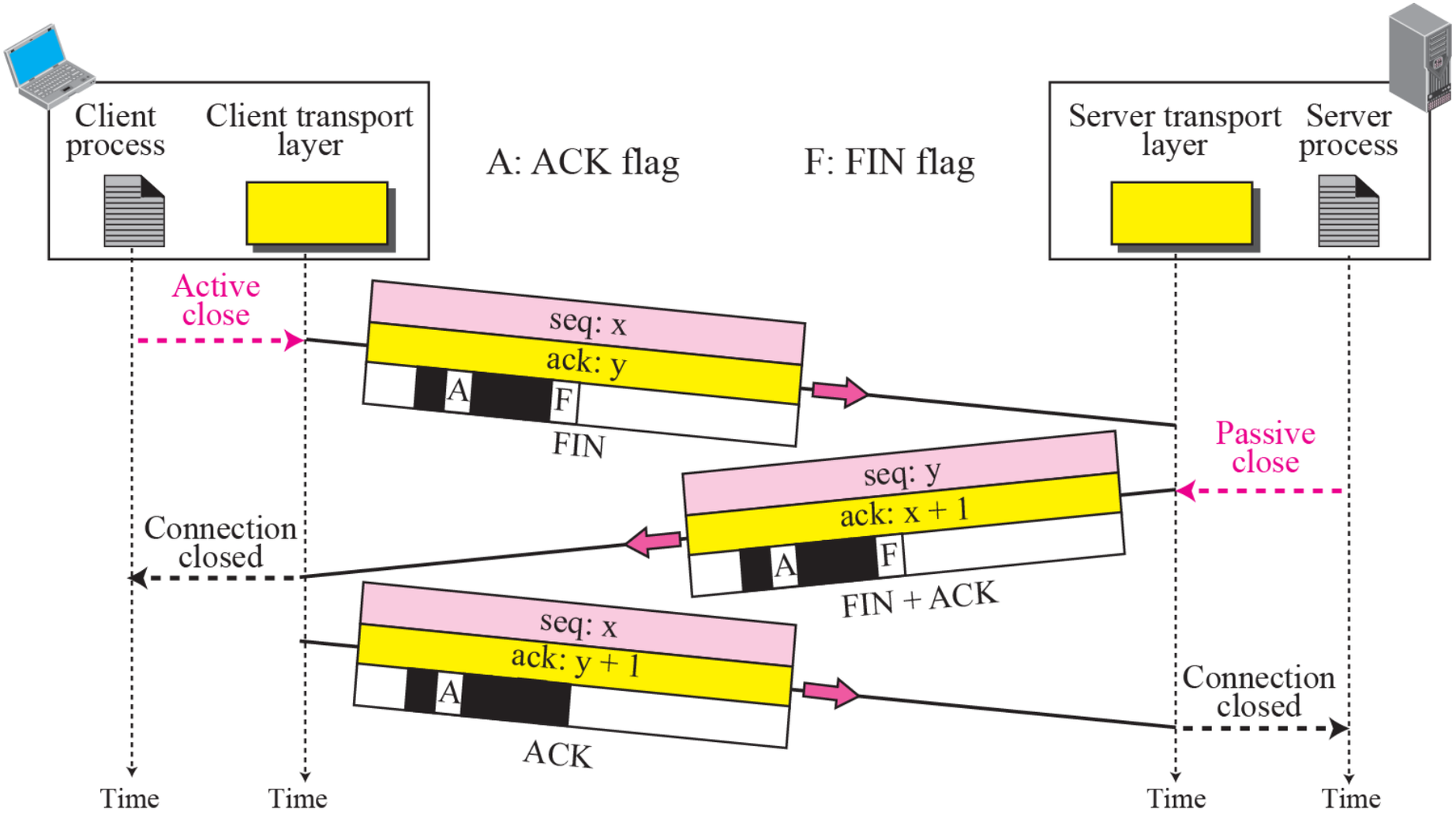
## Figure 15.10 *Data Transfer*




Pushing data

Urgent data

**Figure 15.11** *Connection termination using three-way handshake*



**Note**

***The FIN segment consumes one sequence number if it does not carry data.***

**Note**

***The FIN + ACK segment consumes one sequence number if it does not carry data.***

# Chapter 3
# Transport Layer
## TCP - Flow Control

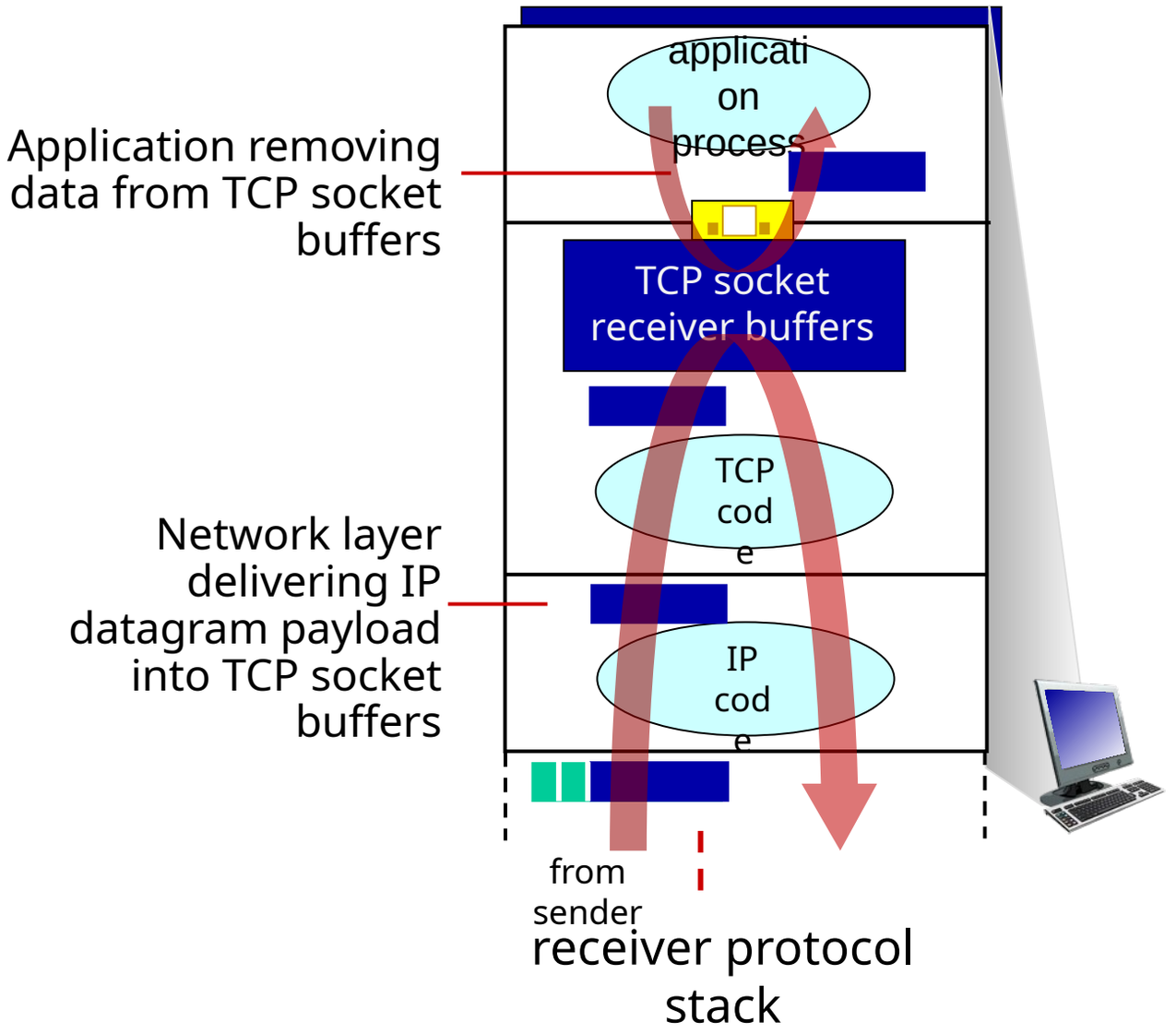
# Chapter 3: roadmap

- Transport-layer services
- Principles of reliable data transfer
- Connection-oriented transport: TCP
  - segment structure
  - reliable data transfer
  - flow control
  - connection management
- Principles of congestion control
- TCP congestion control



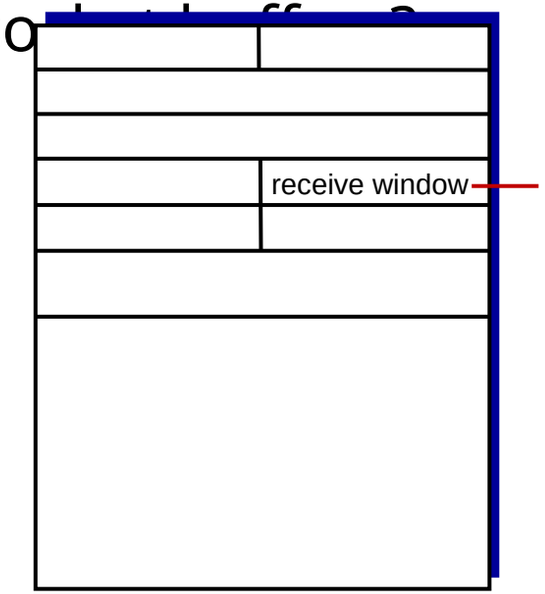
# TCP flow control

*Q:* What happens if network layer delivers data faster than application layer removes data from socket buffers?



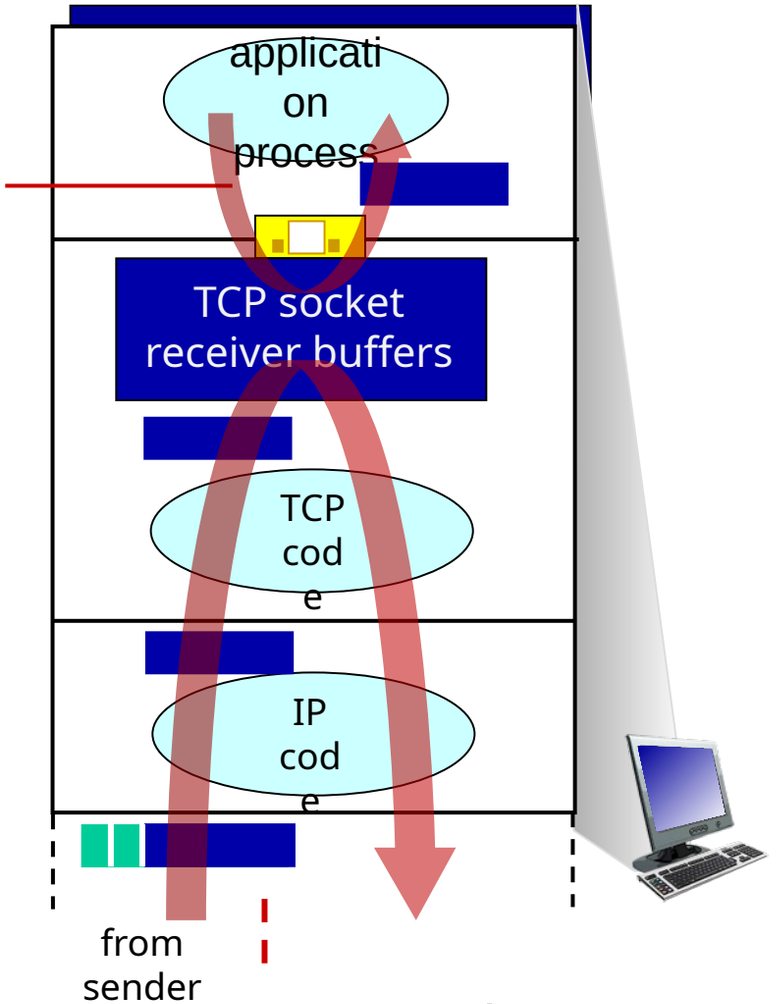
# TCP flow control

*Q:* What happens if network layer delivers data faster than application layer removes data from so




flow control: # bytes receiver willing to accept

Application removing data from TCP socket buffers



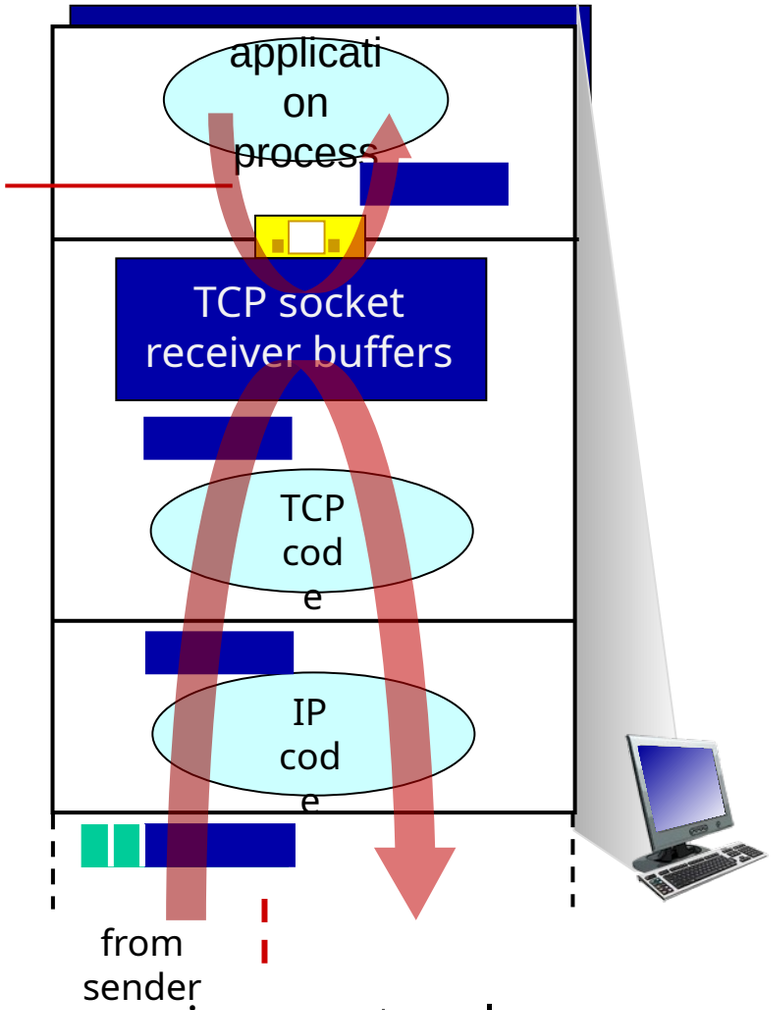

receiver protocol stack

# TCP flow control

*Q:* What happens if network layer delivers data faster than application layer removes data from socket buffers?

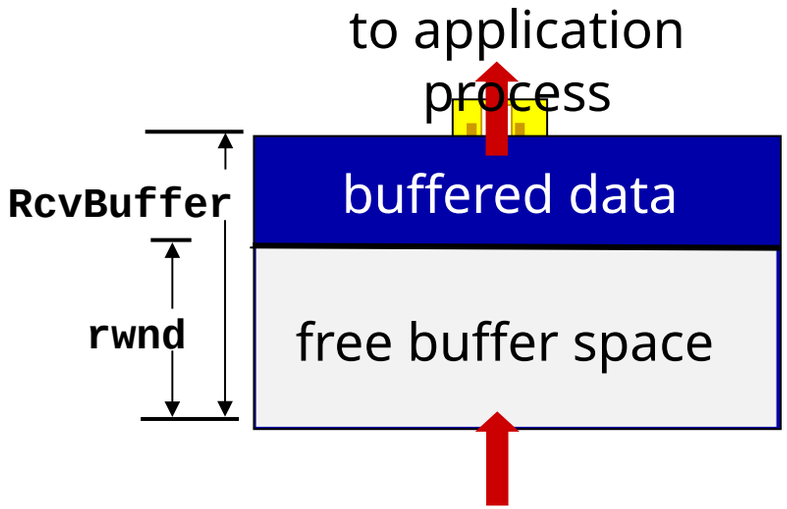
flow control

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast



# TCP flow control

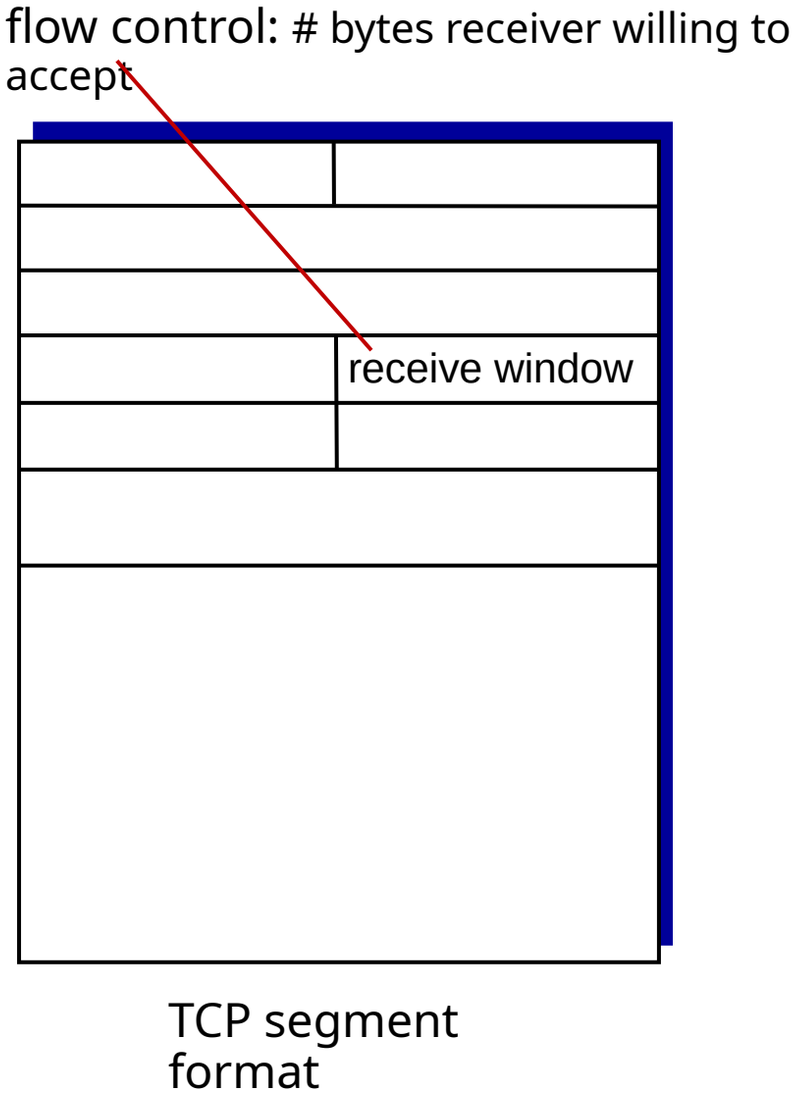
- TCP receiver “advertises” free buffer space in **`rwnd`** field in TCP header
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems auto-adjust **`RcvBuffer`**
- sender limits amount of unACKed (“in-flight”) data to received **`rwnd`**
- guarantees receive buffer will not overflow




TCP receiver-side buffering

# TCP flow control

- TCP receiver “advertises” free buffer space in **`rwnd`** field in TCP header
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems auto-adjust **`RcvBuffer`**
- sender limits amount of unACKed (“in-flight”) data to received **`rwnd`**
- guarantees receive buffer will not overflow




TCP segment format

# 15-6 WINDOWS IN TCP

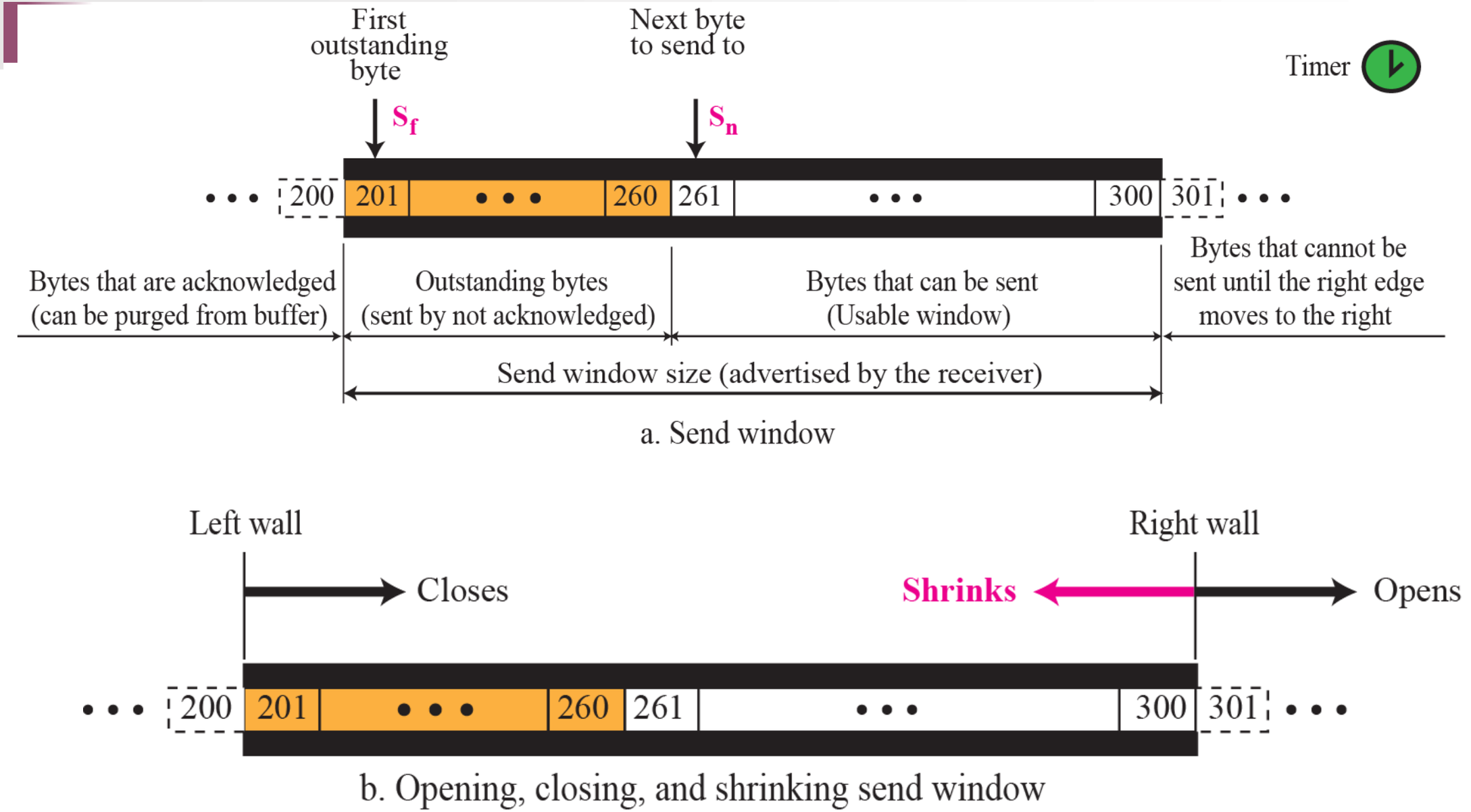
**TCP uses two windows (send window and receive window) for each direction of data transfer, which means four windows for a bidirectional communication.**

**To make the discussion simple, we make an assumption that communication is only unidirectional; the bidirectional communication can be inferred using two unidirectional communications with piggybacking.**

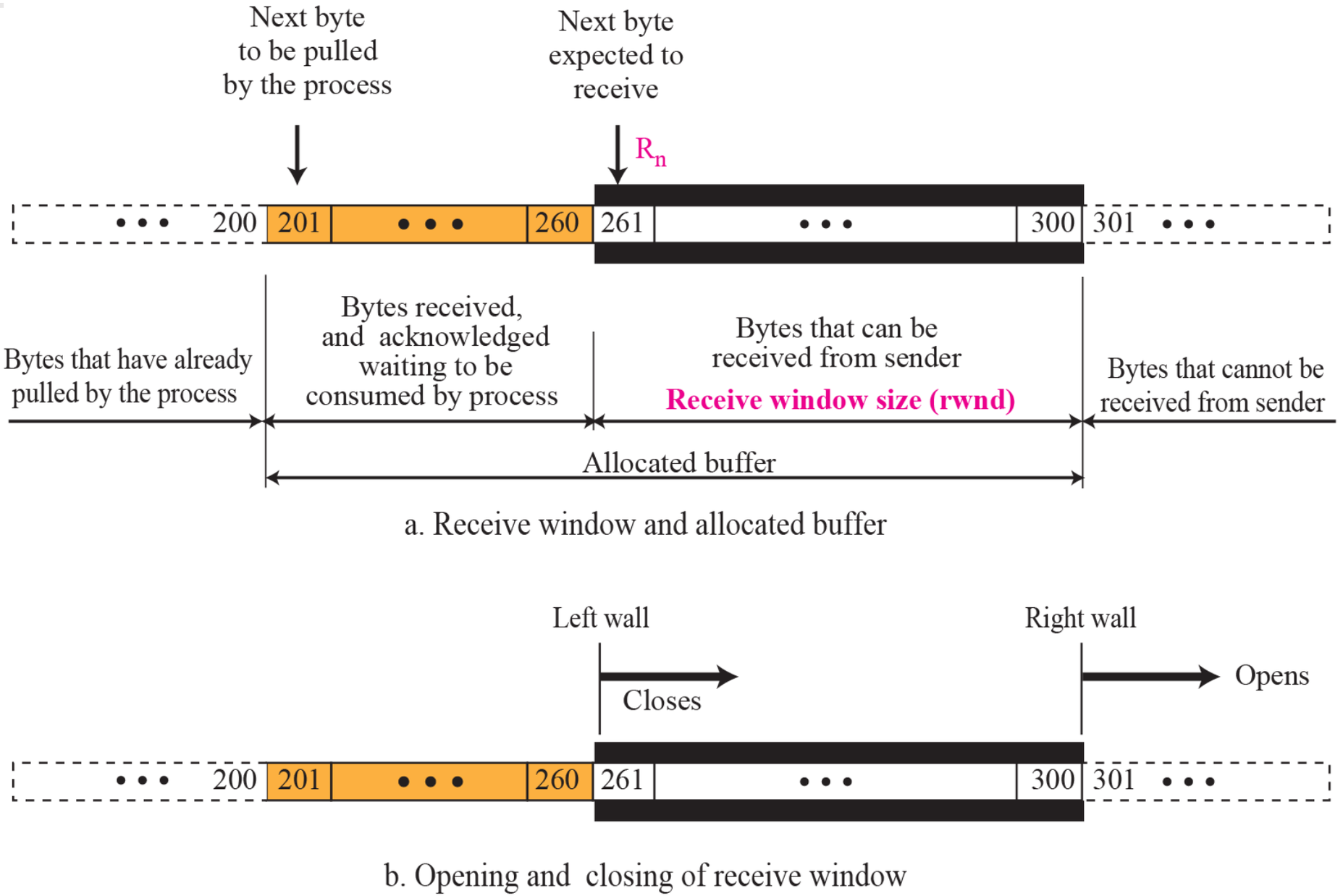
# *Topics Discussed in the Section*

- ✔ **Send Window**
- ✔ **Receive Window**

## Figure 15.22 *Send window in TCP*

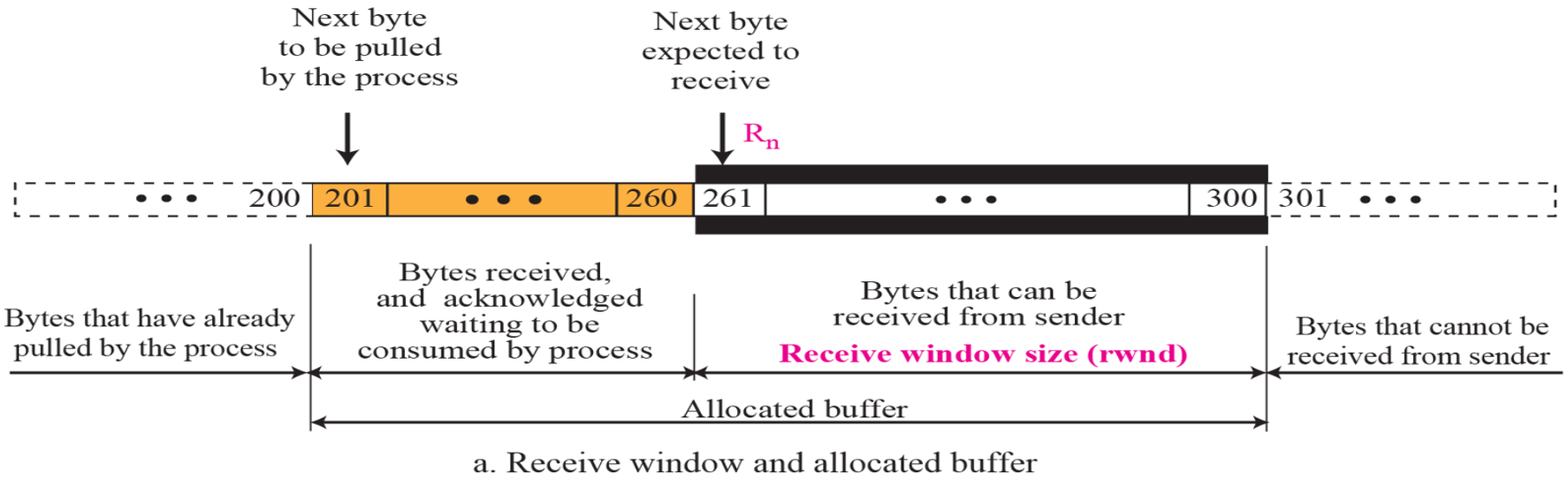


## Figure 15.23 *Receive window in TCP*



# Flow control

- Receiver window




a. Receive window and allocated buffer

- Window can -
  - Open
  - Close
  - Shrink
  - Shutdown

- spare room in buffer

**= advertizedRcvWindow**

**= RcvBuffer-[LastByteRcvd - LastByteRead]**

# TCP Flow control: how it works

- Sender maintain a variable **`RcvWindow`**
- Rcvr advertises spare room by including value of **`RcvWindow`** in segments during ack
- Sender limits unACKed data to **`RcvWindow`**
  - guarantees receive buffer doesn't overflow

$$(\text{LastByteSent} - \text{LastByteAckd}) \leq \text{RecvWindow}$$

$$(Sn - 1) - (Sf - 1) \leq \text{RecvWindow}$$

# Flow control

- The amount of data a source can send before receiving an ACK from the destination
- Whether to send 1 byte of data and wait for ACK or send all bytes and wait for the ACK for the complete message?
- TCP gives a solution in between
  - Sliding window protocol
    - byte oriented

# 15-7 FLOW CONTROL

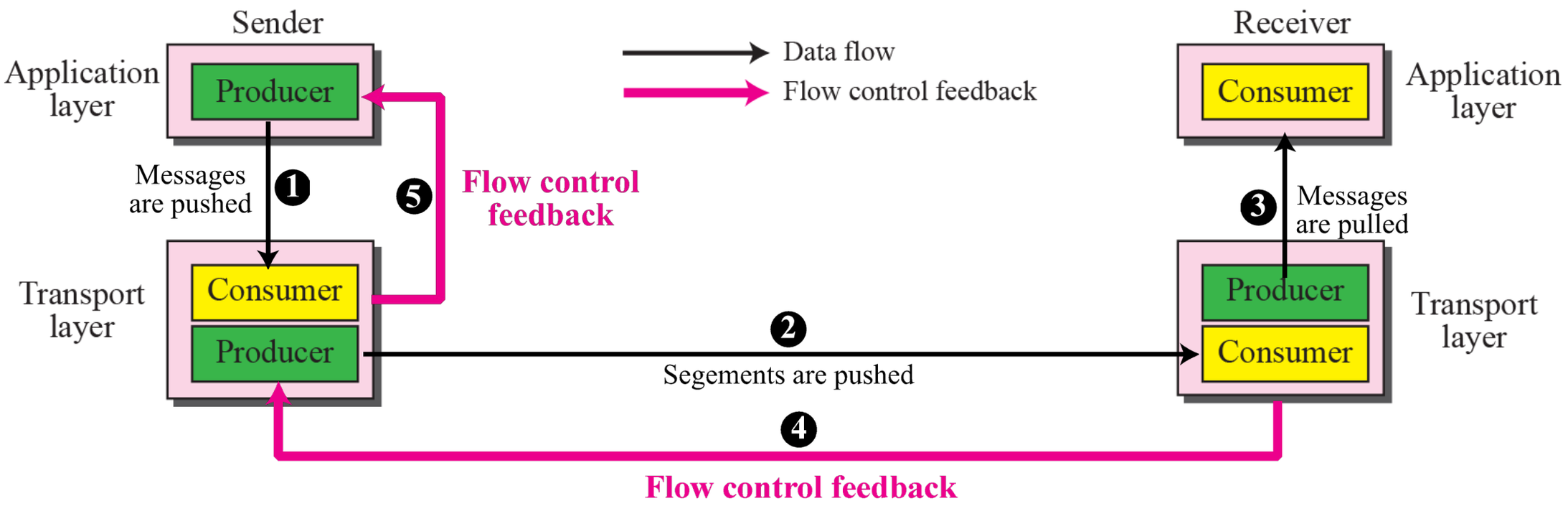
**Flow control balances the rate a producer creates data with the rate a consumer can use the data.**

**Figure 15.24 shows unidirectional data transfer between a sender and a receiver; bidirectional data transfer can be deduced from unidirectional one as discussed in Chapter 13.**

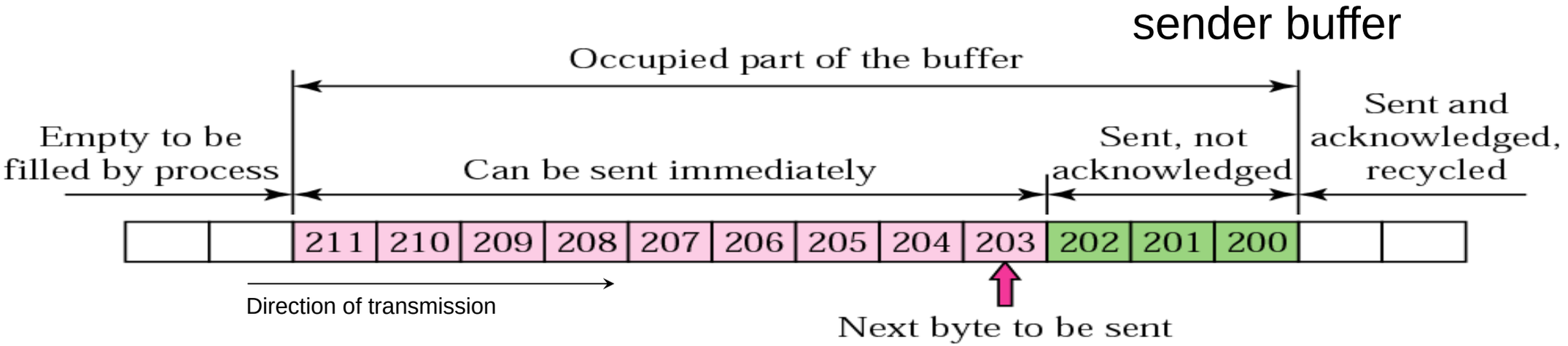
# *Topics Discussed in the Section*

- ✔ **Opening and Closing Windows**
- ✔ **Shrinking of Windows**
- ✔ **Silly Window Syndrome**

## Figure 15.24 *TCP/IP protocol suite*



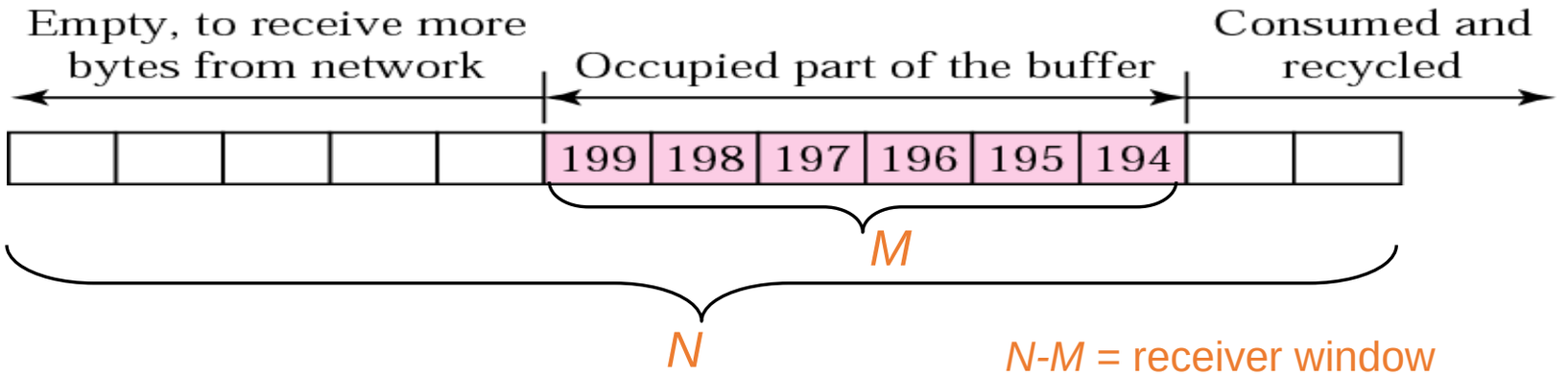
# Flow control



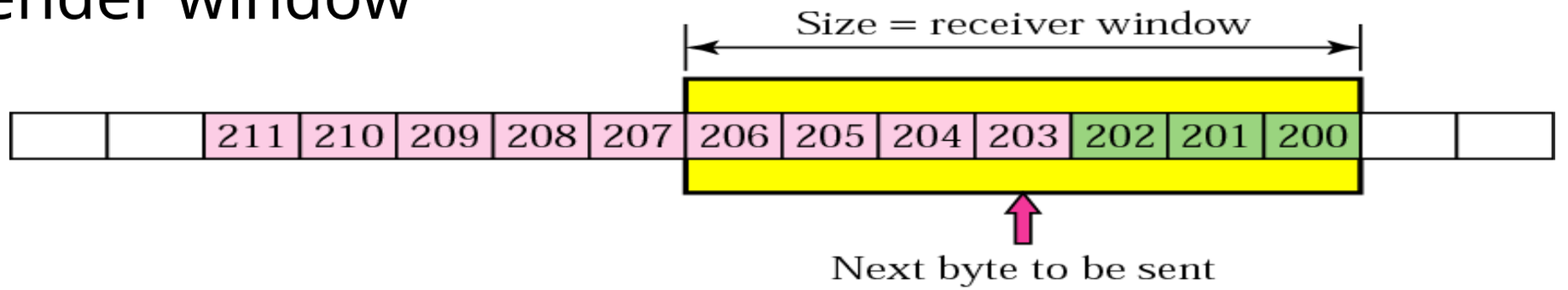

- If no window, a sender can send all bytes without regarding the condition of the receiver
  - if data are consumed too slowly then receiver buffer becomes full -> drop the packet (retransmit)
  - Hence, the sender must adjust itself to the number of the free locations in the receiver buffer

# Flow control

- Receiver window

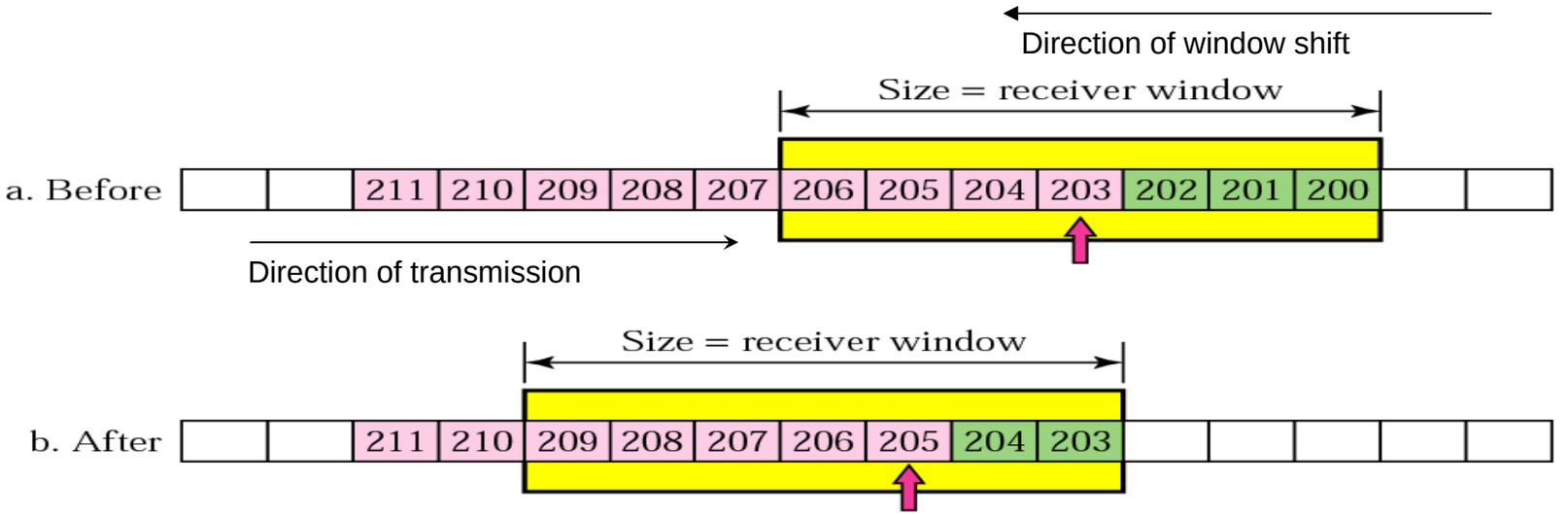



- Sender window

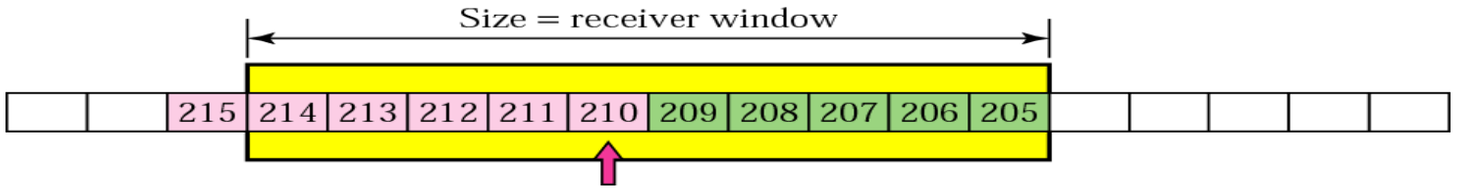


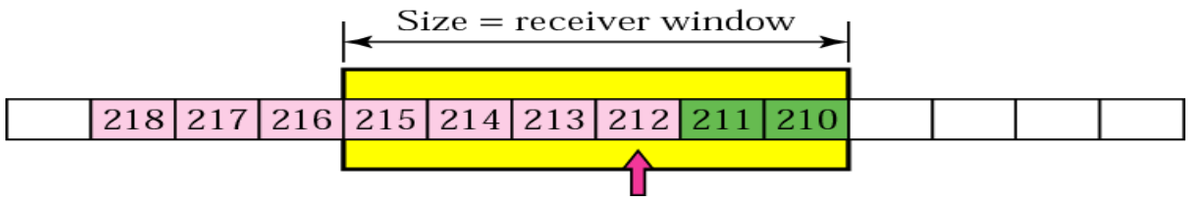
# Flow control

- sliding widow
- ACK is received for bytes 200-202
- the sender can send bytes 205-209



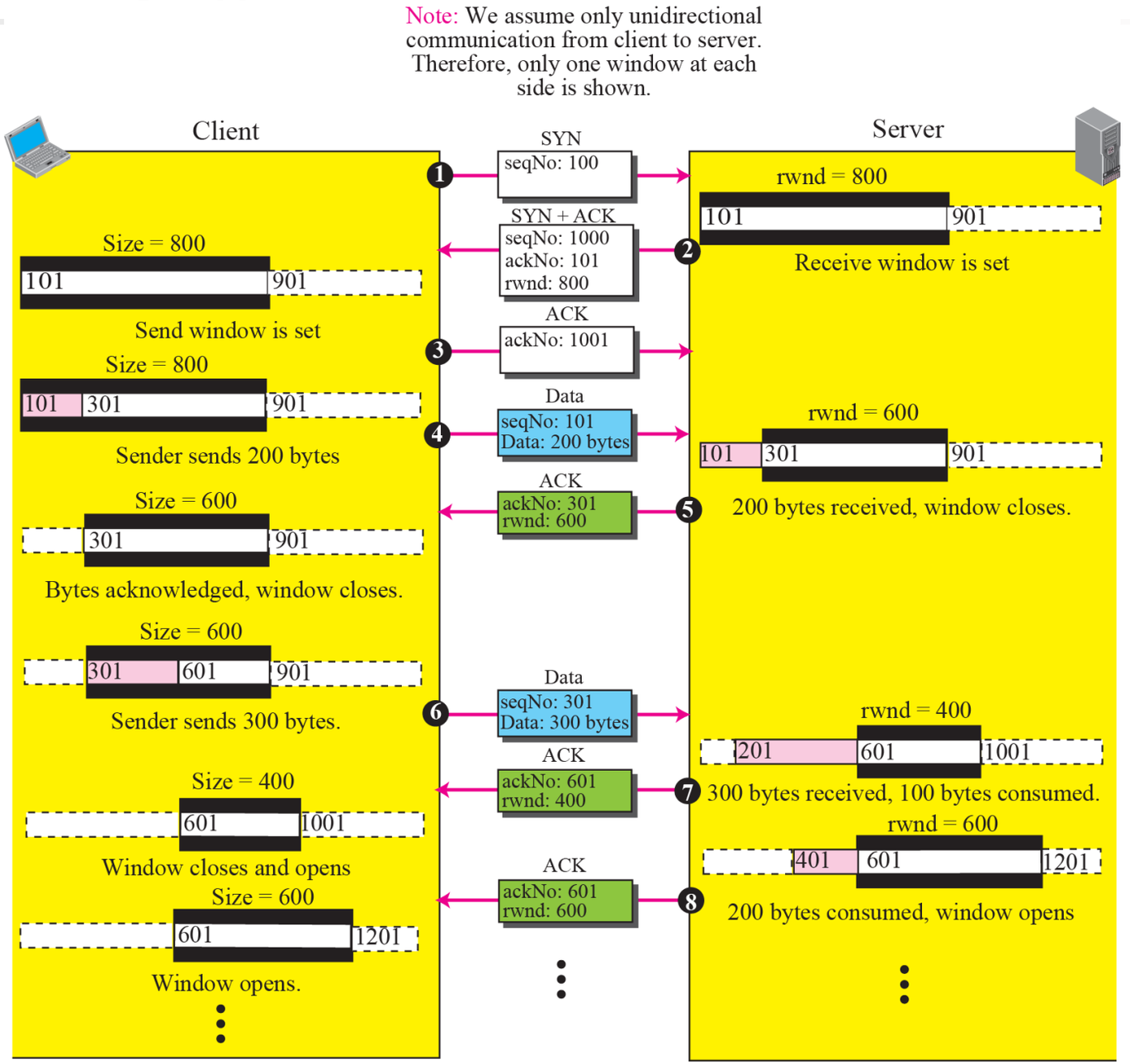

- expanding the sender widow




- shrinking the sender widow

## Figure 15.25 *An example of flow control*

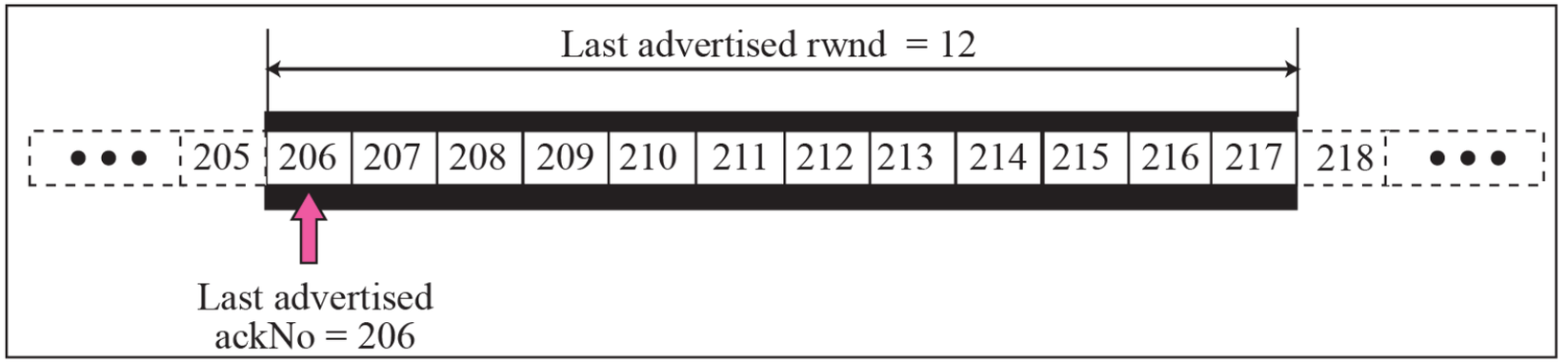


# Example 15.2

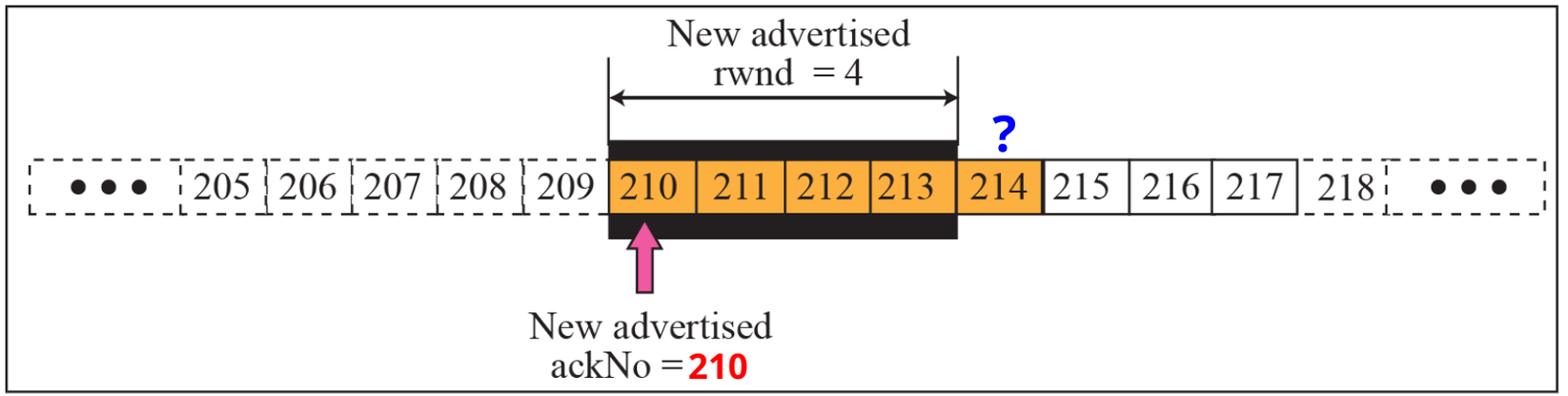
**Figure 15.26 shows the reason for the mandate in window shrinking. Part a of the figure shows values of last acknowledgment and *rwnd*. Part b shows the situation in which the sender has sent bytes 206 to 214. Bytes 206 to 209 are acknowledged and purged. The new advertisement, however, defines the new value of *rwnd* as 4, in which 210 + 4 < 206 + 12. When the send window shrinks, it creates a problem: byte 214 which has been already sent is outside the window. The relation discussed before forces the receiver to maintain the right-hand wall of the window to be as shown in part a because the receiver does not know which of the bytes 210 to 217 has already been sent. One way to prevent this situation is to let the receiver postpone its feedback until enough buffer locations are available in its window. In other words, the receiver should wait until more bytes are consumed by its process.**

## Figure 15.26 *Example 15.2*

**Prevent the shrinking of the send window:**
**new ackNo + new rwnd >= last ackNo + last rwnd**




a. The window after the last advertisement




b. The window after the new advertisement; window has shrunk

# TCP Flow Control

- Problem
  - Receiver advertise **`RecvWindow = 0`**
  - Window shutdown

# Silly Window Syndrome (1)

➢ **Sending data in very small segments**

1. Syndrome created by the Sender
   - Sending application program creates data slowly (e.g. 1 byte at a time)
   - Wait and collect data to send in a larger block
   - How long should the sending TCP wait?

**Solution:** **Nagle's algorithm**

- Nagle's algorithm takes into account (1) the speed of the application program that creates the data, and (2) the speed of the network that transports the data

# Silly Window Syndrome (2)

2. Syndrome created by the Receiver
   - Receiving application program consumes data slowly (e.g. 1 byte at a time)
   - The receiving TCP announces a window size of 1 byte. The sending TCP sends only 1 byte…

**Solution 1: Clark's solution**

   - Sending an ACK but announcing a window size of zero until there is enough space to accommodate a segment of max. size or until half of the buffer is empty

# Silly Window Syndrome (3)

**Solution 2: Delayed Acknowledgement**

- The receiver waits until there is decent amount of space in its incoming buffer before acknowledging the arrived segments
- The delayed acknowledgement prevents the sending TCP from sliding its window. It also reduces traffic.
- Disadvantage: it may force the sender to retransmit the unacknowledged segments
- To balance: should not be delayed by more than 500ms

# TCP

- Question:
  - Is TCP using Selective Repeat or Go-back-*N* flow mechanism?
- Answer:

1. TCP is mostly Selective Repeat because it buffers out-of-order data and retransmits only missing segments.
2. However, it has a GBN-like cumulative acknowledgment scheme, not per-packet ACKs as in Selective Repeat.

# Chapter 3
# Transport Layer
## TCP - Error Control & Congestion Control

# 15-8 ERROR CONTROL

**Error control in TCP is achieved through the use of three tools:**

1. ***checksum***
2. ***acknowledgment***
3. ***time-out***

# *Topics Discussed in the Section*

- ✔ **Checksum**
- ✔ **Acknowledgment**
- ✔ **Retransmission**
- ✔ **Out-of-Order Segments**

**Note**

***ACK segments do not consume sequence numbers and are not acknowledged.***

# Acknowledgement Type

- In the past, TCP used only one type of acknowledgement: Accumulative Acknowledgement (ACK), also namely accumulative positive acknowledgement

- More and more implementations are adding another type of acknowledgement: Selective Acknowledgement (SACK), SACK is implemented as an option at the end of the TCP header.

**Note**

***Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order data are delivered to the process.***

**Note**

***TCP can be best modeled as a Selective Repeat protocol.***

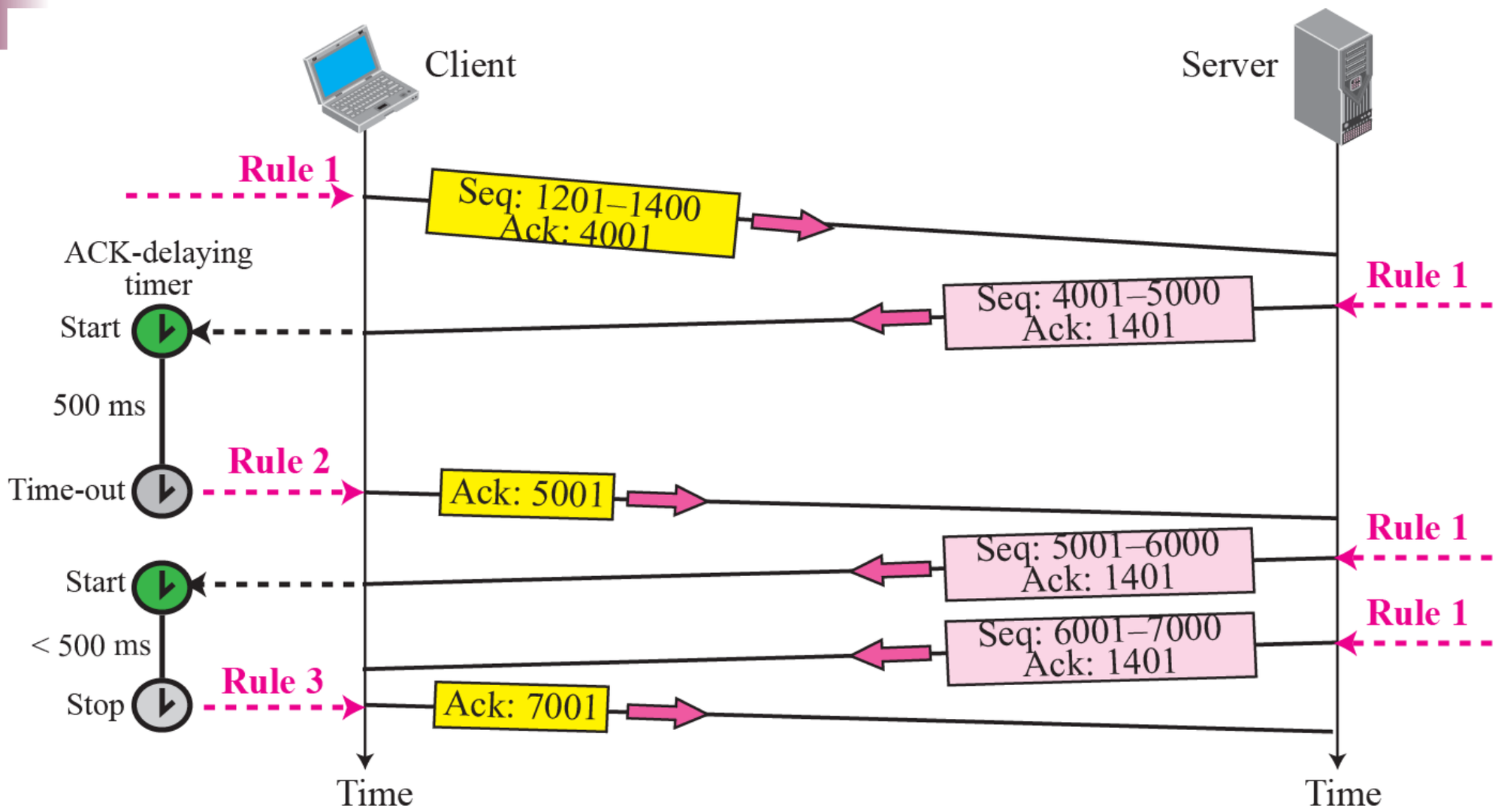
# Rules for Generating ACK (1)

- 1. When one end sends a data segment to the other end, it must include an ACK.  That gives the next sequence number it expects to receive.
- 2. The receiver needs to delay sending (until another segment arrives or 500ms) an ACK segment if there is only one outstanding in-order segment. It prevents ACK segments from creating extra traffic.
- 3. There should not be more than 2 in-order unacknowledged segments at any time. It prevent the unnecessary retransmission.

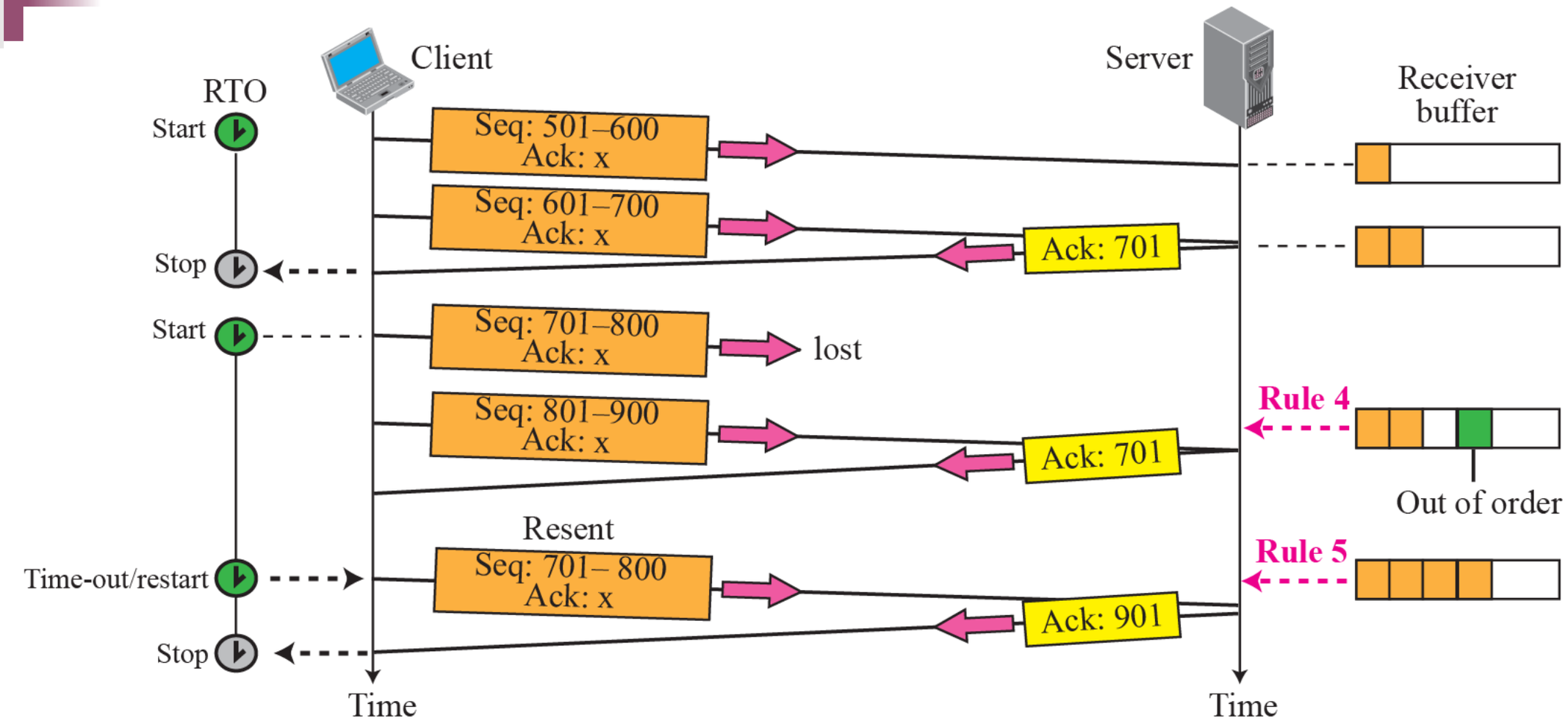
# Rules for Generating ACK (2)

- 4. When a segment arrives with an out-of-order sequence number that is higher than expected, the receiver immediately sends an ACK segment announcing the sequence number of the next expected segment. (for fast retransmission)
- 5. When a missing segment arrives, the receiver sends an ACK segment to announce the next sequence number expected.
- 6. If a duplicate segment arrives, the receiver immediately sends an ACK.

**Figure 15.29** *Normal operation*



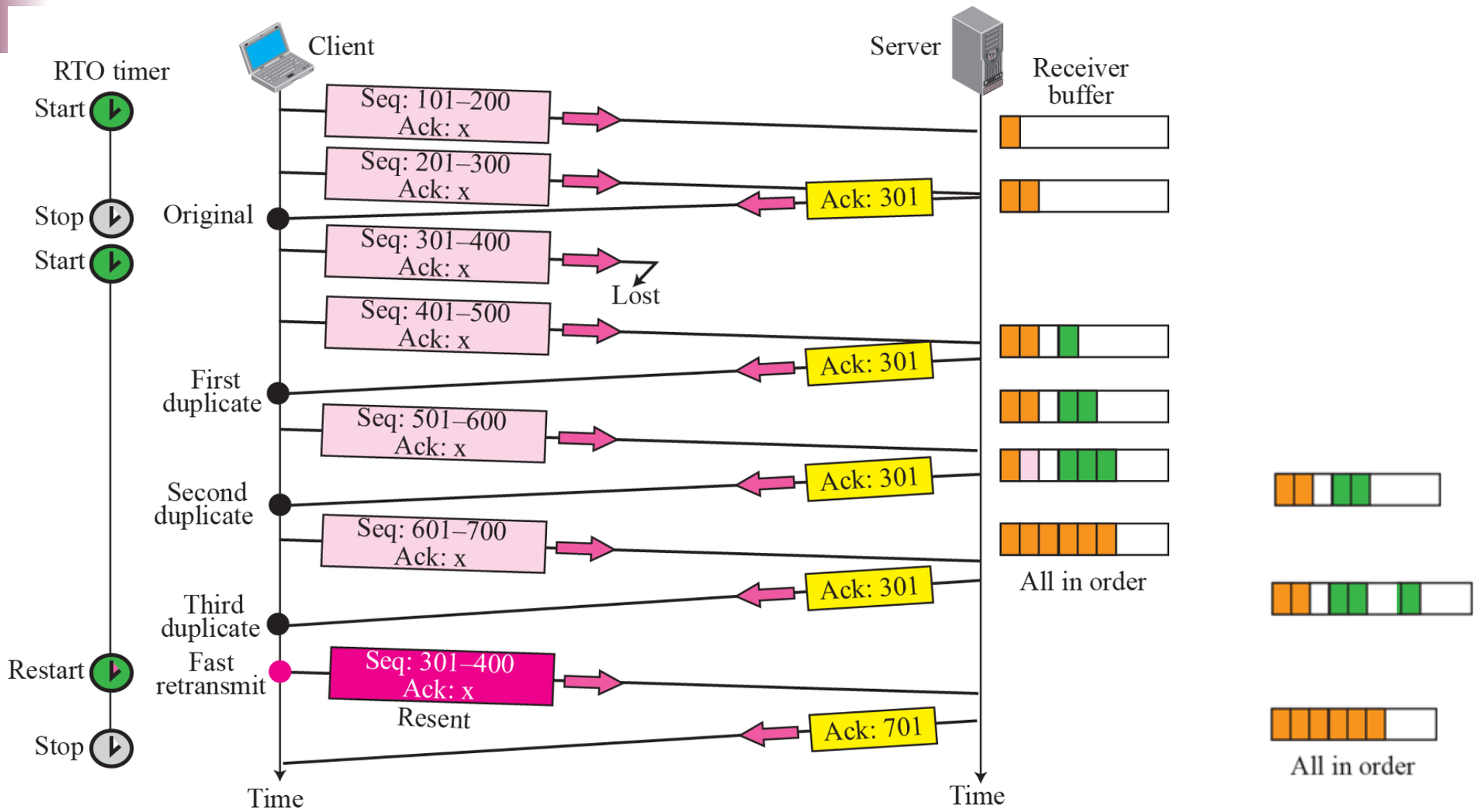
## Figure 15.30 *Lost segment*



**Note**

***The receiver TCP delivers only ordered data to the process.***

## Figure 15.31 *Fast retransmission*



# Fast Retransmit

- Time-out period often relatively long:
  - long delay before resending lost packet
- Detect lost segments via duplicate ACKs.
  - Sender often sends many segments back-to-back
  - If segment is lost, there will likely be many duplicate ACKs.
- If sender receives 3 ACKs for the same data, it supposes that segment after ACKed data was lost:
  - fast retransmit: resend segment before timer expires

**Figure 15.32** *Lost acknowledgment*

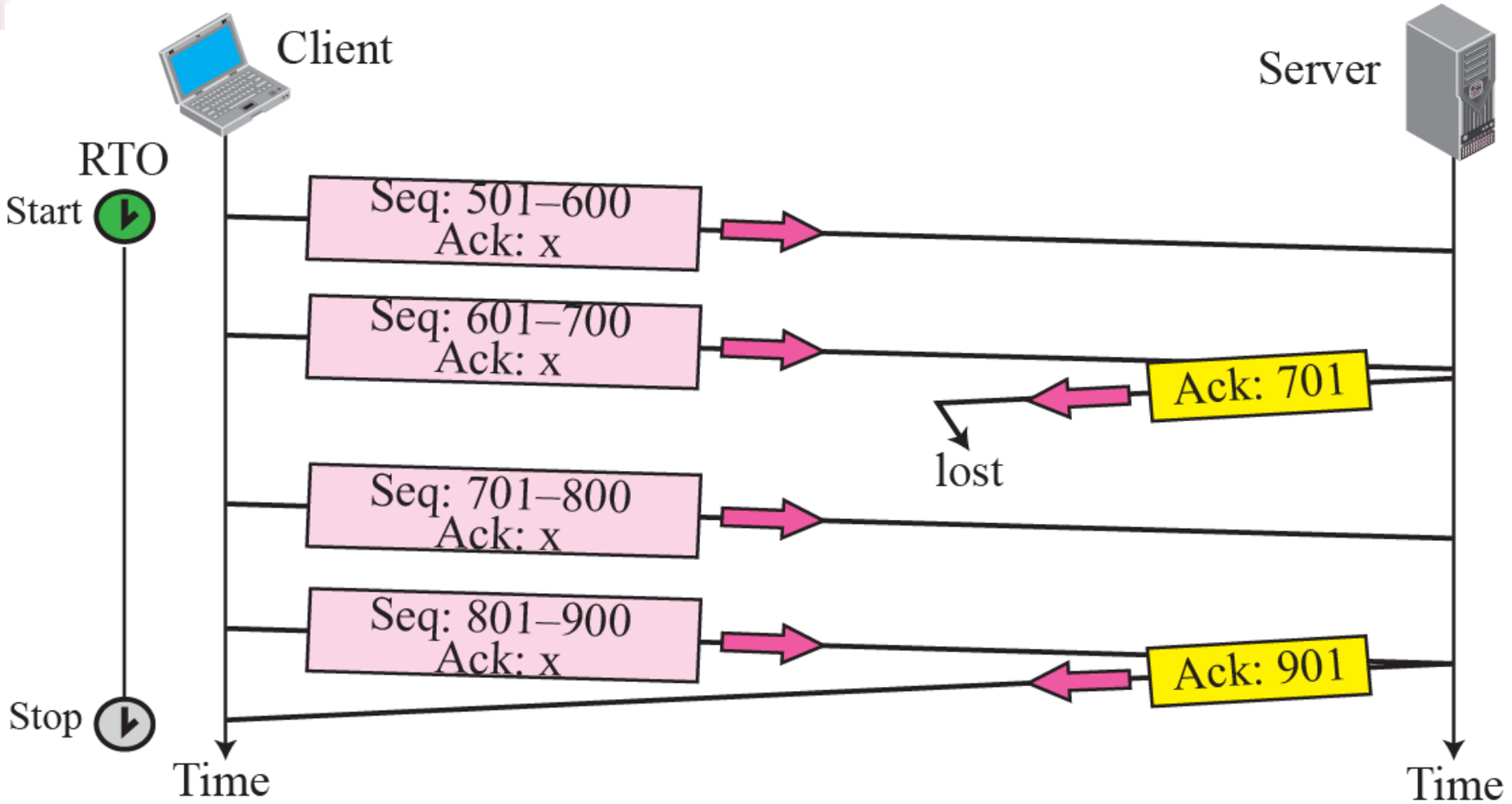
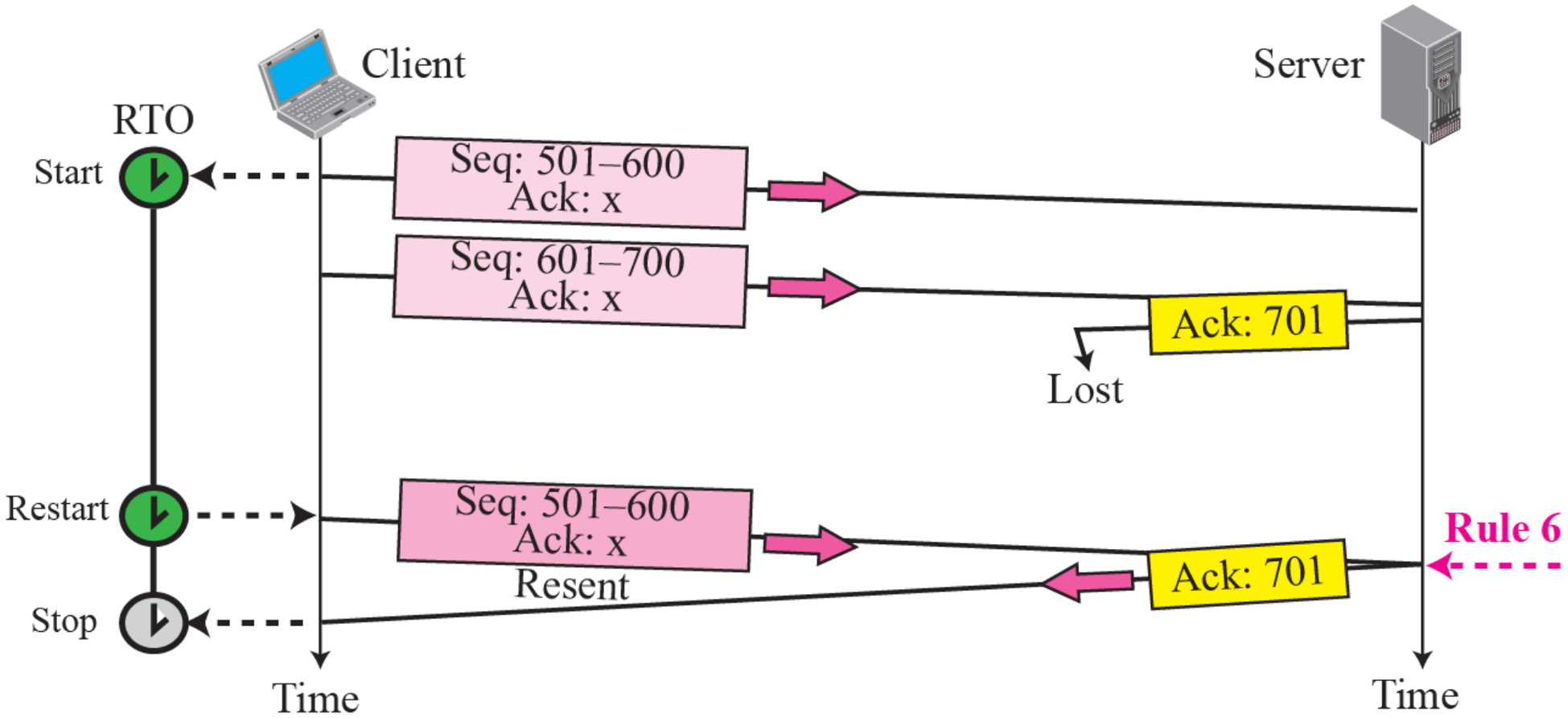


**Figure 15.33** *Lost acknowledgment corrected by resending a segment*



**Note**

***Lost acknowledgments may create deadlock if they are not properly handled.***

# 15-9 CONGESTION CONTROL

**TCP uses**

**1. congestion window and**
**2. congestion policy**

**to avoid congestion and detect and alleviate congestion after it has occurred.**

# *Topics Discussed in the Section*

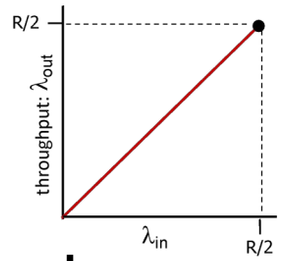
- ✔ **Congestion Window**
- ✔ **Congestion Policy**

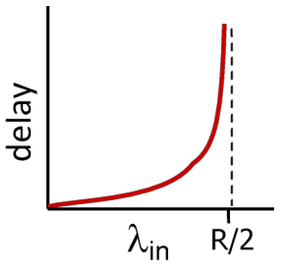
# Congestion

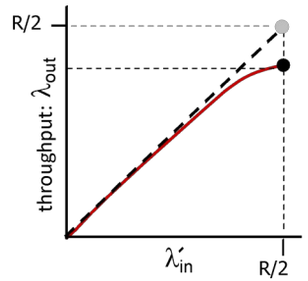
- Informally: “too many sources sending too much data too fast for *network* to handle”
- Appears if the load on the network is greater than the capacity of the network
  - **load:** the number of packets sent to the network
  - **capacity:** the number of packets a network can handle
- manifestations:
  - buffer overflow at routers
  - queuing in router buffers

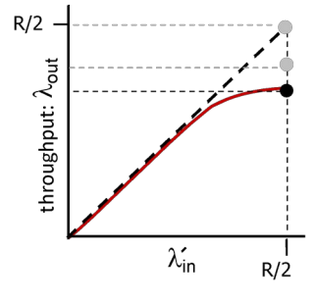
# Causes/costs of congestion: insights

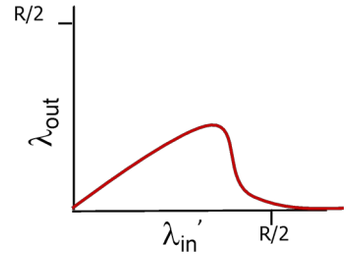
- throughput can never exceed capacity
- delay increases as capacity approached
- loss/retransmission decreases effective throughput
- un-needed duplicates further decreases effective throughput
- upstream transmission capacity / buffering wasted for packets lost downstream



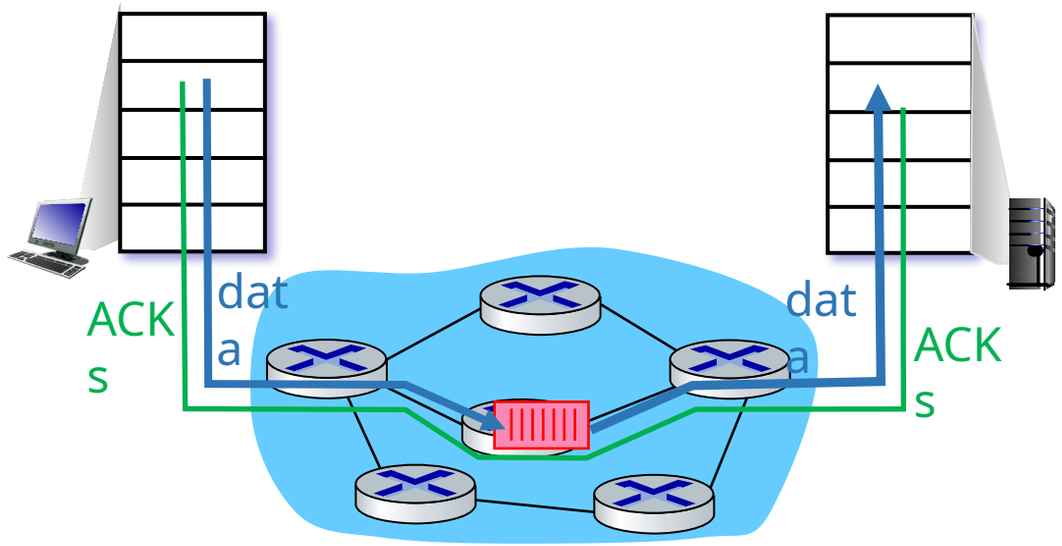



# Approaches towards congestion control

End-end congestion control:

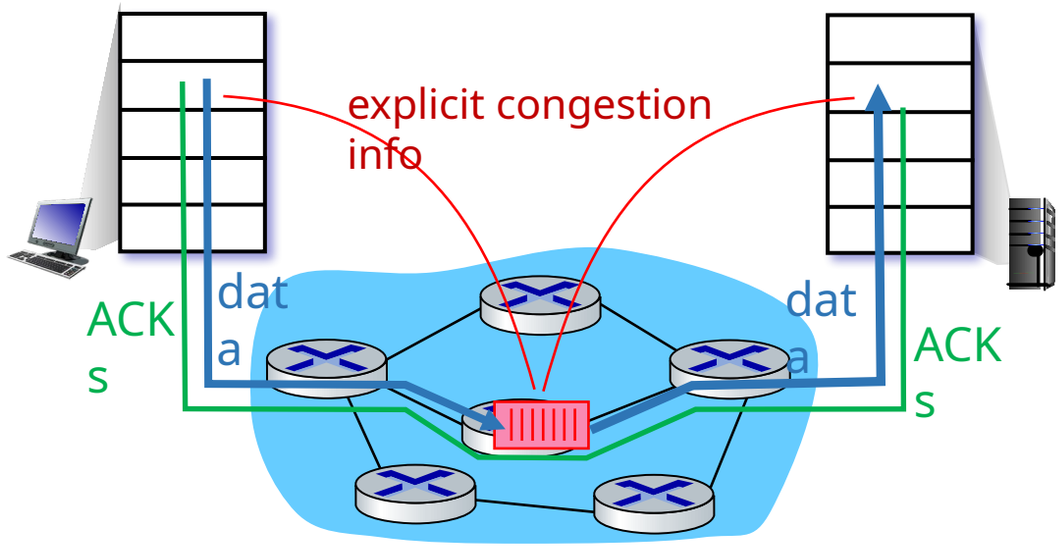
- no explicit feedback from network
- congestion *inferred* from observed loss, delay
- approach taken by TCP



# Approaches towards congestion control

## Network-assisted congestion control:

- routers provide *direct* feedback to sending/receiving hosts with flows passing through congested router
- may indicate congestion level or explicitly set sending rate




- TCP ECN, ATM, DECbit protocols

# TCP Congestion Control

- How does  sender perceive congestion?
  - loss event = timeout *or* 3 duplicate acks
- Retransmission of the lost packets does not solve congestion problem – it aggravates it
- In flow control, sender window size determined by the receiver window – no information about the network congestion
- TCP sender
  - Maintains congestion window **`CongWin`**
    - dynamic function of perceived network congestion
  - Limits sender window to min (receiver window size, congestion window size)

**`LastByteSent-LastByteAcked ≤ min{CongWin,RcvWindow}`**

# TCP Congestion Control

What is the size of **`CongWin?`**

three mechanisms:

a. Slow Start

b. Additive increase (AI)

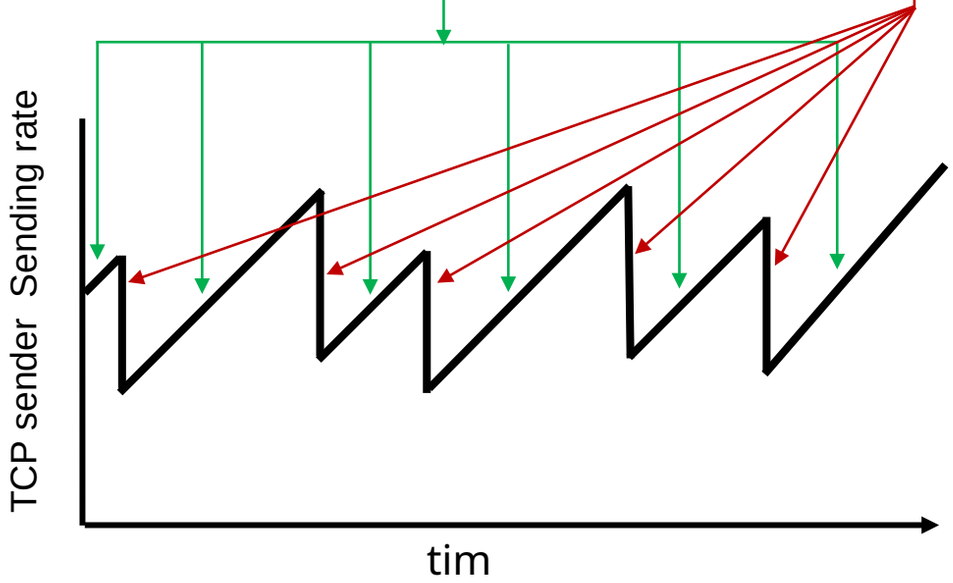
c. Multiplicative Decrease (MD)

# TCP congestion control: AIMD

- *approach:* senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

*Additive Increase*: increase sending rate by 1 maximum segment size every RTT until loss detected

*Multiplicative Decrease*: cut sending rate in half at each loss event




**AIMD** sawtooth behavior: *probing* for bandwidth

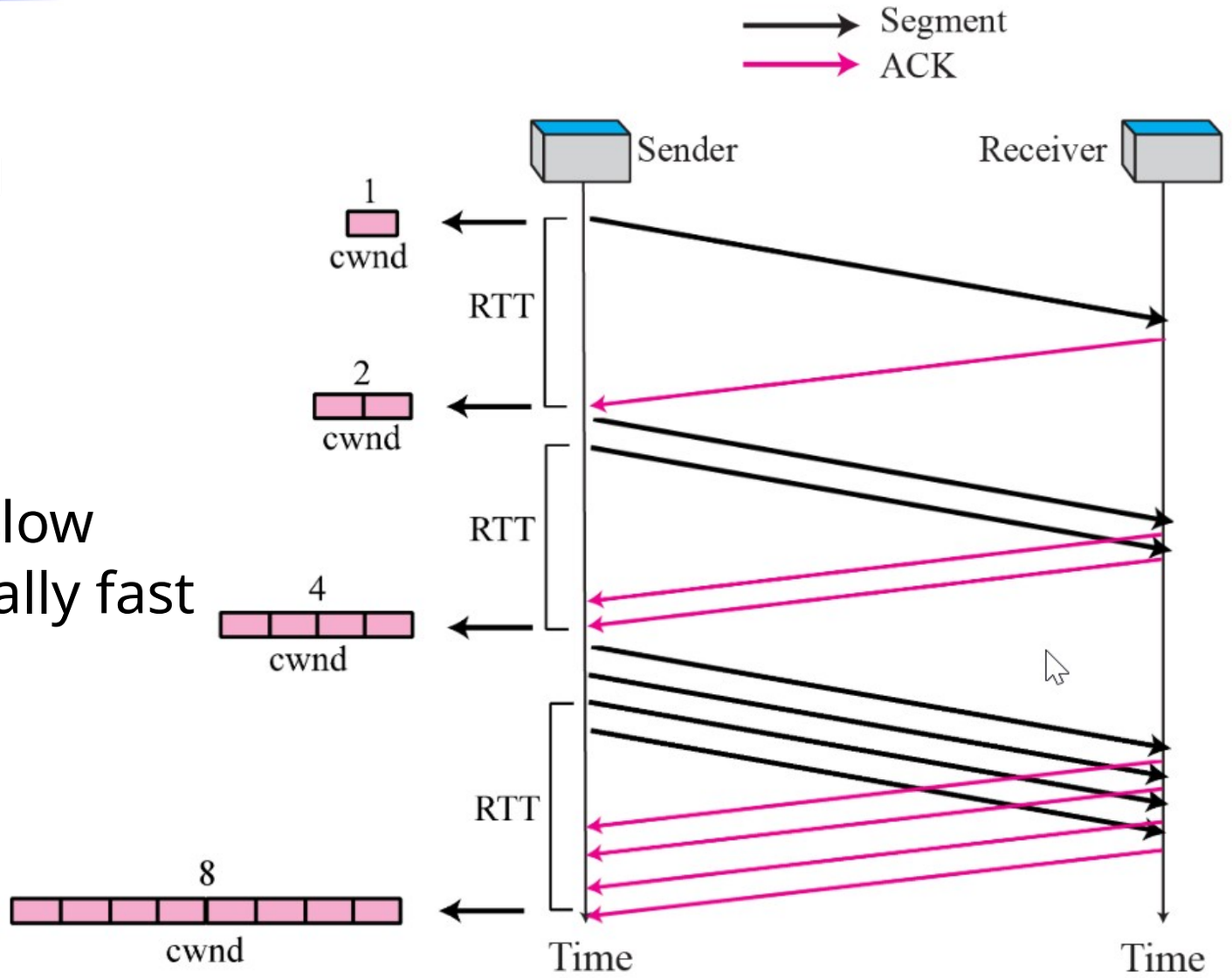
## TCP Slow Start

- When connection begins, **`CongWin`** = 1 MSS
  - Example: MSS = 500 bytes & RTT = 200 msec
  - initial rate = 20 kbps
- available bandwidth may be >> MSS/RTT
  - It is desirable to quickly ramp up to respectable rate
- When connection begins
  - increase rate exponentially fast until
    - first loss event
    - or reaches a threshold (then additive increase)

- MSS = Maximum Segment Size

- done by incrementing **`CongWin`** for every ACK received

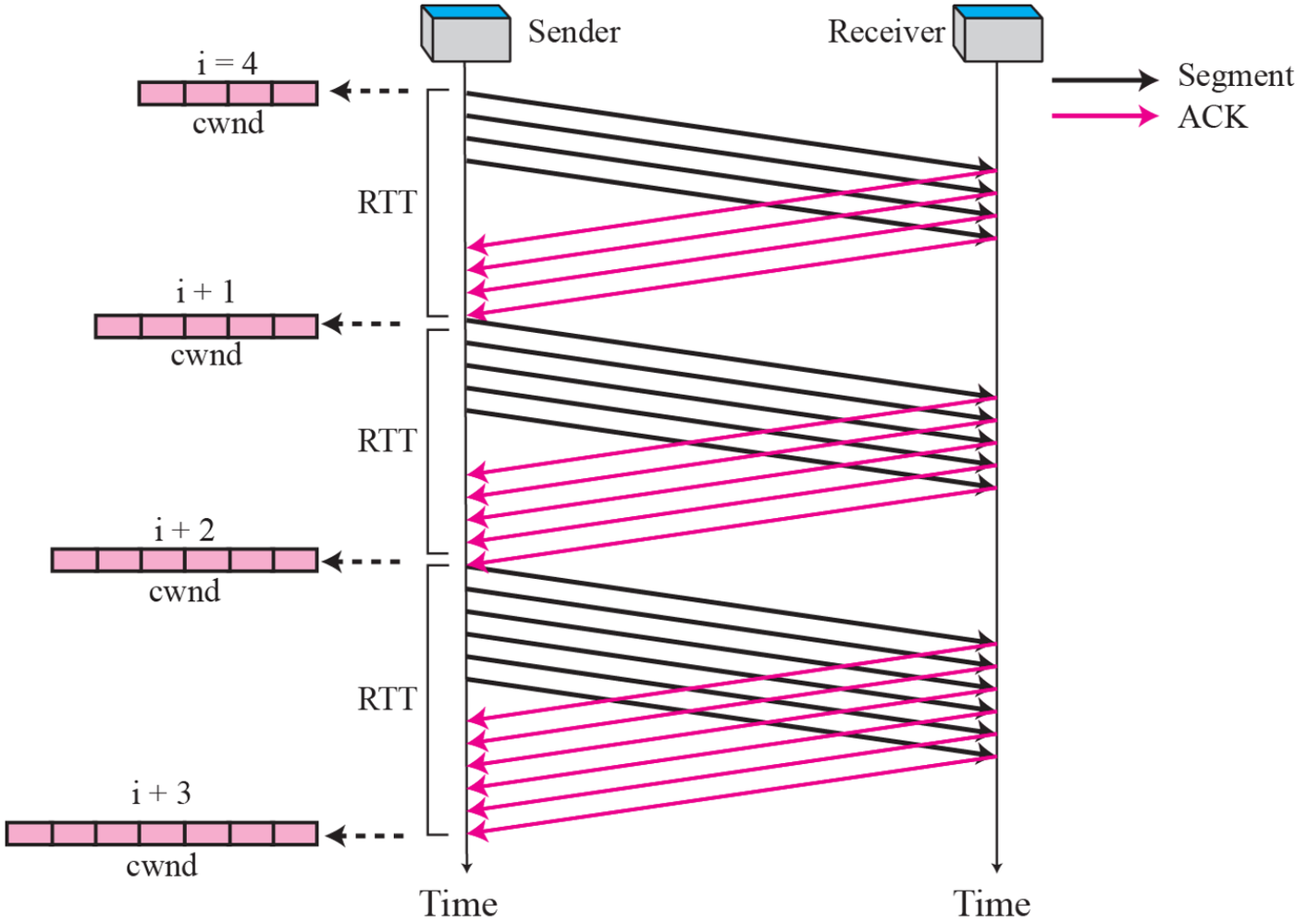
Summary: initial rate is slow but ramps up exponentially fast



# Additive Increase

- for each successfully received ACK, increase the window size by *((1/n) * MSS))*.
  - *n* = current **`CongWin`** size

- In other word, increase 1 MSS after each RTT.



# Multiplicative Decrease

- after loss event reduce or cut **`CongWin`** to
  - half of previous value
  - Or any specific value

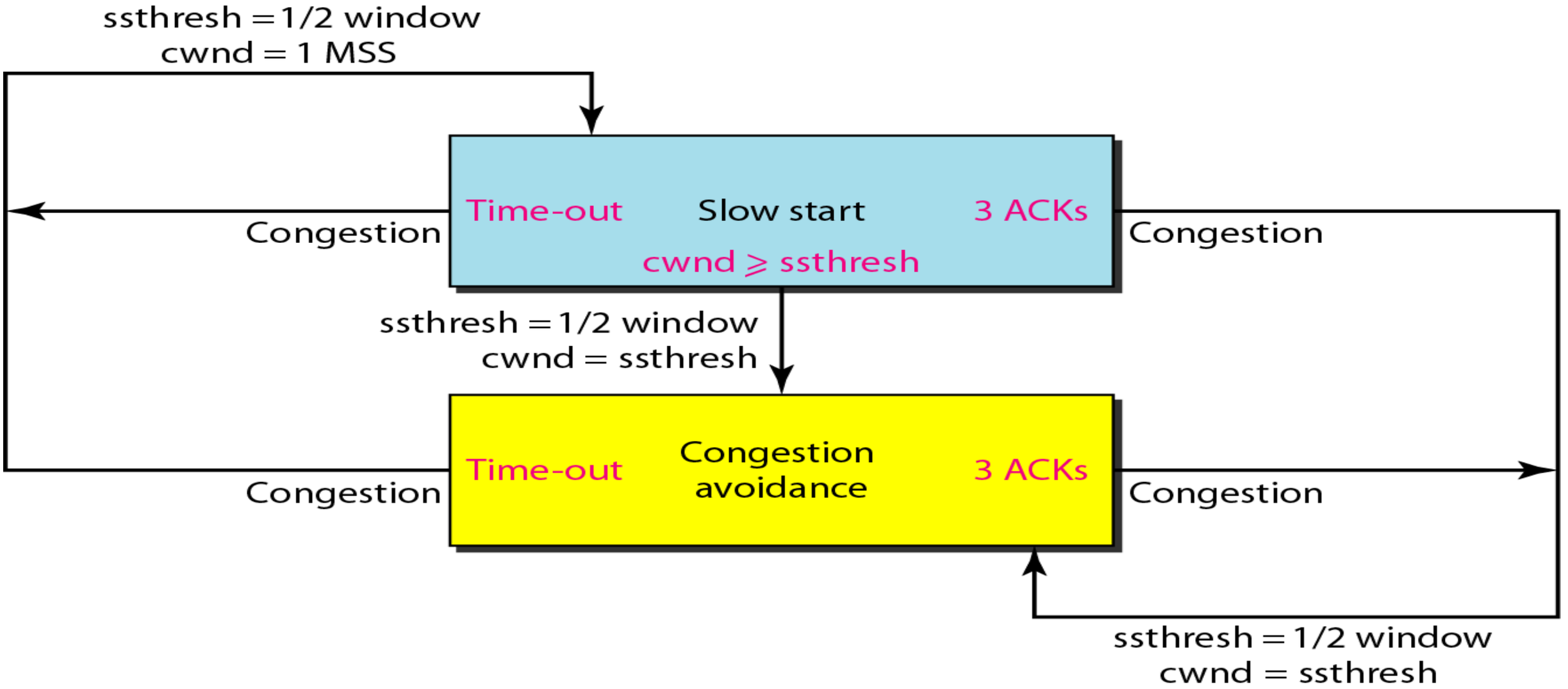
# Congestion control in TCP

1. Slow Start (SS) (Exponential Increase)
   - start with the congestion window (**`CongWin`**) = 1MSS
   - for each successfully received ACK increase the cwnd size by 1 until the **`CongWin`** = threshold value;
2. Additive Increase (AI) (Congestion Avoidance)
   - after that, for each successfully received ACK, increase the window size by 1/*n* segments up to a size of the receiver window. *n*=current **`CongWin`** size
3. Multiplicative Decrease (MD)
   - if a time-out occurs the (TCP Tahoe, TCP Reno)

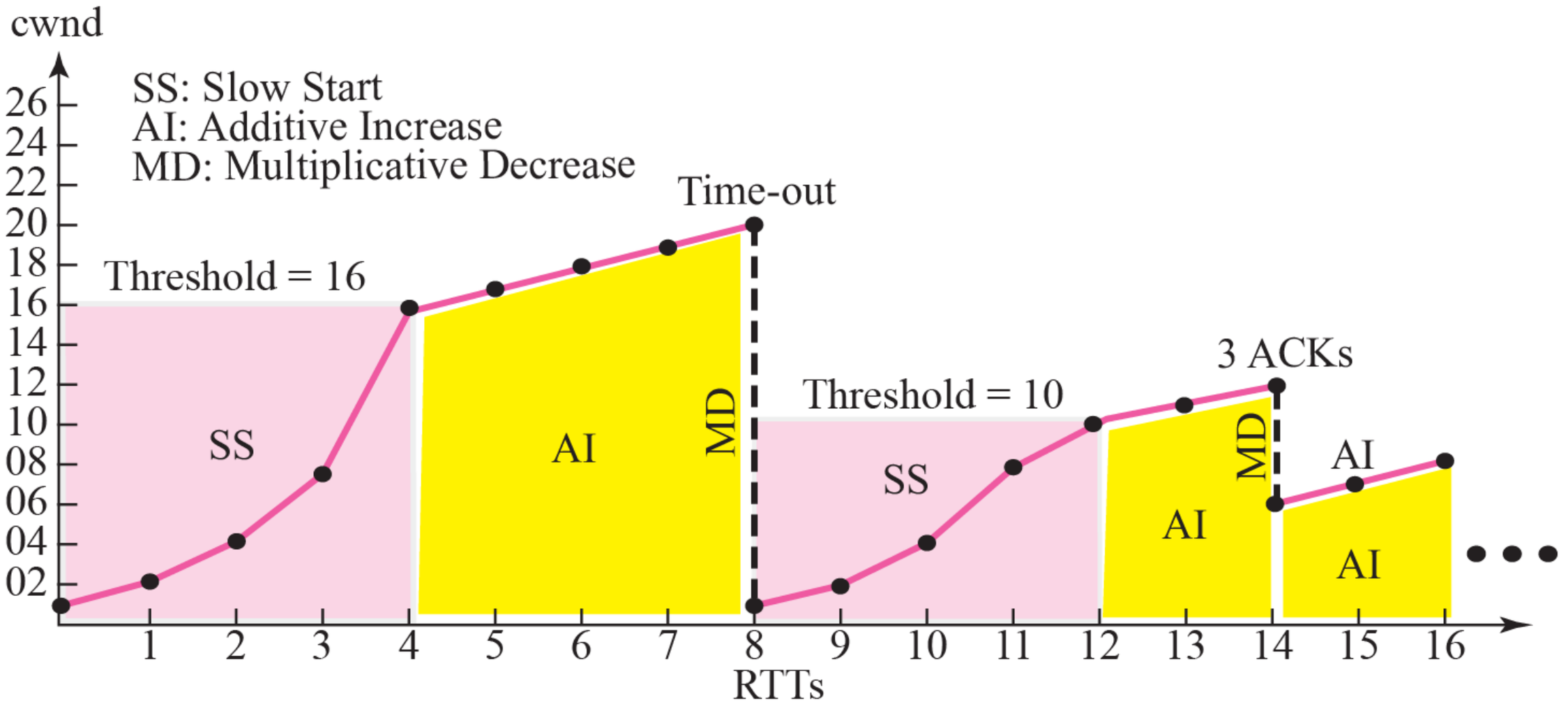
     Threshold = ½ (**`CongWin), CongWin`** = 1 MSS, .
   - if 3 duplicated ACKs received (TCP Reno)

     Threshold = ½ (**`CongWin), CongWin = threshold`**

# Congestion control in TCP



# Congestion control in TCP



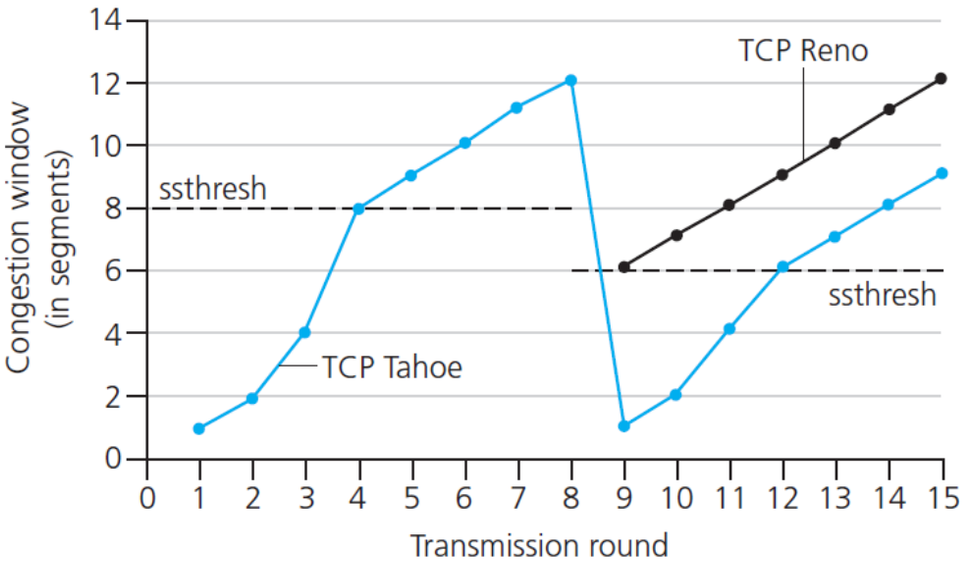
# TCP: Switching from Slow Start to Additive Increase

Q: when should the exponential increase switch to linear?

A: when **cwnd** gets to 1/2 of its value before timeout.

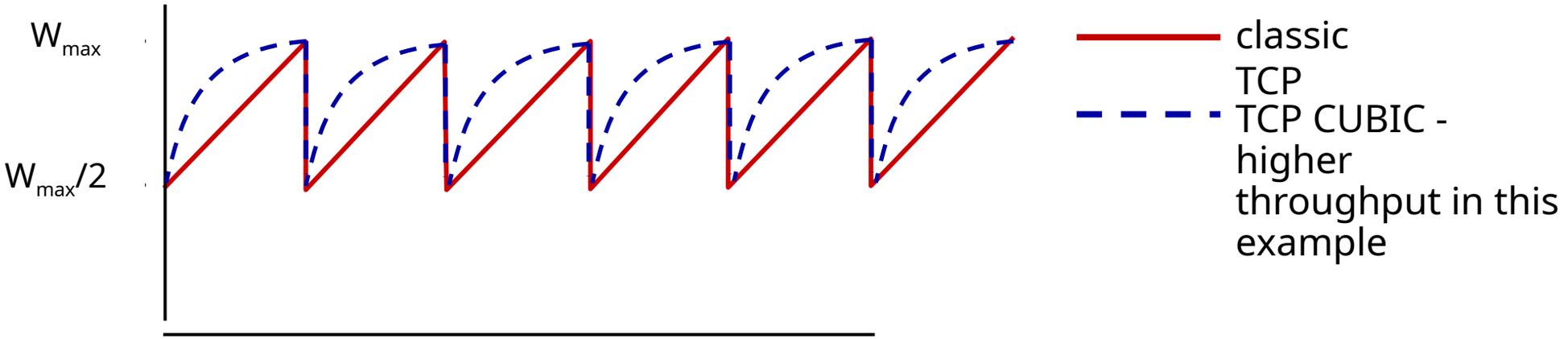
## Implementation:

- variable **ssthresh**
- on loss event, **ssthresh** is set to 1/2 of **cwnd** just before loss event



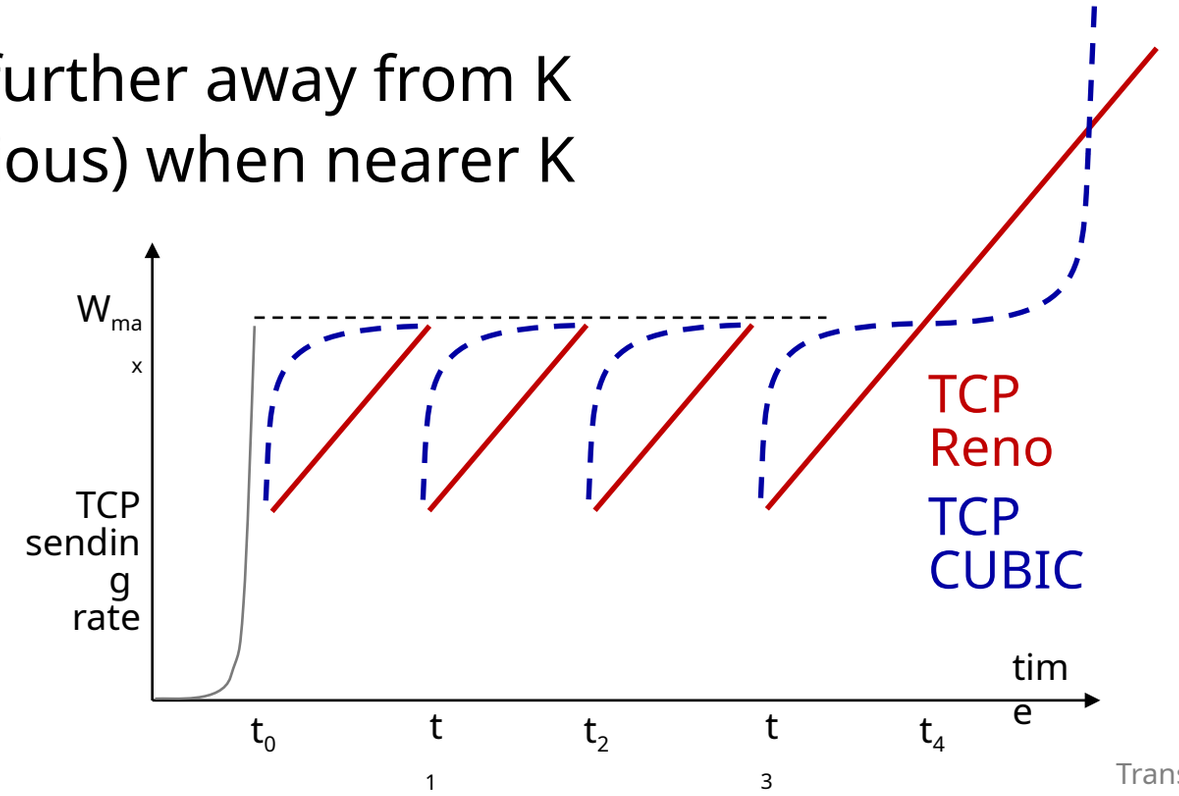
# TCP CUBIC

- Is there a better way than AIMD to "probe" for usable bandwidth?
- Insight/intuition:
  - $W_{max}$: sending rate at which congestion loss was detected
  - congestion state of bottleneck link probably (?) hasn't changed much
  - after cutting rate/window in half on loss, initially ramp to to $W_{max}$ *faster*, but then approach $W_{max}$ more *slowly*



# TCP CUBIC

- K: point in time when TCP window size will reach $W_{max}$
  - K itself is tunable
- increase W as a function of the *cube* of the distance between current time and K
  - larger increases when further away from K
  - smaller increases (cautious) when nearer K
- TCP CUBIC default in Linux, most popular TCP for popular Web servers

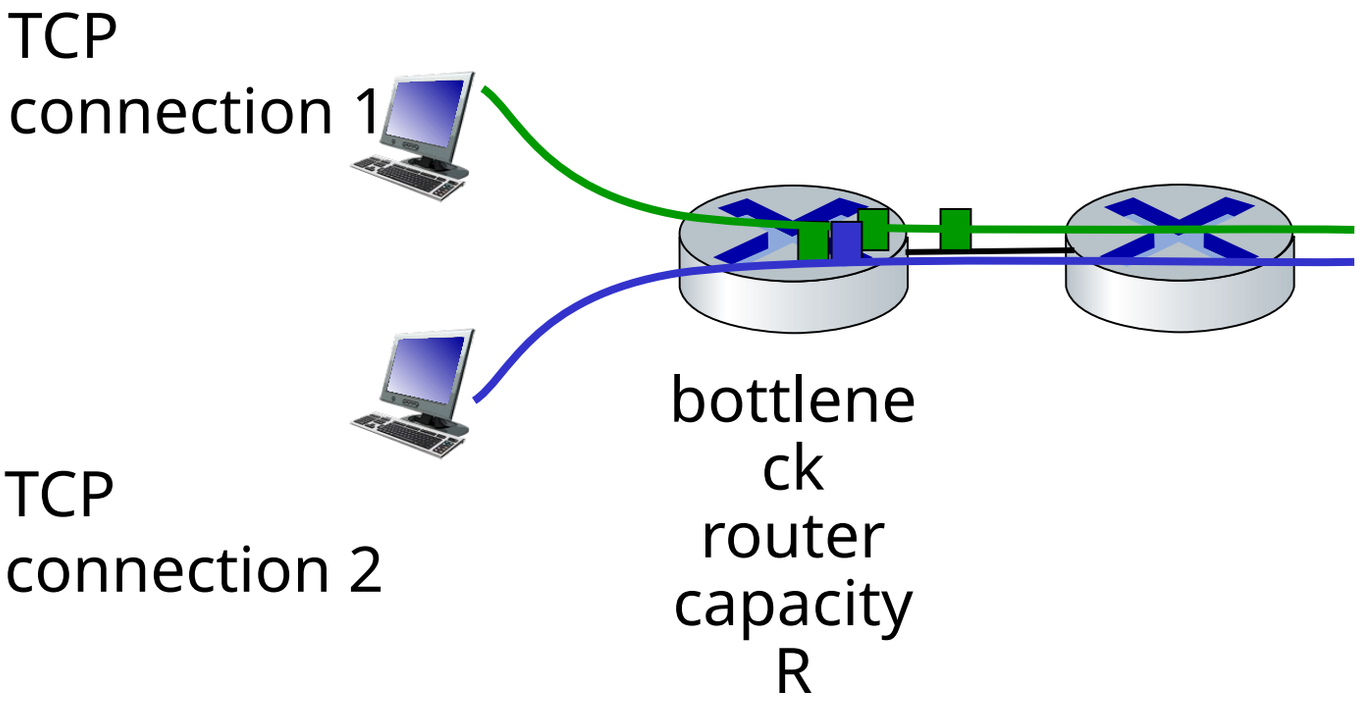


# TCP Evolution Timeline

| | | |
|---|---|---|
| **TCP Tahoe** (late 1980s) | Introduced **slow start**, **congestion avoidance**, and **fast retransmit** | First version to handle congestion dynamically |
| **TCP Reno** (1990) | Added **fast recovery** on top of Tahoe | This became the **classic TCP** used and studied widely |
| **TCP NewReno** (1996) | Improved Reno's fast recovery (better for multiple losses) | Backward-compatible with Reno |
| **TCP Vegas**, **Cubic**, **BBR**, etc. | Later versions with more advanced congestion control | Used in modern systems (e.g., Linux uses **Cubic** by default) |

# TCP fairness

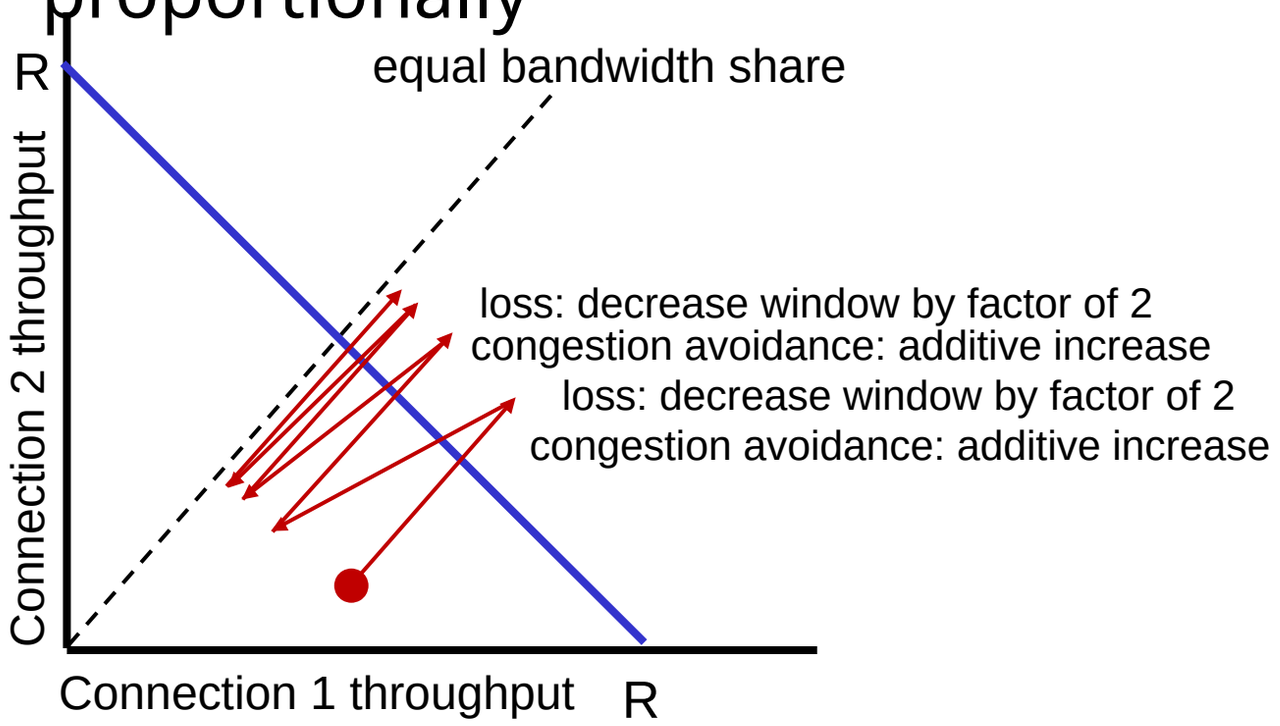
Fairness goal: if *K* TCP sessions share same bottleneck link of bandwidth *R*, each should have average rate of *R/K*



# Q: is TCP Fair?

Example: two competing TCP sessions:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally




*Is* TCP *Fair?*

Yes, under idealized assumptions:

- same RTT
- fixed number of sessions only in congestion avoidance

# Fairness: must all network apps be “fair”?

## Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss
- there is no “Internet police” policing use of congestion control

## Fairness, parallel TCP connections

- application can open *multiple* parallel connections between two hosts
- web browsers do this , e.g., link of rate R with 9 existing connections:
  - new app asks for 1 TCP, gets rate R/10
  - new app asks for 11 TCPs, gets R/2

# Chapter 3
# Transport Layer
## TCP - RTT Estimation

# TCP round trip time, timeout

*Q:* how to set TCP timeout value?

- longer than RTT, but RTT varies!
- *too short:* premature timeout, unnecessary retransmissions
- *too long:* slow reaction to segment loss

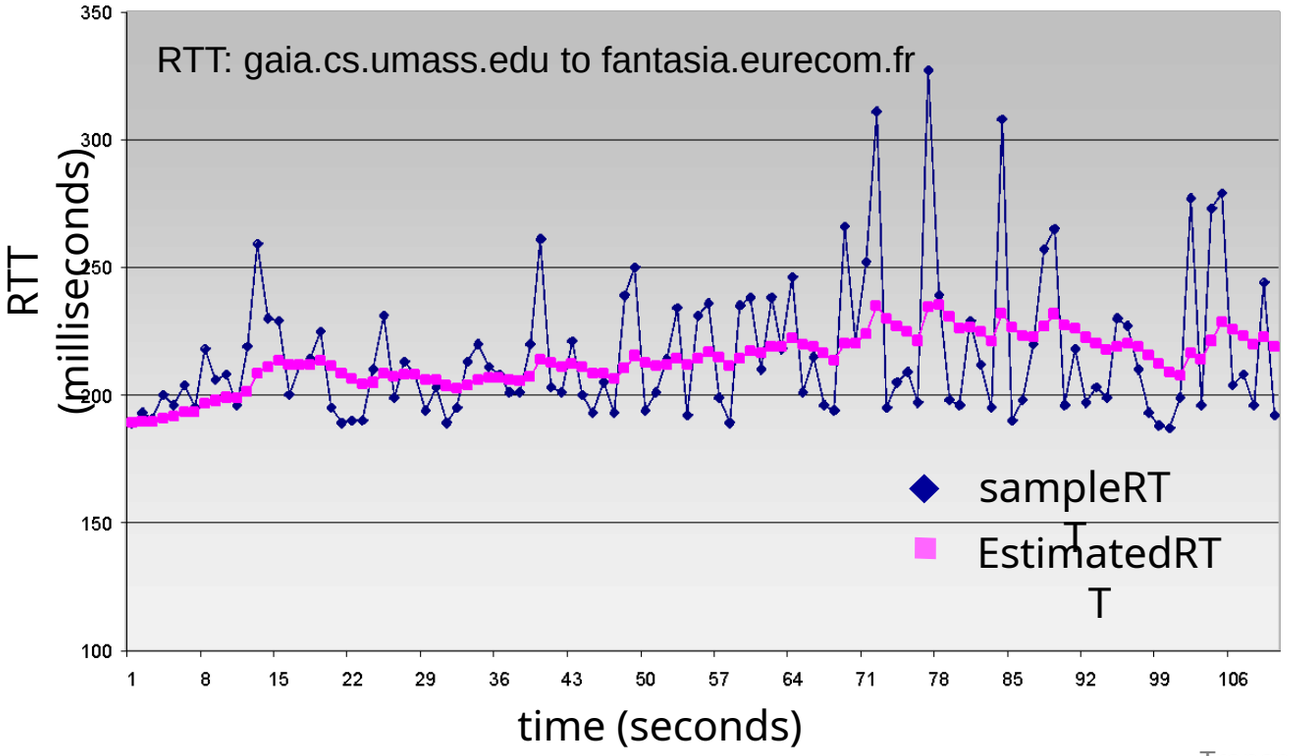
*Q:* how to estimate RTT?

- `SampleRTT`: measured time from segment transmission until ACK receipt
  - ignore retransmissions
- `SampleRTT` will vary, want estimated RTT "smoother"
  - average several *recent* measurements, not just current `SampleRTT`

# TCP round trip time, timeout

**EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT**

- exponential weighted moving average (EWMA)
- influence of past sample decreases exponentially fast
- typical value: α = 0.125



# TCP round trip time, timeout

- timeout interval: **`EstimatedRTT`** plus "safety margin"
  - large variation in **`EstimatedRTT`**: want a larger safety margin

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```



estimated RTT

"safety margin"

- **`DevRTT`**: EWMA of **`SampleRTT`** deviation from **`EstimatedRTT`**:

```
DevRTT = (1-β)*DevRTT + β*|SampleRTT-EstimatedRTT|
```

(typically, β = 0.25)

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

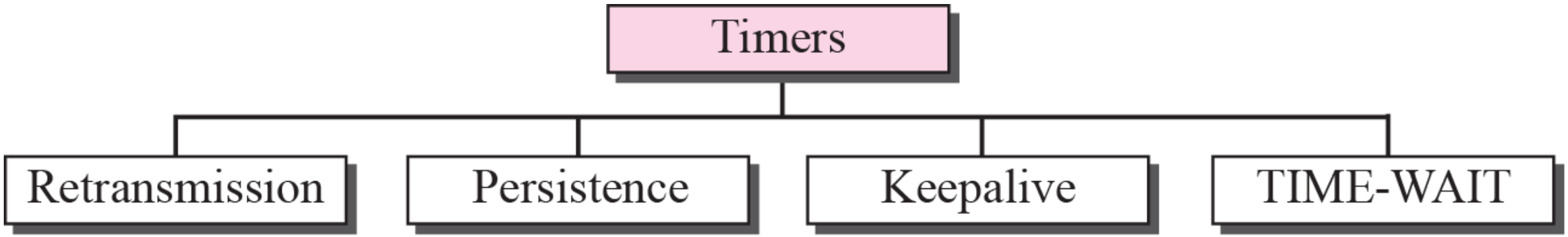
# 15-10 TCP TIMERS

**To perform its operation smoothly, most TCP implementations use at least four timers as shown in Figure 15.38 (slide 83).**

# *Topics Discussed in the Section*

- ✔ **Retransmission Timer**
- ✔ **Persistence Timer**
- ✔ **Keepalive Timer**
- ✔ **TIME-WAIT Timer**

**Figure 15.38** *TCP timers*



# Retransmission timer

1. if an ACK is received before the timer goes off – destroy or reset the timer
2. if the timer goes off before ACK arrives – retransmit the segment & reset the timer

- How to set Retransmission time
  - longer than RTT
    - but RTT varies since paths that IP packets take may differ
  - if too short
    - Premature timeout –> retransmissions -> waste of bandwidth
  - if too large
    - slow reaction to segment loss
    - delay for the application program

Note

***In TCP, there can be only one RTT measurement in progress at any time.***

**Since the segments and their ACKs do not have a 1-1 relationship**

# Calculation of RTO (1)

- **Smoothed RTT: $RTT_S$ (EstimatedRTT)**
  - **Original → No value**
  - **After 1st measurement → $RTT_S = RTT_M$**
  - **2nd … → $RTT_S = (1-\alpha)*RTT_S + \alpha*RTT_M$**

# Calculation of RTO (2)

- **RTT Deviation : $RTT_D$ (DevRTT)**
  - **Original → No value**
  - **After 1st measurement → $RTT_D = 0.5*RTT_M$**
  - **2nd … → $RTT_D = (1-\beta)*RTT_D + \beta*|RTT_S - RTT_M|$**

# Calculation of RTO (3)

- **Retransmission Timeout (RTO)**
  - **Original → Initial value**
  - **After any measurement**

    **→ RTO = $RTT_S + 4RTT_D$**
- **Example 10 (page 322)**
  - **$\alpha = 1/8 = 0.125$**
  - **$\beta = 1/4 = 0.25$**

# Example 15.3

Let us give a hypothetical example. Figure 15.39 shows part of a connection. The figure shows the connection establishment and part of the data transfer phases.

1. When the SYN segment is sent, there is no value for RTTM, RTTS, or RTTD. The value of RTO is set to 6.00 seconds. The following shows the value of these variable at this moment:

$$RTO = 6$$

2. When the SYN+ACK segment arrives, $RTT_M$ is measured and is equal to 1.5 seconds.

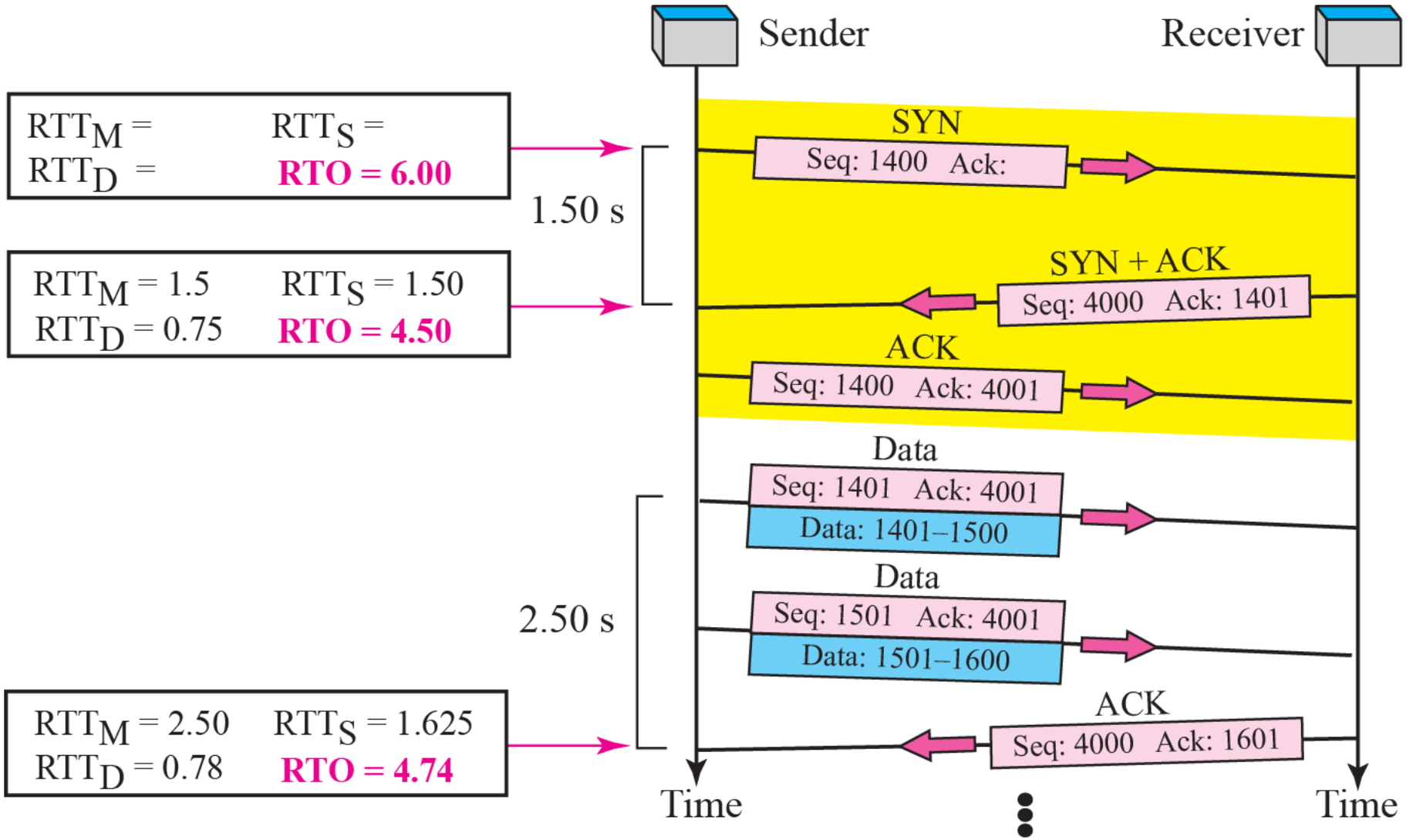
$$RTT_M = 1.5$$
$$RTT_S = 1.5$$
$$RTT_D = (1.5) / 2 = 0.75$$
$$RTO = 1.5 + 4 \times 0.75 = 4.5$$

# Example 15.3 *Continued*

3. **When the first data segment is sent, a new RTT measurement starts. No RTT measurement starts for the second data segment because a measurement is already in progress. The arrival of the last ACK segment is used to calculate the next value of RTTM. Although the last ACK segment acknowledges both data segments (cumulative), its arrival finalizes the value of RTTM for the first segment. The values of these variables are now as shown below.**

$$\text{RTT}_M = 2.5$$
$$\text{RTT}_S = 7/8 \times 1.5 + (1/8) \times 2.5 = 1.625$$
$$\text{RTT}_D = 3/4\ (7.5) + (1/4) \times |1.625 - 2.5| = 0.78$$
$$\text{RTO} = 1.625 + 4 \times 0.78 = 4.74$$

## Figure 15.39 *Example 15.3*



Until a round-trip time (RTT) measurement has been made for a segment sent between the sender and receiver, the sender SHOULD set RTO = 1 second.

Reference: https://www.rfc-editor.org/info/rfc6298/#section-2

Steps:

First Measurement:

1. $RTT_S = RTT_M$

2. $RTT_D = 0.5 * RTT_M$

# Steps:
# All Other Measurement:

1. **$RTT_M$** = Latest measurement of **RTT**
2. Calculate **$RTT_S = (1-\alpha)*RTT_S + \alpha*RTT_M$**
3. Calculate **$RTT_D = (1-\beta)*RTT_D + \beta*|RTT_S - RTT_M|$**
4. Finally Calculate **$RTO = RTT_S + 4RTT_D$**

**Here,**

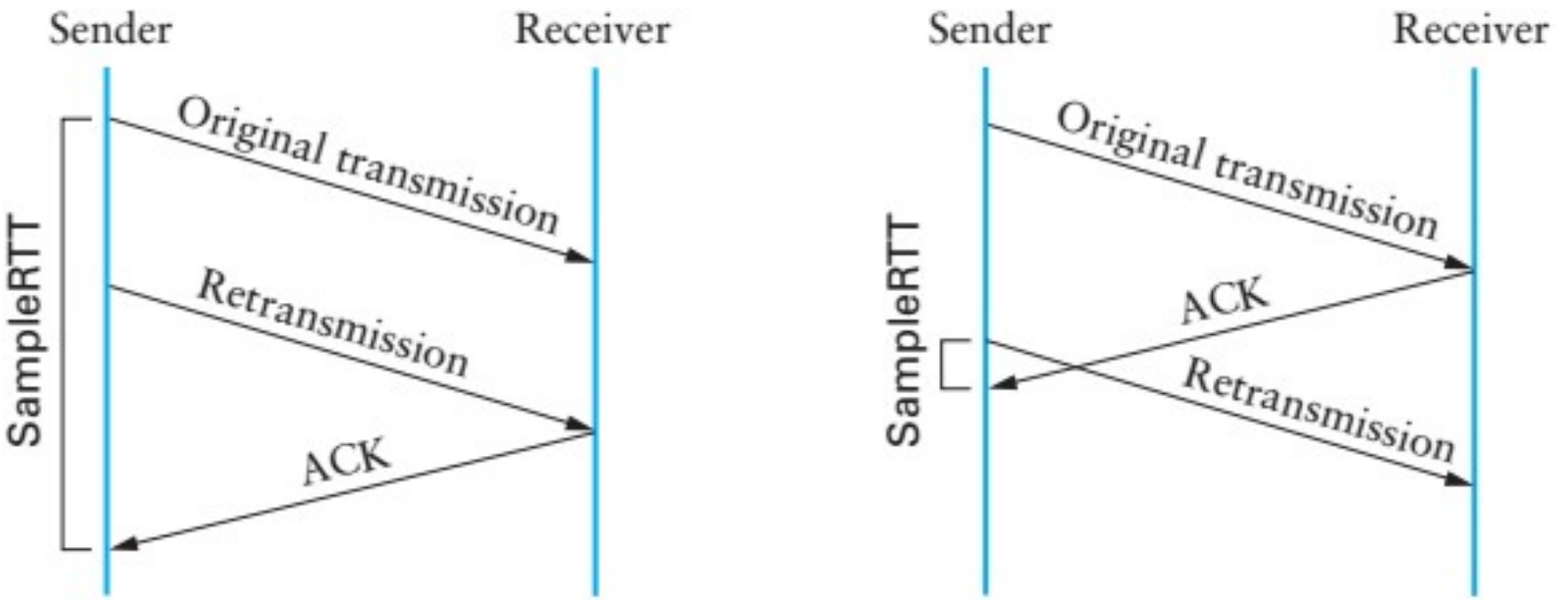
**$\alpha = 0.125$**

**$\beta = 0.25$**

**Note**

***TCP does not consider the RTT of a retransmitted segment in its calculation of a new RTO.***

# Flaw in RTO Calculation




- Karn/Partridge Algorithm
  Doesn’t include retransmitted packet

# Flaw in RTO Calculation

TCP estimates how long it should wait for an **acknowledgment (ACK)** before retransmitting a packet.

It uses two key variables:

- **RTT (Round-Trip Time):** The time from sending a segment until its ACK is received.
- **RTO (Retransmission Timeout):** How long to wait before retransmitting a segment if no ACK arrives.

Normally, TCP updates its RTT estimate using **Jacobson's algorithm**, like:

$$\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$$

$$\text{RTO} = \text{EstimatedRTT} + 4 \times \text{Deviation}$$

But this approach fails when **a packet is retransmitted**.

# Flaw in RTO Calculation

## ❌ Problem

If TCP retransmits a packet and then receives an ACK —

it **can't tell** whether that ACK was for:

1. the **first transmission**, or
2. the **retransmitted copy**.

So the measured **RTT becomes ambiguous** (we don't know which transmission the ACK corresponds to).

Using this ambiguous RTT would **corrupt** the RTO calculation.

# Karn's Algorithm Solution

Karn and Partridge proposed the following rules:

1. **Do not update RTT estimates on retransmitted packets.**

   → Ignore RTT samples for packets that have been retransmitted.

2. **Use exponential backoff for RTO after a retransmission.**

   → When a timeout happens and a packet is retransmitted, **double the RTO**.

$$\mathrm{RTO}_{new} = 2 \times \mathrm{RTO}_{old}$$

This helps TCP back off in case of persistent congestion.

# Karn’s Algorithm Solution

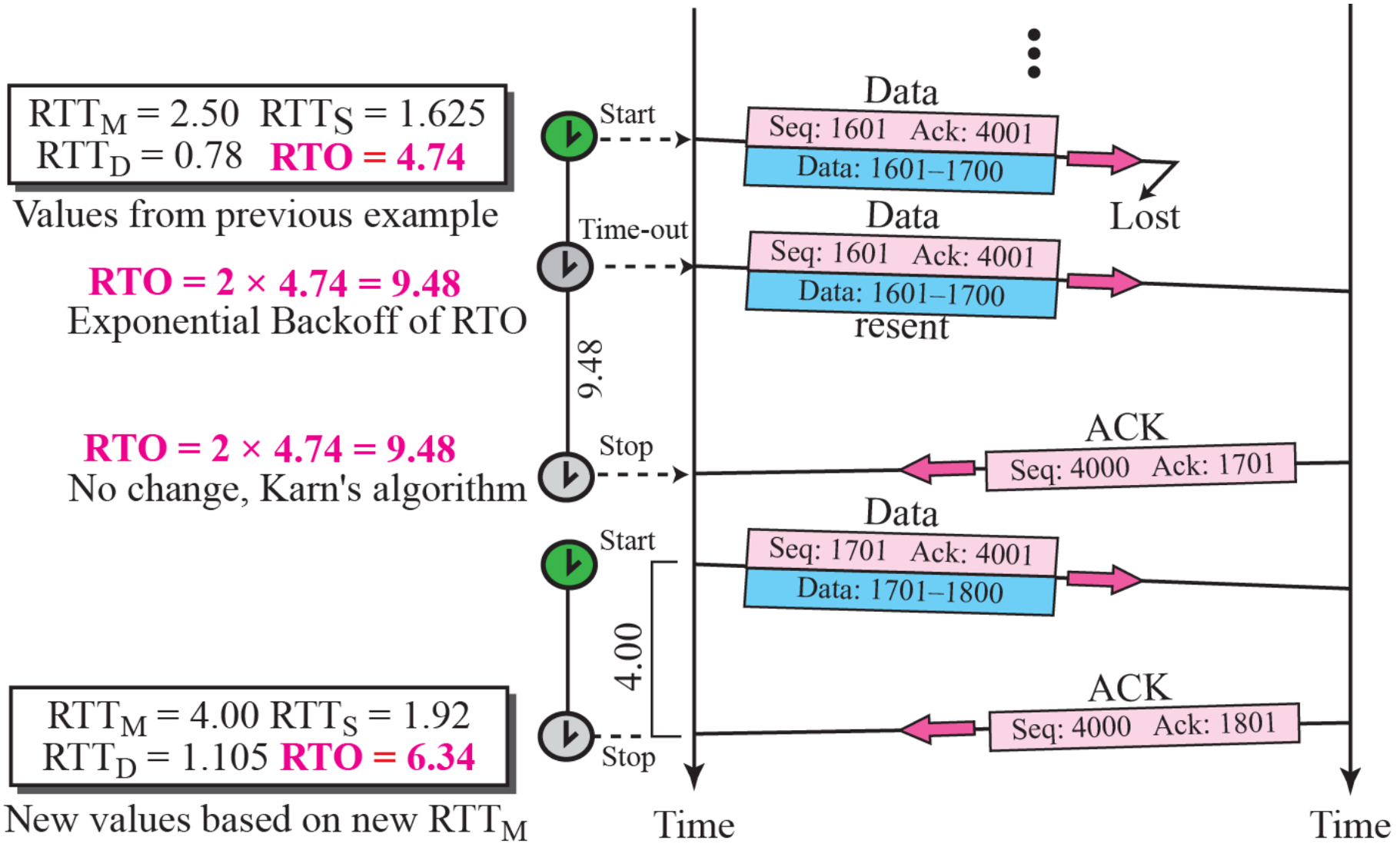
In practice, **TCP caps the RTO — usually around 60 seconds** (depending on implementation), so it doesn’t grow indefinitely.

# Example 15.4

**Figure 15.40 is a continuation of the previous example. There is retransmission and Karn's algorithm is applied.**

**The first segment in the figure is sent, but lost. The RTO timer expires after 4.74 seconds. The segment is retransmitted and the timer is set to 9.48, twice the previous value of RTO. This time an ACK is received before the time-out. We wait until we send a new segment and receive the ACK for it before recalculating the RTO (Karn's algorithm).**

## Figure 15.40 *Example 15.4*



# Persistence timer

- To deal with the zero-size windows
- What if the receiver advertises that the window size is ≠ 0 (by sending ACK) and this ACK is lost?
  - ACK are not acknowledged in TCP
- Start persistence timer
  - when this goes off send a probe (1 byte of data)
  - it is set to the retransmission time & doubled every time a response is not received (until 60s, then sent every 60s)

# Keep alive timer

- to prevent a long idle connection between a client and a server
  - either client or server crash
- usually set to 2h

# Time-Waited Timer

- used during connection termination to allow duplicate FIN segments to be discarded at the destination
- usually 2 times the expected lifetime of a segment (MSL – maximum segment lifetime) typically 120 sec

# Previous Year Question

2.c. In TCP, EstimatedRTT, DevRTT, and TimeoutInterval adapt to changing network conditions. Suppose that three successive **SampleRTT** measurements are recorded as **110 ms, 95 ms, and 120 ms**. Given:

- initial **EstimatedRTT = 100 ms,** initial **DevRTT = 5 ms,** $\alpha = 0.125$, $\beta = 0.25$

i. Compute EstimatedRTT, DevRTT, and TCP TimeoutInterval after each SampleRTT.
ii. Explain **why it is not advisable to use the RTT of retransmitted packets** when updating EstimatedRTT and TimeoutInterval. (14) [CO3]

# Chapter 3
# Transport Layer
## UDP & QUIC

# Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



# UDP: User Datagram Protocol

- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out-of-order to app
- *connectionless:*
  - no handshaking between UDP sender, receiver
  - each UDP segment handled independently of others

**Why is there a UDP?**

- no connection establishment (which can add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
  - UDP can blast away as fast as desired!
  - can function in the face of congestion

# UDP: User Datagram Protocol

- UDP use:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
  - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
  - add needed reliability at application layer
  - add congestion control at application layer

# UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

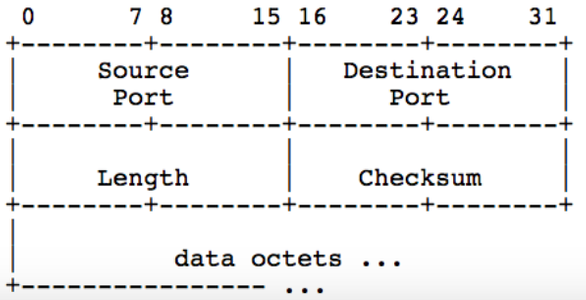
**User Datagram Protocol**

Introduction

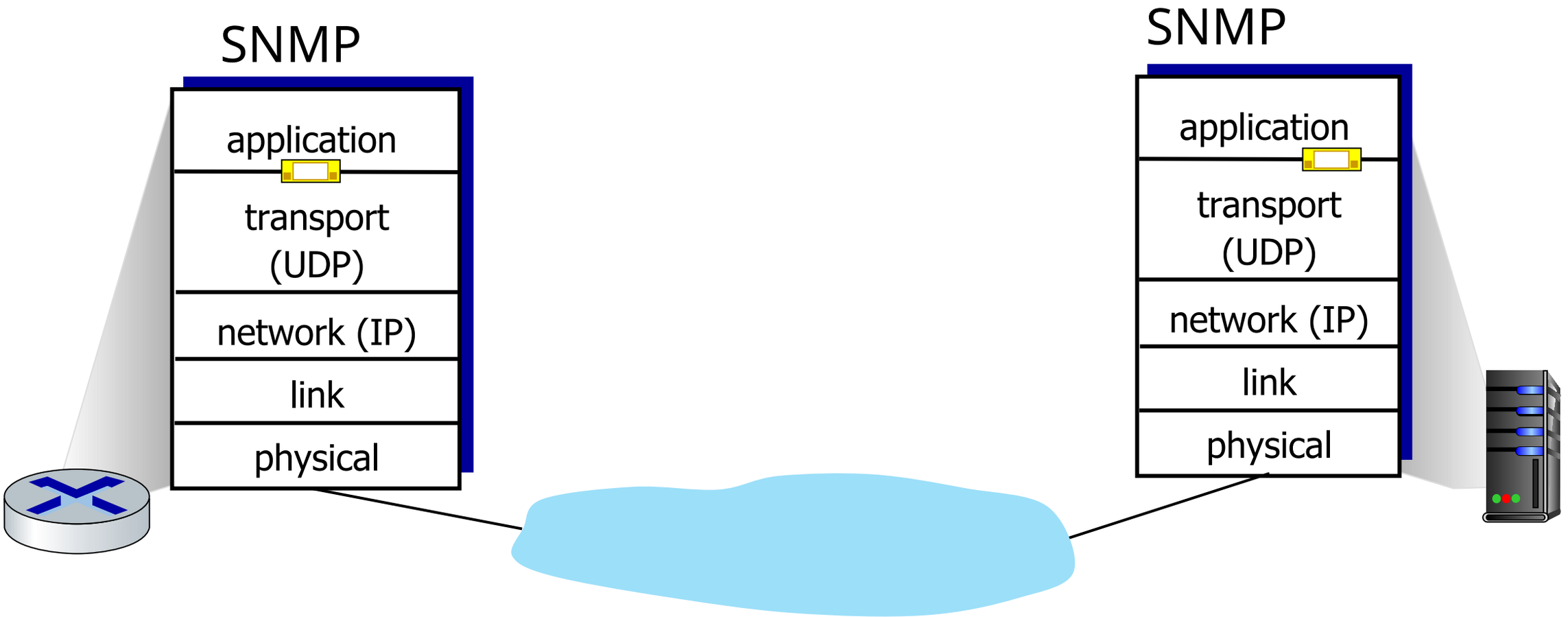
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

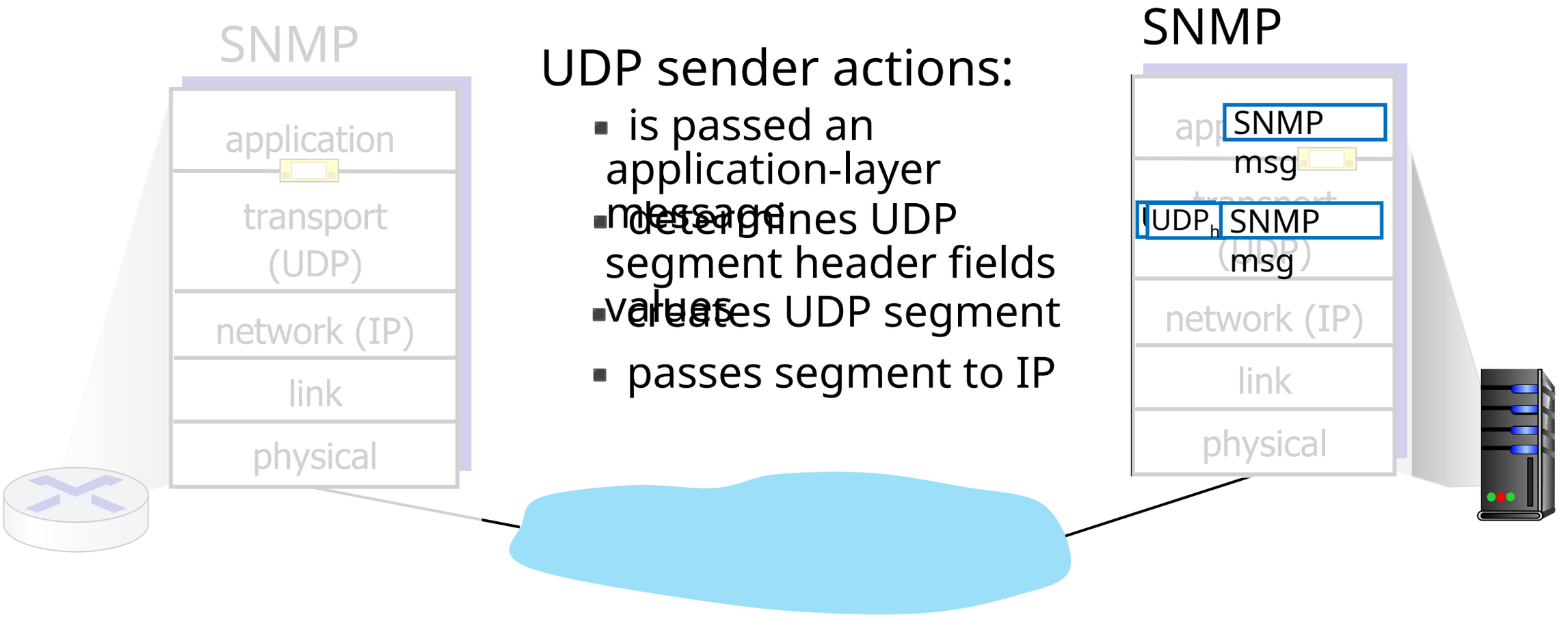
Format



# UDP: Transport Layer Actions



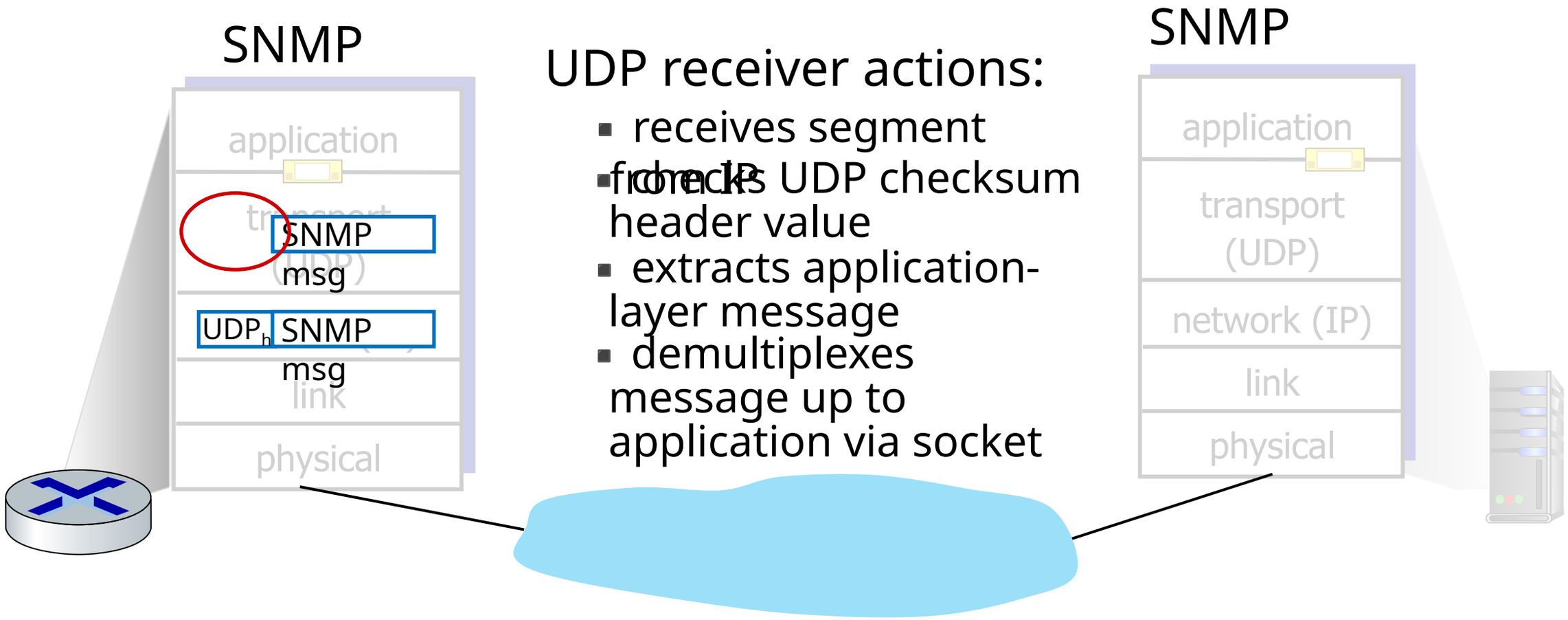
# UDP: Transport Layer Actions




UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

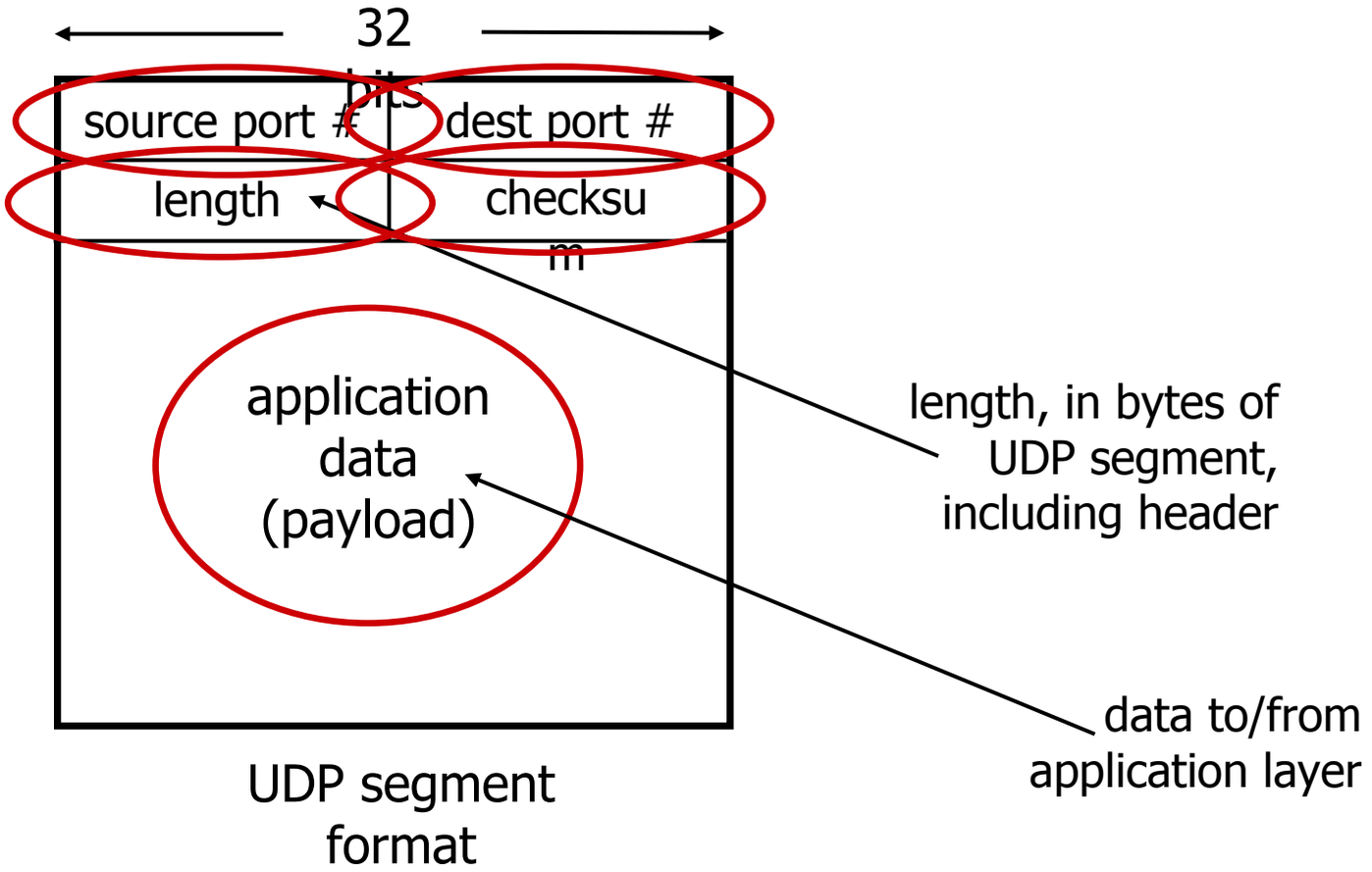
# UDP: Transport Layer Actions




UDP receiver actions:

- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

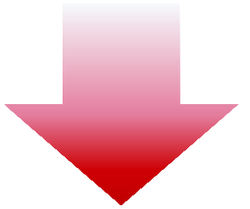
# UDP segment header



# UDP checksum

*Goal:* detect errors (*i.e.*, flipped bits) in transmitted segment

| | 1st number | 2nd number | sum |
|---|---|---|---|
| Transmitted: | 5 | 6 | 11 |



| | | | |
|---|---|---|---|
| Received: | 4 | 6 | 11 |

receiver-computed checksum ≠ sender-computed checksum (as received)



# Internet checksum

*Goal:* detect errors (*i.e.*, flipped bits) in transmitted segment

**sender:**

- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- checksum: addition (one's complement sum) of segment content
- checksum value put into UDP checksum field

**receiver:**

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - not equal - error detected
  - equal - no error detected. *But maybe errors nonetheless?* More later ....

# Internet checksum: an example

example: add two 16-bit integers

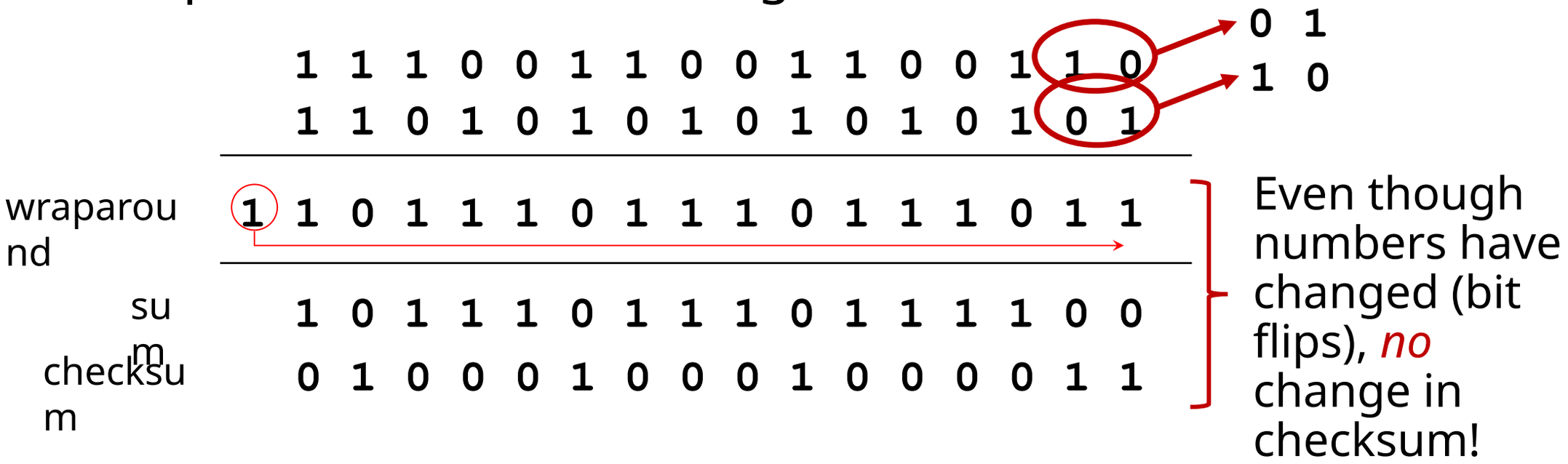
| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| wraparound | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
| sum | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| checksum | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |

*Note:* when adding numbers, a carryout from the most significant bit needs to be added to the result

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

# Internet checksum: weak protection!

example: add two 16-bit integers




Even though numbers have changed (bit flips), *no* change in checksum!

# Summary: UDP

- “no frills” protocol:
  - segments may be lost, delivered out of order
  - best effort service: “send and hope for the best”
- UDP has its plusses:
  - no setup/handshaking needed (no RTT incurred)
  - can function when network service is compromised
  - helps with reliability (checksum)
- build additional functionality on top of UDP in application layer (e.g., HTTP/3)

# Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



# Evolving transport-layer functionality

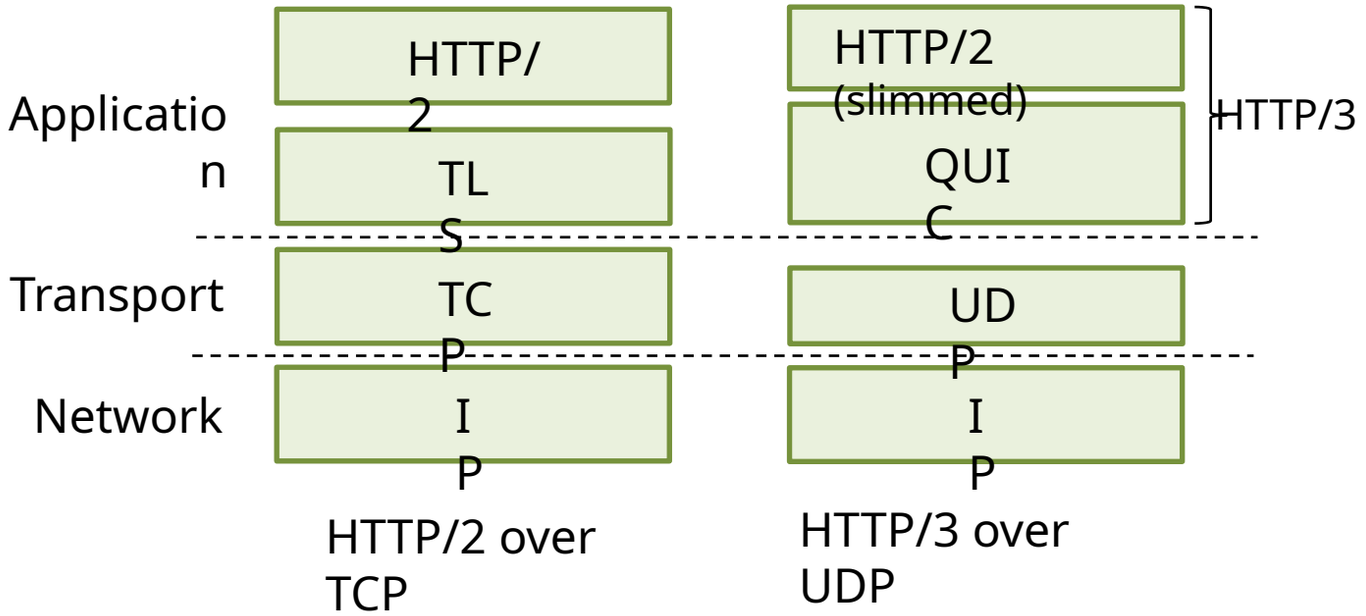
- TCP, UDP: principal transport protocols for 40 years
- different “flavors” of TCP developed, for specific scenarios:

| Scenario | Challenges |
|---|---|
| Long, fat pipes (large data transfers) | Many packets “in flight”; loss shuts down pipeline |
| Wireless networks | Loss due to noisy wireless links, mobility; TCP treat this as congestion loss |
| Long-delay links | Extremely long RTTs |
| Data center networks | Latency sensitive |
| Background traffic flows | Low priority, “background” TCP flows |

- moving transport–layer functions to application layer, on top of UDP
  - HTTP/3: QUIC

# QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
  - increase performance of HTTP
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)



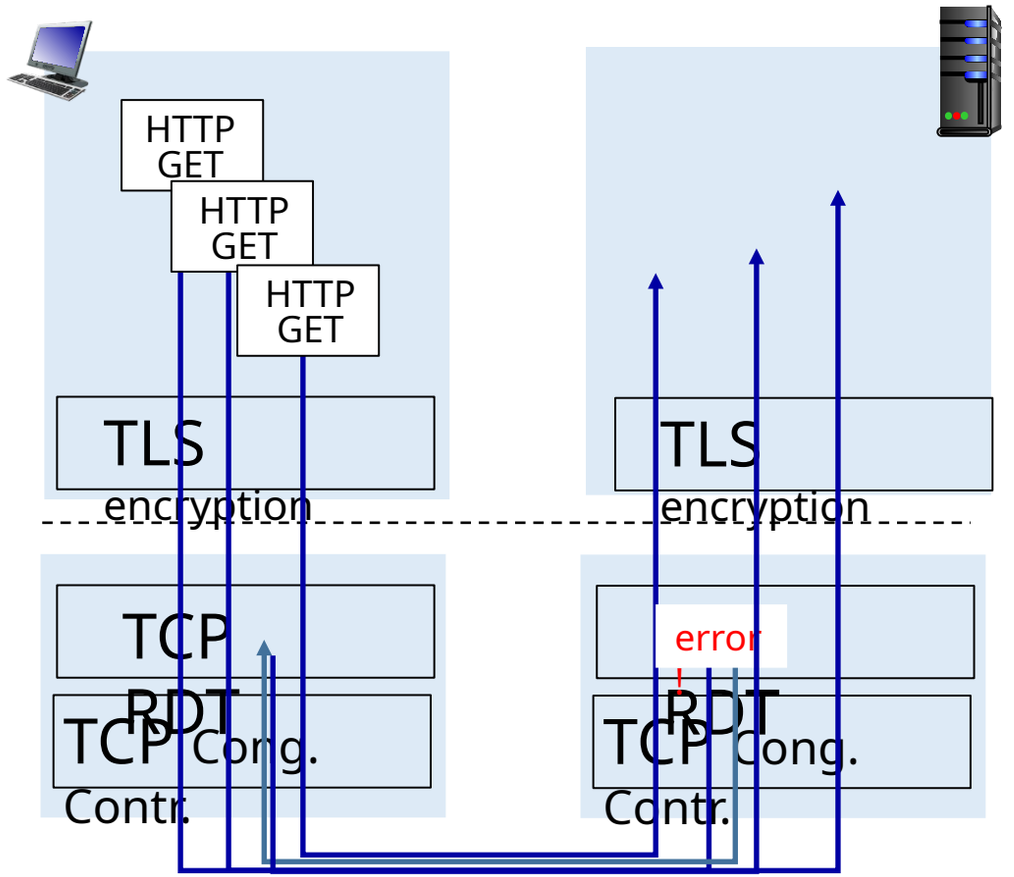
# QUIC: Quick UDP Internet Connections

adopts approaches we've studied in this chapter for connection establishment, error control, congestion control

- **error and congestion control:** "Readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones." [from QUIC specification]
- **connection establishment:** reliability, congestion control, authentication, encryption, state established in one (or even zero) RTT (recall HTTP3 discussion in Chapter 2)

- multiple application-level "streams" multiplexed over single QUIC connection
  - separate reliable data transfer, security
  - common congestion control

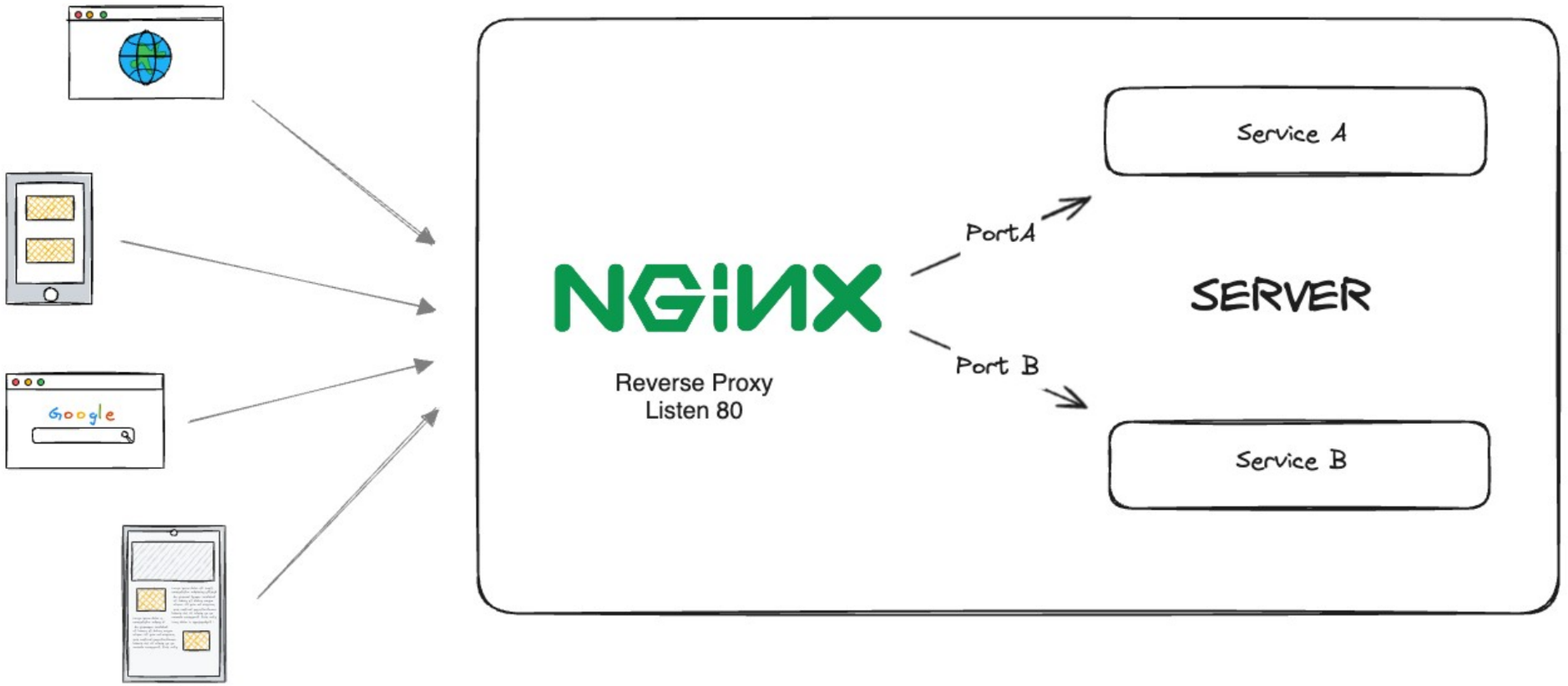
# QUIC: streams: parallelism, no HOL blocking




(a) HTTP 1.1

blocking

# Multiple Web Apps in a Single Server



# Loopback Address

127.0.0.1, commonly known as "localhost“.

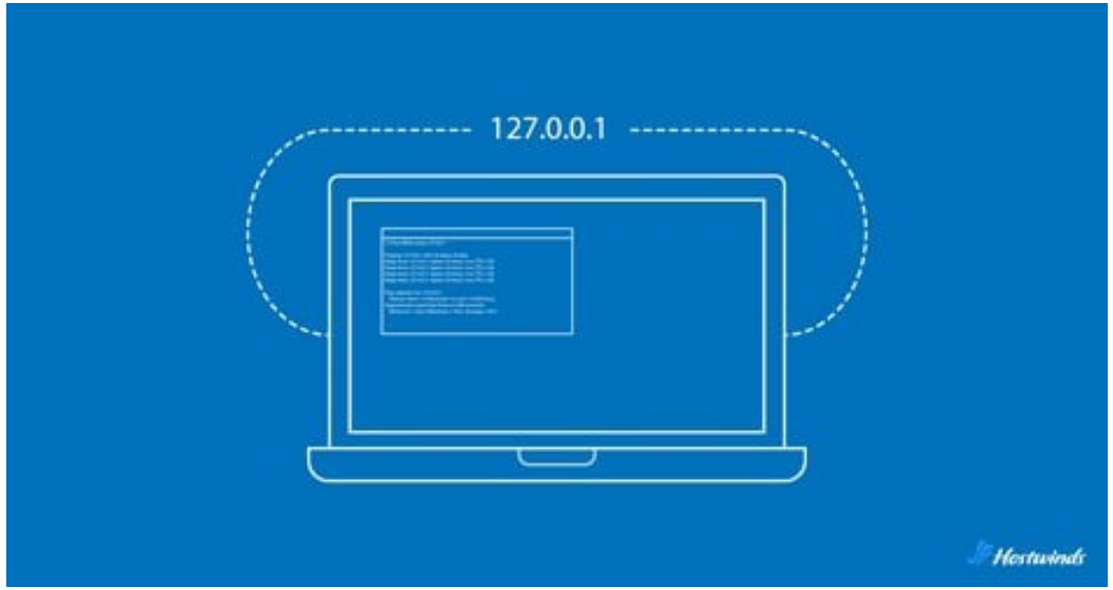
❑**Key Elements of 127.0.0.1**

- **Loopback Interface:** It allows software on the computer to communicate with itself
- **Local Communication:** Any data sent to 127.0.0.1 is looped back to the same computer, meaning it never leaves the device.
- **Universal Address:** 127.0.0.1 is standard across all devices with an IPv4 address, so using it will always point to the local machine.

# Loopback Address

## ❑ IPv4 Loopback Range

- The entire range of IPv4 loopback addresses go from 127.0.0.0 to 127.255.255.255.
- 127.0.0.1 is the most commonly referenced.



# References


1. Explaining QUIC: the protocol that is both very similar to and very different from TCP By Peter Door
2. QUIC Paper
3. QUIC Paper Overview Video
4. What is a reverse proxy?
5. Loopback Address - What is it & How Does it Work?

# Chapter 3
# Transport Layer
## Transport Layer Security (TLS)

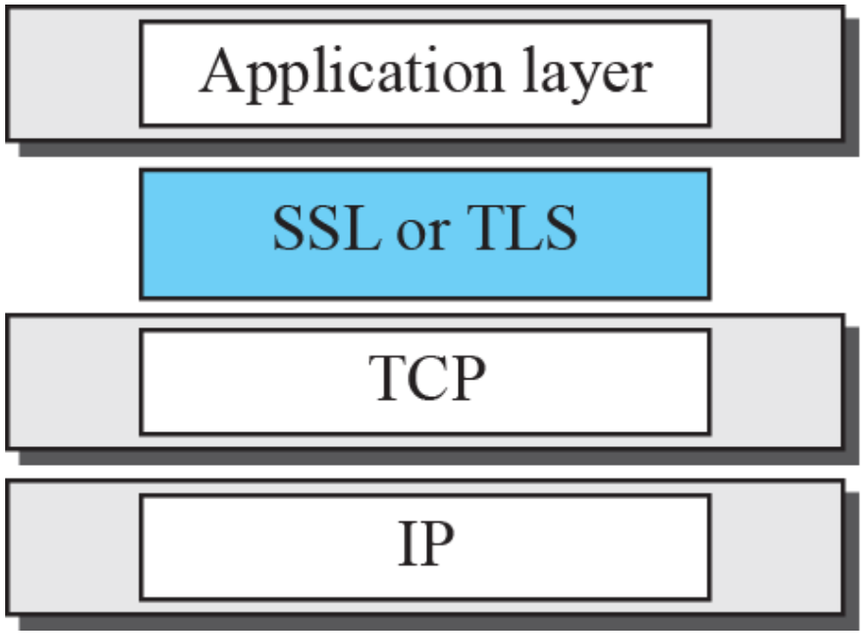
# Transport-layer security (TLS)

- widely deployed security protocol above the transport layer
  - supported by almost all browsers, web servers: https (port 443)
- provides:
  - confidentiality: via *symmetric encryption*
  - integrity: via *cryptographic hashing*
  - authentication: via *public key cryptography*

} *all techniques we have studied!*

- history:
  - Secure Socket Layer (SSL) deprecated [2015]
  - TLS 1.2 RFC 5246 [2008]
  - TLS 1.3: RFC 8846 [2018]

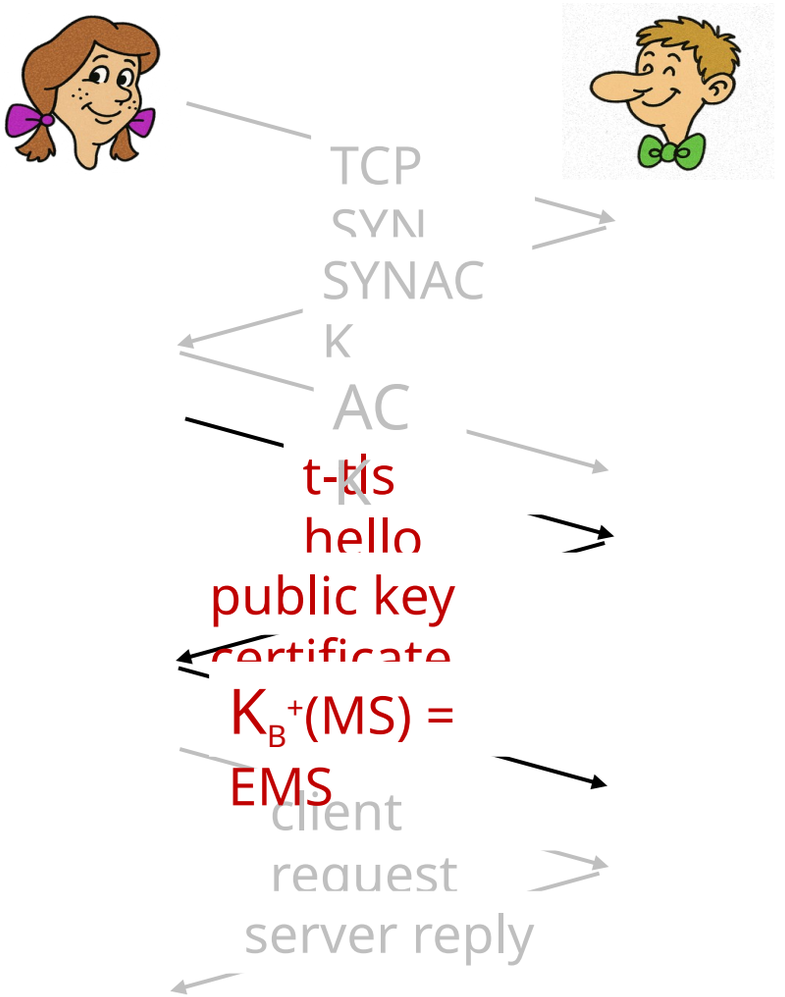
**Figure 30.15** *Location of SSL and TSL in the Internet mode*



# Transport-layer security: what's needed?

- let's *build* a toy TLS protocol, *t-tls,* to see what's needed!
- we've seen the "pieces" already:
  - handshake: Alice, Bob use their certificates, private keys to authenticate each other, exchange or create shared secret
  - key derivation: Alice, Bob use shared secret to derive set of keys
  - data transfer: stream data transfer: data as a series of records
    - not just one-time transactions
  - connection closure: special messages to securely close connection

# t-tls: initial handshake

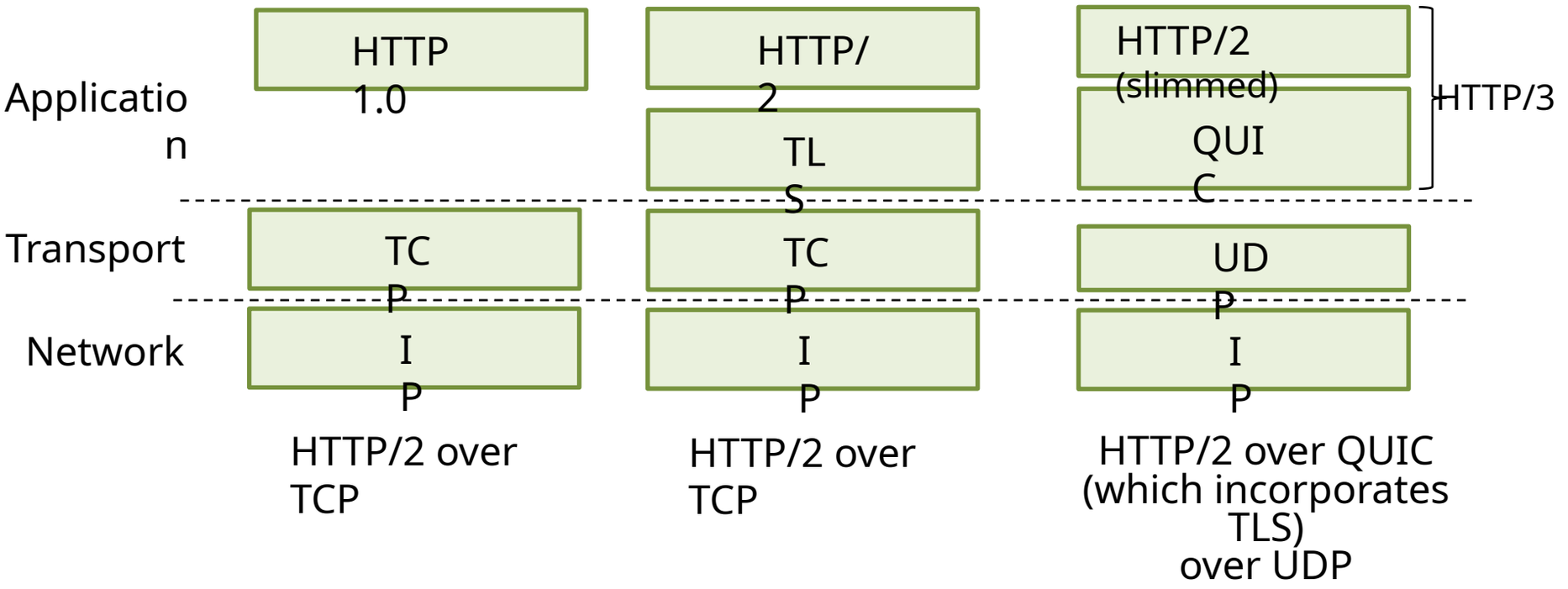



## t-tls handshake phase:

- Bob establishes TCP connection with Alice
- Bob verifies that Alice is really Alice
- Bob sends Alice a master secret key (MS), used to generate all other keys for TLS session
- potential issues:
  - 3 RTT before client can start receiving data (including TCP handshake)

# Transport-layer security (TLS)

- TLS provides an API that *any* application can use
- an HTTP view of TLS:



# TLS 1.2 Handshake

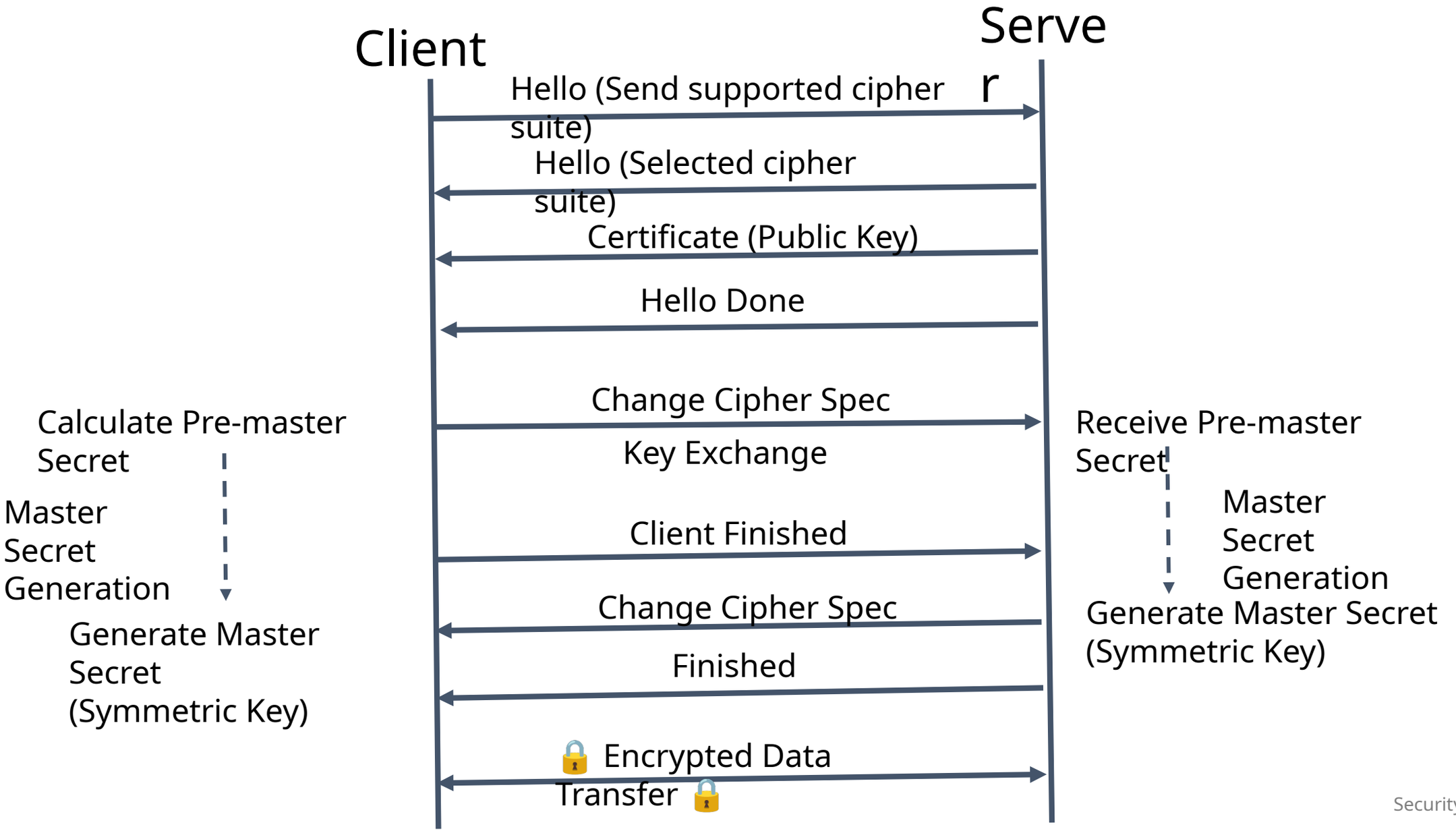


**Figure 30.16** *Calculation of maser key from pre-master secret*

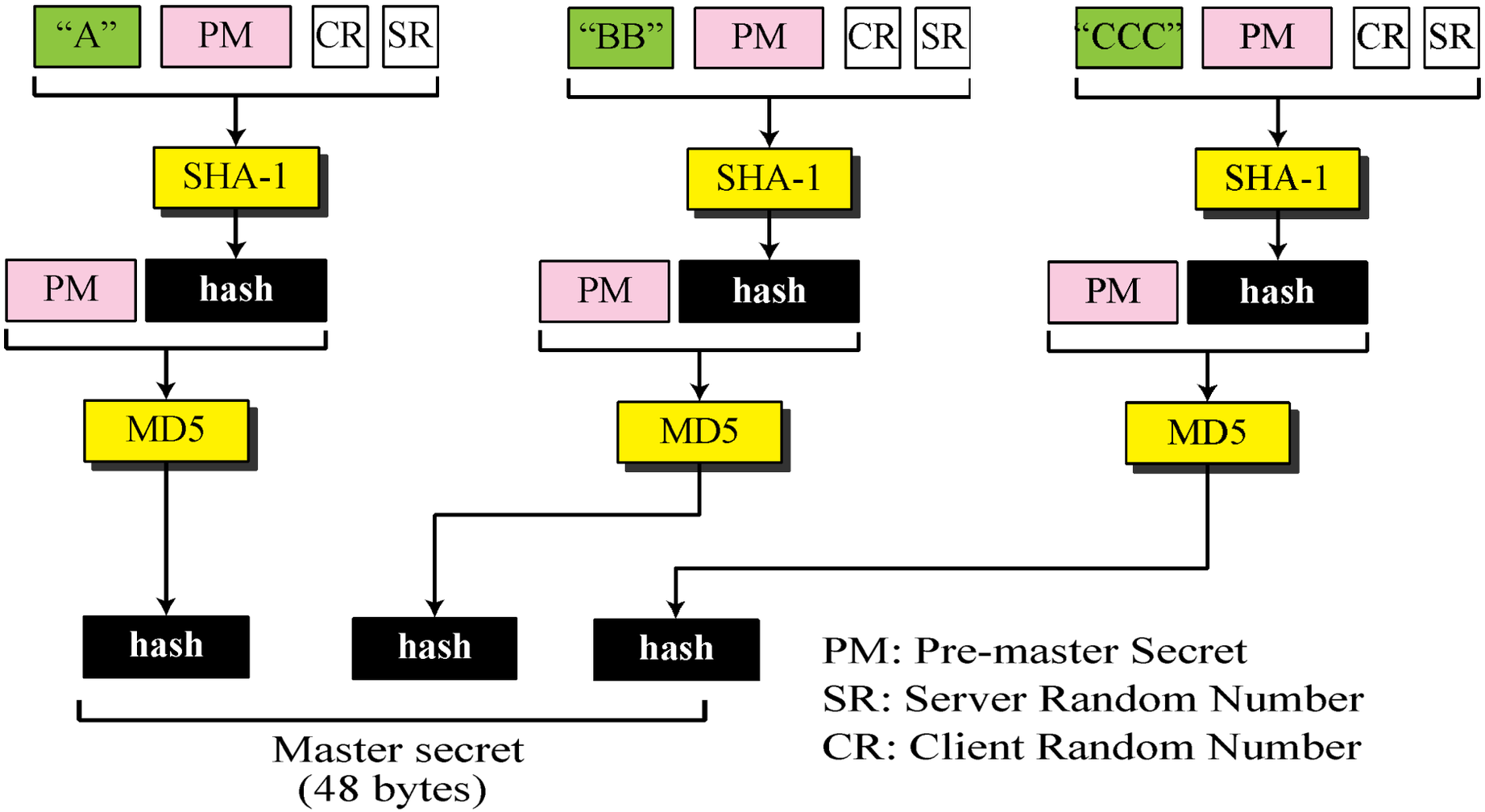


**Figure 30.20** *Handshake protocol*



**Note**

***After Phase I, the client and server know the version of SSL, the cryptographic algorithms, the compression method, and the two random numbers for key generation.***

## Note

***After Phase II, the server is authenticated to the client, and the client knows the public key of the server if required.***

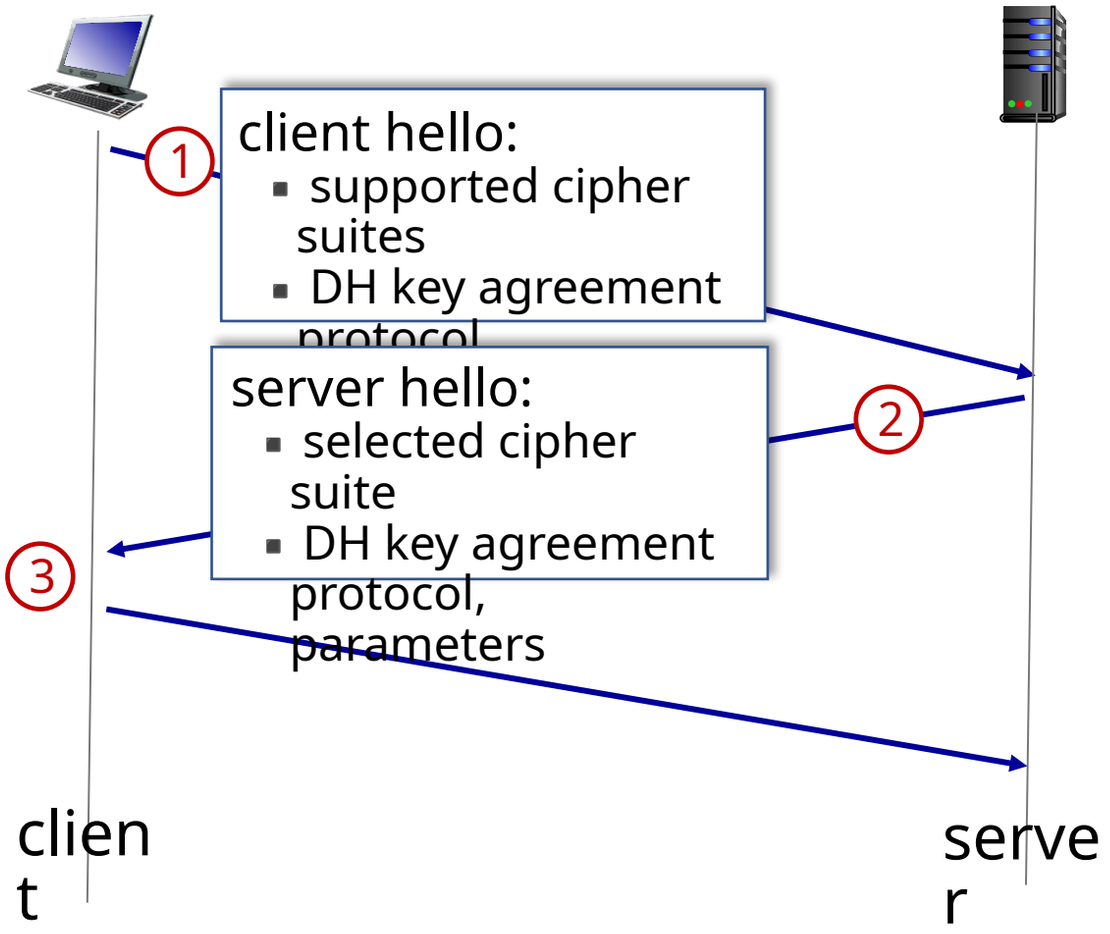
**Note**

***After Phase III, The client is authenticated for the serve, and both the client and the server know the pre-master secret.***

# TLS: 1.3 cipher suite

- "cipher suite": algorithms that can be used for key generation, encryption, MAC, digital signature
- TLS: 1.3 (2018): more limited cipher suite choice than TLS 1.2 (2008)
  - only 5 choices, rather than 37 choices
  - *requires* Diffie-Hellman (DH) for key exchange, rather than DH or RSA
  - combined encryption and authentication algorithm ("authenticated encryption") for data rather than serial encryption, authentication
    - 4 based on AES
  - HMAC uses SHA (256 or 284) cryptographic hash function

# TLS 1.3 handshake: 1 RTT

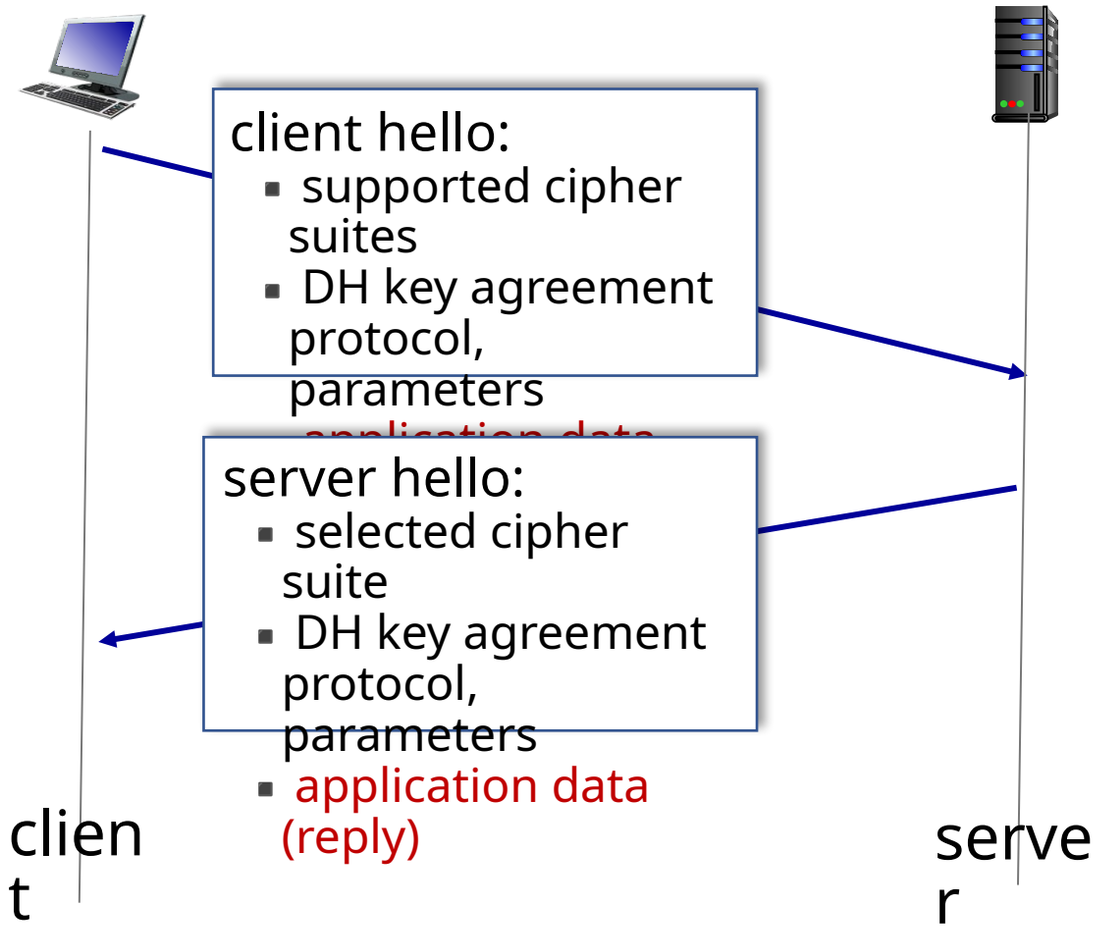



① client TLS hello msg:
- *guesses* key agreement protocol, parameters
- indicates cipher suites it supports

② server TLS hello msg chooses
- key agreement protocol, parameters
- cipher suite
- server-signed certificate

③ client:
- checks server certificate
- generates key
- can now make application request (e.g.., HTTPS GET)

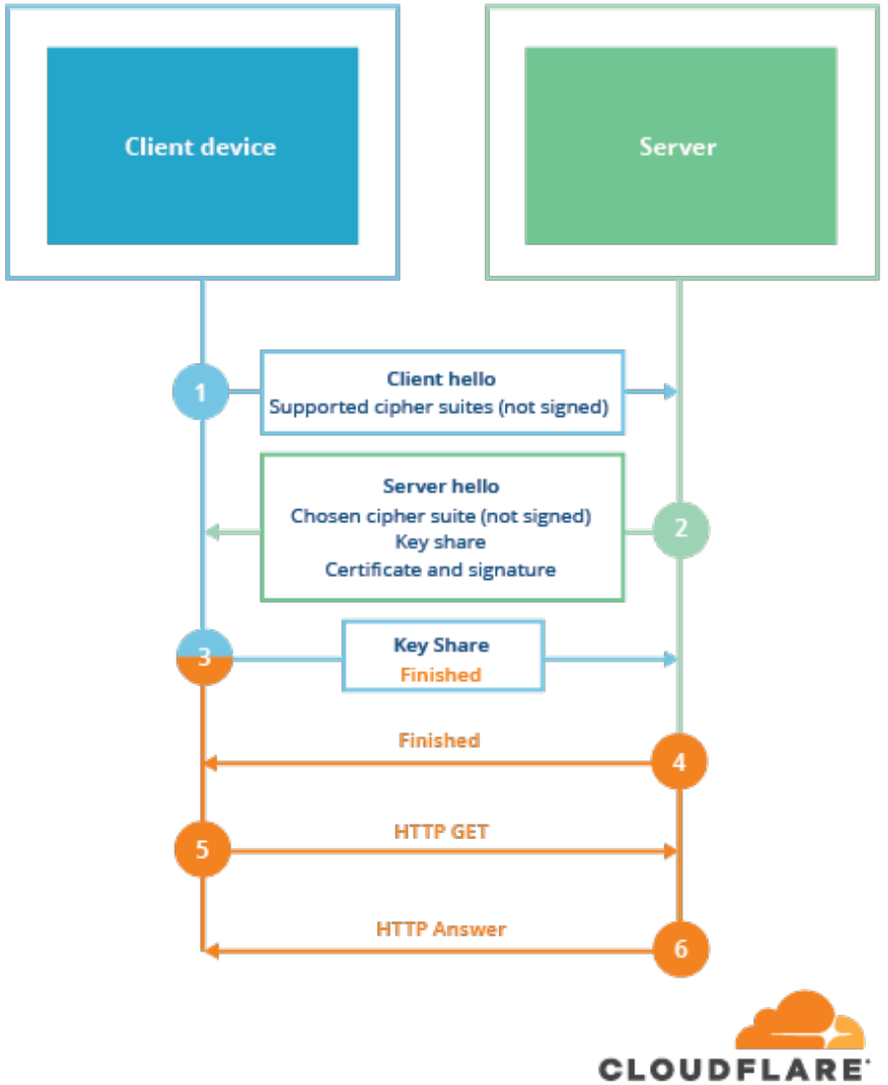
# TLS 1.3 handshake: 0 RTT



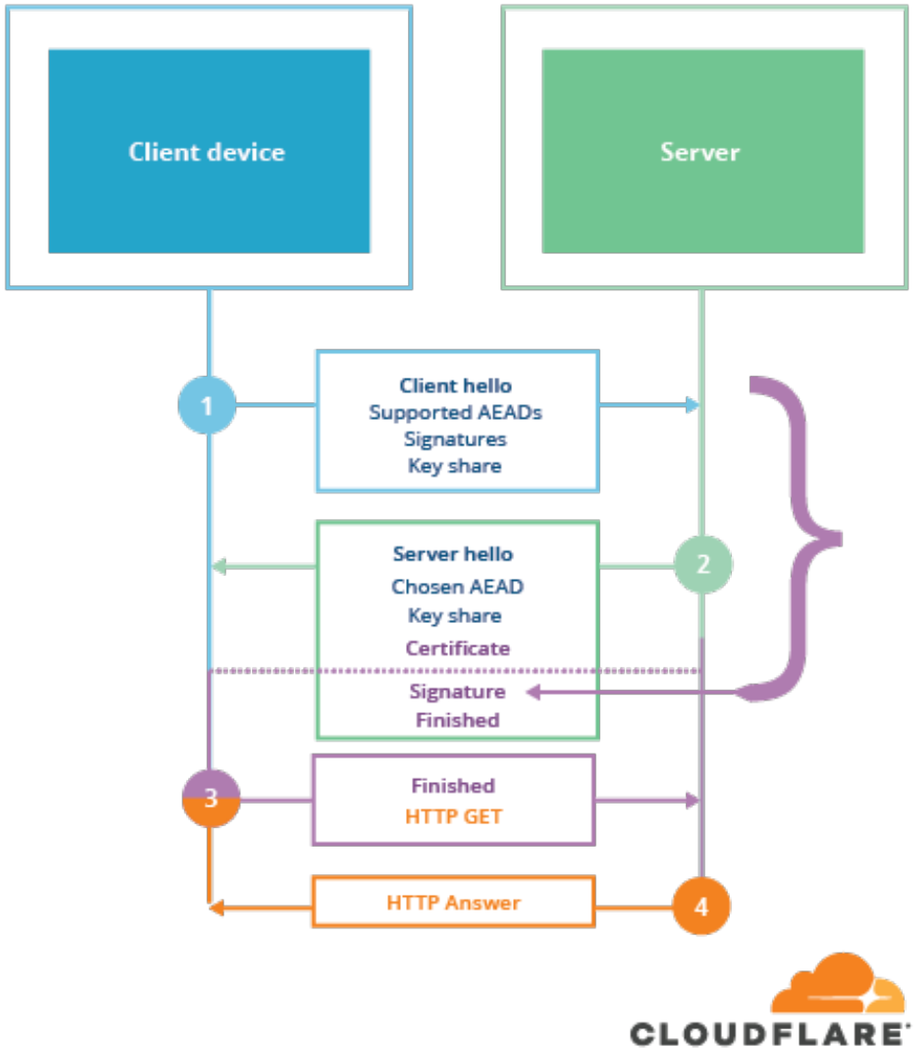

- initial hello message contains encrypted application data!
  - "resuming" earlier connection between client and server
  - application data encrypted using "resumption master secret" from earlier connection
- vulnerable to replay attacks!
  - maybe OK for get HTTP GET or client requests not modifying server state

# TLS 1.2 vs TLS 1.3

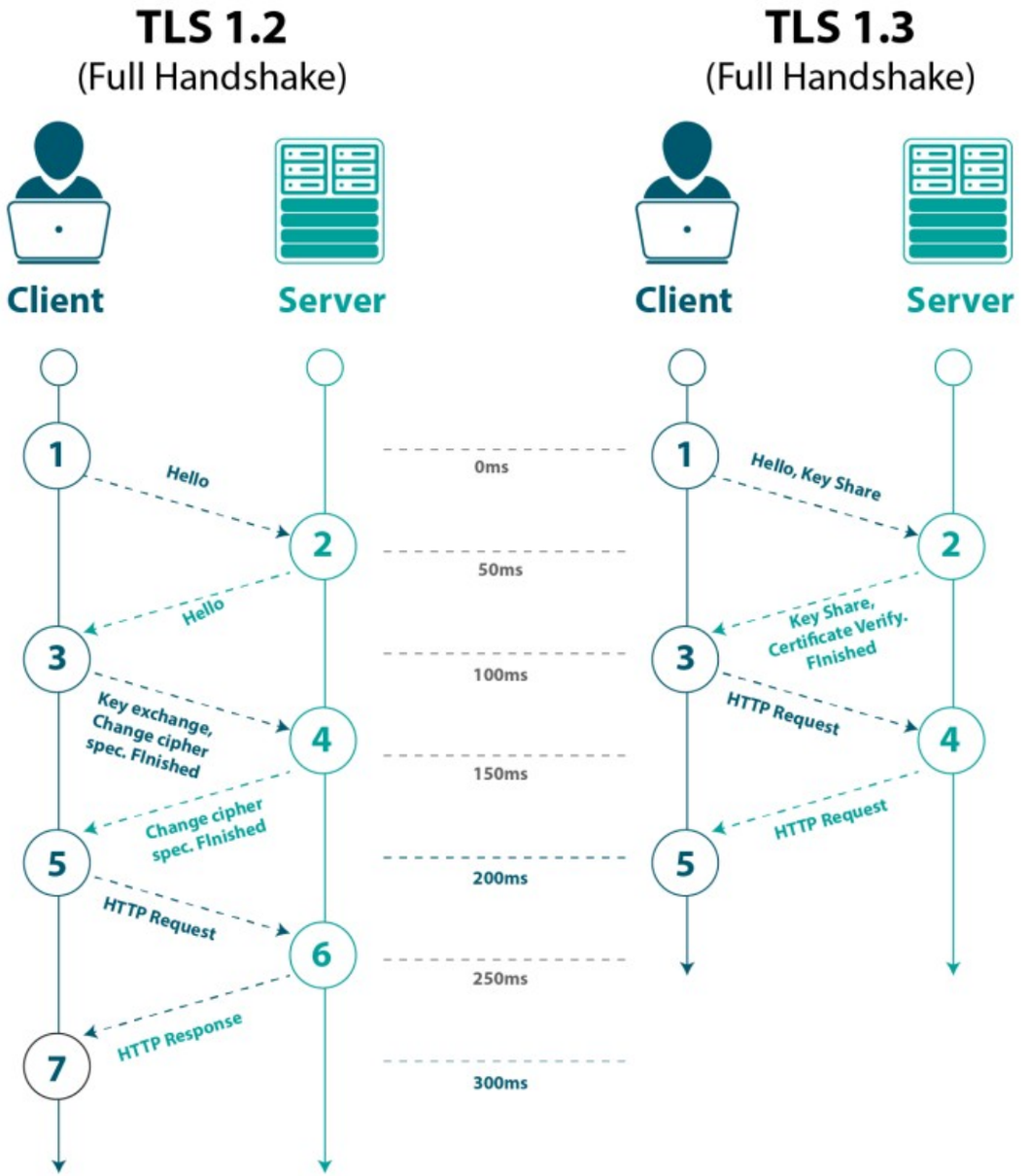
TLS 1.2 ECDHE




TLS 1.3



# TLS 1.2 vs TLS 1.3



# References


1. Computer Networking: A Top-Down Approach (Chapter 8)
2. Breaking Down the TLS 1.2 Handshake
3. TLS 1.3 Handshake
4. https://blog.cloudflare.com/rfc-8446-aka-tls-1-3/